Reference textbook: Discrete and Combinatorial Mathematics, 5th Edition
Ralph P. Grimaldi

Discrete and Combinatorial Mathematics

Full course handout

A4 · Two slides per page · Lin Cing

Contents

1. Foundations: counting, logic, sets, and integers

Chapter 1 · Fundamental principles of counting 2
Chapter 2 · Foundations of logic 19
Chapter 3 · Sets and discrete probability 36
Chapter 4 · Integers and mathematical induction 57
Chapter 5 · Relations, functions, and algorithm analysis 80

2. Relations and automata: languages, states, and orders

Chapter 6 · Languages and finite-state machines 102
Chapter 7 · Relations, orders, equivalence, and FSM minimization 125

3. Combinatorial counting: inclusion–exclusion, generating functions, and recurrences

Chapter 8 · Inclusion–exclusion, derangements, and restricted assignments 148
Chapter 9 · Generating functions 169
Chapter 10 · Recurrence relations 182

4. Graphs and trees: connectivity, optimization, and matching

Chapter 11 · Graph theory 200
Chapter 12 · Trees, coding, and depth-first search 225
Chapter 13. Optimization and matching 251

5. Algebra and applications: rings, Boolean algebras, groups, and finite fields

Chapter 14. Rings and modular arithmetic 268
Chapter 15. Boolean algebra and switching functions 288
Chapter 16. Groups, coding theory and Pólya counting 307
Chapter 17. Finite fields and combinatorial designs 337

Reference appendices: exponentials, matrices, and infinite sets

Appendix 1. Exponential and logarithmic functions 361
Appendix 2. Matrices, matrix operations, and determinants 369
Appendix 3. Countable and uncountable sets 380

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

1.1 Objects and boundary cases

Start with a finite set S. Specify whether objects are distinguishable, positions labeled, order significant, and repetition allowed, and which descriptions count as the same result. Every formula parameter encodes these choices.

Choose three objects from five distinguishable symbols.

Outcome	Order	Repetition	Count
Length-three string of distinct letters	Recorded	Forbidden	$5 \cdot 4 \cdot 3 = 60$
Length-three string	Recorded	Allowed	$5^3 = 125$
Three-element subset	Ignored	Forbidden	$\binom{5}{3} = 10$
Multiset of size three	Ignored	Allowed	$\binom{7}{3} = 35$

Use $0! = 1$. There is one empty sequence and one empty subset. Making no choices has one outcome; a set with no admissible outcome has size zero.

1.2 The sum rule is a disjoint partition

If $S = A_1 \sqcup \cdots \sqcup A_k$, then

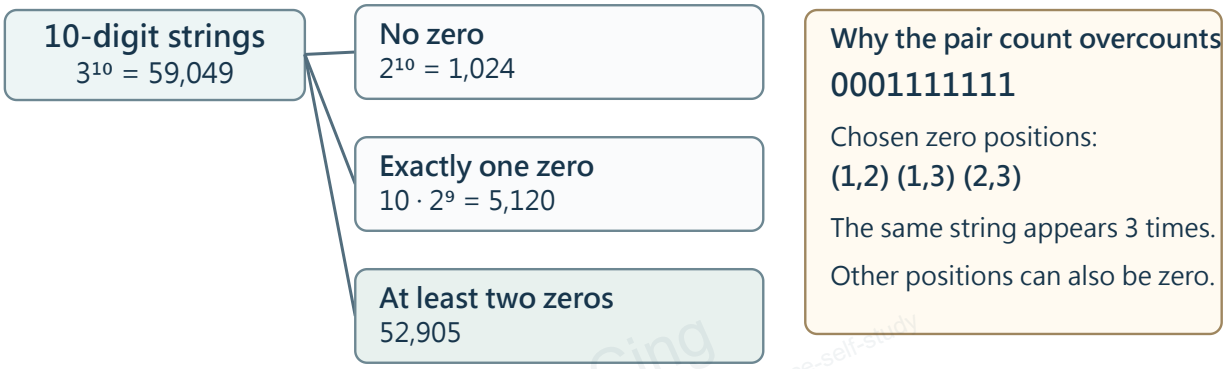
$$|S| = \sum_i |A_i|.$$

Every object lies in exactly one class, so the right side counts it once. Overlaps require repartitioning or subtraction of duplicates; omitted classes cause undercounting.

For $A \subseteq U$, the partition $U = A \sqcup (U \setminus A)$ gives $|A| = |U| - |U \setminus A|$.

A four-digit PIN may begin with zero. Of 10^4 PINs, 9^4 have no 7, leaving $10^4 - 9^4 = 3439$ with at least one 7. For four-digit integers the first digit cannot be zero. Both the universe and complement must then be recounted.

Complement counting: at least two zeros



Ten labeled positions over {0,1,2} give $3^{10} - 2^{10} - 10 \cdot 2^9 = 52\,905$. The tempting $\binom{10}{2}3^8$ counts each word with k zeros $\binom{k}{2}$ times.

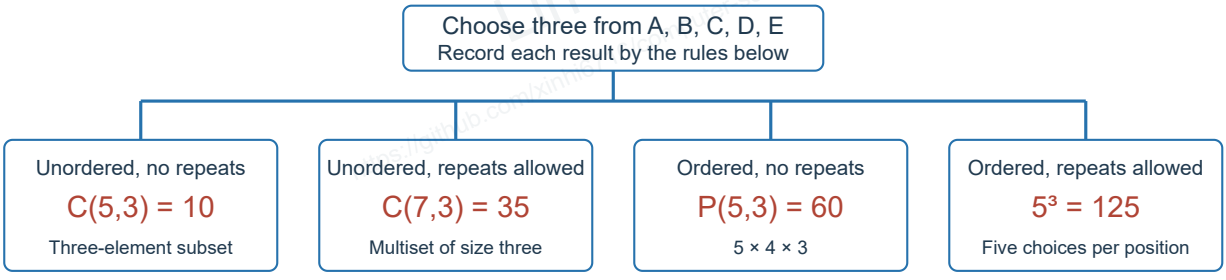
1.3 The product rule needs no probabilistic independence

Suppose every admissible prefix of length $i-1$ has exactly n_i extensions. There are $\prod_{i=1}^{r} n_i$ outcomes of length r. Induction proves this by multiplying each level's node count by its common branching count.

Available objects may depend on previous choices while their number stays fixed. Selecting distinct letters changes the second-step options, but always leaves four choices.

If first choosing A permits two continuations and choosing B permits three, the total is $2 + 3 = 5$. When branching counts differ, sum over prefix classes instead of multiplying an arbitrary count.

Five distinct letters: order and repetition define four different sample spaces



The models count strings, subsets, and multisets. Decide what data the result retains before choosing a formula.

1.4 Division requires equal fibers

Let $f : X \to Y$ be onto and give every $y \in Y$ exactly m preimages, where m is a positive integer. Its disjoint fibers $f^{-1}(y)$ partition X, so

$$|X| = m|Y|, \qquad |Y| = |X|/m.$$

A division argument must say which labels are erased and how often each result was counted. Apparent symmetry does not prove constant multiplicity.

For three-position binary necklaces, identifying rotations gives singleton classes 000 and 111 and size-three classes represented by 001 and 011, four classes in total. The noninteger $2^3/3$ fails because fibers have unequal size. Chapter 16 uses Burnside's lemma for general symmetry counting; here division requires a proof of equal fibers.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

1.5 Permutations and combinations share a forgetful map

For $n \geq 0, 0 \leq r \leq n$, ordered selection of distinct objects gives

$$P(n, r) = n(n-1)\cdots(n-r+1) = n!/(n-r)!.$$

Erasing order produces an r-subset, with exactly $r!$ permutations per subset. Thus

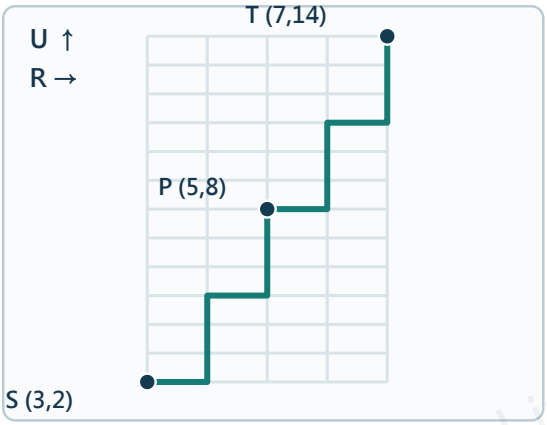
$$\binom{n}{r} = P(n, r)/r! = \frac{n!}{r!(n-r)!}.$$

For integer $n \geq 0$, use $\binom{n}{r} = 0$ if $r < 0$ or $r > n$. Do not arbitrarily put negative upper indices into the factorial formula.

When $r = 0$, both sides have only the empty object and multiplicity $0! = 1$. When $r > n$, selection without repetition is impossible.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Shortest grid paths and complements



All shortest paths
R × 4, U × 12 C(16,4) = 1,820

Paths through P
S → P: R × 2, U × 6
P → T: R × 2, U × 6

Avoiding P
1,820 − C(8,2)² = 1,036

From S=(3,2) to T=(7,14), allow only steps right R and up U. Every shortest path has 4 R steps and 12 U steps, giving $\binom{16}{4} = 1820$. A path through P=(5,8) splits uniquely into two segments, each with 2 R and 6 U steps. Thus $\binom{8}{2}^2 = 784$ paths pass through P, and $1820 - 784 = 1036$ avoid it.

1.6 Circular arrangements and reflection

Arrange $n \geq 1$ distinct objects around a circle, identifying rotations but distinguishing clockwise and counterclockwise order. Anchor a specified object and list the others clockwise. This bijection gives $(n-1)!$.

If reflection is also identified, each rotation class has a different reverse for $n \geq 3$. A reflection exchanges at least one pair of positions and cannot preserve all distinct labels. Exactly two classes merge, giving $(n-1)!/2$.

For $n = 1, 2$, only one circular arrangement exists, and reversal does not give two different rotation classes. Division by two fails. With repeated objects, one also cannot assume that every rotation gives n distinct images.

Five different people at a round table give $4! = 24$ directed circular orders, or 12 when only undirected adjacency matters and reflection is identified.

1.7 Multiset permutations

Let $n = n_1 + \cdots + n_k$, with n_i indistinguishable objects of type i. Temporarily label all copies to obtain $n!$ permutations. Erasing each type's internal labels gives exactly $\prod_i n_i!$ preimages per visible string. Therefore

$$\#\text{strings} = \frac{n!}{n_1! \cdots n_k!}.$$

The internal labels are an auxiliary construction proving equal fiber size.

TRANSMITTER has three T's, two R's, and one each of A, N, S, M, I, E, eleven letters total:

$$11!/(3!2!) = 3\,326\,400.$$

If the three T's are adjacent, contract them into a TTT block. Nine objects remain, with only R repeated, giving $9!/2! = 181\,440$. Contracting and expanding TTT are inverse maps, rather than an arbitrary reduction of the factorial.

1.8 Nonadjacency through gaps

BANANARAMA has five A's and five other letters B, N, N, R, M. Arrange the others in $5!/2! = 60$ ways, producing six gaps including both ends.

Nonadjacent A's occupy distinct gaps. Choose five of the six gaps:

$$60\binom{6}{5} = 360.$$

Deleting all A's uniquely recovers the other-letter arrangement and selected gaps, so no result is lost or duplicated.

For a fixed arrangement of m non-A objects, placing a indistinguishable nonadjacent A's gives $\binom{m+1}{a}$ possibilities, or zero if $a > m + 1$. Distinguishable A's require an additional $a!$ assignments to the selected gaps.

1.9 Compressing nonadjacent positions

Write r nonadjacent positions from $1, \dots, n$ as

$$1 \le x_1 < \cdots < x_r \le n, \quad x_{i+1} \ge x_i + 2.$$

Set $y_i = x_i - (i-1)$. Then

$$1 \le y_1 < \cdots < y_r \le n - r + 1.$$

The inverse $x_i = y_i + i - 1$ restores the restriction. For $r \ge 1, n \ge 2r - 1$, the count is $\binom{n-r+1}{r}$. For $n < 2r - 1$, it is zero; for $r = 0$, only the empty choice exists.

Three nonadjacent positions among eight give $\binom{6}{3} = 20$. Positions $(1, 4, 7)$ compress to $(1, 3, 5)$. The unrestricted $\binom{8}{3}$ includes forbidden adjacent choices.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

1.10 Symmetry, Pascal's identity, and marked subsets

Complementation bijects r-subsets and $(n-r)$-subsets, proving $\binom{n}{r} = \binom{n}{n-r}$.

For $n \ge 1$, classify by membership of a specified element:

$$\binom{n}{r} = \binom{n-1}{r} + \binom{n-1}{r-1}.$$

The two classes are disjoint and exhaustive. The out-of-range convention covers $r = 0, n$.

Count an r-subset with a marked representative, choosing the subset first or representative first:

$$r\binom{n}{r} = n\binom{n-1}{r-1}.$$

The objects are pairs (S, s), $s \in S$. Both descriptions retain the extra role.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

1.11 Vandermonde and hockey-stick identities

Choose an r-subset from disjoint groups A, B of sizes a, b. Classify by its k members from A:

$$\sum_k \binom{a}{k}\binom{b}{r-k} = \binom{a+b}{r}.$$

Inadmissible terms are zero. With $a = 3, b = 4, r = 2$, the left side is $6 + 12 + 3 = 21 = \binom{7}{2}$.

Choose $r + 1$ elements of $\{1, \ldots, n + 1\}$, classified by their largest element $i + 1$. The remaining r come from the first i elements:

$$\sum_{i=r}^{n} \binom{i}{r} = \binom{n+1}{r+1}.$$

The largest element uniquely determines the class, so terms cannot overlap. This is the hockey-stick identity for $0 \le r \le n$.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

1.12 A coefficient proof of the binomial theorem

For nonnegative integer n, with commuting variables or in a commutative ring,

$$(x + y)^n = \sum_{k=0}^{n} \binom{n}{k} x^{n-k} y^k.$$

Choose the k of the n factors supplying y; the others supply x. Choices are k-subsets, hence coefficient $\binom{n}{k}$.

Noncommuting objects need not allow combining cross terms. For matrices, $(A + B)^2 = A^2 + AB + BA + B^2$; the middle becomes $2AB$ only when $AB = BA$.

Setting $x = y = 1$ gives $\sum_k \binom{n}{k} = 2^n$. Setting $x = 1, y = -1$ gives equal numbers of even- and odd-size subsets for $n \ge 1$. Toggling a fixed element's membership is another bijection.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

1.13 Multinomial coefficients and weighted terms

For integers $m \geq 1, n \geq 0, n_i \geq 0$ with $n_1 + \cdots + n_m = n$, assign the n_i factors supplying each x_i. The count is

$$\binom{n}{n_1, \ldots, n_m} = \frac{n!}{\prod_i n_i!}.$$

Therefore

$$(x_1 + \cdots + x_m)^n = \sum_{n_1 + \cdots + n_m = n} \frac{n!}{\prod_i n_i!} \prod_i x_i^{n_i}.$$

Numerical coefficients must also be multiplied. For example,

$$[x^3 y^2 z](2x - y + 3z)^6 = \frac{6!}{3!2!1!}\, 2^3 (-1)^2\, 3 = 1440.$$

A requested total degree other than six has coefficient zero. The homogeneous-degree constraint must hold before the factorial formula applies.

1.14 Weighted sums by marking roles

The sum $\sum_k k\binom{n}{k}$ counts a nonempty subset with a representative. Choose the representative in n ways and let each other person enter or stay out. For $n \geq 1$,

$$\sum_k k\binom{n}{k} = n2^{n-1}.$$

Counting subsets with an ordered pair of distinct representatives gives, for $n \geq 2$,

$$\sum_k k(k-1)\binom{n}{k} = n(n-1)2^{n-2}.$$

As $k^2 = k(k-1) + k$,

$$\sum_k k^2\binom{n}{k} = n(n-1)2^{n-2} + n2^{n-1}.$$

For $n = 0, 1$, inspect the original sum. A form such as $0 \cdot 2^{-1}$ should not hide the counting assumptions.

1.15 Stars and bars as a decodable string

For $n \geq 1, r \geq 0$, nonnegative solutions of $x_1 + \cdots + x_n = r$ correspond to strings of r stars and $n - 1$ bars. Segment i has x_i stars; adjacent bars represent a zero entry.

For example, $(2, 0, 3, 1)$ encodes as

$$** \,|\,| \, *** \,|\, *.$$

Counting stars in each segment uniquely inverts the map. Choose $n - 1$ bar positions from $r + n - 1$:

$$\#\{x_i \geq 0,\ \sum x_i = r\} = \binom{r + n - 1}{n - 1}.$$

For $r = 0$, only the all-zero solution exists; for $n = 1$, only $x_1 = r$. For zero variables, the empty sum is zero, giving one solution for total zero and none otherwise. Handle this separately instead of using a negative upper index.

Stars and bars: decode every group

Four variables, four groups of stars

Six stars encode the total; three bars retain the empty group $x_2 = 0$.

The solution `(2, 0, 3, 1)` becomes `** | | *** | *`. Reading the four groups recovers the tuple uniquely; adjacent bars encode an empty second group. Six stars and three bars occupy nine positions, so there are C(9, 3) = 84 nonnegative integer solutions.

An upper bound: remove the forbidden solutions

All nonnegative solutions
$x_1 + x_2 + x_3 = 7$
$C(9, 2) = 36$

Forbidden: $x_1 \geq 3$
$y_1 + x_2 + x_3 = 4$
$C(6, 2) = 15$

Shift $y_1 = x_1 - 3$, giving $y_1 + x_2 + x_3 = 4$.

Valid = 36 − 15 = 21

Let x_1, x_2, x_3 be nonnegative integers summing to 7. Violating $x_1 \leq 2$ means $x_1 \geq 3$. Set $y_1 = x_1 - 3$ and invert by $x_1 = y_1 + 3$. This bijects forbidden solutions with nonnegative solutions summing to 4, giving 36 − 15 = **21**.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

1.16 Lower bounds, upper bounds, and inequalities

Throughout, n is a positive integer, and r, a_i, x_i are integers.

For $x_i \geq a_i$, put $y_i = x_i - a_i \geq 0$, leaving total $R = r - \sum_i a_i$. There are no solutions if $R < 0$; otherwise there are $\binom{R+n-1}{n-1}$. In particular, entries at least one give $\binom{r-1}{n-1}$ when $r \geq n$.

For nonnegative integers $x_1, \ldots, x_n$, with integers $n \geq 1, r \geq 0$ and $\sum_{i=1}^{n} x_i \leq r$, add slack $x_{n+1} = r - \sum_i x_i \geq 0$. This bijection to $n + 1$ variables totaling r gives $\binom{r+n}{n}$.

One upper bound can use complements. For $x_1 + x_2 + x_3 = 7,\ 0 \leq x_1 \leq 2$, all solutions number $\binom{9}{2} = 36$. Inadmissible solutions have $x_1 \geq 3$. Substituting $y_1 = x_1 - 3$ leaves total four and $\binom{6}{2} = 15$ solutions. The answer is $36 - 15 = 21$. Multiple violations can overlap and require Chapter 8's inclusion-exclusion.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

1.17 Labeled shelves and internal order

Place m distinct books on $n \geq 1$ labeled shelves, ordered left to right on each, with none empty. Permute all books in $m!$ ways and choose $n-1$ cuts among $m-1$ internal gaps:

$$m!\binom{m-1}{n-1}, \qquad m \geq n.$$

Concatenating shelves in label order uniquely recovers the permutation and cuts. For $m < n$, no placement exists.

If empty shelves are allowed, choose a length vector $x_1 + \cdots + x_n = m, x_i \geq 0$, then cut a book permutation to those lengths:

$$m!\binom{m+n-1}{n-1}.$$

Merely assigning each book to a shelf, without recording internal order, gives n^m. These models retain different data despite both being book-distribution problems.

1.18 Runs and positive compositions

A run is a maximal consecutive block of one symbol. For example,

$$110001101 = 11\,|\,000\,|\,11\,|\,0\,|\,1$$

has five runs, three of 1's and two of 0's. Suppose there are $a \geq 1$ ones and $b \geq 1$ zeros.

- For $2k$ runs, either symbol starts, with k runs of each. The count is $2\binom{a-1}{k-1}\binom{b-1}{k-1}$.
- For $2k+1$ runs, initial and final symbols agree. The count is

$$\binom{a-1}{k}\binom{b-1}{k-1} + \binom{a-1}{k-1}\binom{b-1}{k}.$$

Run lengths are positive. Their compositions and the starting symbol uniquely determine the word.

For $a = 5, b = 4$, exactly five runs give $6 \cdot 3 + 4 \cdot 3 = 30$. A nonempty single-symbol word has one run; the empty word has zero. Neither satisfies the two-positive-count assumption.

1.19 Exact counts in nested loops

Each innermost iteration increments the counter once.

```
counter = 0
for i = 1,...,n
    for j = 1,...,i
        for k = 1,...,j
            counter += 1
```

The objects are $1 \le k \le j \le i \le n$, rather than three independent choices of size n. The bijection

$$(k, j, i) \longmapsto (k, j + 1, i + 2)$$

produces three-element subsets of $\{1, \dots, n + 2\}$, giving $\binom{n+2}{3}$.

For $n = 8$, successive outer-loop counts are $1, 3, 6, 10, 15, 21, 28, 36$, totaling 120, equal to $\binom{10}{3}$. For $n = 0$, the empty loop gives zero.

1.20 Three distribution models

Fix m objects and $n \ge 1$ labeled boxes.

Object identities and recorded data	Empty boxes	Count
Distinguishable objects, destination only	Allowed	n^m
Indistinguishable objects, box counts only	Allowed	$\binom{m+n-1}{n-1}$
Distinguishable objects, internal linear order too	Allowed	$m!\binom{m+n-1}{n-1}$

If boxes are also indistinguishable, their labels are erased. Empty boxes and equal-size boxes cause unequal fibers, so division by $n!$ is not generally valid. Distinguishable objects partitioned into unlabeled nonempty blocks lead to Chapter 5's Stirling numbers. Unlabeled groups of indistinguishable objects lead to Chapter 9's integer partitions.

Keep the models distinct; the latter tools are not synonyms for stars and bars.

1.21 Catalan objects through admissible prefixes

For n pairs of parentheses, encode opening as $+1$, closing as -1, and let h_t be prefix height. Admissibility requires

$$h_0 = h_{2n} = 0, \qquad h_t \geq 0 \text{ for every } t.$$

Total height zero is insufficient. The word `())(()` has heights $1, 0, -1, 0, 1, 0$ and fails at step three.

There are $\binom{2n}{n}$ words with equal symbol counts. Subtract exactly those ever becoming negative. Equal counts do not justify declaring half admissible.

For $n = 0$, the empty word is admissible, so $C_0 = 1$. The reflection proof first takes $n \geq 1$.

1.22 Reflection and its inverse

For an inadmissible balanced word, take the first time t with $h_t = -1$. Swap opening and closing symbols in positions $1, \ldots, t$. That prefix had one extra closing symbol, so the full transformed word has $n + 1$ openings, $n - 1$ closings, and endpoint height two.

Conversely, any word with these counts must first reach $+1$, since it ends at two. Swap its prefix through that time. The result is balanced and first reaches -1 there. These operations are inverse, hence a bijection.

Reflect the prefix at the first crossing

position t	0	1	2	3	4	5	6
original	—	(	)	)	(	(	)
height h	0	1	0	-1	0	1	0
reflected	—	)	(	(	(	(	)
height h	0	-1	0	+1	+2	+3	+2

Flip steps 1–3; leave steps 4–6 unchanged

First −1 and first +1 both occur at t = 3; flip the same prefix to reverse.

In the original-word row, `())(()` first becomes negative at $t = 3$. Reflection gives `)((((`, whose first $+1$ also occurs at $t = 3$. Only the specified prefix is reflected.

1.23 The Catalan formula and values

The reflection target contains $\binom{2n}{n-1}$ words, so

$$\begin{aligned} C_n &= \binom{2n}{n} - \binom{2n}{n-1} \\ &= \left(1 - \frac{n}{n+1}\right)\binom{2n}{n} = \frac{1}{n+1}\binom{2n}{n}. \end{aligned}$$

The first terms are $1, 1, 2, 5, 14, 42, 132$.

For $n = 3$, the admissible words are

$$((())), \quad (()()), \quad (())(), \quad ()(()), \quad ()()().$$

Of $\binom{6}{3} = 20$ balanced words, $\binom{6}{2} = 15$ are inadmissible, leaving five. For $n = 6$, the count is $924 - 792 = 132$, rather than $924/2$.

1.24 Unique decomposition at the first return

A nonempty admissible word starts with an opening parenthesis. Its matching closing parenthesis is exactly the first return to height zero. Thus the word has the unique form

$$(\,U\,)\,V,$$

where U, V are admissible. If U has i pairs, V has $n - 1 - i$. Conversely the construction remains admissible, giving

$$C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i}, \qquad C_0 = 1.$$

For $n = 3$, $C_3 = C_0C_2 + C_1C_1 + C_2C_0 = 2 + 1 + 2 = 5$.

Dynamic programming computes the first n terms in $O(n^2)$ integer additions and multiplications and $O(n)$ stored integers. This is an arithmetic-operation count; growing-integer multiplication does not have constant bit cost.

1.25 Ordered full binary trees and parenthesizations

Every internal node of an ordered full binary tree has exactly a left and right child; leaves have none. With n internal nodes there are $n+1$ leaves. A single leaf proves the base $n=0$. Joining subtrees with i, j internal nodes gives $n = i + j + 1$, and induction gives $(i+1) + (j+1) = n+1$ leaves.

Encode a leaf by the empty word. If left and right subtrees encode as U, V, encode the parent as $(U)V$. First-return decomposition uniquely decodes both subtrees. Induction bijects trees with n internal nodes and admissible words with n pairs.

Place $n+1$ factors in fixed order at the leaves, with internal nodes representing binary multiplication. This gives C_n full parenthesizations, without permuting factors:

$$((ab)c)d,\ (a(bc))d,\ (ab)(cd),\ a((bc)d),\ a(b(cd)).$$

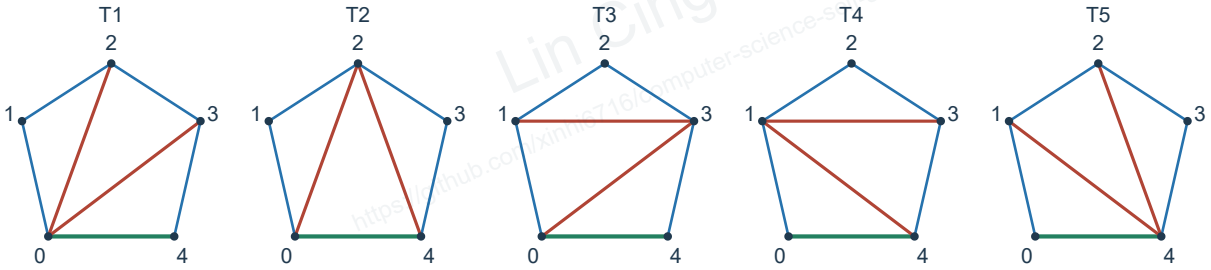
These are the five parenthesizations of four factors. Even if multiplication is associative, the evaluation structures differ.

1.26 Triangulations of a convex polygon

Fix a labeled convex $(n+2)$-gon, $n \geq 1$, with root edge 0–$(n+1)$. Each triangulation has a unique triangle on that edge. Its third vertex $i+1$ splits the rest into two smaller rooted polygons with counts C_i, C_{n-1-i}.

Conversely, the third vertex and two smaller triangulations uniquely reconstruct the original. The same Catalan recurrence results. A degenerate one-edge subproblem has count one, corresponding to C_0; a triangle has one triangulation. Equal initial values and recurrence give C_n.

Five triangulations of one labeled pentagon, each with two noncrossing diagonals



Green root edge 0–4. Its unique triangle splits the polygon into two smaller rooted polygons.

A pentagon has $C_3 = 5$ triangulations. Rotated labeled positions remain different; there is no further division by five.

1.27 Reflection with unequal endpoint counts

Let $a \geq b \geq 0$. There are $\binom{a+b}{b}$ words with a steps $+1$ and b steps -1. Reflect an inadmissible word through its first height -1. The target has $a+1$ positive and $b-1$ negative steps.

Its endpoint height is $a-b+2 \geq 2$, so the first visit to $+1$ provides the same inverse. If $b=0$, no word is inadmissible and the second term is zero by convention. Hence

$$\binom{a+b}{b} - \binom{a+b}{b-1} = \frac{a-b+1}{a+1}\binom{a+b}{b}.$$

For $a=6, b=4$, this gives $210-120=90$. Setting $a=b=n$ recovers Catalan numbers. For $a<b$, the endpoint is already negative and the answer is zero. The formula's assumptions then fail.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

1.28 Two counts of the same team

From six professors and nine doctoral students, select two professors and three students, then a leader:

$$\binom{6}{2}\binom{9}{3} \cdot 5 = 15 \cdot 84 \cdot 5 = 6300.$$

Alternatively select the leader first and classify by role:

$$\text{Professor leader: } 6\binom{5}{1}\binom{9}{3} = 2520,$$

$$\text{Student leader: } 9\binom{6}{2}\binom{8}{2} = 3780.$$

The disjoint exhaustive classes again total 6300. The three students have no ordered offices, so no extra $3!$ appears. The leader is an explicit extra mark that must be retained.

Both counts describe a team together with its leader; a team alone is a different object.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

1.29 A formula needs an encoding and inverse

The chapter's checkable relationships use disjoint classes, prefix extensions, equal fibers, forgetting order, gap compression, stars and bars, run lengths, Catalan first returns, and reflection.

For a new problem, give the value and explain how every admissible object is encoded, why encoding is unique, how to recover the object, and how empty and minimal cases behave. Finite enumeration can expose omissions or duplicates. General formulas still require bijections, partitions, or double-counting proofs.

References and attribution

Lin Cing, handwritten discrete mathematics notes. Ralph P. Grimaldi, Discrete and Combinatorial Mathematics, 5th ed., Ch. 1.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

2.1 Propositions, syntax, and valuations

Propositional variables p, q take values in $\{T, F\}$. A valuation $v : \{p, q, \ldots\} \to \{T, F\}$ determines a formula's truth recursively. Well-formed syntax comes first. Parentheses or a binding convention must fix the structure of `p → q ∧ r`.

Successive implications associate to the right. Still parenthesize complicated formulas. The implication $p \to q$ is false only for $p = T, q = F$. Thus "if it rains, the ground is wet" does not promise dry ground without rain.

Natural-language conditionals may suggest causation. Propositional logic retains only their truth tables; causal claims require another model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.1.1 Evaluating a syntax tree

List all atomic propositions and fix parentheses. For

$$\varphi = \neg p \lor (q \land r),$$

the root is $\lor$, with subtrees $\neg p$ and $q \land r$. Use precedence

$$\neg > \land > \lor > \to > \leftrightarrow .$$

Explicit parentheses still prevent visual ambiguity. For $v(p) = F, v(q) = T, v(r) = F$, evaluate inside out:

$$v(\neg p) = T, \qquad v(q \land r) = T \land F = F,$$
$$v(\varphi) = T \lor F = T.$$

A formula with n atoms has 2^n truth-table rows. Each row assigns every atom, then evaluates the tree. The same syntax accepts different valuations. Changing parentheses changes the tree and may change the formula despite similar text.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.2 Truth conditions of five connectives

For every valuation,

p	q	$p \wedge q$	$p \vee q$	$p \to q$	$p \leftrightarrow q$	$p \oplus q$
T	T	T	T	T	T	F
T	F	F	T	F	F	T
F	T	F	T	T	F	T
F	F	F	F	T	T	F

Negation $\neg p$ reverses truth. Inclusive or is $\vee$; exclusive or is $\oplus$. Natural-language "or" may be ambiguous, so mathematical notation must specify which one. For implication, identify its unique false case before filling the other three.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.3 Tautologies, equivalence, and counterexamples

A formula φ is a tautology if every valuation makes it true, and satisfiable if at least one does. These quantifiers differ. Define $\varphi \equiv \psi$ to mean equal truth values under every valuation. Check equivalence with a full truth table or truth-condition rewrites for arbitrary valuations:

$$p \to q \equiv \neg p \vee q \equiv \neg(p \wedge \neg q).$$

One valuation with different truth values refutes an equivalence claim.

The table compares three formulas under the same p, q valuations. Evaluate each row, then check the classifications and satisfiability.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.3 Tautologies, equivalence, and counterexamples (continued)

p	q	p ∨ ¬p	p ∧ ¬p	p ∧ q
T	T	T	F	T
T	F	T	F	F
F	T	T	F	F
F	F	T	F	F
Class		Tautology	Contradiction	Contingency
Satisfiable?		Yes	No	Yes

Satisfiability is weaker than tautology. The formula $p \wedge q$ is satisfiable but false when $v(p) = F$.

2.4 Logical laws and algebraic rewrites

Common laws include double negation, De Morgan, distribution, absorption, idempotence, commutativity, and associativity:

$$\neg(p \wedge q) \equiv \neg p \vee \neg q, \qquad p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r).$$

These are tautology schemata. State the schema and its location at every rewrite. The false rewrite $p \vee (q \wedge r) \equiv (p \vee q) \wedge r$ is refuted by $p = T, q = F, r = F$.

Truth tables are finite checks. Algebraic rewrites preserve visible structure but need a valid reason at every step.

2.4.1 Duality, substitution, and traceable simplification

For formulas using $\wedge, \vee, T_0, F_0$, form the dual by interchanging $\wedge, \vee$ and T_0, F_0, leaving atoms fixed. For example,

$$P = (p \vee q) \wedge T_0, \qquad P^d = (p \wedge q) \vee F_0.$$

The duality theorem says a tautology $P \Leftrightarrow Q$ gives a tautology $P^d \Leftrightarrow Q^d$. Duality differs from complementation, which also negates literals.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Duality by structural induction

For a structural-induction proof, reverse every atom in a valuation v to obtain v^c. Then

$$\neg P(v) = P^d(v^c).$$

Atoms and constants satisfy this directly. For a conjunction, the induction hypothesis and De Morgan give

$$\neg(A \wedge B) = \neg A \vee \neg B = A^d(v^c) \vee B^d(v^c),$$

which evaluates its dual. The disjunction case is symmetric. If $P(v) = Q(v)$ for every v, negate both sides and let $w = v^c$ range over all valuations to get $P^d(w) = Q^d(w)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Substitution and equivalent replacement

The substitution theorem permits consistent replacement of atoms in a tautology by compound formulas. In $p \to q \Leftrightarrow \neg p \lor q$, replace p by $r \lor s$ and q by $t \land u$. If $P \Leftrightarrow Q$, one or more occurrences of P inside another formula can likewise be replaced by Q.

To justify substitution, treat the compound formulas' truth values as the original atoms' valuation. The tautology holds at that valuation; consistent substitution preserves the truth relationships at all positions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Simplification and absorption

For example,

$$
\begin{aligned}
& \neg((p \lor q) \land r) \lor \neg q \\
\Leftrightarrow & (\neg(p \lor q) \lor \neg r) \lor \neg q && \text{De Morgan} \\
\Leftrightarrow & ((\neg p \land \neg q) \lor \neg r) \lor \neg q && \text{De Morgan} \\
\Leftrightarrow & \neg q \lor \neg r && \text{commutativity, associativity, absorption.}
\end{aligned}
$$

For the last step, rearrange to $(\neg q \lor (\neg p \land \neg q)) \lor \neg r$, then absorb $\neg p \land \neg q$ into $\neg q$. Match each main rewrite to its reason. Absorption requires the specific forms $A \lor (A \land B) \equiv A$ and $A \land (A \lor B) \equiv A$. Merely sharing a subformula is insufficient: $A \lor (A \lor B) \equiv A \lor B$, which need not equal A.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.5 Implication, converse, and inferential validity

The converse of $p \to q$ is $q \to p$; the contrapositive is $\neg q \to \neg p$ and is necessarily equivalent to the original.

An argument $\Gamma \therefore \varphi$ is valid when every valuation satisfying all premises also satisfies the conclusion. Modus ponens, $p, p \to q \therefore q$, is valid: if both premises are true, q cannot be false. Its conditional form $(p \land (p \to q)) \to q$ is a tautology.

Affirming the consequent, $q, p \to q \therefore p$, is invalid: $p = F, q = T$ makes both premises true and the conclusion false. Truth of premises and validity are different requirements; true premises alone do not make an argument sound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.6 Inference rules and proof structure

Besides modus ponens, common rules include modus tollens, hypothetical syllogism, disjunctive syllogism, and resolution. Resolving clauses $(p \lor A)$, $(\neg p \lor B)$ gives $A \lor B$. Each row below expresses a tautological implication under arbitrary substitution.

Rule	Premises	Conclusion
Modus ponens	$P, P \to Q$	Q
Modus tollens	$\neg Q, P \to Q$	$\neg P$
Hypothetical syllogism	$P \to Q, Q \to R$	$P \to R$
Disjunctive syllogism	$P \lor Q, \neg P$	Q
Conjunction introduction	P, Q	$P \land Q$
Simplification	$P \land Q$	P
Addition	P	$P \lor Q$
Resolution	$P \lor A, \neg P \lor B$	$A \lor B$
Constructive dilemma	$P \to Q, R \to S, P \lor R$	$Q \lor S$
Destructive dilemma	$P \to Q, R \to S, \neg Q \lor \neg S$	$\neg P \lor \neg R$

A complete inference trace

For $p \to q, q \to r, \neg r, s \lor p \therefore s$, the full derivation is:

Step	Statement	Reason
1	$p \to q$	Premise
2	$q \to r$	Premise
3	$\neg r$	Premise
4	$s \lor p$	Premise
5	$\neg q$	Modus tollens, 2 and 3
6	$\neg p$	Modus tollens, 1 and 5
7	s	Disjunctive syllogism, 4 and 6

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Counterexamples and inference sequences

Counterexample checking also belongs to proof assessment. The invalid $p \to q, q \therefore p$ fails at $p = F, q = T$. Validity differs from factual truth of premises. Do not reverse an implication without justification.

An inference proof is a finite sequence of premises, substitution instances of known tautologies, and rule applications to earlier lines. Treating every implication as a biconditional is a common mistake. Larger arguments can eliminate implications, convert to CNF, and apply resolution, connecting to Chapter 15.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Resolution: negate the conclusion

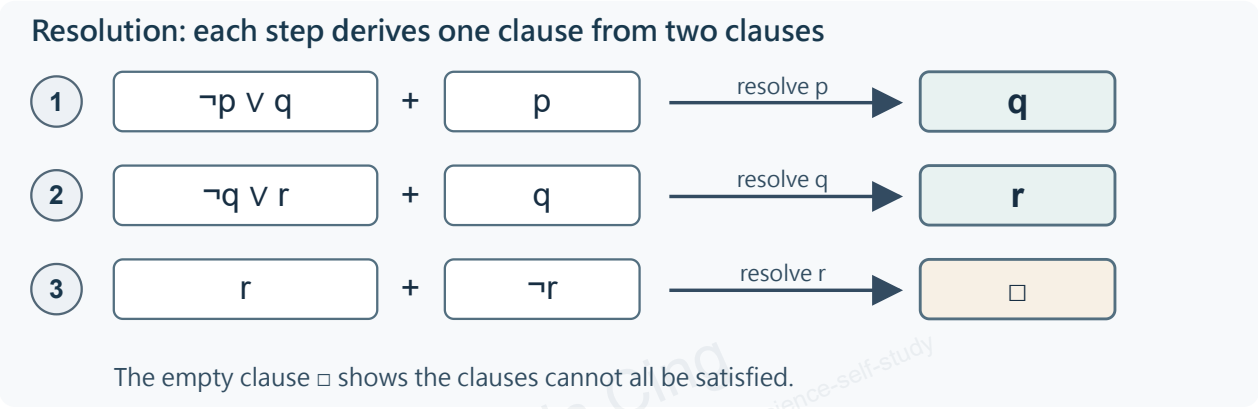
$\Gamma \models r$ exactly when $\Gamma \cup \{\neg r\}$ is unsatisfiable. A valuation making all premises true and r false is a counterexample to validity.

Let $\Gamma = \{p \to q,\ q \to r,\ p\}$. Rewrite the implications as clauses and add $\neg r$:

Clause	Source
$\neg p \lor q$	$p \to q$
$\neg q \lor r$	$q \to r$
p	Premise
$\neg r$	Negated conclusion

Resolve complementary literals. Deriving the empty clause proves that these clauses cannot all be true.

Resolution: derive the empty clause



Resolve p with $\neg p \lor q$ to obtain q; resolve q with $\neg q \lor r$ to obtain r; resolve r with $\neg r$ to obtain the **empty clause** $\square$. It is false under every valuation, so the premises and negated conclusion cannot all hold. The inference is valid.

2.7 Full quantifier forms

Fix an ambient domain U and let $D \subseteq U$. Unbounded quantifiers on the right range over U; bounded quantifiers range over D:

$$(\forall x \in D) P(x) \iff \forall x\,(x \in D \to P(x)),$$
$$(\exists x \in D) P(x) \iff \exists x\,(x \in D \wedge P(x)).$$

Negation gives

$$\neg \forall x P(x) \equiv \exists x \neg P(x), \qquad \neg \exists x P(x) \equiv \forall x \neg P(x).$$

Semantically, $\exists x P(x)$ is true when at least one domain element is a witness. A constructive proof may exhibit one; in classical logic, a nonconstructive proof may instead derive a contradiction from the absence of witnesses. A direct universal proof handles an arbitrary fixed element. If $D = \varnothing$, the bounded universal is true and the bounded existential false; both expansions still hold.

Free variables and implicit quantifiers

Distinguish open and closed statements. Over integers, $p(x) : x + 2 \text{ is even}$ has free variable x, so is not itself a proposition. Substituting three makes it false and four makes it true. In $\forall x\, p(x)$, the quantifier binds x, producing a closed statement. Declare the universe before writing $\exists x P(x)$; changing integers to positive integers or reals may change a statement's truth.

Implicit quantifiers should be restored in proofs. "The sum of two odd integers is even" means

$$\forall m \forall n\,(m, n \text{ odd} \to m + n \text{ even}).$$

This prevents substituting one example for a general argument. Bounded universals use implication and bounded existentials conjunction; exchanging them changes truth.

Negating nested quantifiers

$\neg\exists x\ \forall y\ (P(x,y) \wedge x < y)$	Original statement
$\forall x\ \exists y\ \neg(P(x,y) \wedge x < y)$	Negate quantifiers: $\exists \leftrightarrow \forall$
$\forall x\ \exists y\ (\neg P(x,y) \vee \neg(x < y))$	De Morgan's law
$\forall x\ \exists y\ (\neg P(x,y) \vee y \leq x)$	Real total order: $\neg(x < y) \Leftrightarrow y \leq x$

Let x, y range over the real numbers and P be any binary predicate. The last step uses the total order of the reals. In a general partial order, $\neg(x < y)$ need not imply $y \leq x$, so retain the negated relation.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

2.8 Nested quantifiers and order

Over reals, $\forall x \exists y (y > x)$ is true, permitting $y = x + 1$, whereas $\exists y \forall x (y > x)$ is false. The first allows dependence on x; the second requires a common witness.

$\forall x\ \exists y$ (x+y=0)

For each x, choose a y

witness：y=-x

y may depend on x

True over the reals

$\exists y\ \forall x$ (x+y=0)

Choose one y for all x

If y0 works, x=0 forces y0=0

Then x=1 forces 1=0, a contradiction

False over the reals

The binding question is whether a witness may depend on an earlier quantified variable.

Likewise $\forall x \exists y (x + y = 0)$ uses $y = -x$. A fixed y in $\exists y \forall x (x + y = 0)$ must equal zero at $x = 0$ and minus one at $x = 1$, an impossibility.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Quantifier order and dependency

Quantifiers of the same kind commute:

$$\forall x \forall y P(x, y) \Leftrightarrow \forall y \forall x P(x, y),$$
$$\exists x \exists y P(x, y) \Leftrightarrow \exists y \exists x P(x, y).$$

Mixed quantifiers do not generally commute. One valid direction is

$$\exists x \forall y P(x, y) \Rightarrow \forall y \exists x P(x, y).$$

A common witness works separately for each y; separately available witnesses need not coincide. Quantification relates closely to selecting functions. A pairing table illustrates dependence on finite domains.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Quantifier distribution: one finite counterexample

Domain D = {−1, 1}; P(x): x > 0, Q(x): x ≤ 0

x	P(x)	Q(x)	P(x) ∨ Q(x)	P(x) ∧ Q(x)
−1	F	T	T	F
1	T	F	T	F

Witnesses
P: 1
Q: −1

∀x(P(x) ∨ Q(x)) = T
(∃x P(x)) ∧ (∃x Q(x)) = T
(∀x P(x)) ∨ (∀x Q(x)) = F
∃x(P(x) ∧ Q(x)) = F

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Each element satisfies P or Q, but neither predicate holds for every element. The two existential witnesses are 1 and -1; the premises do not require a shared witness. T means true and F means false.

2.8.1 Quantifier rules and side conditions

Do not merely erase quantifiers. Explain how elements are obtained.

Rule	Form	Condition
Universal specification	$\forall x P(x) \therefore P(c)$	c denotes an element of the domain
Universal generalization	$P(c) \therefore \forall x P(x)$	c is arbitrary and fixed; no special property is used
Existential specification or elimination	Temporarily assume $P(c)$ from $\exists x P(x)$ in a subproof	c is fresh and cannot remain free in the final conclusion
Existential generalization	$P(c) \therefore \exists x P(x)$	c is a domain element or specified witness

Conditions for generalization and elimination

Arbitrary but fixed does not mean selecting a special object and generalizing it. An arbitrary triangle T cannot additionally be assumed equilateral. Universal generalization requires that c not occur free in any undischarged assumption on which the derivation depends. Assuming $P(c)$ alone does not license $\forall x P(x)$.

For existential elimination, derive C under temporary $P(c)$ with fresh c. Discharge it only if c occurs free neither in C nor in any other undischarged assumption. One cannot export $P(c)$ as a proved fact about a specified external constant.

The premises $\exists x P(x), \exists x Q(x)$ cannot share one fresh name to conclude $\exists x (P(x) \land Q(x))$. Over integers, evenness and oddness each have witnesses but no common one. Use separate fresh names unless equality of witnesses is established.

Quantified inference: example and counterexample

For example,

$$\forall x(P(x) \to Q(x)), P(a) \therefore Q(a).$$

Universal specification gives $P(a) \to Q(a)$, then modus ponens gives $Q(a)$. In contrast, all squares are rectangles and r is a rectangle do not imply r is a square:

$$\forall x(\mathrm{Square}(x) \to \mathrm{Rectangle}(x)), \quad \mathrm{Rectangle}(r).$$

Inferring $\mathrm{Square}(r)$ affirms the consequent inside quantified notation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.9 Uniqueness, vacuous truth, and boundaries

Choose y fresh, so it is not free in P. In $P(y)$, substitute y for the free occurrences of x without variable capture; rename bound variables first if needed. Expand unique existence as

$$\exists! x P(x) \iff \exists x \big(P(x) \land \forall y(P(y) \to y = x)\big).$$

If no domain element satisfies P, the statement $\forall x(P(x) \to Q(x))$ remains true vacuously. It asserts no existence. Every element of the empty set satisfies any specified predicate vacuously.

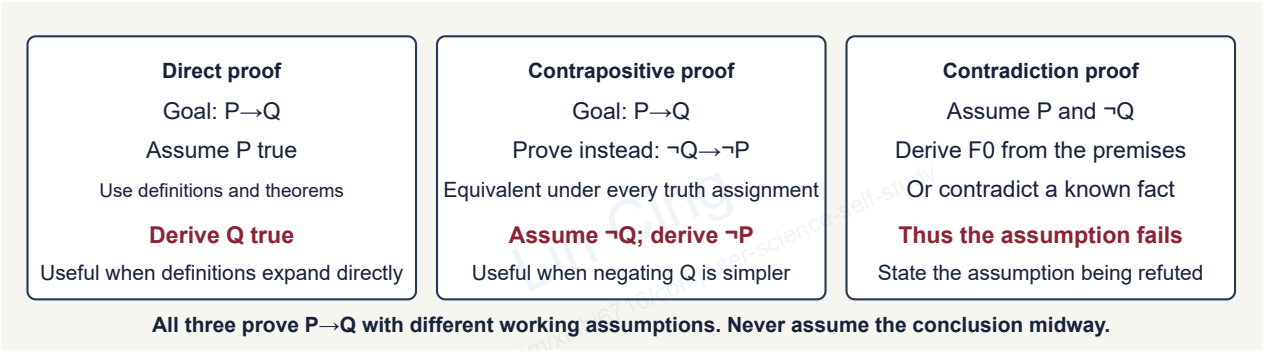
Definitions often use "if and only if." Defining even n by $n = 2k$ for some integer k gives two descriptions of one definition. For a claim $x \in A \iff x \in B$, however, prove both membership implications. Obtain

$$\forall x \, (x \in A \leftrightarrow x \in B)$$

and apply extensionality to conclude $A = B$. One direction ordinarily gives only one inclusion. Uniqueness needs both existence and at most one solution; equality of two hypothetical solutions proves only the latter.

2.10 Direct, contrapositive, and contradiction proofs

To prove $p \to q$, a direct proof assumes p and derives q; a contrapositive proof assumes $\neg q$ and derives $\neg p$; a contradiction proof assumes $p \wedge \neg q$ and derives an impossibility. All suffice logically, though their difficulty differs. Track active assumptions throughout and never silently assume the desired conclusion.



Consider

$$\forall m \in \mathbb{Z}\,(m^2 \text{ even} \to m \text{ even}).$$

Fix an arbitrary integer m and prove its contrapositive. If m is not even, it is odd, so $m = 2k + 1$ for an integer k. Then

$$m^2 = (2k+1)^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1.$$

2.10 Direct, contrapositive, and contradiction proofs (continued)

The expression in parentheses is an integer, so m^2 is odd. This proves the universal odd-square implication; contraposition proves the theorem.

For a direct proof, even m, n have $m = 2a, n = 2b$ for integers a, b. Hence

$$m + n = 2a + 2b = 2(a + b)$$

is even. Universal generalization yields the claim for all integer pairs. Name witnesses and state algebraic steps precisely enough for independent checking.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.11 Cases, existence, and counterexamples

A case proof must cover all cases allowed by the hypotheses and prove each. Cases may overlap, provided none is omitted; residue classes are naturally disjoint. One counterexample refutes a universal claim. Refuting existence requires proving no witness exists, often by universal negation.

"Every integer above one is odd or prime" fails at four, which is neither. Nine is odd, so it satisfies that disjunction despite being composite. It refutes the different claim "every odd integer above one is prime." Formalize first.

To prove $n^2 + n$ even for every integer, use exhaustive parity cases:

1. If $n = 2k$, then $n^2 + n = 4k^2 + 2k = 2(2k^2 + k)$.
2. If $n = 2k + 1$, then $n^2 + n = (4k^2 + 4k + 1) + (2k + 1) = 2(2k^2 + 3k + 1)$.

Both parentheses contain integers. The cases cover every integer, so universal generalization finishes the proof.

A constructive proof of $\exists n \in \mathbb{Z}, n^2 = n + 2$ supplies $n = 2$, since $2^2 = 2 + 2$. An existence proof by contradiction instead negates the quantified assertion and derives a contradiction.

2.11 Cases, existence, and counterexamples (continued)

For the equivalence $n \text{ even} \Leftrightarrow n^2 \text{ even}$, the forward direction uses $n = 2k$, giving $n^2 = 2(2k^2)$. The reverse uses the earlier odd-square contrapositive. Only after both directions may one write the biconditional. A counterexample must meet every hypothesis and violate the conclusion.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.12 Formalizing the theorem

Before proving a statement, specify its domain and variable types, hidden hypotheses such as nonzero inputs or connectivity, its logical form, both directions when required, and counterexample or empty-set boundaries.

For integers a, b with $0 < b \leq a$, $\gcd(a, b) = \gcd(a - b, b)$. For any positive integer d, if $d \mid a$ and $d \mid b$, then $d \mid (a - b)$; conversely, if $d \mid (a - b)$ and $d \mid b$, then $d \mid a$. Thus the pairs have the same positive common divisors and the same greatest one. At the boundary $a = b$, $a - b = 0$ and $\gcd(0, b) = b$. Exercises and full solutions are separate PDFs. Chapter 4 applies divisibility and proof methods to number theory.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.13 Constructing DNF and CNF from any truth table

Fix a positive integer $n \geq 1$ and atoms $p_1, \ldots, p_n$. For each valuation v making φ true, form the minterm

$$m_v = \bigwedge_{i=1}^{n} \begin{cases} p_i, & v(p_i) = T, \\ \neg p_i, & v(p_i) = F. \end{cases}$$

Each literal forces its coordinate, so m_v is true exactly at v. Thus at any valuation w, the disjunction $\bigvee_{v:\varphi(v)=T} m_v$ is true exactly when w is a true row of φ. This proves canonical DNF. An empty disjunction is F.

For each false row v, take $c_v = \neg m_v$, the disjunction of reversed literals. It is false only at v. Their conjunction $\bigwedge_{v:\varphi(v)=F} c_v$ excludes precisely all false rows, giving canonical CNF. An empty conjunction is T.

For $\varphi = (p \leftrightarrow q) \vee r$, valuations 000 through 111 give T, T, F, T, F, T, T, T. False rows 010 and 100 give

$$\varphi \equiv (p \vee \neg q \vee r) \wedge (\neg p \vee q \vee r).$$

Its six true-row minterms simplify to $(\neg p \wedge \neg q) \vee (p \wedge q) \vee r$. Normal forms guarantee existence, rather than minimal terms or gates.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.14 Functional completeness of NAND

Define $N(p,q) = \neg(p \wedge q)$. By idempotence, double negation and De Morgan:

$$\neg p = N(p,p), \quad p \wedge q = N(N(p,q),N(p,q)), \quad p \vee q = N(N(p,p),N(q,q)).$$

Every finite-variable Boolean function has a DNF. Replace NOT, AND and OR bottom-up in its syntax tree using these constructions; structural induction preserves the truth function.

With an input p, $T = N(p,N(p,p))$ and $F = N(T,T)$, so constants are covered. With no inputs or constant source, a binary gate cannot start a circuit; provide a constant input.

References and attribution

Lin Cing, handwritten discrete mathematics notes. Ralph P. Grimaldi, Discrete and Combinatorial Mathematics, 5th ed., Ch. 2.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

3.1 Elements, subsets, and extensionality

Sets retain neither repetitions nor order, so $\{1, 2, 2\} = \{2, 1\}$. Extensionality states

$$A = B \iff \forall x(x \in A \leftrightarrow x \in B).$$

The inclusion $A \subseteq B$ means $\forall x(x \in A \Rightarrow x \in B)$; proper inclusion $A \subsetneq B$ also requires $A \neq B$. Vacuous truth gives $\varnothing \subseteq A$. Membership $\varnothing \in A$ requires the empty set actually to be an element.

For $A = \{\varnothing, \{1\}\}$, there are two elements, with $\varnothing \in A, \{1\} \in A$, but $1 \notin A$. Also $\{\varnothing\} \subseteq A$, whereas $\{1\} \not\subseteq A$ since $1 \notin A$. Braces change the object being discussed.

Set equality is equivalent to the two inclusions. One $x \in A \setminus B$ refutes $A \subseteq B$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Membership and subset: different questions

A = {1, 2, {1, 2}, 3}

S = {1, 2}

S ∈ A: yes, S is itself a listed element

S ⊆ A: yes, both 1 and 2 belong to A

B = {1, 2}

The same S = {1, 2}

S ∈ B: no, B lists only 1 and 2

S ⊆ B: yes, both 1 and 2 belong to B

∅ has 0 elements; {∅} has 1 element. They are different.

Let $S = \{1, 2\}$. For $S \in A$, treat the entire S as one object and check whether it is listed in A. For $S \subseteq A$, check each element of S, namely $1, 2$. Both claims can hold, or only one can hold.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.2 Powersets contain subsets

Define $\mathcal{P}(A) = \{S : S \subseteq A\}$. For $A = \{a, b\}$,

$$\mathcal{P}(A) = \{\varnothing, \{a\}, \{b\}, \{a, b\}\}.$$

When $|A| = n$ is finite, subsets biject with n-bit membership vectors, giving $|\mathcal{P}(A)| = 2^n$. In particular $\mathcal{P}(\varnothing) = \{\varnothing\}$ has one element.

For any A, B,

$$\mathcal{P}(A \cap B) = \mathcal{P}(A) \cap \mathcal{P}(B),$$

since membership on the left means $S \subseteq A \cap B$, equivalently $S \subseteq A$ and $S \subseteq B$, exactly membership on the right.

Generally $\mathcal{P}(A \cup B) \neq \mathcal{P}(A) \cup \mathcal{P}(B)$. With $A = \{a\}, B = \{b\}, a \neq b$, the mixed subset $\{a, b\}$ belongs only to the left. The right side is always included in the left.

3.2.1 Gray code changes one element at a time

Membership vectors can follow reflected Gray-code order. Starting with $G_0 = (\epsilon)$, define

$$G_n = (0G_{n-1},\ 1\,\mathrm{reverse}(G_{n-1})).$$

Prefixes distinguish the two halves. Inductively, each half contains all suffixes without duplication, so together they list all 2^n vectors.

Positions	Membership vectors for three elements	Subsets for bits a,b,c
0,1	000,001	$\varnothing, \{c\}$
2,3	011,010	$\{b, c\}, \{b\}$
4,5	110,111	$\{a, b\}, \{a, b, c\}$
6,7	101,100	$\{a, c\}, \{a\}$

Within each half, adjacent suffixes differ in one bit. At the join, suffixes both equal the last member of G_{n-1}, and only the prefix differs. The first vector is 0^n and the last 10^{n-1}, so they also differ in one bit for $n \geq 1$. This cyclic adjacency order differs from increasing binary numerical order. Karnaugh maps later use this property.

3.3 Set operations and a fixed universe

Fix a universe U, with all sets subsets of it:

$$
\begin{aligned}
A \cup B &= \{x \in U : x \in A \lor x \in B\},\\
A \cap B &= \{x \in U : x \in A \land x \in B\},\\
A \setminus B &= \{x \in U : x \in A \land x \notin B\},\\
A^c &= U \setminus A,\\
A \triangle B &= (A \setminus B) \cup (B \setminus A).
\end{aligned}
$$

Symmetric difference retains elements belonging to exactly one set and corresponds to XOR.

Complements depend on the universe. For $A = \{1, 2\}$, its complement in $\{1, 2, 3\}$ is $\{3\}$, but in $\{1, 2, 3, 4\}$ it is $\{3, 4\}$. An unspecified universe leaves a complement expression incomplete.

3.4 Set laws and their duals

Property	Union form	Intersection form
Commutativity	$A \cup B = B \cup A$	$A \cap B = B \cap A$
Associativity	$(A \cup B) \cup C = A \cup (B \cup C)$	$(A \cap B) \cap C = A \cap (B \cap C)$
Idempotence	$A \cup A = A$	$A \cap A = A$
Identity	$A \cup \varnothing = A$	$A \cap U = A$
Bounds	$A \cup U = U$	$A \cap \varnothing = \varnothing$
Absorption	$A \cup (A \cap B) = A$	$A \cap (A \cup B) = A$
Distribution	$A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$	$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$
Complement	$A \cup A^c = U$	$A \cap A^c = \varnothing$
De Morgan	$(A \cup B)^c = A^c \cap B^c$	$(A \cap B)^c = A^c \cup B^c$

Translate arbitrary-element membership into Chapter 2's propositional equivalences to prove every law, rather than testing particular sets.

3.4 Set laws and their duals (continued)

The paired laws are dual. Interchanging $\cup, \cap$ and $U, \varnothing$, retaining complements, exchanges the axioms with equally valid ones. Thus every universal identity derived from them has a valid dual. Accidental equalities of special sets do not qualify. For $A = \{1, 2\}, B = C = \{1\}, D = \varnothing$, $A \cap B = C \cup D$, but $A \cup B \neq C \cap D$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.5 Element proofs of set equality

For arbitrary $x \in U$,

$$\begin{aligned} x \in A \cap (B \cup C) &\iff (x \in A) \wedge [(x \in B) \vee (x \in C)] \\ &\iff [(x \in A) \wedge (x \in B)] \vee [(x \in A) \wedge (x \in C)] \\ &\iff x \in (A \cap B) \cup (A \cap C). \end{aligned}$$

The middle step uses propositional distribution; extensionality then proves equality. Similarly,

$$x \in (A \cup B)^c \iff \neg[(x \in A) \vee (x \in B)] \iff x \in A^c \cap B^c.$$

A two-inclusion proof must take an arbitrary element in each direction. One specific Venn diagram alone does not establish the result for arbitrary sets.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Cancellation for union and symmetric difference

Union does not cancel

U = {1}, A = {1}, B = ∅, C = {1}

A ∪ B = {1} = A ∪ C

Yet B ≠ C

The common A cannot be deleted

Symmetric difference does cancel

Assume A ▵ B = A ▵ C

Take symmetric difference with A:

A ▵ (A ▵ B) = A ▵ (A ▵ C)

Associativity and A ▵ A = ∅ yield B = C

Symmetric difference is $X \triangle Y = (X \setminus Y) \cup (Y \setminus X)$. Its associativity follows pointwise from XOR on membership bits; using $\varnothing \triangle B = B$ gives the cancellation shown. The union example has equal unions but $B \neq C$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A set equality: prove both directions

For arbitrary sets A, B, C,

$$(A \cap B) \cup C = A \cap (B \cup C) \quad \Longleftrightarrow \quad C \subseteq A.$$

Necessity: If the equality holds, take any $x \in C$. It belongs to the left side and hence the right side, which is a subset of A. Thus $x \in A$, proving $C \subseteq A$.

Sufficiency: If $C \subseteq A$, then $A \cup C = A$. Distributivity gives $(A \cap B) \cup C = (A \cup C) \cap (B \cup C) = A \cap (B \cup C)$. Both directions establish the equivalence.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.6 Indexed families and empty families

For $A_i \subseteq U, i \in I$,

$$\bigcup_{i \in I} A_i = \{x \in U : \exists i \in I, x \in A_i\}, \quad \bigcap_{i \in I} A_i = \{x \in U : \forall i \in I, x \in A_i\}.$$

If $I = \varnothing$, no existential witness exists, so the empty union is $\varnothing$. The universal condition is vacuous, so the empty intersection relative to fixed U is U.

Quantifier negation gives indexed De Morgan laws:

$$\left(\bigcup_i A_i\right)^c = \bigcap_i A_i^c, \qquad \left(\bigcap_i A_i\right)^c = \bigcup_i A_i^c.$$

For $A_n = \{1, \ldots, n\} \subseteq \mathbb{N}^+$, their union is $\mathbb{N}^+$, whereas their intersection is $\{1\}$. The existential and universal membership requirements differ.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

3.7 Cartesian products and partitions

Ordered pairs satisfy $(a, b) = (c, d) \iff a = c, b = d$. Define $A \times B = \{(a, b) : a \in A, b \in B\}$, of finite size $|A||B|$. Generally $A \times B$ and $B \times A$ are different sets; swapping coordinates supplies a bijection, not equality.

A partition $\{B_i\}$ has nonempty pairwise disjoint blocks with union U. Every element belongs to exactly one. The empty universe has the empty family as its partition; adding empty blocks is forbidden.

For $\{1, 2, 3, 4, 5, 6\}$, parity gives two blocks, and residues modulo three give three. Total probability also uses partitions, with additional positive-probability requirements for conditional denominators.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

3.8 Membership regions can be empty

Membership in three sets A, B, C is a three-bit pattern. For each $\epsilon \in \{0,1\}^3$, intersect each set or its complement according to that pattern, producing eight possible regions.

Each element has exactly one pattern. Regions are disjoint and cover U, but some may be empty. Only nonempty regions are partition blocks and atoms of the finite Boolean algebra generated by the sets.

For $A = B = C = \varnothing, U \neq \varnothing$, only region 000 is nonempty, rather than eight nonempty atoms. A Venn diagram may label geometric regions zero. Their shapes do not imply existence of elements.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.9 Indicators and inclusion-exclusion

Define $\mathbf{1}_A(x) = 1$ on A, zero elsewhere. Pointwise,

$$\mathbf{1}_{A\cap B} = \mathbf{1}_A \mathbf{1}_B, \qquad \mathbf{1}_{A\cup B} = \mathbf{1}_A + \mathbf{1}_B - \mathbf{1}_{A\cap B}.$$

Summing over finite U gives

$$|A \cup B| = |A| + |B| - |A \cap B|.$$

For three sets, an element belonging to $k = 1, 2, 3$ sets receives coefficient $k - \binom{k}{2} + \binom{k}{3} = 1$; for $k = 0$, it receives zero. Hence

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|.$$

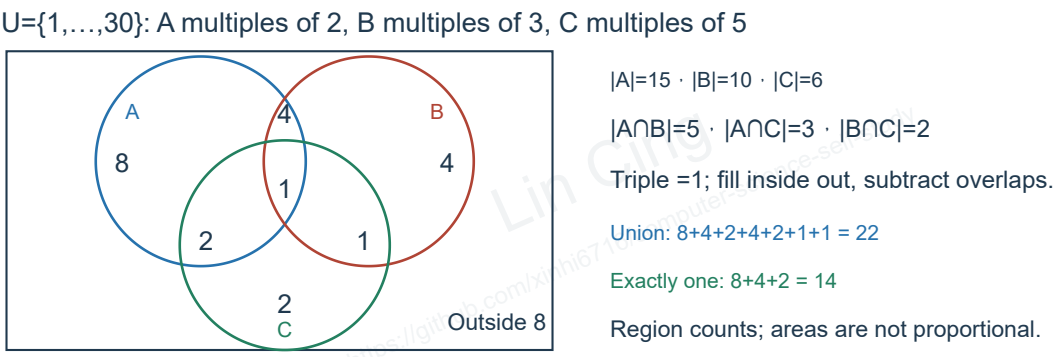
This pointwise proof does not depend on circle geometry. Chapter 8 extends it to arbitrary finite families.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.10 Checking all eight region counts

In $U = \{1, \dots, 30\}$, let A, B, C be multiples of two, three, and five. Their sizes are 15,10,6; pairwise intersections have sizes 5,3,2; the triple intersection contains only 30 and has size one.

Subtract the triple first, obtaining AB-only 4, AC-only 2, BC-only 1. Then A-only is 8, B-only 4, C-only 2, and outside 8.



The union has 22 elements, exactly-one 14, exactly-two 7, and exactly-three 1. The check $14 + 7 + 1 + 8 = 30$ matches the universe and all regions are nonnegative.

3.11 The discrete probability model

Fix a nonempty finite or countable sample space Ω, with outcome masses $p_\omega \geq 0$ summing to one. All subsets are events, with

$$P(A) = \sum_{\omega \in A} p_\omega.$$

Nonnegative sums can be rearranged, so disjoint unions add their masses. This proves normalization $P(\Omega) = 1$ and countable additivity.

Conversely, nonnegativity, normalization, and countable additivity yield this representation on countable Ω by setting $p_\omega = P(\{\omega\})$. No construction for arbitrary uncountable event spaces is needed here.

Only when finitely many outcomes are equally likely does $P(A) = |A|/|\Omega|$. A countably infinite space cannot give each singleton equal probability. Positive common mass makes the sum diverge; zero common mass sums to zero.

3.12 Basic consequences of the axioms

Since $P(\Omega) = P(\Omega \sqcup \varnothing) = P(\Omega) + P(\varnothing)$, $P(\varnothing) = 0$. The partition $\Omega = A \sqcup A^c$ gives $P(A^c) = 1 - P(A)$.

If $A \subseteq B$, then $B = A \sqcup (B \setminus A)$ gives

$$P(B) - P(A) = P(B \setminus A) \geq 0,$$

proving monotonicity. Split two events into their exclusive parts and intersection to obtain

$$P(A \cup B) = P(A) + P(B) - P(A \cap B).$$

As $P(A \cup B) \leq 1$,

$$P(A \cap B) \geq P(A) + P(B) - 1,$$

also combining with nonnegativity. If $P(A) = 0.7, P(B) = 0.5$, their intersection is at least 0.2 without any independence assumption.

3.13 Union bounds and continuity

Set $B_1 = A_1$ and $B_i = A_i \setminus \bigcup_{j<i} A_j$. These events are disjoint, retain the same union, and satisfy $B_i \subseteq A_i$. Therefore

$$P\left(\bigcup_i A_i\right) = \sum_i P(B_i) \leq \sum_i P(A_i).$$

The union bound holds for finite or countable families without independence.

For increasing $A_1 \subseteq A_2 \subseteq \cdots$, partition by successive differences. The first n terms sum to $P(A_n)$, giving

$$P(\bigcup_n A_n) = \lim_n P(A_n).$$

Apply this to complements of a decreasing sequence and use $P(A_n^c) = 1 - P(A_n)$ to get $P(\bigcap_n A_n) = \lim_n P(A_n)$.

3.14 Equally likely outcomes and unequal value probabilities

Two fair independent coin tosses have ordered outcomes $\Omega = \{HH, HT, TH, TT\}$, each of probability $1/4$. For the head count X,

$$P(X = 0) = 1/4, \quad P(X = 1) = 1/2, \quad P(X = 2) = 1/4.$$

Value one has two preimages, so the three values are not each assigned $1/3$.

Likewise two independent fair dice have 36 equally likely ordered pairs. Sum two has one preimage, while sum seven has six. Sampling two distinct objects without replacement instead uses ordered distinct pairs, excluding diagonal (i, i).

Specify outcomes and masses first, then sum preimages of events or random-variable values.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.15 Conditioning restricts and renormalizes

For $P(B) > 0$, define

$$P(A \mid B) = P(A \cap B)/P(B).$$

Restrict outcomes to B and normalize by $P(B)$. This remains a probability model on B, preserving nonnegativity, unit total mass, and disjoint additivity.

The multiplication formula is

$$P(A \cap B) = P(B)P(A \mid B).$$

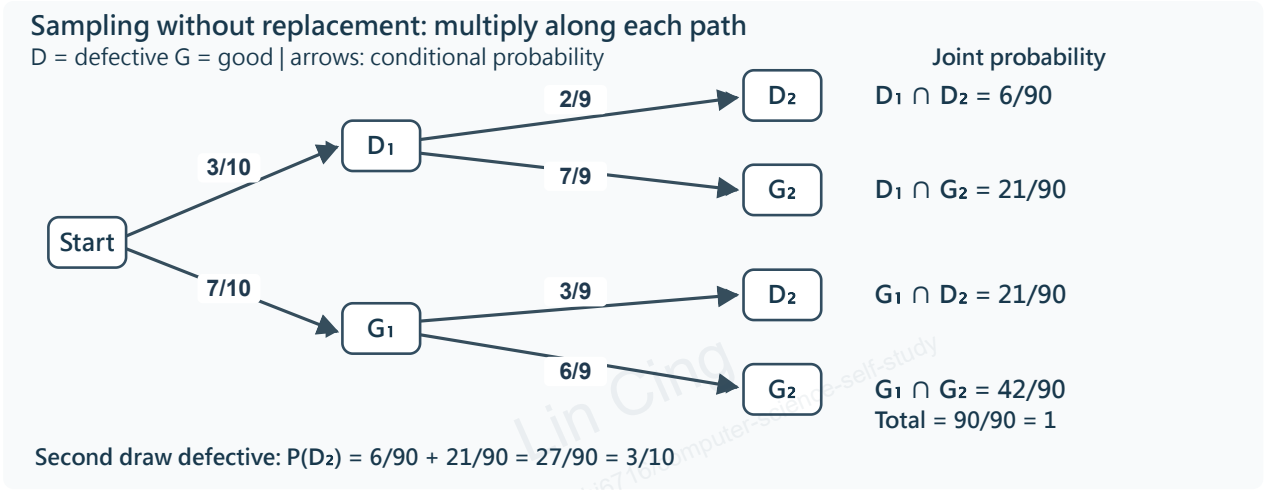
For an integer $n \geq 1$, repeated application gives the chain rule (the product is empty and equals 1 when $n = 1$)

$$P(A_1 \cap \cdots \cap A_n) = P(A_1) \prod_{i=2}^{n} P(A_i \mid A_1 \cap \cdots \cap A_{i-1}).$$

Every conditioning prefix used must have positive probability. If one prefix has probability zero, the full intersection has probability zero; undefined conditionals need not be multiplied.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.16 A complete sampling-without-replacement tree



Draw twice without replacement from ten items, three defective. $P(D_2) = 3/10$, but $P(D_2 \mid D_1) = 2/9$; the draws are not independent.

3.17 Partition proofs of total probability and Bayes

Let $B_1, B_2, \ldots$ partition Ω, with $P(B_i) > 0$ for every block used in conditioning. Since $A = \bigsqcup_i (A \cap B_i)$,

$$P(A) = \sum_i P(A \mid B_i) P(B_i).$$

Omit zero-probability blocks, whose intersections also have probability zero. Do not write an undefined conditional times zero.

For $P(A) > 0$ and fixed j,

$$P(B_j \mid A) = \frac{P(A \cap B_j)}{P(A)} = \frac{P(A \mid B_j) P(B_j)}{\sum_i P(A \mid B_i) P(B_i)}.$$

Bayes' denominator includes all disjoint sources of A. Keeping only the numerator's class incorrectly forces the posterior to one.

3.18 A Bayes example and its denominator

Assume the model $P(D) = 0.02, P(+ \mid D) = 0.95, P(+ \mid D^c) = 0.10$. Then

$$P(+) = 0.95(0.02) + 0.10(0.98) = 0.117,$$
$$P(D \mid +) = 0.019/0.117 = 19/117 \approx 16.24\%.$$

Multiplying these probabilities by 10,000 gives expected frequencies, not additional observations.

Bayes: expected counts for 10,000 people under the assumed test model

Test result	D: disease	D^c: no disease	Total
Positive +	190	980	1170
Negative −	10	8820	8830
Total	200	9800	10000

Given positive: green row

P(D|+) = 190/1170

= 19/117 ≈ 16.24%

P(+|D) = 190/200 = 95%

Both numerators are 190. P(D|+) divides by 1170 positives; P(+|D) divides by 200 disease cases.

The low prior makes the non-D population large. Expected false positives number 980, exceeding the 190 true positives. The 95% is $P(+ \mid D)$, rather than $P(D \mid +)$.

3.19 Independence differs from disjointness

Events are independent iff $P(A \cap B) = P(A)P(B)$. For $P(B) > 0$, this is equivalent to $P(A \mid B) = P(A)$. The product definition also applies to zero-probability events.

Disjoint events with positive probabilities cannot be independent: their intersection has probability zero but the product is positive. Independent events may overlap substantially. Independence also preserves complements:

$$P(A \cap B^c) = P(A) - P(A \cap B) = P(A)(1 - P(B)).$$

Swap roles and complement again to obtain all four complement combinations.

In the uniform 30-integer model, $P(A \cap B) = 5/30 = (15/30)(10/30)$, so multiples of two and three are independent despite nonempty intersection.

3.20 Pairwise independence need not be mutual

Mutual independence of $A_1, \ldots, A_n$ requires, for every nonempty index subset J,

$$P(\bigcap_{j \in J} A_j) = \prod_{j \in J} P(A_j).$$

Pairwise independence checks only size-two subsets.

On uniform $\Omega = \{00, 01, 10, 11\}$, let A mean first bit one, B second bit one, and C unequal bits.

Outcome	00	01	10	11
A	0	0	1	1
B	0	1	0	1
C	0	1	1	0

Each has probability $1/2$, and each pair intersects in one outcome, probability $1/4$. They are pairwise independent. The triple intersection is empty, with $0 \neq 1/8$, so mutual independence fails.

3.21 Random variables and induced distributions

A discrete random variable is a function $X : \Omega \to \mathbb{R}$. The event $\{X = x\}$ is its preimage $\{\omega : X(\omega) = x\}$. Nonempty fibers partition Ω, giving

$$p_X(x) = P(X = x) \geq 0, \qquad \sum_x p_X(x) = 1.$$

Different functions may have the same distribution yet different joint behavior with other variables.

For a fair bit B, set $X = B, Y = 1 - B$. Each is uniform on $\{0, 1\}$, but they are always opposite and dependent. An additional independent fair bit Z has a different joint distribution with X.

3.22 Expectation and LOTUS by grouping

If $E|X| = \sum_x |x| p_X(x) < \infty$, then

$$E[X] = \sum_x x p_X(x) = \sum_{\omega \in \Omega} X(\omega) p_\omega.$$

Group equal-value fibers to pass between forms. Absolute convergence permits rearrangement. A nonnegative variable may have expectation $+\infty$, but formulas involving subtraction here assume the needed expectations finite.

For $E|g(X)| < \infty$, the same grouping yields

$$E[g(X)] = \sum_x g(x) p_X(x).$$

This is LOTUS, the law of the unconscious statistician. No separate distribution for $g(X)$ is needed.

An expectation need not be a possible or most likely value. A fair $X \in \{0, 1\}$ has mean $1/2$, while $E[X^2] = 1/2 \neq (E[X])^2 = 1/4$.

3.23 Linearity needs no independence

For $E|X|, E|Y| < \infty$, sum on the same space:

$$\begin{aligned} E[aX + bY] &= \sum_\omega (aX(\omega) + bY(\omega)) p_\omega \\ &= aE[X] + bE[Y]. \end{aligned}$$

No factorization of joint masses occurs, so independence is unnecessary. Induction extends this to finite sums.

For event indicator I_A, $E[I_A] = P(A)$. Thus a count $N = \sum_i I_{A_i}$ has $E[N] = \sum_i P(A_i)$, even with dependent events.

The equality $E[XY] = E[X]E[Y]$ needs extra conditions. Discrete independence permits factoring $p_{X,Y}(x, y) = p_X(x) p_Y(y)$ and separating the product sum.

3.24 Variance and covariance

For $E[X^2] < \infty$ and $\mu = EX$,

$$\operatorname{Var}(X) = E[(X-\mu)^2] = E[X^2] - 2\mu E[X] + \mu^2 = E[X^2] - \mu^2.$$

Squares give nonnegativity. In a finite discrete distribution, variance is zero exactly when every positive-mass value agrees. Also $\operatorname{Var}(aX+b) = a^2 \operatorname{Var}(X)$.

For finite second moments,

$$\operatorname{Cov}(X,Y) = E[(X-EX)(Y-EY)] = E[XY] - EX\,EY.$$

Expanding a square gives

$$\operatorname{Var}(X+Y) = \operatorname{Var}(X) + \operatorname{Var}(Y) + 2\operatorname{Cov}(X,Y).$$

Independence gives zero covariance; the converse generally fails. Linearity of expectation does not justify unconditional variance addition.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

3.25 Two variance calculations for one PMF

Let masses at 0,1,2 be $c, 2c, 3c$. Normalization gives $6c = 1$, so

$$EX = 0(1/6) + 1(2/6) + 2(3/6) = 4/3,$$
$$E[X^2] = 0 + 1(2/6) + 4(3/6) = 7/3.$$

Thus variance is $7/3 - 16/9 = 5/9$. Direct centering gives

$$\frac{1}{6}\left(-\frac{4}{3}\right)^2 + \frac{2}{6}\left(-\frac{1}{3}\right)^2 + \frac{3}{6}\left(\frac{2}{3}\right)^2 = \frac{16+2+12}{54} = \frac{5}{9}.$$

The mean $4/3$ lies outside the support, which does not prevent computing squared deviations from it.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

3.26 Hypergeometric sampling without replacement

A population of N distinguishable objects contains K defects. Uniformly sample an n-subset without replacement, and let X count defects. Then

$$P(X=k)=\frac{\binom{K}{k}\binom{N-K}{n-k}}{\binom{N}{n}},\quad \max(0,n-(N-K))\le k\le \min(n,K).$$

Choose defective and good identities separately in the numerator. Vandermonde proves normalization. Assume integers N,K,n satisfy $N\ge 1, 0\le K\le N, 0\le n\le N$; the displayed support is for integer k.

For $N=10,K=3,n=2$:

k	0	1	2
$P(X=k)$	$21/45=7/15$	$21/45=7/15$	$3/45=1/15$

Hence $EX=3/5, E[X^2]=11/15, \operatorname{Var}(X)=28/75$. Dependence makes this different from Binomial$(2,3/10)$.

3.27 Negative covariance without replacement

For the same ten objects with three defects, define position indicators I_1, I_2. Each has expectation $3/10$, so $EX=3/5$ by linearity, without independence. However,

$$E[I_1I_2]=\frac{3}{10}\frac{2}{9}=\frac{1}{15},\qquad \operatorname{Cov}(I_1,I_2)=\frac{1}{15}-\frac{9}{100}=-\frac{7}{300}.$$

Each indicator has variance $21/100$, so

$$\operatorname{Var}(X)=2\frac{21}{100}+2\left(-\frac{7}{300}\right)=\frac{28}{75}.$$

Drawing one defect reduces the remaining defect proportion. This creates negative covariance and reduces count variance without changing the identical marginal defect rates at the two positions.

3.28 Bernoulli and binomial assumptions

A Bernoulli(p) variable $I \in \{0, 1\}$ has $P(I = 1) = p$, mean p, and variance $p(1 - p)$.

Suppose $I_1, \ldots, I_n$ are mutually independent, so every binary outcome tuple's joint mass factors, and all have the same $p \in [0, 1]$. Set $X = \sum I_i$. Each size-k set of success positions has probability $p^k(1 - p)^{n-k}$, with $\binom{n}{k}$ such sets. Hence

$$P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}.$$

This is Binomial(n, p), $n \geq 0$. Interpret $p = 0, 1$ by the deterministic endpoint distributions.

Linearity and independence eliminating cross covariances give

$$EX = np, \qquad \operatorname{Var}(X) = np(1 - p).$$

Equal marginals alone, or even pairwise independence alone, do not guarantee this full PMF.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.29 Geometric probabilities and tails

Within this chapter's countable model, take $\Omega = \{1, 2, \ldots\}$, let $T(t) = t$ denote the first-success time, and choose $0 < p \leq 1$, $q = 1 - p$, with

$$P(T = t) = q^{t-1} p, \quad t = 1, 2, \ldots.$$

When $p = 1$, set $P(T = 1) = 1$ directly. For $0 < p < 1$, the geometric series gives total mass $p \sum_{j \geq 0} q^j = 1$. This distribution reproduces the first-success time of independent Bernoulli trials: the first $t - 1$ fail and the tth succeeds. We do not claim that this countable space contains an entire infinite independent trial sequence.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Geometric tails and expectation

For an integer $k \geq 0$, the tail is $P(T > k) = q^k$, where $q^0 = 1$. If $0 < p < 1$, then $P(T > k) > 0$, so the conditional chance of success next, given no success yet, is

$$P(T = k + 1 \mid T > k) = \frac{q^k p}{q^k} = p.$$

This is the hazard of the first-success model. For $p = 1$, $T = 1$ almost surely; the conditional for $k \geq 1$ is not defined because its conditioning event has probability zero.

For positive integer-valued T, the pointwise identity $T = \sum_{k \geq 0} \mathbf{1}_{\{T > k\}}$ gives

$$ET = \sum_{k \geq 0} P(T > k) = \sum_{k \geq 0} q^k = 1/p.$$

The exchange uses nonnegative terms. At $p = 0$, success never occurs; this is not a normalized geometric distribution on finite positive integers.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

3.30 Geometric second moments

Pointwise, $T^2 = \sum_{k=0}^{T-1} (2k + 1)$, giving

$$E[T^2] = \sum_{k \geq 0} (2k + 1) q^k.$$

For $S = \sum_{k \geq 0} k q^k$, subtracting qS gives $S - qS = \sum_{k \geq 1} q^k = q/(1 - q)$, so $S = q/(1 - q)^2$. Therefore

$$E[T^2] = \frac{2q}{(1 - q)^2} + \frac{1}{1 - q} = \frac{2 - p}{p^2},$$

$$\operatorname{Var}(T) = \frac{1 - p}{p^2}.$$

For $p = 1/4$, mean is four and variance twelve. For $p = 1$, $T = 1$, with mean one and variance zero. Counting failures before success instead gives $T - 1$, whose mean is one less.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

3.31 Indicator proofs of Markov and Chebyshev

For nonnegative Y and $a > 0$, $Y \geq a\mathbf{1}_{\{Y \geq a\}}$ pointwise, hence

$$P(Y \geq a) \leq E[Y]/a.$$

This is Markov's inequality. Finite expectation gives a useful finite bound, which can also be capped at one.

For X with finite variance, take $Y = (X - EX)^2$, $a = t^2, t > 0$:

$$P(|X - EX| \geq t) \leq \operatorname{Var}(X)/t^2.$$

Chebyshev needs no special distribution or independence. It uses a nonnegative square and finite second moment. A right side above one remains valid but is uninformative.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

3.32 Joint, marginal, and conditional PMFs

For discrete X, Y, let $p(x, y) = P(X = x, Y = y)$. Summing disjoint fibers gives

$$p_X(x) = \sum_y p(x, y), \qquad p_Y(y) = \sum_x p(x, y).$$

For $p_Y(y) > 0$, define $p_{X|Y}(x|y) = p(x, y)/p_Y(y)$ and $E[X|Y = y] = \sum_x x\, p_{X|Y}(x|y)$, assuming the required absolute sums finite.

Grouping by the partition induced by Y gives total expectation:

$$E[X] = \sum_y E[X|Y = y] p_Y(y).$$

No independence is needed. Zero-probability values need no conditional definition and are omitted from the sum.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

3.33 Why the total-variance cross term vanishes

Assume $E[X^2] < \infty$, and write $m(y) = E[X|Y = y]$. Conditional on $Y = y$,

$$X - EX = (X - m(y)) + (m(y) - EX).$$

After squaring and taking conditional expectation, the cross term is

$$2(m(y) - EX)E[X - m(y)|Y = y] = 0.$$

Thus the conditional squared deviation is $\mathrm{Var}(X|Y = y) + (m(y) - EX)^2$. Average over Y:

$$\mathrm{Var}(X) = E[\mathrm{Var}(X|Y)] + \mathrm{Var}(E[X|Y]).$$

The first term averages within-group variances. The second measures variation of group means. Averaging only within-group variances misses the second contribution.

All terms are finite: nonnegative conditional variance gives $m(y)^2 \le E[X^2|Y = y]$. Averaging yields $E[m(Y)^2] \le E[X^2] < \infty$, avoiding undefined infinity subtraction.

Total variance: compute within and between groups

Conditional groups

Group probability	Conditional mean	Within-group variance
Y = 0 : 2/3	E[X | Y = 0] = 1/4	Var(X | Y = 0) = 3/16
Y = 1 : 1/3	E[X | Y = 1] = 3/4	Var(X | Y = 1) = 3/16

Overall mean = 5/12

Weighted within-group variance = 3/16

Variance of group means = 1/18

Total variance = 3/16 + 1/18 = 35/144

Check via the marginal distribution：P(X = 1) = 5/12; P(X = 0) = 7/12

Let $P(Y = 0) = 2/3$, $P(Y = 1) = 1/3$, $X \mid Y = 0 \sim \mathrm{Bernoulli}(1/4)$, and $X \mid Y = 1 \sim \mathrm{Bernoulli}(3/4)$. Both conditional variances are $3/16$, so their weighted average is $(2/3)(3/16) + (1/3)(3/16) = 3/16$.

Between-group variance and marginal check

First calculate the overall mean and the two deviations:

$$\mu = \frac{2}{3}\frac{1}{4} + \frac{1}{3}\frac{3}{4} = \frac{5}{12}, \qquad \frac{1}{4} - \mu = -\frac{1}{6}, \quad \frac{3}{4} - \mu = \frac{1}{3}.$$

Between-group variance is the weighted average of squared deviations, with weights $2/3$ and $1/3$:

$$\operatorname{Var}(E[X \mid Y]) = \frac{2}{3}\frac{1}{36} + \frac{1}{3}\frac{1}{9} = \frac{1}{18}.$$

Adding the within-group term from the preceding page gives

$$\operatorname{Var}(X) = \frac{3}{16} + \frac{1}{18} = \frac{35}{144}.$$

Marginal check: $P(X = 1) = 5/12$, so $X \sim \operatorname{Bernoulli}(5/12)$ and its variance is $(5/12)(7/12) = 35/144$, matching the decomposition.

3.34 Connecting proofs and calculations

Prove set identities through arbitrary-element predicates; check Venn counts through nonempty-region partitions. Probability then assigns outcome masses. Geometry or the number of possible values cannot establish equal likelihood.

Check positive denominators and partitions before conditioning. Specify event independence, pairwise independence, or mutual independence. Expectation is linear without independence; variance retains covariance. Every distribution requires its stated support and experiment.

References and attribution

Lin Cing, handwritten discrete mathematics notes. Ralph P. Grimaldi, Discrete and Combinatorial Mathematics, 5th ed., Ch. 3.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

4.1 Divisibility and well-ordering

For integers a, b, write $a \mid b$ if $b = ak$ for some integer k. Negative divisors are allowed; this relation is not a numerical quotient. For every $m, n \in \mathbb{Z}$,

$$a \mid b,\ a \mid c \Rightarrow a \mid (mb + nc).$$

The well-ordering principle says every nonempty subset of positive integers has a least element. It does not apply directly to every nonempty integer set; all negative integers have no least member. Here $0 \mid b$ holds exactly when $b = 0$, so $0 \mid 0$ is allowed. The reference textbook instead requires the divisor in the divisibility relation to be nonzero. Division algorithms still require a nonzero divisor.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.2 The precise form of weak induction

To prove $\forall n \geq n_0, P(n)$, prove $P(n_0)$, then for arbitrary fixed $k \geq n_0$, assume $P(k)$ and derive $P(k+1)$. This covers the successor chain starting at n_0. The base is essential; a skipped or incorrect starting point may invalidate the claim.

For the false claim "all integers $n \geq 2$ are even," moving from $2k$ to $2(k+1)$ covers only even integers. It does not prove the required implication for an arbitrary index. The step may use $P(k)$ repeatedly or split into cases according to k, while keeping k arbitrary and fixed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.2.1 Induction follows from well-ordering

Induction is not confidence gained by checking many successors. Suppose $P(n_0)$ and $P(k) \to P(k+1)$ for every $k \geq n_0$. Let

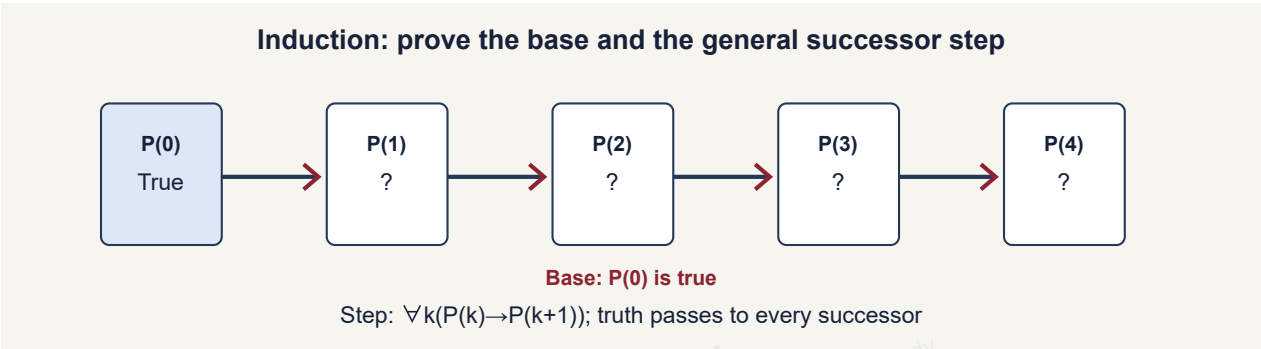
$$S = \{n \in \mathbb{Z} : n \geq n_0 \wedge \neg P(n)\}.$$

If the theorem fails, S is nonempty. Its translate $\{n - n_0 + 1 : n \in S\}$ is a nonempty positive-integer set, so well-ordering gives a least counterexample m. The true base excludes $m = n_0$. Therefore $m - 1 \geq n_0$, and minimality gives $P(m-1)$. The induction step gives $P(m)$, contradicting $m \in S$. Thus S is empty.

Without a base, this argument cannot exclude failure at n_0. Without the step, it cannot pass to the successor.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The induction hypothesis and domino model



Assuming $P(k)$ is a conditional assumption for one arbitrary fixed index, not an assumption of the whole theorem. The completed step proves an implication.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.2.2 Complete induction examples and boundaries

For $n \geq 1$, prove

$$1 + 3 + \cdots + (2n - 1) = n^2.$$

At $n = 1$, both sides equal one. Assume the formula at fixed $k \geq 1$. The next term is $2(k + 1) - 1 = 2k + 1$, and

$$1 + 3 + \cdots + (2k - 1) + (2k + 1) = k^2 + 2k + 1 = (k + 1)^2.$$

This is the next-index claim, completing induction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Induction with a later starting point

For $P(n) : 4n < n^2 - 7$, $n \geq 6$, the base gives $24 < 29$. Assume $4k < k^2 - 7$ with $k \geq 6$. Then

$$4(k + 1) = 4k + 4 < k^2 - 3 < k^2 + 2k - 6 = (k + 1)^2 - 7,$$

where the second inequality uses $2k > 3$. The claim fails at one, since $4 < -6$ is false, so its start cannot be changed to one.

Writing $P(k + 1)$ on scratch paper helps find a route from $P(k)$. The formal proof must start with established assumptions and end at the target, without assuming the target.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.3 Strong induction and least counterexamples

To prove $P(n)$, strong induction assumes every earlier $P(j)$, $n_0 \leq j < n$. It is useful for integer factorization, graph-removal operations, and divide-and-conquer structures. Weak induction is a special case.

A least-counterexample proof forms $S = \{n \geq n_0 : \neg P(n)\}$, assumes it nonempty, and applies well-ordering to derive a smaller counterexample or contradiction. This method differs from contraposition and must explicitly justify nonemptiness.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.3.1 Equivalence of weak and strong induction

Assume weak induction valid. Suppose $P(n_0)$ holds and all preceding statements imply $P(n)$ for each $n > n_0$. Set

$$Q(n) = P(n_0) \wedge P(n_0 + 1) \wedge \cdots \wedge P(n).$$

The base $Q(n_0)$ holds. Assuming $Q(k)$ supplies every earlier P; the given strong step yields $P(k + 1)$, hence $Q(k + 1)$. Weak induction proves every $Q(n)$, and $Q(n) \Rightarrow P(n)$ proves the desired result. Conversely, strong induction includes the most recent hypothesis $P(k)$, so it permits every weak-induction proof. It broadens the usable history without increasing logical proof strength.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Strong induction for a two-term recurrence

For a complete example , let $a_0 = 1, a_1 = 2$, and

$$a_n = 2a_{n-1} + 3a_{n-2} \quad (n \geq 2).$$

Claim $a_n = (3^{n+1} + (-1)^n)/4$. At zero and one, the formula gives $(3 + 1)/4 = 1$ and $(9 - 1)/4 = 2$. Fix $k \geq 1$ and assume the formula through k. Then

$$\begin{aligned} a_{k+1} &= 2a_k + 3a_{k-1} \\ &= \frac{6 \cdot 3^k + 3 \cdot 3^k + 2(-1)^k - 3(-1)^k}{4} \\ &= \frac{9 \cdot 3^k - (-1)^k}{4} = \frac{3^{k+2} + (-1)^{k+1}}{4}. \end{aligned}$$

The two-term recurrence needs two bases to start the dependence on $k - 1$. Every strong proof can be rewritten as weak induction on a prefix conjunction.

Strong induction: stamps of values 3 and 8

Three residue chains: add 3 at each step

We claim every integer $n \geq 14$ can be formed using stamps worth 3 and 8. The base cases are $14 = 3 + 3 + 8$, $15 = 5 \cdot 3$, and $16 = 8 + 8$. Adding 3 preserves feasibility; the three bases cover the three residues.

Stamp example: the inductive step

Fix an integer $n \geq 17$, and assume every value $14, 15, \ldots, n-1$ can be formed. Since $14 \leq n-3 < n$, the hypothesis applies to $n-3$. Add one 3-value stamp to obtain n.

The bases cover $14, 15, 16$, and the step covers every $n \geq 17$. Checking only 14 would not start the other two residue chains.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.4 Recursive definitions

A recursive definition specifies bases and a rule, such as

$$F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2} \quad (n \geq 2).$$

Existence and uniqueness both need justification. If every new value depends only on smaller indices, finite iteration determines it uniquely. Negative-index references or insufficient initial values can leave a sequence undefined.

More generally, let $r \geq 1$ be an integer, let S be a nonempty set of values, and choose bases $a_0, \ldots, a_{r-1} \in S$. Require f to be defined on every pair of an integer $n \geq r$ and an S-valued prefix of length n, with values in S. Then

$$a_n = f(n, a_0, \ldots, a_{n-1}) \quad (n \geq r)$$

defines exactly one sequence. The recursion theorem constructs the infinite sequence: each finite prefix extends uniquely by the total function f, and these prefixes are compatible, so their entries determine one sequence. If another sequence has the same bases and rule, induction on prefix length shows all its entries agree; hence it is unique.

Recursive initial values and uniqueness

For Fibonacci bases $b_0 = 0, b_1 = 1$, iteration gives $b_2 = 1, b_3 = 2, b_4 = 3, b_5 = 5, b_6 = 8, b_7 = 13$. Without the two bases, both $0, 1, 1, 2, \ldots$ and $2, 5, 7, 12, \ldots$ satisfy the same recurrence. Nor does $d_n = d_{n+1}$ without other boundaries uniquely determine an infinite sequence. Recursion describes a definition method; uniqueness depends on well-founded structure and initial conditions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Harmonic and Lucas sequences: starting values

Set $H_0 = 0$ and, for integers $n \geq 1$, define $H_n = H_{n-1} + 1/n$. Successive values are:

n	0	1	2	3	4
H_n	0	1	$3/2$	$11/6$	$25/12$

For example, $H_4 = H_3 + 1/4 = 11/6 + 1/4 = 25/12$. The value $H_0 = 0$ is the empty-sum convention; it does not introduce a $1/0$ term.

The Lucas sequence starts at $L_0 = 2, L_1 = 1$ and satisfies $L_n = L_{n-1} + L_{n-2}$ for $n \geq 2$, giving $L_2 = 3, L_3 = 4, L_4 = 7$. Its recurrence matches Fibonacci's, but its starting values differ, so the recurrence alone does not make the sequences equal.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Eulerian numbers: classify permutations by descents

For integers $n \geq 1$, let $A(n, k)$ count permutations of $1, \ldots, n$ with exactly k descents, where a descent is an adjacent position with $\pi_i > \pi_{i+1}$. Use $A(0, 0) = 1$; for $n \geq 1$, set $A(n, k) = 0$ when $k < 0$ or $k \geq n$, and set $A(0, k) = 0$ for $k \neq 0$. Thus $A(1, 0) = 1$.

For $n \geq 2$, insert the largest element n, separating the cases by the old descent count:

- A permutation with k descents has $k + 1$ preserving positions: inside any descent or at the end.
- A permutation with $k - 1$ descents has $n - k$ increasing positions: at the start or inside any ascent.

Deleting the largest element uniquely recovers the old permutation and insertion position. The cases are disjoint and exhaustive, giving

$$A(n, k) = (k + 1)A(n - 1, k) + (n - k)A(n - 1, k - 1).$$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Eulerian recurrence on three elements

Using $A(2, 0) = A(2, 1) = 1$ and the out-of-range convention:

$$A(3, 0) = 1, \quad A(3, 1) = 2(1) + 2(1) = 4, \quad A(3, 2) = 1.$$

Permutation	Descents	Permutation	Descents
123	0	231	1
132	1	312	1
213	1	321	2

Hence $1 + 4 + 1 = 6 = 3!$. In general every permutation has exactly one descent count, so the row sum is $n!$ by a disjoint classification; it is not a conclusion based only on the first three rows.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

4.5 Recursive algorithms and structural induction

Recursive factorial needs an explicit base return and calls with strictly smaller arguments remaining inside the domain. Correctness can be proved by induction on argument size.

For $\mathrm{bitsum}(0) = 0$, define for $n > 0$,

$$\mathrm{bitsum}(n) = \mathrm{bitsum}(\lfloor n/2 \rfloor) + (n \bmod 2).$$

The nonnegative argument strictly decreases, so recursion terminates. A trace is

$$\mathrm{bitsum}(45) = 1 + \mathrm{bitsum}(22) = 1 + 0 + \mathrm{bitsum}(11) = \cdots = 4.$$

The full remainder sequence is $1, 0, 1, 1, 0, 1$, matching four ones in $45 = (101101)_2$. For correctness, strong induction assumes all smaller arguments give their binary digit sums, then adds the last binary digit in the displayed recurrence. Termination and correctness are separate claims. Recursion trees or the master theorem analyze complexity.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.6 The division algorithm

For integer a and positive integer d, unique integers q, r satisfy

$$a = qd + r, \qquad 0 \le r < d.$$

For existence, choose $q = \lfloor a/d \rfloor, r = a - qd$. For uniqueness, two representations give $r - r' = (q' - q)d$, while $-d < r - r' < d$, forcing zero.

Programming-language remainder rules for negative inputs can differ. In particular C/C++ `%` follows its quotient convention rather than the nonnegative mathematical convention for $a \bmod d$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.6.1 A well-ordering proof of division

The floor construction is short; well-ordering explains existence of a least nonnegative remainder directly. For fixed $a \in \mathbb{Z}, d > 0$, form

$$S = \{a - dq : q \in \mathbb{Z}, a - dq \geq 0\}.$$

It is nonempty: $q_0 = -|a| - 1$ gives $a + d(|a| + 1) > 0$, since $d \geq 1$. Let its least member be $r = a - dq$. If $r \geq d$, then $r - d = a - d(q + 1)$ is a smaller nonnegative member, contradiction. Thus $0 \leq r < d$.

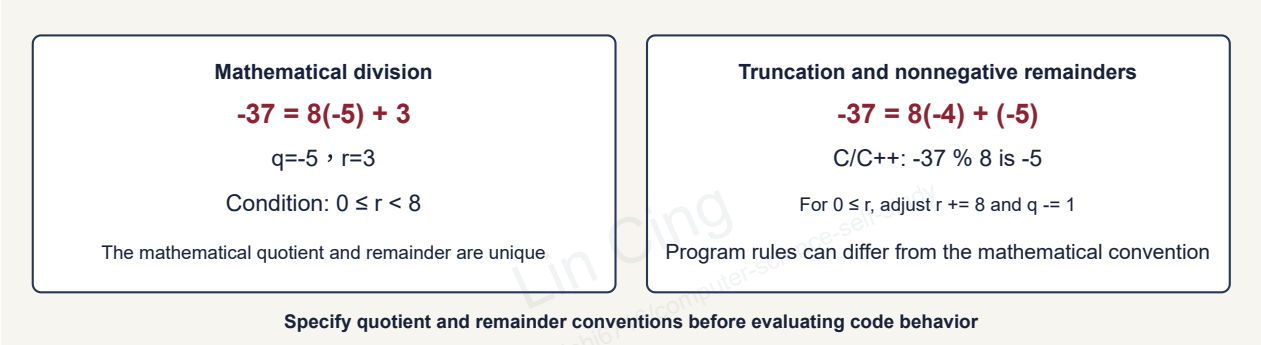
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Uniqueness of quotient and remainder

For another representation $a = dq' + r'$ in the same range,

$$r - r' = d(q' - q), \qquad -d < r - r' < d.$$

Only zero in that interval is divisible by d, so $r = r'$. Since $d > 0$, also $q = q'$.



Exhibiting one pair proves existence but not uniqueness. Excluding a second pair proves uniqueness but not existence.

4.6.2 Base representations and two's complement

For nonnegative integer n, repeated division in integer base $b \geq 2$ gives

$$n = r_0 + r_1 b + \cdots + r_k b^k, \qquad 0 \leq r_i < b.$$

For example,

$$47 = 3 \cdot 15 + 2, \quad 15 = 3 \cdot 5 + 0, \quad 5 = 3 \cdot 1 + 2, \quad 1 = 3 \cdot 0 + 1.$$

Reading remainders upward gives $47 = (1202)_3$, checked by $27 + 18 + 0 + 2 = 47$.

For positive input, each quotient $\lfloor n/b \rfloor < n$, so division terminates at zero. Substitution back proves existence with a nonzero leading digit. Any two expansions have equal last digits modulo b. Subtract that digit, divide by b, and repeat to prove uniqueness. Write zero as one digit 0. Negative integers use a separate minus sign rather than a sum of nonnegative digits times positive powers.

Two's complement and overflow

For eight-bit two's complement , write $6 = (00000110)_2$, invert bits to 11111001, and add one:

$$-6 = (11111010)_2.$$

Precisely, w bits represent

$$-b_{w-1} 2^{w-1} + \sum_{i=0}^{w-2} b_i 2^i,$$

with range $[-2^{w-1}, 2^{w-1} - 1]$. When the top bit is one, subtract 2^w from unsigned value U. For minus six in eight bits, $U = 256 - 6 = 250$. The top bit has negative weight, rather than being an attached sign label.

Inverting and adding one again recovers 00000110. Fixed-width signed addition may discard the left carry, yet equal-sign operands producing the opposite sign indicate overflow. Eight-bit $127 + 1$ produces 10000000, interpreted as minus 128. Bit patterns and mathematical integer values must be distinguished. Base representation proceeds low digit first, and two's complement requires a fixed width and range convention.

4.7 Primes and irreducibles require full definitions

An integer $p > 1$ is prime when its only positive divisors are one and itself. One is a unit, not prime; zero is not prime either.

In an integral domain R, a prime element p must be nonzero and a nonunit, and satisfy

$$p \mid ab \Rightarrow p \mid a \text{ or } p \mid b \qquad (a, b \in R).$$

A unit has a multiplicative inverse. An irreducible is also nonzero and a nonunit, and every factorization $p = ab$ has a unit factor.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Prime element conditions and examples

Element	Can the divisibility implication alone hold?	Prime element?
0	Yes in a domain, since $ab = 0$ forces a zero factor	No, nonzero condition fails
Units such as ± 1 in integers	Yes, units divide everything	No, nonunit condition fails
2 in integers	Yes, by Euclid's lemma	Yes, and irreducible
4 in integers	No, it divides $2 \cdot 2$ but neither factor	No; reducible as $2 \cdot 2$

A prime is irreducible: if $p = ab$ and $p \mid a$, put $a = pc$ and cancel nonzero p to get $1 = cb$, making b a unit; the other case is symmetric. The converse holds in integers but may fail in other domains. For trial division of positive n, a factorization $n = ab, 1 < a \le b$ gives $a \le \sqrt{n}$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.7.1 Prime divisors and infinitely many primes

For composite $n > 1$, its nontrivial divisors $1 < d < n$ form a nonempty set. Let p be the least. If $p = ab$ with $1 < a < p$, then a is a smaller nontrivial divisor of n, contradiction. Thus p is prime. Writing $n = pk$, minimality gives $p \leq k$, so $p^2 \leq n$. Every composite has a prime divisor at most $\sqrt{n}$, explaining the trial-division stopping condition.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Infinitely many primes

If all primes were $p_1, \ldots, p_t$, let

$$N = p_1 \cdots p_t + 1.$$

Since $N > 1$, either N is prime and is its own prime divisor, or it is composite and the preceding argument gives a prime divisor. Every listed prime leaves remainder one on dividing N, so none divides it, contradiction. Thus infinitely many primes exist.

Allowing one as prime would destroy uniqueness through arbitrary factors of one. Zero is also excluded. For $n = 2$, no trial candidates $2 \leq d \leq \sqrt{n}$ exist, correctly declaring it prime. The test assumes $n > 1$ and does not apply to one.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.8 GCD and the Euclidean algorithm

For integers a, b not both zero, their gcd is the greatest positive common divisor. For $b > 0$,

$$\gcd(a, b) = \gcd(b, a \bmod b).$$

If $a = qb + r$, the common divisors of a, b are exactly those of b, r. Successive division reduces remainders to zero, leaving the previous positive remainder as gcd.

A zero input returns the absolute value of the other in constant steps. For two nonzero inputs normalized positive, remainders at least halve every two steps. This gives $O(\log \min(|a|, |b|) + 1)$ divisions, not the bit complexity of large-integer arithmetic.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.8.1 The least positive linear combination

For a, b not both zero, form

$$S = \{ax + by : x, y \in \mathbb{Z}, ax + by > 0\}.$$

If $a \neq 0$, one of $a, -a$ is positive; if $a = 0$, one of $b, -b$ is positive. Thus S is nonempty. Let its least member be $d = ax_0 + by_0$.

Divide $a = dq + r$, $0 \leq r < d$. Then

$$r = a - dq = a - (ax_0 + by_0)q = a(1 - x_0 q) + b(-y_0 q).$$

If $r > 0$, this is a smaller positive member of S, contradiction. Hence $d \mid a$, and similarly $d \mid b$. Every common divisor c divides $ax_0 + by_0 = d$; every positive common divisor is therefore at most d. This proves $d = \gcd(a, b)$ and existence of a Bézout representation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Euclidean invariants and substitution

For Euclidean correctness ,

$$d \mid a \text{ and } d \mid b \iff d \mid b \text{ and } d \mid r$$

follows in the two directions from $r = a - qb$ and $a = qb + r$. The common-divisor set is invariant. The final nonzero remainder is its gcd, and strict remainder descent proves termination.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Euclidean invariants and substitution (continued)

Euclidean algorithm and Bézout substitution

Division steps

1. 1071 = 2 × 462 + 147
2. 462 = 3 × 147 + 21
3. 147 = 7 × 21 + 0

Remainders decrease: 147 → 21 → 0

Common divisors stay unchanged

C(a,b) = set of positive common divisors

C(1071, 462) = C(462, 147)

= C(147, 21) = C(21, 0)

r = a − qb gives the same common divisors

Remainder 0: last nonzero remainder = gcd = 21

Back-substitution: replace 147 with 1071 − 2 × 462

21 = 462 − 3 × 147 → 21 = 7 × 462 − 3 × 1071

Coefficients: 1071 × (−3) + 462 × 7 = 21

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Calling d the least positive combination does not alone prove maximality. One must show both that it is a common divisor and that all common divisors divide it.

4.9 Bézout and linear Diophantine equations

For a, b not both zero, there are integers s, t with

$$\gcd(a, b) = sa + tb.$$

Extended Euclid retains these coefficients; reversing its division steps constructs them. They may be negative and are not unique.

For $84 = 2 \cdot 30 + 24$, $30 = 24 + 6$,

$$6 = 30 - 24 = 30 - (84 - 2 \cdot 30) = 3 \cdot 30 - 84.$$

Thus $84x + 30y = 12$ is solvable, since six divides twelve. Multiply by two to obtain $-2 \cdot 84 + 6 \cdot 30 = 12$, with $(x, y) = (-2, 6)$. If $d = sa + tb$ and $g = \gcd(a, b)$, then for every integer k,

$$d = (s + k\,b/g)a + (t - k\,a/g)b.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Solvability of linear Diophantine equations

More generally, for a, b not both zero,

$$ax + by = c$$

has integer solutions iff $g = \gcd(a, b)$ divides c. If $c = g\ell$, multiply $g = sa + tb$ by ℓ to construct a solution. Conversely, every common divisor divides the left side of any solution, hence divides c.

Handle $a = b = 0$ separately: every integer pair solves it when $c = 0$, and none when $c \neq 0$. This convention leaves $\gcd(0, 0)$ undefined. Chapter 14 uses Bézout to test modular invertibility.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.9.1 GCD, LCM, and common multiples

For positive integers a, b, the lcm is their least positive common multiple. We prove

$$\gcd(a,b)\,\operatorname{lcm}(a,b) = ab.$$

Write $a = gA, b = gB$, where $g = \gcd(a,b)$ and $\gcd(A,B) = 1$. The candidate $L = gAB = aB = bA$ is a common multiple.

Any positive common multiple has $M = au = bv$, giving $Au = Bv$. Bézout gives $sA + tB = 1$. Multiply by v: both terms on the left are divisible by A, the second because $Bv = Au$. Thus $A \mid v$; write $v = Aw$. Then $M = gB(Aw) = Lw$, so L divides every positive common multiple and is least. Finally $gL = g^2AB = ab$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

LCM example and domain

For $\gcd(168, 456) = 24$,

$$\operatorname{lcm}(168, 456) = 168 \cdot 456/24 = 3192.$$

Indeed $168 = 24 \cdot 7, 456 = 24 \cdot 19$, with coprime 7 and 19. Every common multiple must include their product after removing the common factor.

The displayed identity uses positive inputs. Other integer, sign, and zero conventions must be specified first. For example $\gcd(0, n) = |n|$, while $\operatorname{lcm}(0, n)$ requires its own convention.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.9.2 Prime and irreducible can differ

In a commutative integral domain with identity, both definitions require a nonzero nonunit. Prime means $p \mid ab \Rightarrow p \mid a$ or $p \mid b$; irreducible means $p = ab \Rightarrow$ at least one factor is a unit.

For prime $p = ab$, primality gives $p \mid a$ or $p \mid b$. In the first case $a = pc$ gives $p = pbc$; cancel nonzero p in the domain to obtain $1 = bc$, so b is a unit. The other case is symmetric.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Irreducible integers are prime

In integers the converse holds. For any irreducible integer p, let $q = |p| > 1$. The numbers differ by the unit ± 1, so q is irreducible and $p \mid z$ iff $q \mid z$. It therefore suffices to prove primality for a positive irreducible; write it as $p > 1$. For $p \mid ab, p \nmid a$, its only positive divisors imply $\gcd(p, a) = 1$. Bézout gives $up + va = 1$. Multiply by b: $upb + vab = b$. Both terms are divisible by p, giving $p \mid b$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A counterexample in a general integral domain

The converse fails in

$$R = \mathbb{Z}[\sqrt{-5}] = \{a + b\sqrt{-5} : a, b \in \mathbb{Z}\}.$$

Define $N(a + b\sqrt{-5}) = a^2 + 5b^2$. Multiplication expands to give $N(uv) = N(u)N(v)$; nonzero elements have positive integer norms. Norm one occurs exactly at ± 1, precisely the units, since inverse norms must multiply to one.

If $3 = uv$ with both factors nonunits, their norms multiply to nine and must both equal three. But $a^2 + 5b^2 = 3$ has no integer solution: nonzero b gives at least five, and $b = 0$ requires $a^2 = 3$. Thus three is irreducible. It is false that all nonzero norms below nine are units; two has norm four and $\sqrt{-5}$ norm five.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why three is not prime in this domain

In the same domain,

$$(2 + \sqrt{-5})(2 - \sqrt{-5}) = 9$$

is divisible by three, while neither factor is: comparing integer coefficients would require three to divide two and ± 1. Three is not prime in this domain. The integer equivalence cannot be transferred to arbitrary domains. The prime divisibility implication alone admits zero and units, so the nonzero and nonunit conditions are essential.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.10 The fundamental theorem of arithmetic

Every integer $n \geq 2$ is a product of primes, uniquely up to order. For existence, the base two is already prime. Assume all integers from two to $n-1$ factor. If n is prime, stop. Otherwise $n = ab$ with $1 < a, b < n$. Strong induction factors both, and concatenation factors n.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Euclid's lemma and uniqueness

For uniqueness, first prove Euclid's lemma without using unique factorization. If prime $p \mid ab$ and $p \nmid a$, then $\gcd(p, a) = 1$, so Bézout gives $up + va = 1$. Multiplication by b gives $upb + vab = b$. Both left terms are divisible by p, so $p \mid b$.

Extend the lemma to any finite product by induction on its length. The one-factor case is immediate. If $p \mid (a_1 \cdots a_m)a_{m+1}$, the lemma gives divisibility of the last factor or the first product. In the latter case, the induction hypothesis supplies a divisible factor among the first m.

Now suppose

$$n = p_1 \cdots p_r = q_1 \cdots q_s$$

are prime factorizations. The extended lemma makes p_1 divide some q_j, so the two positive primes are equal. Reorder and cancel them, then repeat. If one list ends before the other, a remaining product of primes greater than one would equal one, impossible. Thus the prime lists match with multiplicities, proving uniqueness up to order. In a unique factorization domain, factorizations are unique up to order and multiplication by units; a general integral domain need not have unique factorization.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.10.1 Standard form, divisor counts, and squares

Write the unique factor multiplicities as

$$n = p_1^{\alpha_1} \cdots p_r^{\alpha_r}, \qquad \alpha_i \geq 1.$$

Every positive divisor uniquely has $d = p_1^{\beta_1} \cdots p_r^{\beta_r}, 0 \leq \beta_i \leq \alpha_i$. Independent exponent choices and uniqueness give

$$\tau(n) = \prod_i (\alpha_i + 1).$$

A divisor is a square iff all its exponents are even, so square divisors number

$$\prod_i \left(\left\lfloor \frac{\alpha_i}{2} \right\rfloor + 1 \right).$$

For $360 = 2^3 \cdot 3^2 \cdot 5$, $\tau(360) = 4 \cdot 3 \cdot 2 = 24$. Square exponents for two are 0,2; for three 0,2; and for five only zero. The four square divisors are $1, 4, 9, 36$.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Square divisors and odd divisor counts

If $n = m^2$, every prime exponent is even; conversely halving even exponents constructs a square root. Also $\tau(n)$ is odd iff every $\alpha_i + 1$ is odd, equivalently every α_i even, so iff n is a square.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Trailing zeros of 100!: count factors by level

Counting factors of 5 in 100!

5^1	$\lfloor 100 / 5 \rfloor = 20$	5, 10, …, 100
5^2	$\lfloor 100 / 25 \rfloor = 4$	25, 50, 75, 100
5^3	$\lfloor 100 / 125 \rfloor = 0$	125 > 100

Each multiple of 5 contributes a factor of 5; each multiple of 25 contributes one more. Thus $v_5(100!) = 20 + 4 + 0 = 24$. The 50 even factors supply at least 50 factors of 2, enough to pair with all 24 factors of 5, so there are **24 trailing zeros**.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.11 Proof defects and boundaries

"Every $n \geq 2$ is a product of two integers greater than one" is false at every prime. The corrected conditional for nonprime n needs a precise composite definition.

A strong-induction factorization proof cannot invoke $P(1)$ before establishing its actual base $n = 2$. Every recursive reference must lie in an established or strictly smaller part of the domain.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.12 Checks and exercises

Check the quantified starting point, bases, decreasing measure, uniqueness of division, gcd invariant, and assumptions for prime-factorization uniqueness. The separate exercise PDF provides questions, and the solution PDF gives their derivations.

References and attribution

Lin Cing, handwritten discrete mathematics notes. Ralph P. Grimaldi, Discrete and Combinatorial Mathematics, 5th ed., Ch. 4.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Ordered pairs, products, and relation types

Ordered pairs satisfy $(a, b) = (c, d) \iff a = c \land b = d$. Coordinates in $A \times B = \{(a, b) : a \in A, b \in B\}$ are ordered. A binary relation from A to B is a subset $R \subseteq A \times B$.

For $A = \{p, q\}, B = \{0, 1, 2\}$, there are six product points and $2^6 = 64$ relations. Finite sizes m, n generally give mn points and 2^{mn} relations. The empty product still has the one subset $\varnothing$.

Define $\operatorname{Dom} R = \{a : \exists b, (a, b) \in R\}$ and $\operatorname{Ran} R = \{b : \exists a, (a, b) \in R\}$. The inverse relation $R^{-1} = \{(b, a) : (a, b) \in R\}$ always exists but need not be a function.

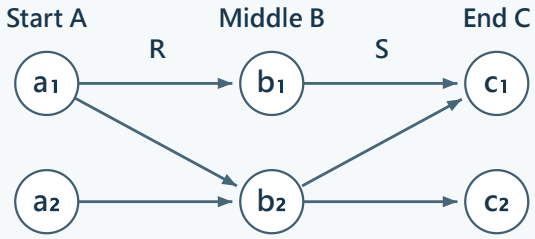
Elements $((a, b), c)$ of $(A \times B) \times C$ differ from $(a, (b, c))$ of $A \times (B \times C)$. Natural bijections do not make their ordered-pair sets literally equal.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Composing relations through witnesses

Relation composition: R first, then S

Two different intermediate witnesses yield one ordered pair



Two witnesses for (a1, c1)

a1 → b1 → c1
a1 → b2 → c1

Existence: 1 ∨ 1 = 1
Path count: 1 + 1 = 2

Arrows are ordered pairs; a pair is not repeated when it has multiple paths.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Let $A = \{a_1, a_2\}$, $B = \{b_1, b_2\}$, and $C = \{c_1, c_2\}$. Follow $R = \{(a_1, b_1), (a_1, b_2), (a_2, b_2)\}$ and then $S = \{(b_1, c_1), (b_2, c_1), (b_2, c_2)\}$. A pair (a, c) belongs to $S \circ R$ exactly when some $b \in B$ satisfies $(a, b) \in R$ and $(b, c) \in S$.

Boolean matrix product, entry by entry

Use rows a_1, a_2 and columns b_1, b_2 for M_R; use rows b_1, b_2 and columns c_1, c_2 for M_S. Boolean multiplication uses AND followed by OR:

$$M_R = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, \quad M_S = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}, \quad M_{S \circ R} = M_R \odot M_S = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}.$$

Input and output	Terms through b_1, b_2	Result
(a_1, c_1)	$(1 \wedge 1) \vee (1 \wedge 1)$	1
(a_1, c_2)	$(1 \wedge 0) \vee (1 \wedge 1)$	1
(a_2, c_1)	$(0 \wedge 1) \vee (1 \wedge 1)$	1
(a_2, c_2)	$(0 \wedge 0) \vee (1 \wedge 1)$	1

Thus $S \circ R = A \times C$. Two witnesses make the first Boolean entry 1, since it records existence; the ordinary arithmetic product gives 2 there, counting intermediate paths. With the row convention the matrices are $M_R \odot M_S$, whereas the relation is $S \circ R$; both mean R first, then S.

Function existence and uniqueness are separate

The statement $f : A \to B$ requires exactly one output for each input:

$$\forall a \in A \; \exists b \in B : (a, b) \in f,$$

$$\forall a, b, c : \; ((a, b), (a, c) \in f) \Rightarrow b = c.$$

The relation $\{(p, 0), (q, 0)\}$ is a function. Omitting q violates existence; assigning both zero and one to p violates uniqueness. Distinct inputs may share outputs.

The domain is A, codomain B, and image $f(A)$. Codomain is part of the type. Squaring from reals to reals and from reals to nonnegative reals has different surjectivity.

For every B, one empty function $\varnothing \to B$ exists. No nonempty domain maps to the empty set. Partial functions allow undefined inputs and are not total functions on the original domain.

Images and preimages need no inverse function

For $X \subseteq A, Y \subseteq B$,

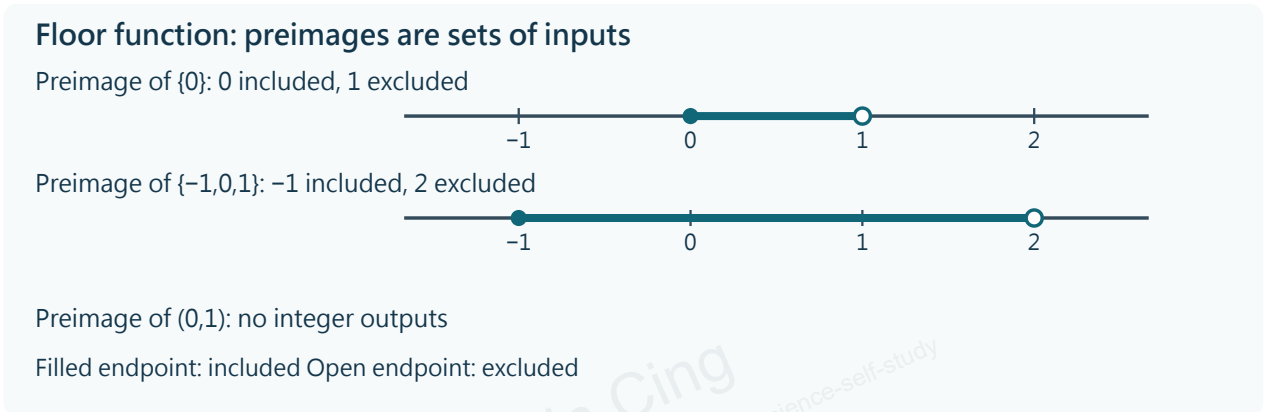
$$f(X) = \{f(x) : x \in X\}, \qquad f^{-1}(Y) = \{a \in A : f(a) \in Y\}.$$

Preimages exist even when f is not invertible. Integer squaring has preimages $f^{-1}(\{4\}) = \{-2, 2\}$ and $f^{-1}(\{-1\}) = \varnothing$.

Elementwise, $a \in f^{-1}(Y \cap Z) \iff f(a) \in Y \wedge f(a) \in Z$, proving preservation of intersections. Replacing conjunction with disjunction or negation proves preservation of unions and complements relative to the stated domain and codomain.

Images satisfy only $f(X \cap W) \subseteq f(X) \cap f(W)$. Under squaring, $X = \{-2\}, W = \{2\}$ give empty left side and right side $\{4\}$. If f is injective, the two preimages for a shared image coincide, proving the reverse inclusion and equality.

Preimages of the floor function



Let $g : \mathbb{R} \to \mathbb{R}$, $g(x) = \lfloor x \rfloor$. From $k \le x < k + 1$, $g^{-1}(\{0\}) = [0, 1)$ and $g^{-1}(\{-1, 0, 1\}) = [-1, 2)$. Every output is an integer, so $g^{-1}((0, 1)) = \varnothing$. Here $g^{-1}(S)$ denotes a set preimage. Since g is neither injective nor surjective, it has no inverse function $\mathbb{R} \to \mathbb{R}$.

Injectivity, surjectivity, and their negations

Injectivity requires $\forall a, a' \in A, f(a) = f(a') \Rightarrow a = a'$. Its negation is one collision between distinct inputs. Surjectivity requires $\forall b \in B\ \exists a \in A : f(a) = b$; its negation is an output missed by every input. A bijection has both properties.

For $f : \mathbb{Z} \to \mathbb{Z}$, $f(n) = 3n - 7$, cancellation proves injectivity. The image is $2 + 3\mathbb{Z}$, so it is not onto integers. Restricting the codomain to that image makes it bijective, with inverse $m \mapsto (m + 7)/3$.

For finite sets, injection gives $|A| \leq |B|$ and surjection the reverse inequality. If both sizes are n, an injection's n different outputs fill the codomain. A surjection's n nonempty fibers can each contain only one of the n inputs. Thus injection, surjection, and bijection are equivalent in this equal-finite-size case, including zero.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fibers separate a partition from its labels

For $b \in f(A)$, let $F_b = f^{-1}(\{b\})$. Fibers are nonempty, disjoint, and cover A, because every input has exactly one output.

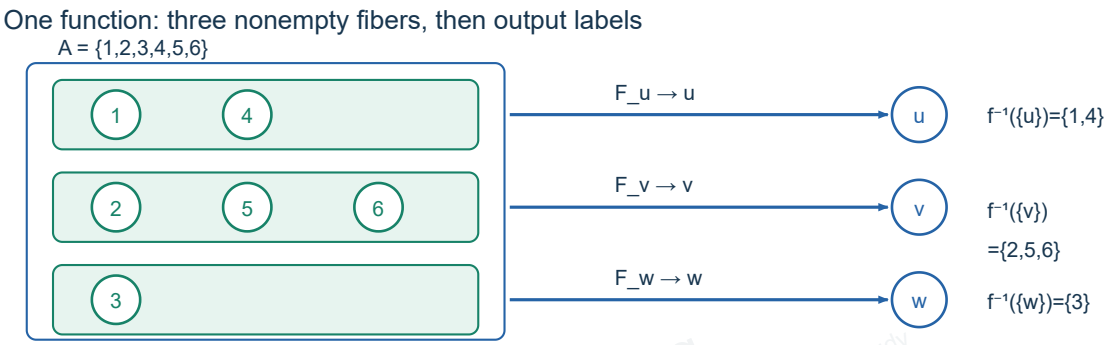
With $\pi(a) = F_{f(a)}$, $\bar{f}(F_b) = b$, and inclusion $\iota : f(A) \hookrightarrow B$,

$$f = \iota \circ \bar{f} \circ \pi.$$

The middle map bijects blocks with the image. Injectivity means all fibers have at most one point; surjectivity means every codomain point has a nonempty fiber.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Example: fibers of six inputs



The fibers $\{1,4\},\{2,5,6\},\{3\}$ carry labels u,v,w. First send an input to its block, then its block to the output.

Equivalence relations and partitions determine each other

Define $a \sim a' \iff f(a) = f(a')$. Equality proves reflexivity, symmetry, and transitivity, with fibers as equivalence classes.

For any equivalence relation, each a lies in $[a]$. If $[a] \cap [b]$ contains c, then $a \sim c \sim b$, giving $a \sim b$. Any $x \sim a$ then satisfies $x \sim b$, and conversely, so the classes agree. Classes therefore partition the set.

Conversely, define equivalence by membership in the same block of a partition. Unique block membership proves reflexivity and symmetry. If a, b share a block and b, c share one, their blocks intersect at b and thus coincide, proving transitivity. These constructions are inverse. For the empty set, its empty partition gives the same conclusion.

Quotient formulas must be independent of representatives

To define $h : A/\sim \to C$ by $h([a]) = g(a)$, the necessary and sufficient condition is $a \sim a' \Rightarrow g(a) = g(a')$. Necessity follows because equal classes are one input. Sufficiency follows because any representative gives the same unique output.

The map $[a]_6 \mapsto [a]_3$ is well-defined, since divisibility by six implies divisibility by three. The proposed $[a]_4 \mapsto [a]_3$ fails: $[0]_4 = [4]_4$, but their outputs $[0]_3, [1]_3$ differ. Substitution on every integer does not alone define a function on classes.

This is well-definedness, logically before injectivity. Different representatives of the same input are not two different colliding inputs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Counting functions, injections, and partial functions

For finite $|A| = m, |B| = n$, functions number n^m. Ordered distinct output choices give

$$(n)_m = n(n-1)\cdots(n-m+1) = \frac{n!}{(n-m)!} \quad (m \le n)$$

injections, zero if $m > n$. Equal-size bijections number $n!$. The empty domain gives one function, using combinatorial convention $0^0 = 1$; nonempty-to-empty gives zero.

For partial functions with arbitrary defined subdomain, each input has n output choices or an undefined choice, giving $(n+1)^m$. Of these, n^m are total. Requiring nonempty defined subdomain excludes the one wholly undefined function.

Relations number 2^{mn} by independent cell membership. Functions instead choose exactly one cell per input row, so arbitrary subsets overcount them.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Stirling numbers and boundary cases

The second-kind Stirling number $S(m, k)$ counts partitions of m labeled elements into k unlabeled nonempty blocks. Set $S(0, 0) = 1$, and set $S(m, 0) = 0$ for $m > 0$, $S(0, k) = 0$ for $k > 0$, and $S(m, k) = 0$ for $k > m$.

Classify by the last element. As a singleton, deleting it leaves $S(m - 1, k - 1)$ choices. Otherwise partition the others into k blocks and choose one to join, giving $kS(m - 1, k)$. These reversible, disjoint, exhaustive constructions prove

$$S(m, k) = S(m - 1, k - 1) + kS(m - 1, k) \quad (m, k \geq 1).$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Stirling recurrence: a numerical table

$m\backslash k$	1	2	3	4
1	1	0	0	0
2	1	1	0	0
3	1	3	1	0
4	1	7	6	1
5	1	15	25	10
6	1	31	90	65

For example $S(6, 3) = 15 + 3 \cdot 25 = 90$. Showing four columns does not limit the possible block count to four.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two formulas for surjections

A surjection has exactly n nonempty fibers. Partition the domain, then bijet its blocks to named outputs:

$$\operatorname{Surj}(m,n) = n!S(m,n).$$

Alternatively weight each function by alternating subsets of its missed outputs. If it misses t values, its net weight is $\sum_{j=0}^{t}(-1)^j\binom{t}{j} = (1-1)^t$, one exactly when $t=0$. Exchange finite sums to get

$$\operatorname{Surj}(m,n) = \sum_{j=0}^{n}(-1)^j\binom{n}{j}(n-j)^m.$$

This proves the formula directly rather than treating later inclusion-exclusion as a black box. For $m<n$, no surjection exists and the sum is zero; for equal positive sizes it is $n!$. The empty-to-empty case has one empty function.

Six items and three named destinations

Send six labeled items to u,v,w, using every destination. Partition then label to obtain $3!S(6,3) = 6\cdot 90 = 540$. Unnamed nonempty groups would number only ninety.

Check by omitted destinations: all $3^6 = 729$, subtract three choices of an empty destination each leaving $2^6 = 64$, and restore three constant functions missing two destinations. Missing all three is impossible for a nonempty domain. Thus $729 - 192 + 3 = 540$.

With independent uniform choices, the probability is $540/729 = 20/27$. Unequal probabilities preserve the integer count but invalidate direct division by 729. The displayed fibers $\{1,4\},\{2,5,6\},\{3\}$ represent one such function; swapping output names changes it.

Classifying functions by image size

Choose the k-point image, then map onto it. Image sizes give disjoint exhaustive classes:

$$n^m = \sum_{k=0}^{\min(m,n)} \binom{n}{k} k! S(m,k) = \sum_k (n)_k S(m,k).$$

For $m = 4, n = 3$, image sizes one, two, three contribute 3, $\binom{3}{2} \cdot 2! \cdot 7 = 42$, $3! \cdot 6 = 36$, totaling 81.

First-kind numbers $c(m,k)$ instead count permutations with k cycles. In the recurrence below $m, k \geq 1$ are integers; set $c(0,0) = 1$, and use 0 for out-of-range indices and the remaining zero-size cases. A new element forms a singleton cycle or is inserted after one of the $m-1$ old elements, giving

$$c(m,k) = c(m-1,k-1) + (m-1)c(m-1,k).$$

The constructions are reversible, with the same zero-size and out-of-range boundaries as S. For example $c(3,1) = 2$, unlike $S(3,1) = 1$.

Closure before unary and binary operations

A unary operation is $u : A \to A$; here a binary operation follows the closed convention $* : A \times A \to A$. The reference also permits terminology $A \times A \to B$, checking closure separately; repeated operations here explicitly require it.

Addition is closed on $\mathbb{N}_0$; subtraction is not, since $2 - 5 \notin \mathbb{N}_0$, though it is closed on integers. Division is not total on $\mathbb{R}^2$ because zero denominators are undefined.

Union and intersection are closed binary operations on $\mathcal{P}(U)$, and complement is unary relative to fixed U. A sequence is a function $\mathbb{N}_0 \to A$, whose input is its index rather than the previous value.

Commutativity, associativity, and identities

Commutativity requires $a * b = b * a$, associativity $(a * b) * c = a * (b * c)$, for all inputs. A symmetric operation table establishes only commutativity, not the three-input associativity condition.

Real averaging $a * b = (a + b)/2$ commutes but is not associative: $(0 * 0) * 4 = 2$, whereas $0 * (0 * 4) = 1$. Integer subtraction is closed but neither commutative nor associative.

A two-sided identity satisfies $e * a = a = a * e$. If e, e' are both identities, $e = e * e' = e'$, proving uniqueness without associativity. Left identities alone need not be unique: for $a * b = b$, every element is a left identity, and a set with more than one element has no right identity.

The minimum operation on an ordered set has its maximum as identity when one exists; on all integers it has none. The domain matters as much as the operation's name.

Free cells in operation tables

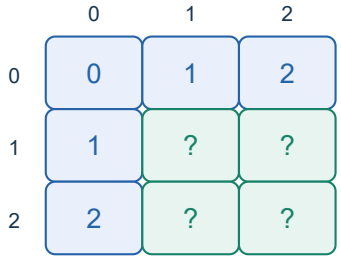
Let $|A| = n \geq 1$. A closed binary operation assigns one of n values to each of n^2 input pairs, giving n^{n^2} tables.

For commutativity, the n diagonal and $n(n-1)/2$ upper off-diagonal cells are free; symmetry fixes the lower half. Thus $n(n+1)/2$ cells are chosen independently, giving $n^{n(n+1)/2}$ tables.

A specified two-sided identity e fixes its row and column, each with n cells; their intersection (e, e) was counted twice, so $2n - 1$ cells are fixed. The remaining $n^2 - (2n - 1) = (n - 1)^2$ cells give $n^{(n-1)^2}$ tables. A two-sided identity is unique, so specified-identity classes are disjoint; requiring some identity gives $n \cdot n^{(n-1)^2}$ tables.

Example: an operation table with a fixed identity

Identity 0 fixes row 0 and column 0



Fixed: 0*x = x = x*0

Free: four cells, three values each

$3^4 = 81$ operations

May be nonassociative; need not be groups.

For $A = \{0, 1, 2\}$ with identity zero, four cells give 81 operations. Requiring commutativity leaves three independent cells and gives 27. Associativity has not been imposed, so such tables need not define groups.

Floor, ceiling, and negative boundaries

The definitions $\lfloor x \rfloor = k \iff k \le x < k + 1$ and $\lceil x \rceil = k \iff k - 1 < x \le k$ imply $\lceil x \rceil = -\lfloor -x \rfloor$ and $\lfloor x + t \rfloor = \lfloor x \rfloor + t$ for integer t.

Write $x = p + \alpha, y = q + \beta$, with integer p, q and fractional parts in $[0, 1)$. Then

$$\lfloor x + y \rfloor = \lfloor x \rfloor + \lfloor y \rfloor + \lfloor \alpha + \beta \rfloor.$$

The last term is zero or one, so floor is not additive. With $x = -1.4, y = 0.7$, the left side is minus one while the separate floors sum to minus two. Truncation toward zero also differs from floor on negative inputs.

The convention $0! = 1$ counts an empty permutation. Real $\log_b x$ requires $x > 0, b > 0, b \ne 1$; positive-cost comparisons commonly also require $b > 1$.

Generalized pigeonhole bounds are tight

Place $N \geq 0$ objects into $k \geq 1$ boxes. If every box had fewer than $\lceil N/k \rceil$, the total would be at most $k(\lceil N/k \rceil - 1) < N$, contradiction. Some box has at least that many. The zero-object case holds; zero boxes cannot serve as a divisor.

Write $N = qk + r, 0 \leq r < k$. Put $q + 1$ in r boxes and q in the others, attaining the bound. Thus it cannot generally be improved by one. To force at least $t \geq 1$ together, the minimum total is $k(t - 1) + 1$.

The impossibility of injections from larger finite sets to smaller ones is the one-per-box version. Boxes must cover all objects and their number must be correctly bounded. The theorem asserts existence without identifying the box.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pigeonhole principle: a 21-game consecutive block

Prefix-sum collision: a consecutive block of 21 games

First list: $a_1 < \cdots < a_{77} = 132$

Second list: $a_1+21 < \cdots < a_{77}+21 = 153$

154 entries lie in 1...153; each list is strictly increasing, so a collision crosses lists.

$a_j = a_i + 21 \Rightarrow i < j \Rightarrow$ days i+1 through j contain 21 games

The collision determines i and j; no particular schedule is assumed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Over 77 days, 132 games are played, with at least one every day. Let a_i be the cumulative games through day i. Both 77-entry lists lie in $\{1, \ldots, 153\}$ and are strictly increasing, so their 154 entries force a cross-list collision $a_j = a_i + 21$. Strict increase implies $i < j$, and days $i + 1$ through j contain exactly $a_j - a_i = 21$ games.

Prefix residues produce a contiguous interval

Given an integer $m \geq 1$ and integers $a_1, \ldots, a_m$, let $s_0 = 0$ and $s_j = \sum_{t=1}^{j} a_t$. Including the empty prefix gives $m + 1$ prefixes, each with a residue modulo m, but only m possible residues.

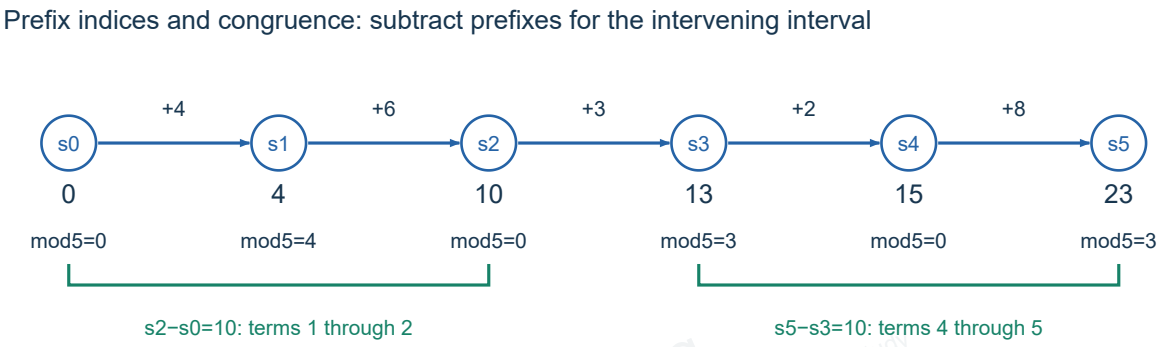
The pigeonhole principle gives distinct indices, which we order as $0 \leq i < j \leq m$, with $s_i \equiv s_j \pmod{m}$. Subtracting cancels the first i terms:

$$s_j - s_i = a_{i+1} + a_{i+2} + \cdots + a_j \equiv 0 \pmod{m}.$$

Because $i < j$, the contiguous segment contains at least one term, and its sum is divisible by m.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Example: prefix residues and a contiguous interval



For $4, 6, 3, 2, 8$, prefixes are $0, 4, 10, 13, 15, 23$, with residues $0, 4, 0, 3, 0, 3$ modulo five. Positions zero and two give $4 + 6 = 10$; three and five give $2 + 8 = 10$. The interval begins at $i + 1$, not i. Prefix zero includes intervals starting at the first item.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Odd-part boxes force divisibility

Let $n \geq 1$ be an integer. Every positive integer uniquely has form $2^t u$ with odd u. Among $n+1$ different choices from $\{1, \ldots, 2n\}$, classify by the n possible odd parts $1, 3, \ldots, 2n-1$. Two choices $2^r u, 2^s u$ share a box. Their distinctness gives $r \neq s$, so the lower power divides the higher.

The bound is tight. The n integers $n+1, \ldots, 2n$ have every larger element below twice every smaller one, excluding divisibility between distinct choices. Parity alone would not suffice: odd numbers need not divide each other. Choose classes whose shared membership forces the desired relation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A two-label proof for monotone subsequences

Let r, s be positive integers and consider a finite sequence of distinct reals. Let I_j, D_j be longest strictly increasing and decreasing subsequence lengths ending at position j. If $i < j, a_i < a_j$, appending gives $I_j > I_i$; if $a_i > a_j$, it gives $D_j > D_i$. Thus every position's label pair differs.

If no increasing subsequence has length r and no decreasing one length s, all labels lie in $[r-1] \times [s-1]$, allowing at most $(r-1)(s-1)$ positions. A longer sequence must contain one of them.

Ten distinct reals therefore contain a length-four monotone subsequence. Nine do not guarantee it: $3, 2, 1, 6, 5, 4, 9, 8, 7$ has longest lengths three in both directions. Subsequences retain order without needing contiguity.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Composition types, associativity, and iteration

For $f : A \to B, g : B \to C$, define $(g \circ f)(a) = g(f(a))$. More generally $f(A) \subseteq \operatorname{Dom} g$ suffices, but specify the final domain and codomain.

Compatible triple compositions evaluate as $h(g(f(a)))$ with either parenthesization, proving associativity. Identity maps satisfy $f \circ \mathrm{id}_A = f = \mathrm{id}_B \circ f$. Composition need not commute: on integers, $f(x) = x + 2, g(x) = 3x$ give $gf(x) = 3x + 6$, $fg(x) = 3x + 2$.

For a self-map, define $f^0 = \mathrm{id}_A$, $f^{t+1} = f \circ f^t$. This exponent counts compositions, not numerical powers of values. The example gives $f^t(x) = x + 2t$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The exact condition for injective composition

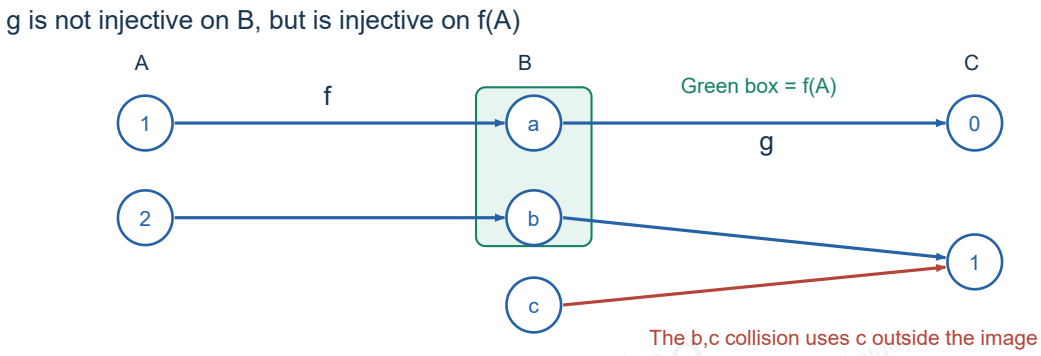
For $f : A \to B$ and $g : B \to C$, the composite $g \circ f$ is injective iff f is injective and g is injective on $f(A)$.

Forward, $f(a) = f(a')$ implies $g(f(a)) = g(f(a'))$; composite injectivity gives $a = a'$, so f is injective. For any $y, y' \in f(A)$, choose $y = f(a)$ and $y' = f(a')$. If $g(y) = g(y')$, composite injectivity gives $a = a'$ and hence $y = y'$; thus g is injective on the image.

Conversely, for any $a, a' \in A$, if $g(f(a)) = g(f(a'))$, injectivity of g on $f(A)$ gives $f(a) = f(a')$. Injectivity of f then gives $a = a'$, which is exactly injectivity of the composite.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Example: a collision outside the image



With $f(1) = a, f(2) = b$ and $g(a) = 0, g(b) = g(c) = 1$, g is not globally injective but is injective on $\{a, b\}$, so the composite is injective. Inferring global outer-map injectivity overlooks the restricted range being used.

The image condition for surjective composition

The exact condition is $g(f(A)) = C$. Thus surjective composition forces $g : B \to C$ onto, but need not force $f : A \to B$ onto. Take $A = \{1\}, B = \{a, b\}, C = \{0\}$, with $f(1) = a, g(a) = g(b) = 0$.

If g is additionally injective, each $b \in B$ has $g(f(a)) = g(b)$ for some a by composite surjectivity. Cancelling g proves $f(a) = b$, hence onto.

Two surjections compose to a surjection by finding a preimage under g first, then under f. Two injections compose by cancelling the outer map first, then the inner. Preimage search runs opposite to function evaluation.

Invertibility is equivalent to bijectivity

If $gf = \mathrm{id}_A$, $fg = \mathrm{id}_B$, the first proves injectivity of f, and the second supplies preimage $g(b)$ for every b, proving surjectivity.

Conversely, a bijection gives each b exactly one preimage, defining $g(b)$ and both identity equations. Any other inverse h satisfies $h = h(fg) = (hf)g = g$, so the inverse is unique. Compatible bijections obey $(g \circ f)^{-1} = f^{-1} \circ g^{-1}$, undoing the outer map first.

Squaring on nonnegative reals is invertible with nonnegative square root. From all reals to nonnegative reals it is onto but not invertible because of sign collisions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Left inverses, right inverses, and empty sets

A left inverse $\ell : B \to A$ with $\ell f = \mathrm{id}_A$ forces injectivity. Conversely, if f is injective and $A \neq \varnothing$, take unique preimages on $f(A)$, sending unused outputs to a fixed $a_0 \in A$. The injection $\varnothing \to B \neq \varnothing$ has no left inverse because no $B \to \varnothing$ exists.

A right inverse r with $fr = \mathrm{id}_B$ forces surjectivity. For a finite surjection, choose one point in each nonempty fiber. The universal infinite version requires the axiom of choice, whereas unique preimages of a bijection require no arbitrary choices.

On $\mathbb{N}_0$, let $f(n) = n + 1, \ell(m) = \max(m - 1, 0)$. Then $\ell f = \mathrm{id}$, but $f\ell(0) = 1 \neq 0$. If both a left and right inverse exist, $\ell = \ell(fr) = (\ell f)r = r$, making them one two-sided inverse.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Infinite cardinality and finite intuition

On $\mathbb{N}_0$, doubling is injective but not onto; flooring division by two is onto but not injective. Equinumerous sets do not turn a specified injection into a surjection.

Doubling bijects natural numbers with evens. A bijection to integers is $0 \mapsto 0, 2k-1 \mapsto k, 2k \mapsto -k$ for $k \geq 1$. Positive, negative, and zero outputs each occur once, and their signs reconstruct unique indices.

No map $F : A \to \mathcal{P}(A)$ is onto. If it were, the set $D = \{a : a \notin F(a)\}$ would equal $F(d)$ for some d, giving $d \in D \iff d \notin D$, contradiction. This general diagonal proof is not finite enumeration. Cardinality asks whether some bijection exists, not whether every existing map has the same properties.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Quantifiers and constants in growth bounds

For eventually nonnegative costs and eventually positive comparison functions,

$$f \in O(g) \iff \exists c > 0, N\, \forall n \geq N : 0 \leq f(n) \leq cg(n).$$

A lower bound $\Omega(g)$ instead requires $f(n) \geq cg(n)$, and $\Theta(g)$ requires positive lower and upper constants past one threshold. Signed-function versions use absolute values rather than negative execution times.

Constants depend on the functions but cannot vary with n. Big-O is an upper bound, not a synonym for worst case; best, average, and worst costs each admit growth bounds. Two-sided bounds imply both upper and lower bounds. Conversely, use the maximum of their thresholds to prove $\Theta(g) = O(g) \cap \Omega(g)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Proving growth with general inequalities

For $n \geq 1$, $4n^2 \leq 4n^2 + 3n + 2 \leq 9n^2$, giving $\Theta(n^2)$ witnesses 4,9,1. To prove $n^2 \notin O(n)$, any proposed fixed c, N fails at integer $n > \max(c, N)$, since $n^2 > cn$. A few large examples do not refute all constants.

Transitivity follows by $f \leq cg \leq cdh$ beyond both thresholds. Nonnegative cost bounds may similarly be added or multiplied after their thresholds.

For fixed bases $a, b > 1$, $\log_a n = \log_b n / \log_b a$, a positive constant multiple, hence the same $\Theta(\log n)$ class. A varying base such as $\log_n n = 1$ cannot be ignored.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fix input size and cost model first

An array's input size may be its element count n. A positive integer N has binary length $b = \lfloor \log_2 N \rfloor + 1$; cost linear in N may be exponential in b.

A fixed-word model may count indexing, comparison, and addition at unit cost. Arbitrary-precision arithmetic and string comparisons require bit or character costs. An operation count is not automatically a time measurement under every representation.

For the legal inputs of a fixed size, define worst cost as the supremum, possibly $+\infty$, and best cost as the infimum; these are a maximum or minimum only when attained. For example, an array of one arbitrarily long string can require unbounded character comparisons; include total character length in the size to distinguish these inputs. Average cost requires a distribution $p(x)$, with $E[T] = \sum p(x)T(x)$; the sum may be infinite. "Usually" alone supplies no distribution. Correctness also needs an input contract, output specification, invariant, and termination proof. Before comparing speed, establish that the procedures solve the same problem.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Linear search and its full correctness contract

For $A[0..n-1]$ and target x, return its first equal index, or n if absent. Short-circuit evaluation checks the index before reading the array.

```
i=0
while i<n and A[i]!=x:
    i=i+1
return i
```

At each loop start, $0 \le i \le n$ and every earlier element differs from x. This holds initially on an empty prefix. Entering the body establishes $A[i] \neq x$, so incrementing preserves it. Nonnegative $n - i$ decreases each iteration, proving termination.

At exit with $i < n$, equality holds and the invariant proves it is the first match. At $i = n$, the invariant proves absence. At $n = 0$, return zero without any array access.

Best, worst, and average linear-search costs

Count element comparisons. For $n \ge 1$, a first match at position i, index $i-1$, costs i; absence costs n. Best is one and worst n. If absence probability is $0 \le q < 1$ and successful positions are conditionally uniform,

$$E[C] = (1-q)\frac{n+1}{2} + qn.$$

When $q = 1$, conditioning on success is undefined because success has probability zero; every search fails, so directly $E[C] = n$, agreeing with the numerical value of the displayed expression at $q = 1$. For $n = 4, q = 1/3$, each successful position has probability $1/6$, contributing $1/6, 2/6, 3/6, 4/6$, and absence contributes $4/3$, totaling three. With all successful mass at the first cell, the average can instead be one when every search succeeds.

For arbitrary position probabilities $p_1, \dots, p_n$ and absence probability q summing to one, the formula is $\sum i p_i + nq$. Sorting, index construction, and variable-length comparison costs are not included.

Nested-loop ranges determine the cost

```
for i=1..n:
    for j=1..i: op()
```

The count $\sum_{i=1}^{n} i = n(n+1)/2$ lies between $n^2/2$ and n^2 for $n \geq 1$, hence $\Theta(n^2)$. Empty input gives zero operations, though program control still has constant cost.

If each inner loop instead starts $j = 1$, calls `op()` while $j \leq n$, and doubles j, it takes $\lfloor \log_2 n \rfloor + 1$ iterations per outer iteration, giving $\Theta(n \log n)$ for $n \geq 2$. A fixed c-iteration inner loop gives $cn = \Theta(n)$. Two nested loops do not by themselves determine a quadratic power. Also prevent index overflow and retain the stated operation model.

Binary exponentiation and its invariant

For a in a set with associative multiplication and identity one, and integer $N \geq 0$, define $a^0 = 1$.

```
r=1; b=a; e=N
while e>0:
    if e is odd: r=r*b
    b=b*b
    e=floor(e/2)
return r
```

The invariant is $rb^e = a^N$. At even $e = 2t$, the new product is $r(b^2)^t = rb^{2t}$. At odd $e = 2t+1$, it is $(rb)(b^2)^t = rb^{2t+1}$. Associativity suffices because factors are powers of the same b, without general commutativity. Initialization satisfies the invariant; at $e = 0$, it gives the answer.

For positive e, flooring its half strictly decreases it, proving termination. At $N = 0$, the loop is skipped and returns one. The final unused square may be omitted as an optimization, while preserving initialization and multiplication rules.

Thirteen's bits and the actual iteration states

The bits of $13 = (1101)_2$ read low first are 1,0,1,1. To compute 3^{13}:

Initial e	b	Initial r	Multiply into r?	Final r
13	3	1	Yes, low bit 1	3
6	9	3	No, low bit 0	3
3	81	3	Yes, low bit 1	243
1	6561	243	Yes, low bit 1	1594323

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Multiplication count and bit cost

Four squarings and three extra multiplications total seven. Generally $N \geq 1$ gives $b = \lfloor \log_2 N \rfloor + 1$ iterations, and popcount extra multiplications, at most $2b$ multiplications in total.

This is not a logarithmic bit-time proof for unbounded integers. Answer length can grow with N. Modular computation with positive modulus keeps intermediate values bounded, but large-modulus multiplication and reduction still need their own cost analysis.

References and attribution

Lin Cing, handwritten discrete mathematics notes. Ralph P. Grimaldi, Discrete and Combinatorial Mathematics, 5th ed., Ch. 5.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Symbols, finite words, and the empty word

An alphabet Σ is a finite nonempty set of symbols. A symbol is one indivisible token, even if its printed label has several characters. A word $w = a_1 \cdots a_n$ is a finite sequence of length n. The empty word ε has length zero and differs from a blank-space symbol.

Define $\Sigma^0 = \{\varepsilon\}$, Σ^n as all length-n words, $\Sigma^* = \bigcup_{n \geq 0} \Sigma^n$, and $\Sigma^+ = \bigcup_{n \geq 1} \Sigma^n$. Thus $\Sigma^* \setminus \Sigma^+ = \{\varepsilon\}$, with $\varnothing \neq \{\varepsilon\}$.

For $|\Sigma| = s$, independent position choices give $|\Sigma^n| = s^n$. Listing by length and a fixed within-length symbol order enumerates all words, so Σ^* is countably infinite. Each word being finite is compatible with infinitely many words.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Concatenation, prefixes, and reversal

Concatenation places sequences in order, with $|uv| = |u| + |v|$, identity ε, and associativity. It generally does not commute, as $01 \neq 10$. A common prefix cancels: deleting its $|x|$ positions in $xu = xv$ yields $u = v$.

A prefix satisfies $w = uv$, a suffix $w = vu$, and a substring $w = xuy$. Substrings are contiguous; subsequences only preserve position order. Thus 01 is not a prefix of 001, but is a suffix, substring, and subsequence.

Define $\varepsilon^R = \varepsilon$, $(aw)^R = w^R a$. Induct on $|u|$ to prove $(uv)^R = v^R u^R$. The empty case holds. Writing $u = ax$, the step gives $((ax)v)^R = (xv)^R a = v^R x^R a = v^R u^R$. Reversing positions twice also gives $(w^R)^R = w$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Language concatenation need not multiply cardinalities

A language is any subset $L \subseteq \Sigma^*$, empty, finite, or infinite. Define $AB = \{uv : u \in A, v \in B\}$. The concatenation map $A \times B \to AB$ is onto, but a word may have different split points. Thus finite languages only guarantee $|AB| \leq |A||B|$.

For $A = \{a, aa\}, B = \{\varepsilon, a\}$, the four pairs produce a, aa, aa, aaa, only three words. The duplicate is a resulting word, not a pair.

The language identity is $\{\varepsilon\}$, so $A\{\varepsilon\} = \{\varepsilon\}A = A$. In contrast $A\varnothing = \varnothing A = \varnothing$. Concatenation generally does not commute: singleton languages produce ab versus ba.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Language equalities and one-way inclusions

Both $(AB)C$ and $A(BC)$ contain exactly words abc with factors in their respective languages, proving associativity. A suffix in $B \cup C$ belongs to one of those languages, giving $A(B \cup C) = AB \cup AC$; right distribution follows similarly.

Only $A(B \cap C) \subseteq AB \cap AC$ is automatic. A shared decomposition suffices on the left, while the right may use two decompositions. With $A = \{\varepsilon, a\}, B = \{\varepsilon\}, C = \{a\}$, the left is empty and the right $\{a\}$.

Inclusion $A \subseteq B$ gives $A^n \subseteq B^n$, since every factor remains permitted. Inductively the zero powers both equal $\{\varepsilon\}$, and concatenation preserves inclusion at the next step.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Kleene star counts factors, not character length

Set $L^0 = \{\varepsilon\}$, $L^{n+1} = LL^n$, $L^* = \bigcup_{n\geq 0} L^n$, and $L^+ = \bigcup_{n\geq 1} L^n$. The exponent counts language factors, whose lengths need not be one.

We have $\varnothing^* = \{\varepsilon\}$, $\varnothing^+ = \varnothing$, and $\{\varepsilon\}^* = \{\varepsilon\}^+ = \{\varepsilon\}$. For general L, its positive closure may contain the empty word. The alphabet's Σ^+ excludes it because each symbol has positive length.

The star contains ε and L, and merging finite factor lists proves closure under concatenation. Conversely, induction on factor count shows every concatenation-closed set containing ε, L contains every L^n, hence L^*. This proves its minimality.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Star identities by splitting and regrouping

A nonempty factor list can be split before its first or after its last factor, proving $LL^* = L^*L = L^+$, even with empty-word factors. Merging two lists proves $L^*L^* \subseteq L^*$; concatenating with ε proves the reverse. Flattening finite lists of finite lists similarly gives $(L^*)^* = L^*$, $(L^*)^+ = L^*$, $(L^+)^* = L^*$.

$$(A \cup B)^* = (A^* \cup B^*)^* = (A^*B^*)^*.$$

For the first equality, flatten each starred factor into factors from $A \cup B$; conversely every individual factor belongs to one star. For the second, expand each A^*B^* factor into two blocks. Conversely write an A^* factor as $u\varepsilon$, or a B^* factor as εv. Both inclusions need justification; stars do not cancel by appearance.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Soundness and completeness of recursive languages

Start at word zero and permit adding one on either end, taking only words generated by finitely many applications. Exactly the words with one zero result. Soundness holds because rules preserve the zero count. Every target 1^i01^j is generated by i left additions and j right additions, proving completeness.

Define balanced parentheses by $\varepsilon \in D$ and $u, v \in D \Rightarrow (u)v \in D$. The intended words have nonnegative prefix height and final height zero. Induction on generation proves soundness: enclosing u raises its heights by one, returns to zero, then appends admissible v.

For completeness, induct on length. A nonempty admissible word starts with an opening parenthesis. Its first height-zero return uniquely splits it as $(u)v$, with both shorter pieces admissible, hence generated by induction. Requiring the least generated set excludes arbitrary extra words unsupported by the rules.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Grammar symbols and one derivation

Consider $G = (N, \Sigma, P, S)$, with nonterminals $N = \{S, A, B\}$, terminals $\Sigma = \{a, b\}$, and start symbol S. Its productions P are

$$S \to AB, \qquad A \to aA \mid a, \qquad B \to b.$$

Each step replaces one nonterminal using one production. Apply $A \to aA$ once, then terminate A with $A \to a$, and replace B by b:

$$S \Rightarrow AB \Rightarrow aAB \Rightarrow aaB \Rightarrow aab.$$

The intermediate form aAB still contains nonterminals; aab contains only terminals and is a generated word.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The generated language: both inclusions

The grammar generates exactly

$$L(G) = \{a^k b : k \geq 1\}.$$

Every generated word has this form. The sole start rule gives AB. Each $A \to aA$ adds one a and retains A. A terminal derivation must eventually use $A \to a$ to eliminate A, producing at least one a; B can only become one final b. Thus every generated word is $a^k b$ for $k \geq 1$.

Every target word is generated. For any integer $k \geq 1$, apply $S \to AB$, then $A \to aA$ exactly $k-1$ times, followed by $A \to a$ and $B \to b$. The result is $a^k b$; $k = 2$ gives aab. The empty word ε cannot be generated because every terminating derivation contains at least one a and one b.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A Mealy machine outputs a whole stream

A fixed-initial-state Mealy transducer is $(Q, \Sigma, \Gamma, \delta, \lambda, q_0)$, with finite state, input, and output alphabets, and total maps $\delta : Q \times \Sigma \to Q$, $\lambda : Q \times \Sigma \to \Gamma$. Each input emits one symbol; edges are labeled input/output.

Extend by $\delta^*(q, \varepsilon) = q, \lambda^*(q, \varepsilon) = \varepsilon$, and

$$\delta^*(q, wa) = \delta(\delta^*(q, w), a),$$
$$\lambda^*(q, wa) = \lambda^*(q, w)\lambda(\delta^*(q, w), a).$$

Output uses the state before reading the symbol, then the transition updates it. Substituting the new state into the output function can corrupt every step. Finite states summarize history while permitting arbitrarily long input.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A serial adder remembers only carry

Input binary numbers low bit first, with paired symbols $xy \in \{00, 01, 10, 11\}$. State $c \in \{0, 1\}$ stores carry, initially zero. Let $t = x + y + c$, output $z = t \bmod 2$, and update $c' = \lfloor t/2 \rfloor$.

Old carry	00: new/output	01	10	11
0	0/0	0/1	0/1	1/0
1	0/1	1/0	1/0	1/1

After k pairs, the low-part values satisfy $X_k + Y_k = Z_k + 2^k c_k$. Initially all are zero. Multiply the next-bit identity $x + y + c_k = z + 2c_{k+1}$ by 2^k and add to the previous invariant to preserve it. This proves arbitrary input lengths, rather than only eight one-step cases.

Thirteen plus six and the final carry

Input $13 = (1101)_2$ and $6 = (0110)_2$ low first, followed by one padding 00 to emit the final carry.

Bit k	Pair	Old c	t	Output	New c
0	10	0	1	1	0
1	01	0	1	1	0
2	11	0	2	0	1
3	10	1	2	0	1
4, padding	00	1	1	1	0

Output arrives as 11001; reversing gives conventional 10011, nineteen. Before padding, $13 + 6 = 3 + 16 \cdot 1$; after padding carry is zero and output value nineteen. Ignoring the final state after only four inputs would retain just the low value three.

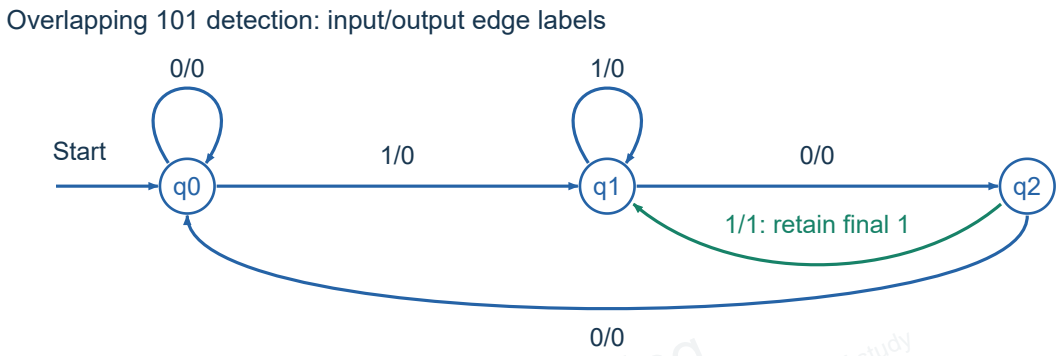
Detecting 101 requires overlap memory

States q_0, q_1, q_2 store the longest suffix matching a proper prefix of 101, respectively $\varepsilon, 1, 10$. Emit one exactly when the latest symbol completes 101.

State	Input 0: new/output	Input 1: new/output
q_0	$q_0/0$	$q_1/0$
q_1	$q_2/0$	$q_1/0$
q_2	$q_0/0$	$q_1/1$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Detecting 101 requires overlap memory (continued)



Reading 10101 gives states $q_0, q_1, q_2, q_1, q_2, q_1$ and output 00101. A completed match returns to q_1, since its final one can start another. Returning to q_0 misses positions three through five. Every table cell preserves the longest matching suffix; induction from empty input proves output one iff the last three input bits are 101.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Position restrictions require a phase

To report matches only at positions divisible by three, also store consumed length modulo three. Use nine product states (q, r), initially $(q_0, 0)$. For input a, let the original table give q', z, set $r' = (r + 1) \bmod 3$, and emit one iff $z = 1 \wedge r' = 0$. The new state is (q', r').

Each component invariant is preserved, proving both conditions. Output for 10101 becomes 00100, and for 101101 it is 001001. Resetting phase only upon matches would measure distance from the previous match, not global position. Nine is a construction upper bound, not an assertion of minimality.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fixed delay, state lower bounds, and output length

A delay of $k \geq 1$ ticks with initial zero padding uses queue state $(b_1, \ldots, b_k) \in \{0, 1\}^k$. On input a, emit old b_1 and shift to $(b_2, \ldots, b_k, a)$, starting at all zeros.

Induction shows the queue stores the latest k inputs, padded on the left when needed. Output t is zero for $t \leq k$, and input $t - k$ afterward. Length-n input gives length-n output, without automatic extra ticks.

For delay two and input 10110, states are $00 \to 01 \to 10 \to 01 \to 11 \to 10$, with output 00101. Two explicit padding inputs are needed to flush the remaining bits.

At least 2^k states are necessary. After any of the distinct length-k histories, append common 0^k. The next output block reproduces the history, so different histories cannot have the same machine state.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

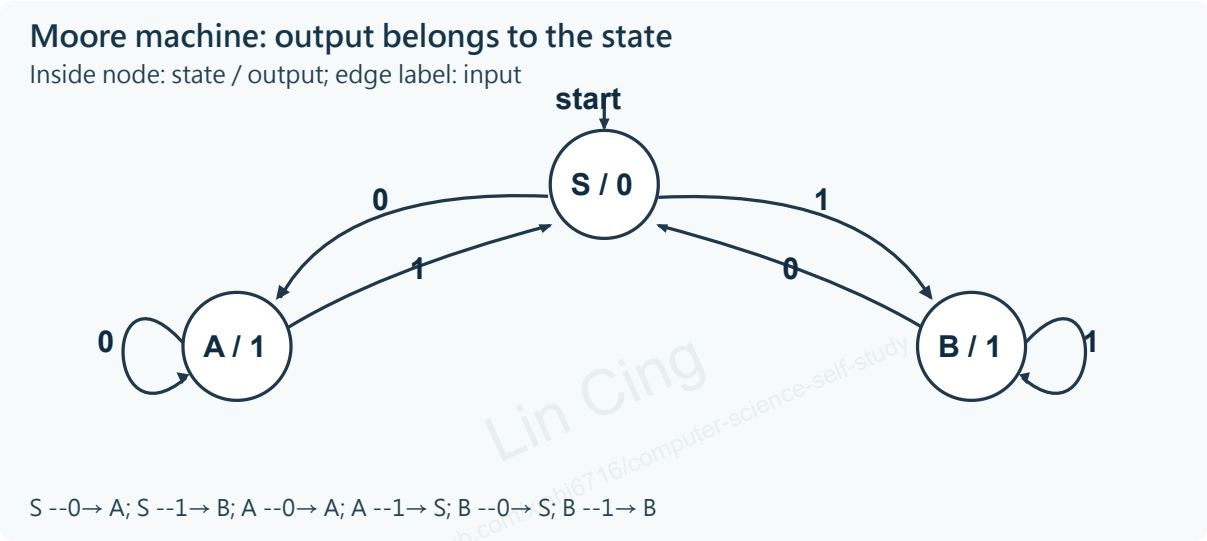
Align Mealy and Moore output timing

A Moore machine's output $h : Q \to \Gamma$ depends only on state. Here each input first transitions, then emits the new state's output; initial output is excluded. Including initial output or emitting before transition introduces a one-symbol shift.

From Moore to Mealy, keep transitions and set $\lambda(q, a) = h(\delta(q, a))$. From Mealy to Moore, states $(q, z) \in Q \times \Gamma$ record the new state and last output; let $h(q, z) = z$, and transition to $(\delta(q, a), \lambda(q, a))$. Add a fresh initial s, choose any $z_0 \in \Gamma$, and set $h(s) = z_0$, keeping the Moore output function total. On its first input, s goes to $(\delta(q_0, a), \lambda(q_0, a))$; under this timing convention, $h(s)$ is not counted in the output stream.

Induction on input length identifies the first component with the original state and second with its last output. Entire streams agree for nonempty input, and both give the empty output on empty input. Finiteness is preserved, though states may increase.

Moore machine: states and outputs



The value inside each node is the state output h; edge labels are inputs. The initial state is S, with $h(S) = 0$ and $h(A) = h(B) = 1$. Every state has exactly one transition for each input.

One path under two output conventions

Read 0011 left to right. For each input, take the transition from the current state, then read the output of the new state.

Time	Initial	Read 0	Read 0	Read 1	Read 1
State	S	A	A	S	B
State output	0	1	1	0	1

Including the initial output gives **01101**, one symbol longer than the input. This chapter emits after each transition and excludes the initial output, giving **1101**, the same length as the input. Fix the convention before comparing Moore and Mealy machines.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Reachable, transient, sink, and submachine

State q is reachable from p if $\delta^*(p, w) = q$ for some $w \in \Sigma^*$, including self-reachability through empty input. This chapter uses that convention; the reference textbook restricts reachability here to distinct states connected by a nonempty transfer string. A state is transient if no nonempty word returns to it. Absence of a self-loop alone is insufficient; a two-edge cycle also returns.

A sink satisfies $\delta(q, a) = q$ for all symbols, hence remains there on all words by induction. A submachine's selected states must be closed under its retained input alphabet; a circled region with outward transitions does not qualify.

Strong connectivity means mutual reachability of every state pair. These transition properties do not determine outputs. A sink of an acceptor may accept or reject according to its membership in the accepting set.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

BFS proves shortest transfer sequences

A transfer sequence is a shortest word from one specified state to another. With unit symbol costs, BFS discovers states by word length. Store a predecessor and input label at first discovery, then backtrack for the word.

State	0	1
S	A	B
A	A	T
B	T	B
T	T	T

BFS from S finds S at distance 0, A and B at distance 1, then T at distance 2. Both 01 and 10 are shortest.

BFS shortestness and cost

A revisited state need not be expanded with a longer prefix: the same state followed by the same suffix gives the same result, so the longer prefix cannot yield a shorter path.

A shortest path between distinct reachable states repeats no vertex; deleting a repeated-vertex cycle would shorten it. Its length is at most $|Q| - 1$. Searching the complete transition table costs $O(|Q||\Sigma|)$ time and $O(|Q|)$ search space. Unequal symbol costs invalidate the unit-cost BFS shortestness argument.

Acceptors differ from stream transducers

A DFA is $M = (Q, \Sigma, \delta, q_0, F)$, with $q_0 \in Q$, $F \subseteq Q$, and total δ. It accepts w iff $\delta^*(q_0, w) \in F$, defining $L(M)$.

Each input has one complete path, and acceptance is decided at its end. Passing an accepting state midway does not suffice. The empty word is accepted iff $q_0 \in F$. A double circle marks acceptance; a source-free arrow marks the initial state.

Mealy output is emitted at every step and has no built-in accepting set. Using final output one as acceptance requires an additional empty-input convention, since empty input emits nothing.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The extended-transition concatenation lemma

With $\delta^*(q, \varepsilon) = q$ and $\delta^*(q, wa) = \delta(\delta^*(q, w), a)$,

$$\delta^*(q, uv) = \delta^*(\delta^*(q, u), v).$$

Induct on $|v|$. The empty suffix is immediate. If $v = xa$, use the induction hypothesis for x, then append the final transition on a. This proves that a common suffix from equal states gives equal results.

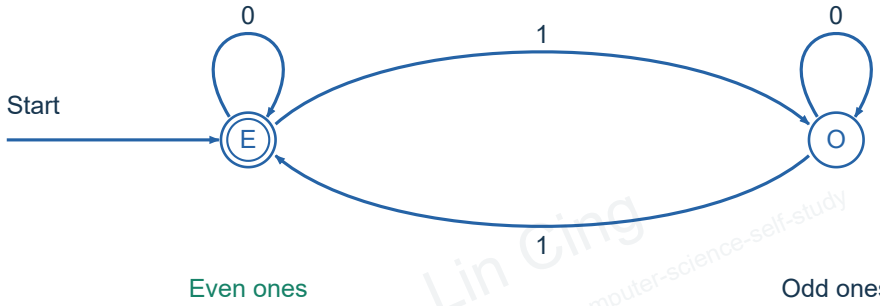
A machine design needs a state invariant proved from empty input through every symbol, then acceptance iff the target property. Finite tests may refute a design but cannot replace this universal proof.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A two-state acceptor for even ones

States E,O record even and odd counts of ones. Start E and accept only E. Zero leaves state unchanged; one toggles it.

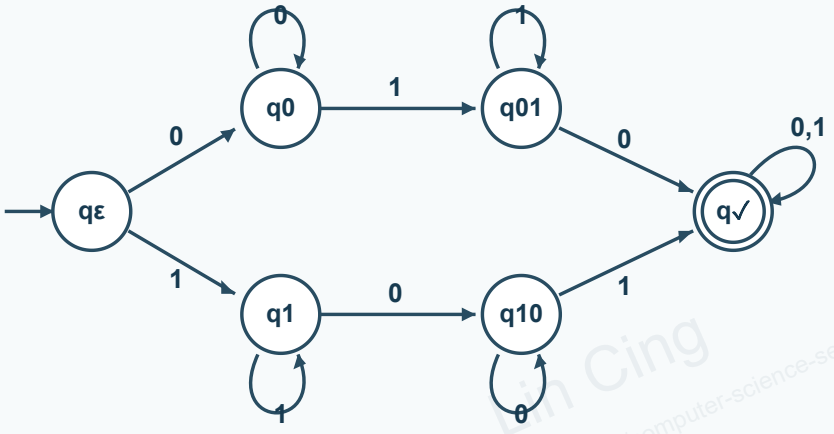
Even ones: E is initial and accepting; edges show inputs



Empty input has zero ones. Appending zero preserves parity, while one flips it, matching all four transitions. Induction proves the state meanings and the desired acceptance equivalence.

The path for 1010 is $E \to O \to O \to E \to E$, accepting. Word 1011 ends at O and rejects, regardless of earlier visits to E.

A DFA for both 01 and 10



Double circle: the only accepting state. Each state has both 0 and 1 transitions.

States record which adjacent changes have occurred; only $q_{\checkmark}$ accepts. Acceptance is decided after the whole word is read, and seeing just one pattern is insufficient.

Trace both patterns

Input	Path	Result
010	$q_\varepsilon \to q_0 \to q_{01} \to q_\checkmark$	Accept
101	$q_\varepsilon \to q_1 \to q_{10} \to q_\checkmark$	Accept
01	$q_\varepsilon \to q_0 \to q_{01}$	Reject
10	$q_\varepsilon \to q_1 \to q_{10}$	Reject

The empty word stays at q_ε, so it is rejected. By induction on input length, q_{01} records that 01 has occurred and 10 has not; the last bit must then be 1, so reading 0 creates the first 10. The case q_{10} is symmetric. Once reached, $q_\checkmark$ retains both patterns, so acceptance is equivalent to having both.

A modulo-three summary of binary values

Allow leading zeros and assign value zero to empty input. Appending bit b changes value v to $2v + b$, so remember only its residue:

r	0	1
0	0	1
1	2	0
2	1	2

Start at residue zero and accept zero. The initial residue and preservation under multiplication by two plus a bit prove the invariant by induction. Requiring nonempty representations instead needs an additional seen-input flag; merely changing the verbal specification would leave the old machine accepting empty input.

Product machines preserve two invariants

For two total DFAs on one alphabet, use states $Q_1 \times Q_2$, paired initial state, and componentwise updates. Induction gives after w exactly the pair of original final states.

Accepting pairs in $F_1 \times F_2$ gives intersection. Accepting $(F_1 \times Q_2) \cup (Q_1 \times F_2)$ gives union. First accepting and second rejecting gives difference; exactly one accepting gives symmetric difference. Substituting the paired-state invariant proves every language identity.

There are at most $|Q_1||Q_2|$ states; discard unreachable pairs. Both acceptors read the same input. This is not feeding one whole-word output into another machine.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A six-state product for multiples ending in one

Store (r, ℓ), residue modulo three and whether the latest bit is one, using empty-input flag $\ell = 0$. Start $(0, 0)$, update on bit b to $((2r + b) \bmod 3, b)$, and accept only $(0, 1)$.

State	0	1
(0,0)	(0,0)	(1,1)
(0,1)	(0,0)	(1,1)
(1,0)	(2,0)	(0,1)
(1,1)	(2,0)	(0,1)
(2,0)	(1,0)	(2,1)
(2,1)	(1,0)	(2,1)

Word 10101 follows $(0, 0) \to (1, 1) \to (2, 0) \to (2, 1) \to (1, 0) \to (0, 1)$, accepting value 21. Word 1010 has value ten and ends in zero, finishing $(1, 0)$ and rejecting. Empty input rejects too because it has no final one.

Complete transitions before complementing

Every word in a total DFA reaches one final state. Replacing F by $Q \setminus F$ therefore complements its language relative to fixed Σ^*.

For partial transitions with missing edges rejecting, flipping circles fails. A sole initial accepting state s on alphabet $\{a\}$, with no edges, accepts only ε. Flipping it yields the empty language instead of the correct a^+.

First add a rejecting sink d, direct every missing edge to it, and loop it on all symbols. This preserves the language. Then flip acceptance, making d accepting and s rejecting, to obtain a^+. Missing-edge rejection is a definite semantics, not a third status between acceptance and rejection.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

NFA acceptance means one complete path exists

An epsilon-NFA can use $\Delta : Q \times (\Sigma \cup \{\varepsilon\}) \to \mathcal{P}(Q)$. Symbols may have zero or multiple successors; epsilon edges consume no input. Acceptance requires a finite path starting at the initial state, whose labels after removing epsilons equal the whole input, and ending accepting.

Not every branch needs success. Reaching an accepting state does not excuse unread input. No available branch means rejection; epsilon cycles do not require infinite execution.

Let $E(S)$ be states reachable from S by zero or more epsilon edges. A work queue with visited states adds each new state once, processing at most $|Q|$ states even with cycles. Induction on reachable-path length proves completeness.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Subset construction and its general proof

The deterministic start is $S_0 = E(\{q_0\})$, with subsets as states and update

$$T(S, a) = E\left(\bigcup_{q \in S} \Delta(q, a)\right).$$

A subset accepts iff it intersects the NFA accepting set. If the empty subset is reachable, retain it as a rejecting sink: every input leaves it empty. If constructing only reachable subsets and the empty subset is unreachable, it need not be created.

Induct on input length. At zero, the subset is precisely epsilon-reachable states. Appending a first takes an a-edge from a currently reachable state, then any epsilon edges, exactly the update. Conversely every updated point has such a path. Thus final acceptance is equivalent to existence of an NFA accepting path.

At most $2^{|Q|}$ subsets occur; construct only reachable ones. NFAs recognize no additional language class, though they may provide smaller descriptions.

An epsilon-NFA for suffix ten

Start s has an epsilon edge to p. State p loops on zero and one and also has a one-edge to q. State q reads zero to accepting r. All unlisted transitions are empty. The machine guesses the start of the last two bits and accepts exactly suffix 10.

NFA for suffix 10: accept if a complete input path ends at r

Initial s takes an epsilon edge; p retains every possible prefix.

Unlisted transitions are empty; only r accepts.

Subset	0	1	Accepting?
S={s,p}	P={p}	Q={p,q}	No
P={p}	P	Q	No
Q={p,q}	R={p,r}	Q	No
R={p,r}	P	Q	Yes

An epsilon-NFA for suffix ten (continued)

Word 1010 follows $S \to Q \to R \to Q \to R$, accepting. After R, reading one loses the old accepting r branch, but p retains a fresh potential match start. Word 0101 ends Q and rejects.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

NFA constructions for union and concatenation

First rename component states disjointly. For union, a fresh initial state epsilon-branches to both initial states; accept their combined accepting set. Every accepting path chooses one component, and any component accepting path extends from the new start, proving union in both directions.

For concatenation, retain the first start and add epsilon edges from each first-machine accepting state to the second start. Accept only second-machine accepting states. Split an accepting path at its cross-machine edge into uv, accepted by the respective components. Conversely concatenate such paths. Empty factors are handled by epsilon paths.

Disjoint renaming prevents accidental returns from the second machine to the first. Merging same-named states or retaining first-component accepting states as final can introduce erroneous acceptance after only the first factor.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Explicit path proofs for star and reversal

For star, add start s and accepting t, with epsilon edges s to t and the old start, and from every old accepting state to t and the old start. Only t accepts. The direct s-to-t path represents zero factors. Every restart begins another original-language factor; conversely concatenate accepting paths for any finite factor list.

For reversal, reverse all edges retaining labels. A fresh start epsilon-branches to every old accepting state; only the old initial state accepts. Reverse each finite old path to reverse its symbol word, proving both language inclusions. If the old accepting set is empty, the new machine has no entry to old states and still accepts nothing.

Together with determinization, these prove regular-language closure under union, concatenation, star, and reversal. Total-DFA constructions prove intersection, difference, and complement.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Regular-expression syntax and meaning

Regular expressions start with $\varnothing, \varepsilon, a$ for $a \in \Sigma$, and finitely apply union $R|S$, concatenation RS, and star R^*. Their languages are respectively empty, singleton empty word, singleton symbol, and the associated language operations.

Star binds before concatenation, which binds before union. Expression $(0|1)^*10$ describes arbitrary binary prefixes followed by 10, matching the preceding NFA. Changing parentheses to $0|1^*10$ changes the language.

Each expression yields an NFA. The three base cases use no accepting path, one epsilon path, or one symbol edge. The union, concatenation, and star constructions handle its syntax tree inductively. This proves existence and gives a terminating algorithm, since a finite tree invokes finitely many finite-state constructions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A path recurrence from automata to expressions

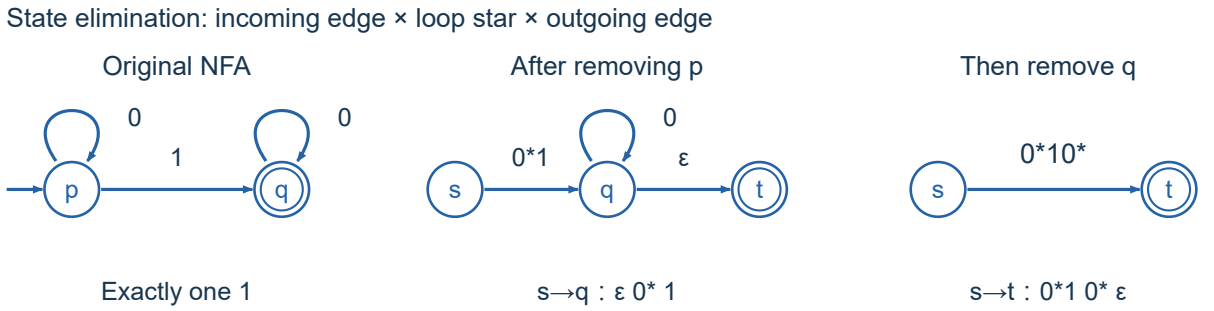
Number NFA states one through n. Let $R_{ij}^{(k)}$ denote paths from i to j whose internal states are among one through k. Base $R_{ij}^{(0)}$ unions direct-edge labels, also adding ε for i=j, and uses $\varnothing$ if no edge exists between distinct states.

$$R_{ij}^{(k)} = R_{ij}^{(k-1)} \mid R_{ik}^{(k-1)} (R_{kk}^{(k-1)})^* R_{kj}^{(k-1)}.$$

Paths without internal k give the first term. Otherwise split at successive visits to k into a first i-to-k segment, zero or more k-to-k segments, and a final k-to-j segment, each internally avoiding k. Conversely concatenating these segments uses only permitted internal states. Empty first or last segments cover endpoints equal to k.

Union the final $R^{(n)}$ from start to each accepting state to obtain the language expression. Round k must use previous-round values rather than overwritten current values. Finitely many rounds terminate, but expanded expressions may be large; no size-preservation claim follows.

A concrete state-elimination example



Star permits zero repetitions, so word 1 must also accept.

The start state p and accepting state q each have a 0-loop; p has a 1-edge to q and there are no other edges. Add a new start s with an ε-edge to p, and an ε-edge from q to a new accepting state t.

State elimination: update each label

Eliminating p creates the s-to-q label $\varepsilon 0^* 1 = 0^* 1$. Eliminating q then gives the s-to-t label $(0^*1)0^*\varepsilon = 0^*10^*$. These words contain exactly one 1, with any number of zeros before and after.

If another s-to-q edge already existed, take the union with the new label; do not overwrite it. The loop star permits zero repetitions; omitting that case would incorrectly reject the word 1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Equivalent states agree on every future suffix

Define $p \equiv q$ when every $w \in \Sigma^*$ has the same acceptance result starting at p and q. This is an equivalence relation; the empty suffix forces equal acceptance marks.

For any symbol a and suffix w, the concatenation lemma rewrites acceptance after $\delta(p, a)$ on w as acceptance after p on aw. Thus $p \equiv q$ implies $\delta(p, a) \equiv \delta(q, a)$. Quotient transitions $[p] \xrightarrow{a} [\delta(p, a)]$ are independent of representatives, as are the initial and accepting classes.

Induction on input length makes the quotient's final class equal the class of the original final state, preserving the language. Nearby drawings or identical one-step labels cannot replace the universal-suffix condition. Chapter 7 computes this partition effectively.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Equivalence and shortest distinguishing words

For two total DFAs on one alphabet, construct a product accepting when exactly one component accepts. Its language is their symmetric difference. The machines are equivalent iff no accepting product state is reachable from the start.

BFS of reachable pairs finds a shortest distinguishing word by backtracking labels. If the initial pair already accepts, the word is empty. Otherwise a shortest path repeats no state and has length at most $|Q_1||Q_2| - 1$. Exhausting reachable pairs without acceptance proves equal results for all words via the projection invariant.

Switching the even-ones machine's accepting state from E to O makes their initial results disagree, so empty input is the shortest counterexample. Exhaustive enumeration of all words below the bound is unnecessary.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pumping lemma and its quantifiers

For a p-state DFA, every accepted word of length at least p has a split $w = xyz$, with $|xy| \leq p$, $|y| \geq 1$, such that xy^tz is accepted for every $t \geq 0$.

The first p input symbols pass through p+1 state positions, so positions $i < j \leq p$ repeat a state. Let x end at i, y contain the next j−i symbols, and z the remainder. The nonempty y labels a loop, traversable zero or arbitrarily many times before the same z, preserving acceptance.

A nonregularity proof must defeat every permitted split, choosing a suitable repetition count, rather than one convenient split. Pumpability is a necessary condition, not a sufficient machine construction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Equal blocks of zeros and ones need unbounded memory

For $L = \{0^n 1^n : n \geq 0\}$, suppose a p-state DFA exists and choose $0^p 1^p$. Every pumping split has a nonempty y entirely inside the initial zeros. Taking t=0 removes zeros but leaves p ones, producing a word outside L, contradiction.

Independently, among $\varepsilon, 0, \ldots, 0^p$, two prefixes $0^i, 0^j$, i<j, reach one state. Append 1^i. The first word belongs to L and the second does not, contradicting identical future behavior.

The impossibility concerns a fixed finite-state machine with unbounded input lengths. Bounding the number of zeros in advance allows finitely many counting states and does not conflict with the result.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Recursive balanced parentheses are not regular

Prefixes with arbitrarily many opening parentheses require arbitrarily many distinct histories. If prefixes with i<j openings reached the same state, appending i closings would balance the first but leave j−i unmatched openings in the second, contradiction. Pigeonhole guarantees such a collision for every proposed finite state count, so the language is not regular.

Its earlier recursive definition remains valid. Recursive generation is different from finite-state memory. A growing stack or counter can retain unbounded unmatched depth; a fixed maximum depth permits a finite-state encoding. The boundary depends on distinguishable histories, not visual graph complexity.

References and attribution

Lin Cing, handwritten discrete mathematics notes. Ralph P. Grimaldi, Discrete and Combinatorial Mathematics, 5th ed., Ch. 6.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Basic properties of relations

For $R \subseteq A^2$, reflexivity means $\forall a \in A, (a, a) \in R$; symmetry means $\forall a, b, aRb \Rightarrow bRa$; and transitivity means $\forall a, b, c, (aRb \land bRc) \Rightarrow aRc$. Irreflexivity forbids every loop and is not the logical negation of reflexivity. Antisymmetry requires $(aRb \land bRa) \Rightarrow a = b$, and differs from asymmetry. For example, $\leq$ is reflexive and transitive without being symmetric.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Asymmetry differs from antisymmetry

Asymmetry requires $aRb \Rightarrow \neg bRa$, including $a = b$, so it forbids loops. It is equivalent to antisymmetry plus irreflexivity. The first forbids opposite edges between distinct points; the second forbids the same-point case.

Equality $\Delta_A = \{(a, a) : a \in A\}$ is both symmetric and antisymmetric. On a nonempty set, $\leq$ is antisymmetric but not asymmetric. The empty relation on nonempty A is symmetric, antisymmetric, transitive, and irreflexive, but not reflexive.

Transitivity allows repeated variables: aRb and bRa force aRa. Checking only three distinct points misses this. On the empty set the unique relation is both reflexive and irreflexive, since neither universal assertion has a counterexample.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Counting finite relations

For $|A| = n$, each ordered pair is optional, giving 2^{n^2} relations.

Requirement	Independent choices	Count
Reflexive	Fixed diagonal; other pairs free	$2^{n(n-1)}$
Symmetric	Free diagonal; opposite directions chosen together	$2^{n+\binom{n}{2}}$
Reflexive and symmetric	Fixed diagonal; free unordered pairs	$2^{\binom{n}{2}}$
Antisymmetric	Free diagonal; neither, left, or right per distinct pair	$2^n 3^{\binom{n}{2}}$
Asymmetric	Same choices without diagonal	$3^{\binom{n}{2}}$
Symmetric and antisymmetric	Only diagonal pairs optional	2^n

Adding reflexivity in the last row leaves equality alone. Transitivity couples choices of different pairs and prevents independent multiplication.

Composition direction and matrices

Use the usual function direction. For $R \subseteq A \times B, S \subseteq B \times C$,

$$(a, c) \in S \circ R \iff \exists b\ (aRb \wedge bSc).$$

Apply R first, then S. Some references use left-to-right notation; the pair definition determines the convention.

Both associative expressions mean $\exists b, c\ (aRb \wedge bSc \wedge cTd)$, proving $T \circ (S \circ R) = (T \circ S) \circ R$. Reversing the route gives $(S \circ R)^{-1} = R^{-1} \circ S^{-1}$.

With source rows and destination columns, $M_R \odot M_S = M_{S \circ R}$. Integer multiplication counts two-step walks; Boolean multiplication asks whether at least one exists. For the following properties, take $R \subseteq A \times A$, let $M = M_R$ use the same ordering for rows and columns, and let I be the identity matrix of that order. Entrywise, reflexivity means $I \leq M$, symmetry $M = M^T$, transitivity $M \odot M \leq M$, and antisymmetry $M \wedge M^T \leq I$.

Zero-one matrices

Define $M_{ij} = 1$ iff $(a_i, a_j) \in R$. Reflexivity means an all-one diagonal; symmetry means $M = M^T$. Boolean composition uses

$$(A \odot B)_{ij} = \bigvee_k (A_{ik} \wedge B_{kj}).$$

The reverse relation has the transposed matrix. Transitive closure is the union of Boolean powers. On $n \geq 1$ vertices, positive reachability uses $R \cup \cdots \cup R^n$: distinct endpoints need at most n−1 edges in a simple path, but a nonempty return cycle may need n. Reflexive-transitive closure uses $I \cup \bigcup_{k=1}^{n-1} R^k$. For $n = 0$, the base set is empty, both R and the identity relation are empty, and $R^+ = R^* = \varnothing$; the displayed $n - 1$ power bound is not used. Boolean addition is OR and multiplication AND, unlike integer multiplication.

When $n = 1$, the union from 1 to 0 is empty, so $R^* = I$.

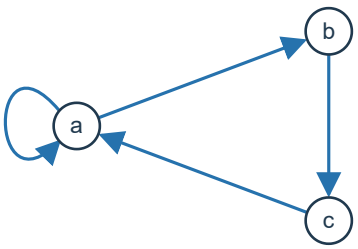
Powers of a three-vertex digraph

Take $a \to a, a \to b, b \to c, c \to a$, indexed a,b,c:

$$M = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}, \quad M^{\odot 2} = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 0 & 0 \\ 1 & 1 & 0 \end{pmatrix}, \quad M^{\odot 3} = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{pmatrix}.$$

The square's bb entry is zero because c does not return directly to b. The cube's bb entry is one via $b \to c \to a \to b$; c also first returns in three steps.

Three-point relation: edges and length-2,3 walk endpoints



Original M: 100 OR 010 = row a, 110

M² rows : 111 / 100 / 110

M³ rows : 111 / 110 / 111

b → c → a → b: first positive return

R⁺ = R ∪ R² ∪ R³: all 9 pairs

OR-ing the three powers gives all ones. The square alone is not the closure. Relations are sets of pairs, allowing loops and opposite edges but no parallel copies of a pair.

Closure existence and minimality

Let R^k contain endpoints of exactly length-k walks and $R^0 = \Delta_A$. Define

$$R^+ = \bigcup_{k \geq 1} R^k, \qquad R^* = \Delta_A \cup R^+.$$

Concatenating positive walks proves R^+ transitive and containing R. If transitive T contains R, induction on walk length gives every $R^k \subseteq T$, so $R^+ \subseteq T$. This proves minimality; R* similarly is the smallest reflexive-transitive superrelation.

On $n \geq 1$ vertices, delete internal cycles between distinct endpoints to leave at most n−1 edges. A shortest positive return is a cycle of at most n edges. Hence

$$R^+ = R \cup \cdots \cup R^n, \qquad R^* = \Delta_A \cup \bigcup_{k=1}^{n-1} R^k.$$

The diagonal of a directed three-cycle shows why the first union cannot always stop at n−1.

When $n = 1$, the union from 1 to 0 is empty, so $R^* = I$.

Warshall's invariant and in-place correctness

Start at $W^{(0)} = M_R$ and number vertices 1,…,n:

$$W_{ij}^{(k)} = W_{ij}^{(k-1)} \vee (W_{ik}^{(k-1)} \wedge W_{kj}^{(k-1)}).$$

An entry is one iff a positive i-to-j walk has internal vertices among 1,…,k. A shortest eligible walk either avoids internal k, or splits at k into two segments internally using earlier vertices. Conversely concatenate such segments. If an endpoint is k, a shortest walk needs no internal revisit, so the first term already retains it.

```
W := copy(M)
for k = 1..n:
    for i = 1..n:
        for j = 1..n:
            W[i,j] := W[i,j] OR (W[i,k] AND W[k,j])
```

At fixed k, its row and column update as $x \vee (x \wedge y) = x$. Needed entries therefore stay unchanged, justifying in-place updates. Time is $O(n^3)$, matrix space $O(n^2)$. For R*, initialize the diagonal to one.

Every round on a four-vertex chain

Use edges a→b,b→c,c→d and indices a,b,c,d.

Round	Intermediate	New pairs	Four matrix rows
0	None	Original	0100 / 0010 / 0001 / 0000
1	a	None	0100 / 0010 / 0001 / 0000
2	b	(a,c)	0110 / 0010 / 0001 / 0000
3	c	(a,d),(b,d)	0111 / 0011 / 0001 / 0000
4	d	None	0111 / 0011 / 0001 / 0000

Round three uses the preceding round's a→c to obtain a→d, rather than only original edges. The acyclic graph's R^+ diagonal is zero; R^* adds loops.

Reflexive closure is $R \cup \Delta$; symmetric closure is $R \cup R^{-1}$. Equivalence closure is $(R \cup R^{-1})^*$. Reversing walks gives symmetry. Every equivalence superrelation contains inverses, identities, and concatenations, so contains this expression. Equivalently, ignore directions, find components, and include every within-component pair.

Every round on a four-vertex chain (continued)

The intersection of a **nonempty** family of relations on a fixed set A preserves reflexivity, symmetry, antisymmetry, and transitivity separately. For antisymmetry, if both aRb and bRa lie in the intersection, choose one relation in the family and use its antisymmetry to conclude a=b. Nonemptiness is needed to choose that relation. The other properties follow by applying their conditions in each member.

The empty-family intersection relative to A^2 is A^2. It is reflexive, symmetric, and transitive, but if A has distinct elements a,b it contains both (a,b) and (b,a), so is not antisymmetric. The nonempty-family condition cannot be omitted. Unions need not preserve transitivity: {(1,2)} and {(2,3)} are individually transitive but their union lacks (1,3).

A union need not preserve transitivity

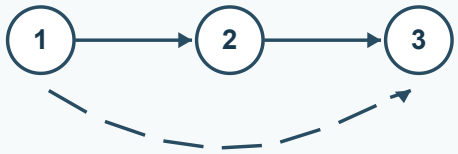
Each is transitive

R = {(1,2)}

S = {(2,3)}

One missing pair disproves transitivity

Union need not be transitive



Missing 1→3

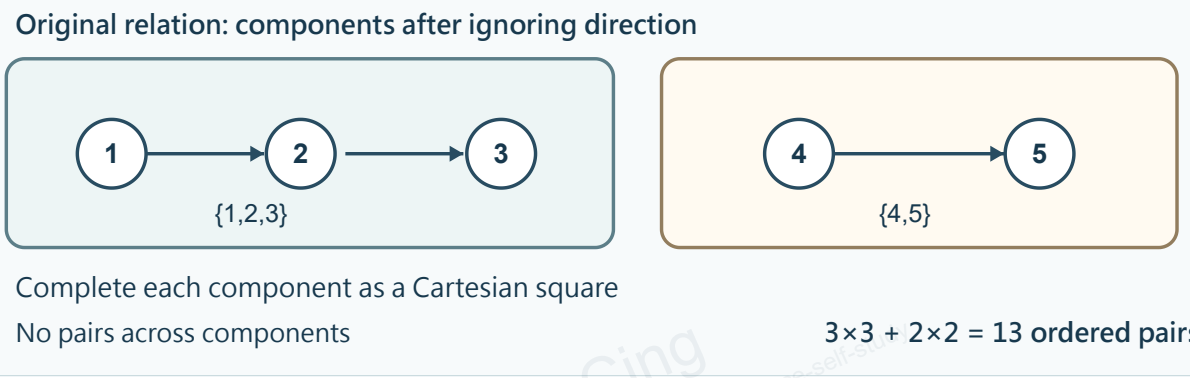
On $A = \{1, 2, 3\}$, let $R = \{(1, 2)\}$ and $S = \{(2, 3)\}$. Neither relation has two composable edges, so each is vacuously transitive. Their union has $1 \to 2 \to 3$ but lacks $(1, 3)$, and is not transitive.

Intersections and the empty family

Suppose each $R_i \subseteq A^2$ is transitive. If $(a, b), (b, c) \in \bigcap_{i \in I} R_i$, both pairs occur in every R_i. Transitivity puts (a, c) in every R_i, hence in the intersection. This proves the claim for a nonempty family.

For $I = \varnothing$, define the intersection relative to A^2 as A^2; it is still transitive. However, if $|A| \geq 2$, it is not antisymmetric. Thus transitivity is preserved even for the empty family, while antisymmetry requires this boundary to be considered separately.

Equivalence closure via connected components



Let $A = \{1, 2, 3, 4, 5\}$ and $R = \{(1, 2), (2, 3), (4, 5)\}$. Ignoring direction gives components $B_1 = \{1, 2, 3\}$ and $B_2 = \{4, 5\}$. The candidate closure is $E = (B_1 \times B_1) \cup (B_2 \times B_2)$.

Why is the closure minimal?

The candidate E relates points in the same component. It is reflexive, symmetric and transitive, and it contains every edge of R.

Conversely, any equivalence relation containing R must contain reversed edges and self-pairs, and transitivity forces it to contain both endpoints of every undirected path. It therefore contains all pairs within both components, hence contains E. Thus E is the smallest equivalence relation containing R, with $3^2 + 2^2 = 13$ ordered pairs and no pairs across components.

Explicitly, $B_1^2 = \{(1, 1), (1, 2), (1, 3), (2, 1), (2, 2), (2, 3), (3, 1), (3, 2), (3, 3)\}$ and $B_2^2 = \{(4, 4), (4, 5), (5, 4), (5, 5)\}$.

Partial orders

A partial order is reflexive, antisymmetric, and transitive; $(A, \preceq)$ is a poset. Two elements are comparable if $a \preceq b$ or $b \preceq a$, otherwise incomparable. A total order makes every pair comparable.

Write $a \prec b$ for $a \preceq b, a \neq b$. Element b covers a if $a \prec b$ and no c satisfies $a \prec c \prec b$. The word partial means not every pair must be comparable.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Strict orders, restrictions, and covers

If $a \prec b \prec c$, transitivity gives $a \preceq c$. Were a=c, antisymmetry would force a=b, a contradiction. Thus the strict relation is irreflexive and transitive. Conversely, adding equality to an irreflexive transitive relation gives a partial order, since opposite comparisons would force a loop.

Restriction to $B \subseteq A$ preserves the three axioms but changes covers. Removing 2 from 1≺2≺4 makes 1–4 a cover. In a finite poset, insert intermediate elements in any noncover a≺b; finiteness prevents endless insertion, giving a chain of covers. The infinite dense order $(\mathbb{Q}, \leq)$ has no covers, so an empty cover graph cannot reconstruct it.

Positive-integer divisibility is a partial order. On nonzero integers, 1 and −1 divide each other without equality, giving only a preorder.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Hasse diagrams

Place smaller elements lower, draw all covers, omit loops, and represent transitivity by upward paths. Only covers receive direct edges; the upward convention permits omitting arrows. If a⩽b has an intermediate c, a–b is not a Hasse edge. Drawing every pair in transitive closure adds redundant lines and hides cover structure.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

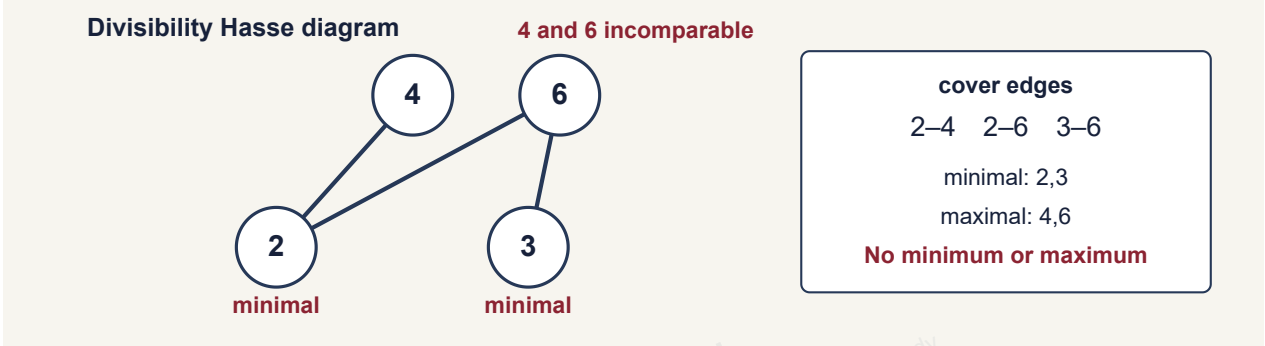
Extremal elements

A minimal element $m \in S$ has no $x \in S$ with $x \prec m$; a maximal element has no strictly larger element. A minimum must satisfy $m \preceq s$ for every $s \in S$, and a maximum requires $s \preceq M$ for every $s \in S$.

There may be several minimal or maximal elements. A minimum or maximum, when it exists, is unique. A minimum is minimal, but minimality alone does not require comparability with every element.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Extremal elements in a divisibility order



For $A = \{2, 3, 4, 6\}$, the three edges are exactly the covers. The minimal elements are 2 and 3; the maximal elements are 4 and 6. The two minimal elements and the two maximal elements are respectively incomparable, so there is neither a minimum nor a maximum.

Finite extrema and a unique maximal element

Every finite nonempty poset has a maximal element. Start anywhere and move strictly upward whenever possible. No vertex repeats, since that would imply x≺x; after at most |A| −1 moves the process stops. Descending similarly finds a minimal element.

Greatest elements are unique: candidates u,v satisfy u⩽v and v⩽u, hence u=v. The least case is dual. If a finite poset has a unique maximal m, ascent from any x ends at a maximal, necessarily m. Thus x⩽m for every x and m is greatest.

Finiteness matters. On $\mathbb{N} \sqcup \{z\}$, give naturals their usual order and compare z only with itself. Naturals always have larger elements; z is uniquely maximal but not greatest. The empty poset has no extrema.

Bounds may lie outside the subset

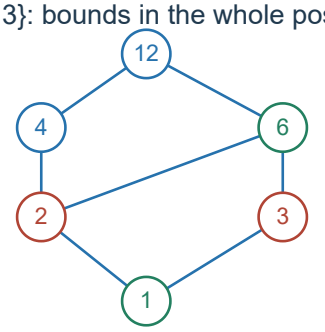
In a poset A, a lower bound l of $S \subseteq A$ satisfies $l \preceq s$ for every $s \in S$; an upper bound u satisfies $s \preceq u$. Bounds are sought in all of A, not necessarily in S.

The greatest lower bound is the maximum among lower bounds; the least upper bound is the minimum among upper bounds. Antisymmetry makes each unique when it exists.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Common bounds among the divisors of 12

S={2,3}: bounds in the whole poset; glb=1, lub=6



Upward means divides; only covers are drawn.

S: red 2,3 (incomparable)

Common lower bounds: {1}; glb=1

Common upper bounds: {6,12}; lub=6

6 ≤ 12; not every upper bound is the lub.

For $S = \{2, 3\}$, the lower bounds are $\{1\}$ and upper bounds are $\{6, 12\}$. Thus glb = 1 and lub = 6, both outside S; S itself has neither a minimum nor a maximum.

In the divisibility order of all positive integers, meet is gcd and join is lcm. Deletion can destroy bounds: in $\{2, 3, 6, 12\}$, 2 and 3 have no common lower bound.

A lattice's universal properties

A nonempty poset is a lattice when every pair has meet a∧b and join a∨b. Common bounds are independent of the pair's order, proving commutativity.

Common lower bounds of a,a∨b are exactly lower bounds of a: one direction requires being below a; the other uses x⩽a⩽a∨b. Hence $a \wedge (a \vee b) = a$. Reversing the order gives dual absorption. Both bracketings of a three-element meet are the glb of {a,b,c}; uniqueness proves associativity, and likewise for joins.

Repeated meet/join over a finite lattice yields bottom/top. Order reversal interchanges upper/lower, minimal/maximal, join/meet. Top and bottom alone need not give a lattice; a lattice need not be distributive. Chapter 15 adds distribution and complements, so Boolean conclusions do not apply to arbitrary posets.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Topological sorting with proof and trace

A finite strict order is a DAG: a cycle would imply $a \prec a$. Conversely, reflexive reachability in a DAG is a partial order because distinct mutually reachable vertices would form a cycle.

Repeatedly output and remove a currently minimal element. Finite descent guarantees one exists. If originally $a \prec b$, then b cannot be minimal until a is removed, so the resulting linear extension preserves every comparison.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Topological sorting the divisors of 12

At each step choose the numerically smallest current minimum:

Already output	Current minima	Choice
None	{1}	1
1	{2,3}	2
1,2	{3,4}	3
1,2,3	{4,6}	4
1,2,3,4	{6}	6
1,2,3,4,6	{12}	12

Cover adjacency lists and indegrees allow initialization and processing each edge once in $O(n+m)$ time. This excludes finding covers from a full relation matrix.

Linear extensions and Catalan words

For an integer $n \geq 1$ and distinct primes p,q, divisors of $p^{n-1}q$ form chains $A_i = p^i$, $B_i = p^i q$, 0≤i<n. Each chain increases with i; across chains, $A_i \preceq B_j$ iff $i \leq j$, and no B lies below an A.

Record an extension as n A's and n B's. Within each chain, occurrence order determines indices uniquely. Legality is equivalent to every prefix having at least as many A's as B's. Before the (j+1)st B, A_j must appear; that also ensures all A_i for $i \leq j$ have appeared. The reflection bijection of Chapter 1 gives

$$C_n = \binom{2n}{n} - \binom{2n}{n-1} = \frac{1}{n+1}\binom{2n}{n}.$$

For n=3, the five words are AAABBB,AABABB,AABBAB,ABAABB,ABABAB. Replace successive A's by 1,2,4 and B's by 3,6,12 to recover all five extensions of the divisor poset of 12.

Equivalence relations and classes

A reflexive, symmetric, transitive relation is an equivalence relation. Define $[a] = \{x : x \sim a\}$. Classes are equal or disjoint and partition A.

If x∈[a]∩[b], symmetry/transitivity give a~b. Any y∈[a] then satisfies y~b, proving [a]⊆[b]; reversing roles proves equality. A representative may label a class arbitrarily, but the class itself is a fixed set.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Integers modulo five

Define a~b iff a−b=5k for an integer k. Reflexivity uses k=0; symmetry negates k; transitivity adds two multiples of five. The classes are

$$[r] = \{5k + r : k \in \mathbb{Z}\}, \qquad r = 0, 1, 2, 3, 4.$$

They partition the integers. The quotient is $\mathbb{Z}/5\mathbb{Z}$. Chapter 14 adds ring operations, whose well-definedness must be proved before asserting that additional algebraic structure.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Equivalence and partition are inverse constructions

Classes cover A since a∈[a]. If x∈[a]∩[b], a~x~b gives a~b. Every y~a then satisfies y~b, proving inclusion and, by reversal, equality. Thus a~b iff [a]=[b], and distinct classes are disjoint.

From a partition, relate elements in the same block. Each element's unique block proves reflexivity; symmetry is immediate. If a,b∈B and b,c∈C, the common b forces B=C, proving transitivity. The block containing a is precisely [a].

Taking classes and then the same-block relation recovers the original relation. Taking the relation and then classes recovers the original blocks. These are inverse bijections. The empty partition has no blocks, including no empty block, and corresponds to the empty relation on the empty set.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Representative independence on quotients

For f:A→B define $a \sim_f b$ iff f(a)=f(b). Equality's properties give equivalence; projection π(a)=[a] is onto the quotient. Set $\bar{f} : A/\!\sim_f \to f(A)$, $\bar{f}([a]) = f(a)$. Changing representatives leaves the value unchanged, so it is well-defined. Equal values imply $a \sim_f b$ and equal classes, proving injection; every image has a preimage, proving surjection. Therefore

$$A/\!\sim_f \longleftrightarrow f(A), \qquad f = \bar{f} \circ \pi.$$

Generally g factors as $\bar{g} \circ \pi$ iff it is constant on every class. Equal projection values prove necessity; defining $\bar{g}([a]) = g(a)$ proves sufficiency. Surjectivity of π makes the factor unique.

Modulo five, [a]↦a mod 5 is well-defined; [a]↦a^2∈Z is not, since [1]=[6] but 1≠36. Quotient operations require the same representative check.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Class sizes, counts, and refinement

For class sizes $s_1, \dots, s_k$, $R = \bigcup_i (B_i \times B_i)$ is a disjoint union. Thus $|R| = \sum_i s_i^2$, with $\sum_i s_i = n$, giving $n \le |R| \le n^2$ and $|R| \equiv n \pmod 2$. Range and parity are only necessary conditions.

S(n,k) counts partitions into k unlabeled nonempty blocks. A new element either forms a singleton or joins one of k old blocks, giving

$$S(n,k) = S(n-1,k-1) + kS(n-1,k), \qquad S(0,0) = 1.$$

Out-of-range values are zero. The Bell number $\sum_k S(n,k)$ counts equivalence relations.

Write $P \preccurlyeq Q$ when every P-block lies in a Q-block. Inclusion gives reflexivity/transitivity. Under mutual refinement, $B \in P$ lies in $C \in Q$, itself in $D \in P$. Nonempty disjoint blocks force $B = D$, then $B = C$; hence $P = Q$. This order is equivalent to $R_P \subseteq R_Q$.

DFA length invariant and recurrence

For a complete DFA $(Q, \Sigma, \delta, q_0, F)$, define

$$p \equiv_k q \iff \forall |w| \le k : [\delta^*(p,w) \in F \iff \delta^*(q,w) \in F].$$

Empty input gives $P_0 = \{F, Q \setminus F\}$, omitting empty blocks. A nonempty word is av, with $|v| \le k$, so

$$p \equiv_{k+1} q \iff (p \equiv_0 q) \wedge \bigwedge_{a \in \Sigma} [\delta(p,a) \equiv_k \delta(q,a)].$$

Partitions only get finer. Signatures may include the current block label and each input's successor-block label; retaining the current label does not over-refine because the new relation is contained in the old.

At a fixed point $P_{k+1} = P_k$, same-block states have same-block successors under every input. Induction on suffix length proves all-word agreement: empty input agrees, and av reaches equivalent successors before applying the induction hypothesis to v. Conversely behaviorally equivalent states never split at any finite round. The stable partition is exactly behavioral equivalence.

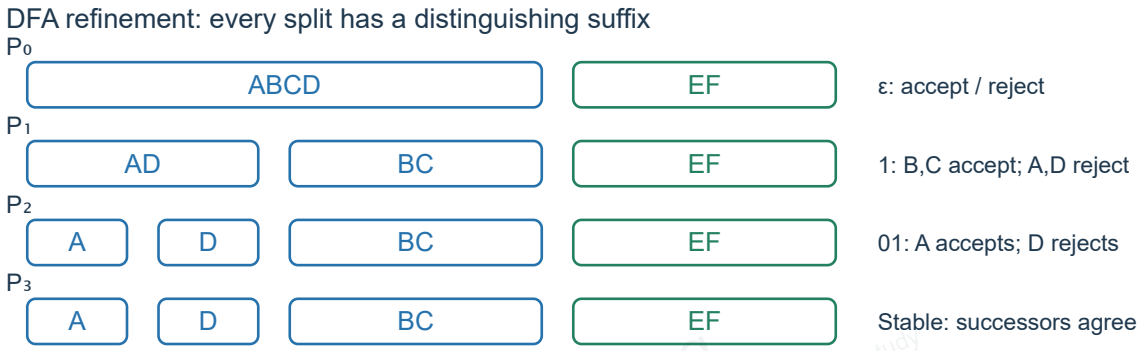
A complete six-state refinement trace

Start at A, accept $\{E, F\}$, and use alphabet $\{0, 1\}$:

State	0	1	Accepts empty input?
A	B	C	No
B	D	E	No
C	D	F	No
D	D	D	No
E	E	E	Yes
F	E	F	Yes

The words $\varepsilon, 0, 1, 00, 01, 11$ reach A, B, C, D, E, F respectively, so all six states are reachable.

Each DFA refinement round



P_1 splits ABCD into AD and BC according to whether input 1 leads to an accepting state. P_2 splits A and D because input 0 reaches different P_1 blocks; $P_3 = P_2$. Stability requires every state in each block to transition into one block for every input.

Quotient DFA and the lower bound

Stable blocks U={A}, V={B,C}, D={D}, T={E,F} give representative-independent $\delta'([p], a) = [\delta(p, a)]$.

Block	0	1	Accepting?
U	V	V	No
V	D	T	No
D	D	D	No
T	T	T	Yes

Empty suffix distinguishes T from each other block; 1 distinguishes U,V and V,D; 01 distinguishes U,D. If v distinguishes successors under a, av distinguishes original states. First-split rounds therefore recover witnesses backward.

Choose an access prefix for each reachable class. If another same-language DFA sends prefixes x,y for two distinct classes to one state, every common suffix has the same acceptance there. A witness w makes exactly one of xw,yw accepted in the original, contradiction. Every equivalent DFA needs at least this many states; the quotient attains the bound.

Refinement cost and unreachable states

Each genuine refinement adds a block, so at most n−|P_0| occur. Signatures cost O(n|Σ|) per round, giving O(n^2|Σ|) with array labels and linear grouping; comparison sorting adds its own cost.

For minimization from a specified initial state, first remove unreachable states by graph search. Otherwise classes describe every state's behavior, including classes the initial state never reaches. For a partial DFA, add one common rejecting sink and complete all missing transitions first. Missing edges cannot alternate between rejection and ignoring input.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Mealy machines have no accepting partition

A Mealy machine $(Q, \Sigma, \Gamma, \delta, \lambda)$ emits λ(q,a) on input a, then moves to δ(q,a). Its entire output satisfies

$$\lambda^*(q, \epsilon) = \epsilon, \quad \lambda^*(q, av) = \lambda(q, a)\lambda^*(\delta(q, a), v).$$

Equivalence compares whole outputs for every word and has no accepting set. Let $p \approx_k q$ mean agreement for lengths ≤k. P_0 is one block Q, and

$$p \approx_{k+1} q \iff \forall a : [\lambda(p, a) = \lambda(q, a) \wedge \delta(p, a) \approx_k \delta(q, a)].$$

Compare outputs for every input, then successor blocks. DFA P_0 compares empty-word acceptance; Mealy P_1 first compares one-symbol outputs. Do not mix these indices.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Mealy merging and output witnesses

Each cell gives next state/output; the initial state is p:

State	0	1
p	q/0	r/0
q	q/0	s/1
r	q/0	s/1
s	s/1	s/1

$P_0 = \{pqrs\}$. Output vectors (0,0), (0,1), (1,1) give $P_1 = \{p\}/\{q, r\}/\{s\}$. The respective successor signatures are (qr,qr), (qr,s), (s,s), so $P_2 = P_1$ and q,r merge.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Distinguishing words and Mealy minimality

Input 1 distinguishes p,q with outputs 0,1; input 0 distinguishes q,s with outputs 0,1. On input 01, p emits 01 and s emits 11.

If t has p's output vector (0,0) but moves to s on input 0, then 00 distinguishes p,t with outputs 00,01. Induction on word length using the output recurrence proves stability correctness.

For minimality, use reachable access prefixes and a distinguishing suffix: different future-output behaviors cannot share a state in an equivalent machine. Compare future outputs after the prefix without mixing in already emitted output.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A seven-state Mealy machine: outputs before successors

Each cell is “next state / output”. Before choosing an initial state, all seven states belong to $P_0 = \{Q\}$ because the empty input produces the empty word.

State	Input 0	Input 1	One-step output vector (0,1)
1	5 / 1	7 / 0	(1,0)
2	7 / 1	2 / 0	(1,0)
3	6 / 1	1 / 0	(1,0)
4	3 / 0	4 / 0	(0,0)
5	3 / 0	5 / 0	(0,0)
6	2 / 1	7 / 0	(1,0)
7	4 / 1	1 / 0	(1,0)

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Refinement by input length

P1 · Outputs for words of length ≤1
{1,2,3,6,7} | {4,5}

P2 · Outputs for words of length ≤2
{1,7} | {2,3,6} | {4,5}

P3 · Outputs for words of length ≤3
{1,7} | {2} | {3,6} | {4,5}

P4 · Outputs for words of length ≤4
{1,7} | {2} | {3} | {6} | {4,5}

States in the same block of P_k give identical output words for every input of length at most k. At $P_4 = P_5$, successors remain in the same blocks. Induction on input length then gives equal output for every finite input.

How successor signatures split blocks

Within each old block compare $(\lambda(q,0),[\delta(q,0)]),(\lambda(q,1),[\delta(q,1)])$. The table omits the identical one-step outputs within each compared block and lists **old successor blocks** in input order 0,1:

Round	Old blocks	States	Successor blocks (0,1)
$P_1 \to P_2$	X={1,2,3,6,7}, Y={4,5}	1,7; 2,3,6	(Y,X); (X,X)
$P_2 \to P_3$	A={1,7}, B={2,3,6}, C={4,5}	2; 3,6	(A,B); (B,A)
$P_3 \to P_4$	A={1,7}, B={2}, C={3,6}, D={4,5}	3; 6	(C,A); (B,A)

Compare signatures **within the same old block**; blocks already separated by output never merge. States within each other block have equal signatures and do not split.

A shortest distinguishing word and the quotient

From states 3 and 6, read the same input 0000:

Initial state	Four-step state path	Output
3	3→6→2→7→4	1111
6	6→2→7→4→3	1110

The states share a block in P_3, so no input of length at most 3 distinguishes them. The word 0000 has length 4 and yields different outputs; it is **a shortest distinguishing word**, not necessarily the only one.

The stable blocks are A={1,7}, B={2}, C={3}, D={6}, E={4,5}. To minimize the machine for a **specified initial state**, remove states unreachable from it and then quotient the reachable machine. These five blocks classify all original states; without an initial state they do not establish that all five are reachable in the minimized machine.

Quotient transitions and outputs

Use the stable blocks A={1,7}, B={2}, C={3}, D={6}, E={4,5} as states. Each cell is “next block / output”:

Block	Input 0	Input 1
A	E / 1	A / 0
B	A / 1	B / 0
C	D / 1	A / 0
D	B / 1	A / 0
E	C / 0	E / 0

For example, on input 0, representatives 1 and 7 of A go to 5 and 4, both in E, and both output 1; on input 1 they both go to A and output 0. The same check works for each block and input: stability makes the quotient independent of the representative.

When can Mealy states merge?

Check equal outputs for every input and successor membership in the same stable blocks. Induction on input length then gives equal future output streams. A split can be traced through successors to a distinguishing word. Once an initial state is specified, remove unreachable states before taking the quotient.

References and attribution

Lin Cing, handwritten discrete mathematics notes. Ralph P. Grimaldi, Discrete and Combinatorial Mathematics, 5th ed., Ch. 7.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Fix the universe and properties

Let finite U have size N, with properties $A_1, \ldots, A_n \subseteq U$. An intersection over J means satisfying at least all properties in J; properties outside J may also hold. Define $S_0 = N$ and

$$S_k = \sum_{\substack{J \subseteq [n] \\ |J|=k}} \left| \bigcap_{i \in J} A_i \right|.$$

This sums intersection sizes, rather than measuring their union. An object satisfying c properties appears $\binom{c}{k}$ times in S_k. This multiplicity explains alternating signs, exact-hit formulas, and truncation errors.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

General proof by indicators

Let I_i(x) indicate membership in A_i. The indicator of no hit is $\prod_i (1 - I_i(x))$. Expand and sum over U:

$$\left| U \setminus \bigcup_i A_i \right| = \sum_{J \subseteq [n]} (-1)^{|J|} \left| \bigcap_{i \in J} A_i \right| = \sum_{k=0}^{n} (-1)^k S_k.$$

The empty intersection is U, supplying N. Finite sums commute and each indicator product counts its intersection. Neither independence nor equal intersection sizes is required. Subtracting from N gives union size $S_1 - S_2 + S_3 - \ldots$. At n=0, the union is empty and the no-hit count N, consistent with empty product one.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why each union member remains once

For c≥1 hits, net multiplicity is

$$\sum_{k=1}^{c}(-1)^{k+1}\binom{c}{k} = 1-(1-1)^{c} = 1.$$

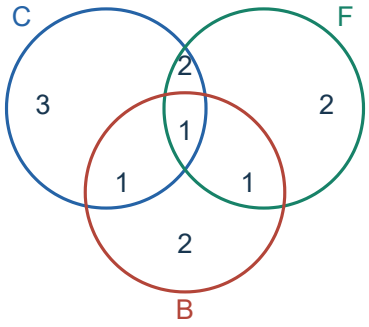
Objects with c=0 enter no positive-order intersection and contribute zero. Four hits contribute 4−6+4−1=1; stopping after pair intersections leaves −2. Multiple overlaps require the full accounting. Individual set sizes alone do not determine intersections; inclusion–exclusion organizes known data, rather than supplying missing data.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Every region for three defects

Among 20 nodes, defects C,F,B have sizes 7,6,5; pair intersections 3,2,2; triple intersection 1. Pair intersections include triple members.

Numbers label disjoint regions, not entire intersections



No hits: 8

N=20 ; |C|=7 · |F|=6 · |B|=5

|C∩F|=3 · |C∩B|=|F∩B|=2

S_1=18 · S_2=7 · S_3=1

At least one: 18−7+1=12

No hits: 20−12=8

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Counting each disjoint region

Exactly two defects give 2,1,1. Only C gives 7−3−2+1=3; only F gives 2 and only B gives 2. Interior regions total 3+2+2+2+1+1+1=12, leaving 8 compliant nodes. Equivalently $S_1=18, S_2=7, S_3=1$ and 20−18+7−1=8. A negative disjoint region would expose inconsistent input intersections, even if an alternating arithmetic expression could still be evaluated.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Truncated alternating binomial sums

For integers c≥1,r≥0, with out-of-range binomials zero,

$$\sum_{k=0}^{r}(-1)^k\binom{c}{k}=(-1)^r\binom{c-1}{r}.$$

Substitute Pascal's identity $\binom{c}{k}=\binom{c-1}{k}+\binom{c-1}{k-1}$. Shift the second sum by one; internal terms cancel, leaving the final term. For r≥c both sides vanish.

Thus an object with c≥1 hits contributes $1-(-1)^r\binom{c-1}{r}$ to the union sum through order r. It is at least one for odd r and at most one for even r; outside objects always contribute zero. This proves the direction rather than guessing from signs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bonferroni bounds

For $T_r = \sum_{k=1}^{r} (-1)^{k+1} S_k$, 1≤r≤n, summing the pointwise inequalities gives

$$r \text{ odd}: \quad |\cup A_i| \le T_r, \qquad r \text{ even}: \quad |\cup A_i| \ge T_r.$$

No-hit bounds reverse on subtracting from N. These bounds require consistent intersection data through order r and do not require independence. In the defect example, T_1=18 is an upper bound and T_2=11 a lower bound. The union lies in [11,18], compliance in [2,9]. Order three gives exact values 12,8. Truncation is already exact when every higher-order intersection is empty.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Probability and weights use linearity

Taking expectations of the same finite indicator identity yields

$$P(\cup_i A_i) = \sum_{\varnothing \ne J \subseteq [n]} (-1)^{|J|+1} P(\cap_{i \in J} A_i).$$

Intersection probabilities need not factor. Mutual independence permits that substitution; pairwise independence alone does not give all higher-order products.

Replace cardinality by $\sum_{x \in A} w(x)$. For nonnegative weights the identity and Bonferroni directions survive by multiplying pointwise inequalities and summing. Signed weights retain the algebraic identity but need not retain inequality directions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Divisibility intersections use lcm

Let M be a nonnegative integer and a, b, d be positive integers. In 1,…,M, let A_d contain multiples of d. Then $A_a \cap A_b = A_{\operatorname{lcm}(a,b)}$, of size $\lfloor M / \operatorname{lcm}(a,b) \rfloor$. Only coprime divisors permit replacing lcm by product.

For M=40 avoiding 2,3,5, single counts are 20,13,8; pair counts 6,4,2; triple 1. Thus

$$40 - (20 + 13 + 8) + (6 + 4 + 2) - 1 = 10.$$

The numbers are 1,7,11,13,17,19,23,29,31,37. Multiples of both 4 and 6 are multiples of 12, not 24. Alternating signs cannot correct a wrongly modeled intersection.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Euler's totient product from inclusion–exclusion

For a positive integer m, the totient φ(m) counts integers in 1,…,m coprime to m. Let distinct prime divisors be p_1,…,p_s. Noncoprimality is equivalent to divisibility by at least one of them, since any common divisor above one has a prime divisor. Every subset product d divides m, so its intersection has exactly m/d members, without floor error. Expanding gives

$$\varphi(m) = m \prod_{p \mid m} (1 - 1/p).$$

For m=30, 30(1/2)(2/3)(4/5)=8, with 1,7,11,13,17,19,23,29. At m=1 the empty product gives φ(1)=1. For a general upper endpoint M, retain floors; this product cannot be applied unconditionally.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bounded nonnegative integer solutions

For m≥1, N,b_i nonnegative integers, count $\sum_i x_i = N, 0 \le x_i \le b_i$. Without upper bounds there are $\binom{N+m-1}{m-1}$ solutions. If every i∈J violates its bound, subtract b_i+1 from x_i. This is a bijection with nonnegative solutions of the reduced total, giving

$$\# = \sum_{J \subseteq [m]} (-1)^{|J|} \binom{N - \sum_{i \in J}(b_i + 1) + m - 1}{m - 1}.$$

A negative remaining total means zero solutions; do not interpret it through generalized negative-upper binomials. For $x_1+x_2+x_3=6$ with bounds 2,3,4, the unrestricted count is 28. Single violations leave totals 3,2,1, giving 10,6,3. Every double violation needs more than six. Thus 28−10−6−3=9.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Surjections exclude missed outputs

For nonnegative integers m and n, there are n^m functions from m points to n points. Let A_i mean output i is missed. Missing a specified k outputs leaves (n−k)^m functions, so

$$\operatorname{Surj}(m, n) = \sum_{k=0}^{n} (-1)^k \binom{n}{k} (n - k)^m = n! S(m, n).$$

The partition expression counts nonempty fibers, and the alternating expression follows from the general proof. For m<n the count is zero; m=n gives n!. For m=0 the unique empty function is surjective only when n=0: the count is 0 for n>0 and 1 for m=n=0. The counting convention $0^0 = 1$ includes the empty function in the formula. Five labeled inputs onto three named outputs give 243−3 • 32+3 • 1−0=150. Exactly two outputs instead requires choosing them first: $\binom{3}{2}(2^5 - 2) = 90$. Deleting a term of the onto-three formula does not change its meaning correctly.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Circular arrangements require valid blocks

For m≥2 disjoint specified pairs among 2m people, identify rotations but distinguish reflections. If k specified pairs are adjacent, contract each pair into a labeled block. There remain 2m−k distinct objects around the circle and two orders inside each block, giving $2^k(2m-k-1)!$. Hence no specified pair adjacent gives

$$\sum_{k=0}^{m}(-1)^k\binom{m}{k}2^k(2m-k-1)!.$$

At m=4 the terms are 5040−5760+2880−768+96=1488. The condition 2m≥4 excludes the two-person circular degeneracy in which two orientations represent one rotation class. If bad pairs share people, independent two-person contractions are invalid.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

No isolated vertices need not mean connected

There are $2^{\binom{n}{2}}$ labeled undirected simple graphs. If specified k vertices must be isolated, only edges among the other n−k vertices remain free, giving $2^{\binom{n-k}{2}}$. Thus the count without isolated vertices is

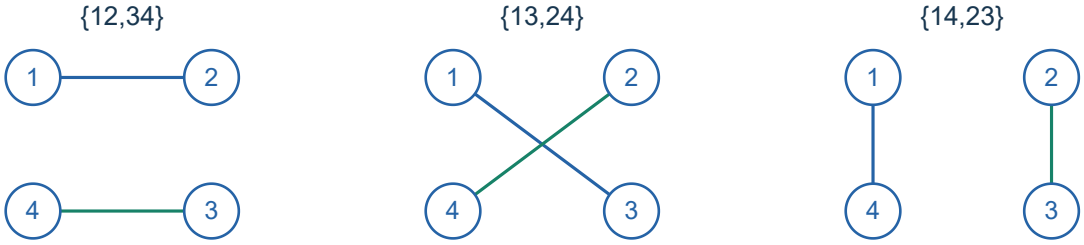
$$\sum_{k=0}^{n}(-1)^k\binom{n}{k}2^{\binom{n-k}{2}}.$$

For n=4 this is 64−32+12−4+1=41. Three are disconnected perfect matchings.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The four-vertex example

Four labeled vertices: exactly three disconnected graphs without isolates



All degrees are 1; crossings are not vertices. Connected graphs: 41−3=38.

Every disconnected no-isolate graph on four vertices has two components of size two, each with its unique edge. These three exhaust the cases, so connected graphs number 41−3=38. Inclusion–exclusion answers the stated bad properties, not a substituted graph property.

Intersection sums and exact-hit counts

Let E_t count objects satisfying exactly t properties. An object with c hits contributes $\binom{c}{k}$ to S_k, so $S_k = \sum_{c=k}^{n} \binom{c}{k} E_c$. Define the finite polynomial $G(z) = \sum_{x \in U} z^{c(x)} = \sum_t E_t z^t$. Expand $z^{c(x)} = [1 + (z-1)]^{c(x)}$ and interchange finite sums:

$$G(z) = \sum_{k=0}^{n} S_k (z-1)^k.$$

This polynomial identity has no convergence condition. At z=1 it gives N; at z=0 the no-hit count. Coefficient comparison gives inversion.

Exact-hit inversion with a pointwise proof

The coefficient of z^t gives

$$E_t = \sum_{k=t}^{n} (-1)^{k-t} \binom{k}{t} S_k.$$

An object with c<t contributes zero, and one with c=t contributes one. For c>t,

$$\sum_{k=t}^{c} (-1)^{k-t} \binom{k}{t} \binom{c}{k} = \binom{c}{t} \sum_{j=0}^{c-t} (-1)^j \binom{c-t}{j} = 0.$$

Here $\binom{k}{t}\binom{c}{k} = \binom{c}{t}\binom{c-t}{k-t}$: both choose t distinguished elements and k−t extra ones. Thus precisely the target objects remain. The cases t=0 and t=n give inclusion–exclusion and E_n=S_n respectively.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

At-least-hit coefficients differ

For 1≤t≤n, let $H_t = \sum_{j=t}^{n} E_j$. Substituting inversion and swapping sums gives the coefficient of S_k as

$$\sum_{j=t}^{k} (-1)^{k-j} \binom{k}{j} = (-1)^{k-t} \binom{k-1}{t-1},$$

by the truncated binomial identity with index k−j. Therefore

$$H_t = \sum_{k=t}^{n} (-1)^{k-t} \binom{k-1}{t-1} S_k.$$

At t=1 this is the union formula. At least zero properties means N directly; do not substitute t=0 into a lower index −1. Exact and at-least formulas cannot be interchanged by merely dropping terms.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Recovering the full three-property distribution

For N=20,S_1=18,S_2=7,S_3=1,

$$E_3 = 1, \quad E_2 = 7 - 3 = 4, \quad E_1 = 18 - 2 \cdot 7 + 3 \cdot 1 = 7, \quad E_0 = 8.$$

The total 8+7+4+1=20. Reconstruct S_1=7+2 • 4+3 • 1=18 and S_2=4+3 • 1=7. At least two gives H_2=S_2−2S_3=5=E_2+E_3. No hits is the union complement, size 8; the triple-intersection complement has size 19 and answers another question.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Derangements and their formula

A permutation π:[n]→[n] is a bijection. It is a derangement if every π(i)≠i; write its count D_n. Let A_i mean i is fixed. Specifying k fixed points leaves (n−k)! permutations, so

$$D_n = \sum_{k=0}^{n} (-1)^k \binom{n}{k} (n-k)! = n! \sum_{k=0}^{n} \frac{(-1)^k}{k!}.$$

The empty permutation gives D_0=1, while D_1=0. No fixed point differs from merely not being the identity. Swapping 1,2 while fixing the rest satisfies only the latter when fixed points remain.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two reversible classes prove the recurrence

For $n \geq 2$ fix $\pi(1)=j \neq 1$, with $n-1$ choices. If $\pi(j)=1$, delete the two-cycle $(1\ j)$, leaving an $(n-2)$-point derangement.

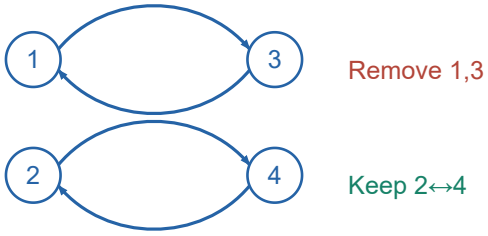
Otherwise let k be the unique predecessor of 1, so $k \neq j$. Contract $k \to 1 \to j$ to $k \to j$ and delete 1. No new fixed point occurs: the only possibility would require $k=j$, already excluded. Conversely, in any derangement of the remaining $n-1$ labels, insert 1 just before j at its unique predecessor. This uniquely restores the second class.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

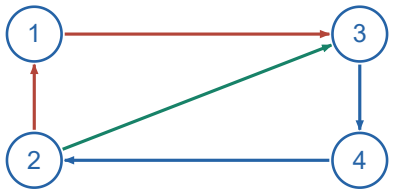
The two derangement cases

Fix $\pi(1)=3$: two-cycle deletion or longer-cycle contraction

Two-cycle: remove (1 3)



Longer cycle: contract 2→1→3 to 2→3



Remove red edges; add green. Inverse: insert 1 before 3.

The classes are disjoint and exhaustive, proving $D_n = (n-1)(D_{n-1} + D_{n-2})$. The chosen j remains the image of 1; do not silently relabel it as another point in the reduced problem.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Integer traces for five-point derangements

$D_0=1, D_1=0$ give $D_2=1, D_3=2, D_4=3(2+1)=9, D_5=4(9+2)=44, D_6=5(44+9)=265$.

k	$\binom{5}{k}(5-k)!$	Sign
0	120	+
1	120	−
2	60	+
3	20	−
4	5	+
5	1	−

Thus $120-120+60-20+5-1=44$, matching the recurrence. A permutation with three fixed points appears in several intersection sums; these rows are not disjoint permutation classes.

Distance from n!/e and nearest integers

The alternating remainder in $e^{-1}=\sum_{k\ge 0}(-1)^k/k!$ gives, for $n\ge 1$,

$$|D_n - n!/e| < 1/(n+1).$$

Even n lies above n!/e, odd n below. For $n\ge 2$ the error is below one half; n=1 uses $1/e<1/2$ directly. Thus

$$D_n = \lfloor n!/e + 1/2 \rfloor \qquad (n \ge 1).$$

At n=0 use $D_0=1$ separately. Uniform-permutation probability $D_n/n!$ tends to e^{-1} rather than equaling it for every finite n. Numeric rounding also needs controlled computational error; the mathematical remainder alone is no floating-point guarantee.

Exactly fixed points and relative derangements

Choose t fixed points, then derange all others, giving $\binom{n}{t} D_{n-t}$. These disjoint exhaustive classes give $n! = \sum_t \binom{n}{t} D_{n-t}$. For n=5, counts at t=0,…,5 are 44,45,20,10,0,1, totaling 120. Exactly n−1 fixed points is impossible: bijectivity forces the last point fixed too.

For two assignment rounds with no person receiving the same item twice, fix any first-round bijection and relabel by it. The second is a relative derangement, giving n!D_n ordered pairs of rounds. It is not D_n^2 because the first round has no original-label restriction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rook polynomials on a specified board

For $B \subseteq [m] \times [n]$, r_k(B) counts k rooks in B using distinct rows and columns; r_0=1 and $R_B(x) = \sum_k r_k(B) x^k$. At most min(m,n) rooks fit.

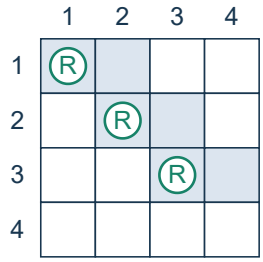
B specifies where rooks are counted. When B later represents forbidden assignment positions, rooks still lie in B to count bad-event intersections; B does not become allowed assignment cells. On a full m×n board, choose k rows, k columns, and a bijection: $r_k = \binom{m}{k}\binom{n}{k}k!$. The full 2×2 board gives $1+4x+2x^2$, not $(1+x)^4$, which would permit attacks.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Every coefficient of the six-cell ladder

In 4×4 take $B=\{(1,1),(1,2),(2,2),(2,3),(3,3),(3,4)\}$.

Rooks on forbidden B: nonattacking cells give compatible bad events



B shaded; R: three rooks

$r_0=1$; $r_1=6$

$r_2=C(6,2)-3-2=10$

$r_3=4$; $r_4=0$

Three-rook column triples

(1,2,3) (1,2,4)

(1,3,4) (2,3,4)

At most one rook per row.

No fourth nonempty row, so four rooks are impossible.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Counting the ladder coefficients

$r_1=6$. Among $\binom{6}{2}=15$ cell pairs, three share rows and two share columns. Distinct cells cannot share both, so $r_2=10$. Three rooks require one in each of the first three rows. Their column triples are exactly (1,2,3),(1,2,4),(1,3,4),(2,3,4), giving $r_3=4$. Row four is empty, so $r_4=0$. Hence $R_B=1+6x+10x^2+4x^3$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Delete a cell or select it

For e=(i,j)∈B, placements either omit e and lie in $B \setminus \{e\}$, or select e and avoid its entire row and column. Call the latter remaining board B_ij. Deleting or restoring e gives reversible disjoint cases:

$$r_k(B) = r_k(B \setminus \{e\}) + r_{k-1}(B_{ij}), \quad R_B = R_{B\setminus\{e\}} + xR_{B_{ij}}.$$

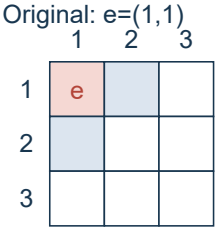
For B={(1,1),(1,2),(2,1)}, choose e=(1,1).

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

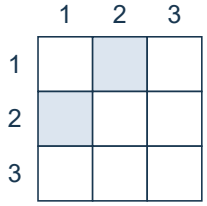
Deletion and selection example

Deletion/contraction: disjoint cases according to whether e is used

Original: e=(1,1)

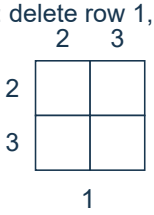


Omit e: delete one cell



1+2x+x²

Select e: delete row 1, column 1



R_B(x) = (1+2x+x²) + x·1 = 1+3x+x²

Omitting e leaves two nonattacking cells with polynomial $(1+x)^2$. Selecting e leaves an empty board, polynomial 1. Thus $R_B=1+2x+x^2+x=1+3x+x^2$. Each branch reduces cell count; empty-board base one ensures termination.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Board products need disjoint rows and columns

If $B=B_1 \cup B_2$ and both row sets and column sets are disjoint, a k-rook placement uniquely splits into i rooks in B_1 and k−i in B_2. Conversely two component placements cannot attack across boards. Hence

$$r_k(B) = \sum_{i=0}^{k} r_i(B_1) r_{k-i}(B_2), \qquad R_B = R_{B_1} R_{B_2}.$$

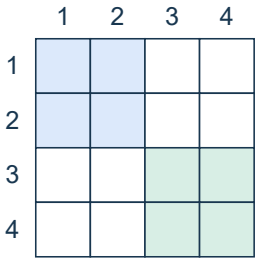
Cell disjointness alone fails. For $B_1=\{(1,1)\}, B_2=\{(1,2)\}$, each gives 1+x, but their product wrongly includes x^2; the union gives 1+2x. Splitting remains unique; combining individually legal placements fails. Row or column relabeling preserves attacks and bijects placements of each size, helping locate genuinely independent blocks.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Full convolution for two 2×2 blocks

Take only the upper-left and lower-right full 2×2 blocks of a 4×4 board. Their rows and columns separate.

Independent rook choices require disjoint rows and columns



Blue B_1: rows 1,2 × columns 1,2

Green B_2: rows 3,4 × columns 3,4

$R_1=R_2=1+4x+2x^2$

$R_B=1+8x+20x^2+16x^3+4x^4$

$r_2=2\cdot1+4\cdot4+1\cdot2=20$

Two rooks: both in B_1, one per block, or both in B_2.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Convolution of the block coefficients

$$R_B = (1 + 4x + 2x^2)^2 = 1 + 8x + 20x^2 + 16x^3 + 4x^4.$$

The x^2 coefficient is 2+4 • 4+2=20, for both rooks in one block or one in each. The x^3 coefficient is 4 • 2+2 • 4=16. Polynomial products count independent choices and require cross terms, not just matching-degree products.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why forbidden-position intersections use rooks

A permutation chooses one cell in each row and column. For each forbidden (i,j)∈B, let its bad event be π(i)=j. If specified k cells share a row or column, their intersection is empty. Otherwise fixed nonattacking cells extend to (n−k)! bijections of remaining rows and columns. There are r_k(B) such cell sets, so

$$N_{\text{avoid}} = \sum_{k=0}^{n} (-1)^k r_k(B)(n-k)!.$$

This counts complete avoidance, not exactly k forbidden hits. A diagonal board gives R_B= (1+x)^n and the derangement formula. An empty board gives n!.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

All ladder-board avoiding permutations

The coefficients 1,6,10,4 at n=4 give

$$24 - 6 \cdot 6 + 10 \cdot 2 - 4 \cdot 1 = 4.$$

The row-ordered permutations are (3,1,2,4),(3,4,1,2),(3,4,2,1),(4,1,2,3). Row one permits only 3,4; row two forbids 2,3; row three forbids 3,4; each result uses every column once. Subtracting bad events independently repeats corrections to the same permutation. Rook coefficients organize jointly possible bad cells; factorials count their full extensions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rectangular injections retain a denominator

Assign m named jobs injectively to n named people, m≤n. Total assignments are (n)_m=n!/(n−m)!. Fixing k nonattacking forbidden cells leaves

$$(n-k)!/(n-m)!$$

injections of the other jobs. Therefore $N_{\text{avoid}} = \sum_{k=0}^{m} (-1)^k r_k(B)(n-k)!/(n-m)!$. Attacking cells still give empty intersections. At m>n there are no injections; return zero without negative factorials. At m=0 the unique empty assignment gives one. At m=n the denominator is 0!=1, recovering the square case.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two jobs and four people

Take B={(1,1),(1,2),(2,2),(2,3)}. Here r_0=1,r_1=4; legal two-rook column pairs by row are (1,2), (1,3),(2,3), so r_2=3.

$$N = 4!/2! - 4 \cdot 3!/2! + 3 \cdot 2!/2! = 12 - 12 + 3 = 3.$$

Job one permits 3,4; job two permits 1,4. Remove the repeated-person choice (4,4), leaving (3,1),(3,4),(4,1). Omitting 2! counts each injection twice by unnecessarily ordering the two unused people.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rook coefficients recover exact-hit distributions

Let h_t count permutations using exactly t cells of B. Since $S_k = r_k(B)(n-k)!$, inversion gives

$$h_t = \sum_{k=t}^{n} (-1)^{k-t} \binom{k}{t} r_k(B)(n-k)!.$$

The ladder has $S_0,\ldots,S_4$=24,36,20,4,0. Thus h_0=4,h_1=36−40+12=8,h_2=20−12=8,h_3=4,h_4=0, totaling 24. Also $S_1=h_1+2h_2+3h_3$=36. The forbidden-cell count r_1=6 differs from the exactly-one-hit permutation count 8; these quantities have different units.

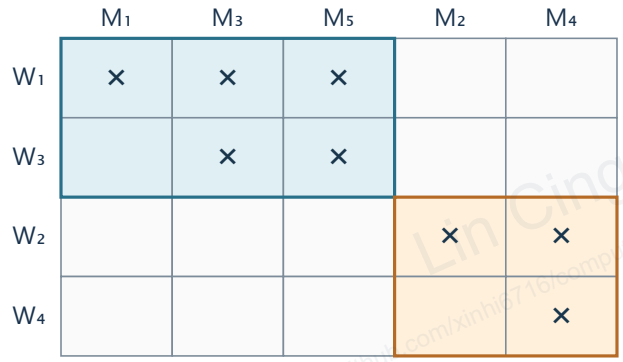
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Restricted matching: split the forbidden board into two blocks

Match each of four people $W_1, \ldots, W_4$ to a different partner among $M_1, \ldots, M_5$. Crosses mark forbidden matches.

Forbidden board: reorder rows and columns into independent blocks

× marks a forbidden match; labels retain their original indices



Block A: 2 rows × 3 columns

$R_a(x) = 1 + 5x + 4x^2$

Block B: 2 rows × 2 columns

$R_\beta(x) = 1 + 3x + x^2$

Restricted matching: coefficients and inclusion–exclusion

The forbidden columns for rows 1–4 are respectively {1,3,5}, {2,4}, {3,5}, {4}. Reordering changes only the display: the blocks share neither rows nor columns, so their rooks cannot attack each other.

Block A has five forbidden cells and four nonattacking two-rook placements; block B has three forbidden cells and one such pair. The product records all ways to choose rooks from both blocks:

$$R(x) = (1 + 5x + 4x^2)(1 + 3x + x^2) = 1 + 8x + 20x^2 + 17x^3 + 4x^4.$$

After fixing k nonattacking forbidden cells, the remaining $4 - k$ rows inject into $5 - k$ columns in $P(5 - k, 4 - k)$ ways. Thus the number of legal assignments is

$$P(5,4) - 8P(4,3) + 20P(3,2) - 17P(2,1) + 4P(1,0)$$
$$= 120 - 192 + 120 - 34 + 4 = \boxed{18}.$$

Here $P(a,b) = a!/(a-b)!$, and $P(1,0) = 1$.

Row-by-row rook dynamic programming

Let $d_i(S)$ count placements in the first i rows using exactly column set S. Initially $d_0(\varnothing) = 1$, others zero. For each row, omit a rook or select one available column j with $(i, j) \in B$:

```
new[...] = 0
for each subset S:
    new[S] += old[S]
    for each j with (i,j) in B and j not in S:
        new[S union {j}] += old[S]
old = new
```

Read only the preceding row's old table; in-place updates could place two rooks in one row.

Why the rook dynamic program works

Every legal placement uniquely determines whether this row has a rook, its column, and the previous placement. This proves completeness without duplication. Finally $r_k = \sum_{|S|=k} d_m(S)$.

There are at most $mn2^n$ additions and $O(2^n)$ counts in two layers. This is arithmetic-operation cost; large-integer bit cost is separate. Transpose to reduce the column dimension when helpful. This is not a polynomial-time claim for large boards.

References and attribution

Lin Cing, handwritten discrete mathematics notes. Ralph P. Grimaldi, Discrete and Combinatorial Mathematics, 5th ed., Ch. 8.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Sequences and formal power series

The ordinary generating function (OGF) of $(a_n)_{n\geq 0}$ is $A(x) = \sum_{n\geq 0} a_n x^n$. With formal indeterminate x, neither numerical substitution nor convergence is required; coefficient comparison remains valid. Numerical real or complex evaluation additionally requires analytic convergence. Finite sequences give polynomials.

For a_n=1, $A(x) = \sum_{n\geq 0} x^n = (1-x)^{-1}$, meaning all coefficients agree. Formal addition and multiplication are defined algebraically. Division requires an invertible constant coefficient in the denominator, equivalent to a nonzero constant over a field.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Which formal operations are legitimate?

A series is a coefficient sequence. Equality means equality at every degree. Thus $1/(1-x) = 1 + x + x^2 + \cdots$ first means $(1-x)(1+x+x^2+\cdots) = 1$, rather than a numerical limit. Convolution uses only finitely many terms at each fixed degree.

Assume coefficients belong to a field. For $A(x) = \sum a_n x^n$ with $a_0 \neq 0$, its inverse $B(x) = \sum b_n x^n$ is determined by

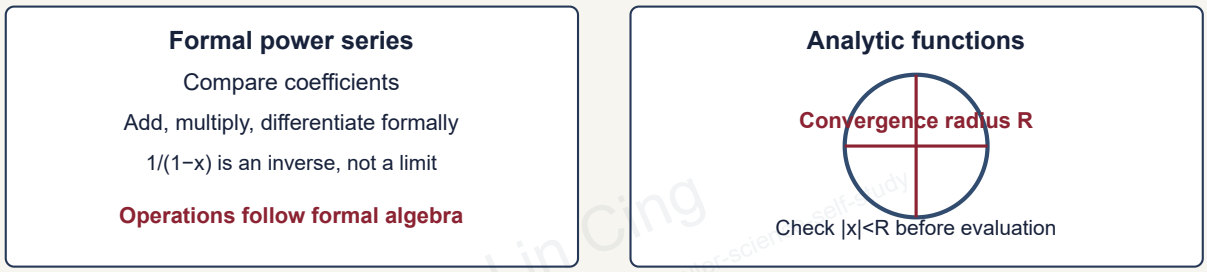
$$b_0 = 1/a_0, \qquad a_0 b_n + \cdots + a_n b_0 = 0, \qquad b_n = -\frac{1}{a_0}\sum_{k=1}^{n} a_k b_{n-k} \quad (n \geq 1).$$

Only earlier b coefficients occur, and the fixed nonzero denominator makes this recursion valid for every n.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Which formal operations are legitimate?: derivation and application

An infinite product is legitimate when only finitely many factors affect each fixed coefficient. In $\prod_{k\geq 1}(1-x^k)^{-1}$, factors k>n equal one through degree n, satisfying this condition.



Coefficient identities are formal; numerical values need analytic conditions

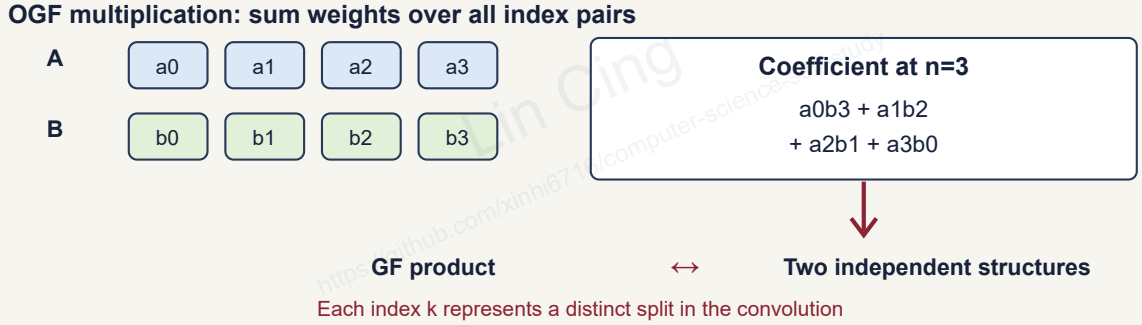
Numerically the geometric series converges for |x|<1. That condition is additional to, rather than a premise of, the formal coefficient identity.

Operations and convolution

For A and B with coefficients a_n,b_n,

$$A(x)B(x)=\sum_{n\geq 0}\left(\sum_{k=0}^{n}a_k b_{n-k}\right)x^n.$$

Multiplying by x^j shifts indices upward by j with j leading zeros; scalar multiplication scales coefficients. Convolution models a split into two independent smaller objects, summed over every possible split size.



Operations and convolution: derivation and application

If objects have additive weight n and count a_n, define $A(x) = \sum a_n x^n$. Coefficients recover the counts once a counting rule or bijection is established. The product rule requires a unique decomposition into an ordered pair (u,v), with additive weights. Nonunique decompositions repeat objects. A partition factor $1/(1-x^k)$ records multiplicity of a fixed part size, so 3+1 and 1+3 do not differ; ordered compositions require a different model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Product models and coin combinations

For objects of unit weight, an allowed multiplicity set S gives factor $\sum_{r \in S} x^r$. If each object of this type has positive integer weight k, the factor is $\sum_{r \in S} x^{kr}$. Independent choices across types multiply; the degree-n coefficient counts total weight n. Coefficients are nonnegative integers counting permitted choices.

Unlimited coins of values 1,2,3 give $1/((1-x)(1-x^2)(1-x^3))$. Identical coins record quantities, not order. Ordered coin sequences instead use $D(x)=x+x^2+x^3$; length k gives $D(x)^k$ and all lengths, including empty, give $1/(1-x-x^2-x^3)$. At total three, unordered choices are 1+1+1,1+2,3; ordered sequences also include 2+1, giving four. The change is order, not invented labels on identical coins.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bounded choices and negative binomial series

Multiplicity at most m gives $1 + x + \cdots + x^m = (1 - x^{m+1})/(1 - x)$; exactly m gives x^m; a minimum a extracts x^a. Three types each allowed 0,1,2 have count $[x^n](1 + x + x^2)^3$. At n=5 only permutations of (2,2,1) contribute, giving three; unrestricted stars and bars $\binom{7}{2} = 21$ ignores the upper bound.

Four unlimited types give $(1 - x)^{-4} = \sum_{n \geq 0} \binom{n+3}{3} x^n$. Generally for positive integer r,

$$(1 - x)^{-r} = \sum_{n \geq 0} \binom{n + r - 1}{n} x^n.$$

The coefficient counts $y_1 + \ldots + y_r = n$ with nonnegative y_i. Arrange n stars and r−1 bars in $\binom{n+r-1}{r-1} = \binom{n+r-1}{n}$ ways, bijectively encoding all allocations. Requiring each type positive first places one of each, giving factor $x^r/(1 - x)^r$ and coefficient $\binom{n-1}{r-1}$ for $n \geq r$. Unlimited and bounded multiplicities require different factors and sample spaces.

Bounded choices: remove the common baseline

At each of ten ordered positions choose one of $5, 8, 11, 14, 17$; count choices with total 80. Write the value at position i as $5 + 3t_i$, where $t_i \in \{0, 1, 2, 3, 4\}$. Positions are distinguishable, so component order matters.

At each position: start with 5, then add a multiple of 3

At position i choose $t_i \in \{0, 1, 2, 3, 4\}$
The corresponding value $5 + 3t_i \in \{5, 8, 11, 14, 17\}$

Baseline for ten positions: 10 × 5 = 50

Remainder: 80 − 50 = 30 = 3 × 10

Hence $t_1 + t_2 + \cdots + t_{10} = 10$; positions are distinguishable.

The total condition is $50 + 3\sum_{i=1}^{10} t_i = 80$, or $\sum t_i = 10$.

Shift and extract the coefficient

Factor x^5 from each factor and put $y = x^3$:

$$(x^5 + x^8 + x^{11} + x^{14} + x^{17})^{10} = x^{50}(1 + x^3 + x^6 + x^9 + x^{12})^{10}.$$

Thus $[x^{80}]$ extracts the x^{30} coefficient of the parenthesized product, equivalently

$$[x^{80}](\cdots)^{10} = [y^{10}](1 + y + y^2 + y^3 + y^4)^{10}.$$

Choosing y^{t_i} from factor i corresponds bijectively to the choice at position i.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Count the bounded coefficient by inclusion–exclusion

Without upper bounds, $t_1 + \cdots + t_{10} = 10$, $t_i \geq 0$, has $\binom{19}{9} = 92378$ solutions. Let A_i mean $t_i \geq 5$. For one specified violation, subtract 5; the remaining total is 5, giving $|A_i| = \binom{14}{9} = 2002$.

For two specified violations, subtracting 5 from each leaves total 0 and exactly one solution. Three violations require total at least 15, so their intersection is empty. Inclusion–exclusion gives

$$\binom{19}{9} - 10\binom{14}{9} + \binom{10}{2} = 92378 - 20020 + 45 = \boxed{72403}.$$

Each single-violation count includes double violations. The 45 restores those over-subtracted choices.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Partial fractions and coefficient extraction

For example,

$$\frac{1}{(1-x)(1-2x)} = \frac{2}{1-2x} - \frac{1}{1-x} = \sum_{n\geq 0}(2^{n+1}-1)x^n,$$

with first coefficients 1,3,7,15. Preserve proper numerator degrees, all repeated-factor powers, and irreducible quadratic terms. A finite truncation cannot determine omitted coefficients.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Partial fractions and coefficient extraction: derivation and application

For P(x)/Q(x), first divide polynomials if deg P≥deg Q. Factor Q over the reals into linear and irreducible quadratic factors. A repeated factor $(1-ax)^m$ requires every term $C_1/(1-ax)+\dots+C_m/(1-ax)^m$. An irreducible quadratic normally requires a linear numerator. Extract using

$$[x^n](1-ax)^{-k} = a^n\binom{n+k-1}{k-1}.$$

An irreducible real quadratic may still simplify:

$$\frac{1}{1+x+x^2} = \frac{1-x}{1-x^3}, \qquad (c_0,c_1,c_2,\dots) = (1,-1,0,1,-1,0,\dots).$$

The numerator 1−x is essential. Expanding $1-(x+x^2)+(x+x^2)^2-\dots$ also works, but every term contributing to a fixed degree must be retained. Rational partial-fraction constants can combine into integer coefficients when the model requires integers.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Formal differentiation and products

Differentiate $(1-x)^{-1} = \sum_{n\ge 0} x^n$:

$$(1-x)^{-2} = \sum_{n\ge 1} n x^{n-1} = \sum_{m\ge 0} (m+1) x^m.$$

Repeating r−1 times yields $(1-x)^{-r} = \sum_{m\ge 0} \binom{m+r-1}{m} x^m$. Formal differentiation only transforms coefficients and requires careful indices, not convergence. Analytic series and derivatives share a convergence radius but may differ at endpoints. Numerical comparisons require the convergence domain; finite polynomial truncations do not supply the missing coefficients. Low-degree expansion and stars-and-bars counts check the identity independently.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Probability-generating coefficients

For nonnegative integer-valued X, define $P_X(z) = \sum_{k\ge 0} \Pr(X=k) z^k$. Coefficients are point probabilities, not densities. Nonnegative coefficients summing to one give P_X(1)=1.

For equal probabilities on 2,4,6,

$$P_X(z) = (z^2 + z^4 + z^6)/3, \qquad E[X] = P_X'(1) = (2+4+6)/3 = 4.$$

Also

$$P_X''(1) = (2 + 12 + 30)/3 = 44/3 = E[X(X-1)],$$
$$E[X^2] = 44/3 + 4 = 56/3, \qquad \operatorname{Var}(X) = 56/3 - 16 = 8/3.$$

For infinite support, such as geometric distributions, differentiation and evaluation at one require justified limits or convergence and need not give a finite derivative. Formal coefficient identities alone do not establish finite moments; check support, normalization, and convergence.

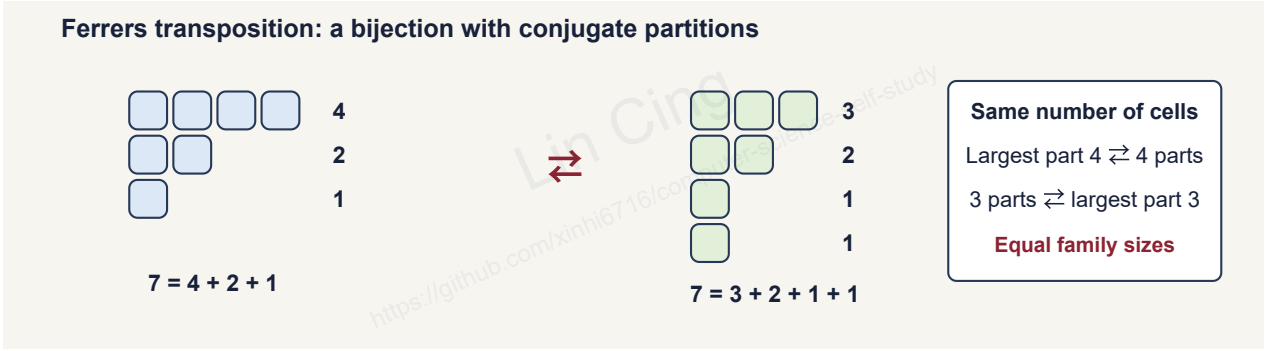
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Integer partitions

The count p(n) of unordered positive-part sums has OGF

$$\prod_{k\geq 1}(1-x^k)^{-1}.$$

Each factor selects a multiplicity of part size k. Thus 3+1=1+3 as partitions. Ordered compositions, including empty, instead give $1/(1-(x+x^2+\cdots))=(1-x)/(1-2x)$; nonempty compositions give x/(1−2x).



At any degree n, only factors k≤n matter, making the infinite product formally valid. Fixed part-size factors remove order by retaining only multiplicity; their sequence in the product is fixed rather than permuting parts.

Bounds on the largest part

Largest part at most m gives $\prod_{k=1}^{m}(1-x^k)^{-1}$. At m=2, choose j twos and n−2j ones, for j=0,…, ⌊n/2⌋, yielding ⌊n/2⌋+1 partitions.

Ferrers transposition exchanges largest part and number of parts. For $\lambda_1\geq\cdots\geq\lambda_\ell>0$, define $\lambda'_j=|\{i:\lambda_i\geq j\}|$. New rows are old column lengths, preserving total cells; $\lambda'_1=\ell$ and the new number of parts is λ_1. A cell (i,j) exists iff $j\leq\lambda_i$ iff $i\leq\lambda'_j$; transposing again restores the original cells. This proves a reversible bijection, so at most m parts and largest part at most m are equinumerous. Distinct parts replace factors by $1+x^k$; at least one of a specified size k uses $x^k/(1-x^k)$. This can be imposed on finitely many specified sizes; no finite partition can contain every positive size.

Odd parts and distinct parts

The product identity

$$\prod_{k\geq 1}(1+x^k)=\prod_{k\geq 1}\frac{1-x^{2k}}{1-x^k}=\prod_{k\geq 1}(1-x^{2k-1})^{-1}$$

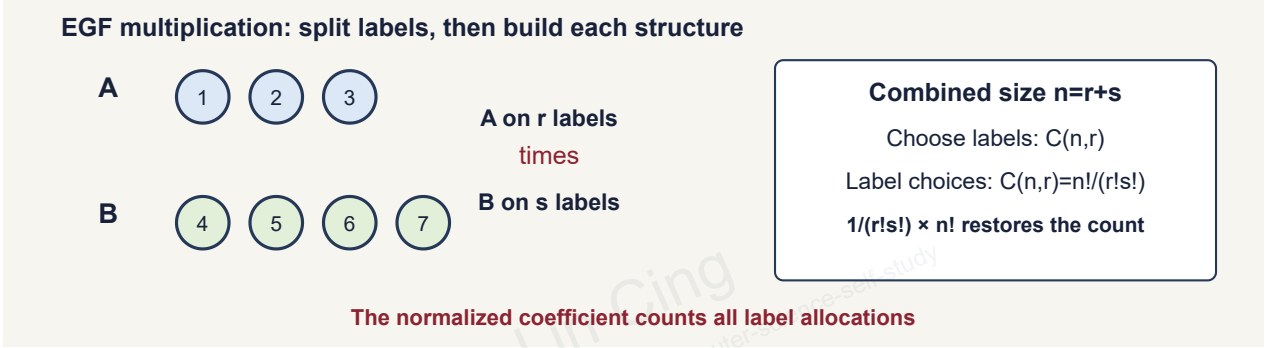
comes from canceling even factors, proving equality of distinct-part and odd-part partition counts.

For an explicit bijection, write each odd part's multiplicity in binary. A nonzero bit 2^j creates one part of size 2^j times that odd part. Unique power-of-two-times-odd decomposition ensures distinct parts and a unique inverse.

For 26=5+5+5+3+3+1+1+1+1+1, multiplicities 3=1+2,2=2,5=1+4 give {5,10},{6},{1,4}, hence 10+6+5+4+1. Reverse 10 to two fives, 6 to two threes, 5 to one five, 4 to four ones, 1 to one one. Odd families cannot collide because their odd factors are unique.

Exponential generating functions

For labeled objects define $\widehat{A}(x)=\sum_{n\geq 0}a_n x^n/n!$. Splitting n=r+s labels into two groups gives $\binom{n}{r}=n!/(r!s!)$, which is exactly the normalization supplied by an EGF product.



If a_r counts A-structures on any r labels and b_s B-structures on s disjoint labels, first choosing the A-label subset then both structures gives $\sum_{r=0}^{n}\binom{n}{r}a_r b_{n-r}$. Algebraically,

$$[x^n](\widehat{A}\widehat{B})=\sum_{r=0}^{n}\frac{a_r b_{n-r}}{r!(n-r)!}=\frac{1}{n!}\sum_{r=0}^{n}\binom{n}{r}a_r b_{n-r}.$$

Multiplication by n! restores the count. The normalization encodes label splitting rather than an arbitrary conversion from an OGF.

Exponential generating functions: derivation and application

Partitions into unlabeled nonempty blocks give $\exp(e^x - 1) = \sum B_n x^n/n!$. Initial Bell numbers are 1,1,2,5. Two labels either share one block or separate; three labels give one full block, three singleton-plus-pair choices, or three singleton blocks, totaling 1+3+1=5.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why SET components must be nonempty

Let $\hat{A}(0)=0$ count nonempty labeled components. Fixed k distinguishable slots have EGF $\hat{A}(x)^k$. Disjoint nonempty label sets make the components distinct objects, so every unordered component set has exactly k! slot orders. Therefore

$$\widehat{C}_k(x) = \widehat{A}(x)^k/k!, \qquad \widehat{C}(x) = \sum_{k\ge 0} \widehat{A}(x)^k/k! = \exp(\widehat{A}(x)).$$

At degree n only $k \le n$ contributes. Empty components would allow arbitrarily many empty blocks and invalidate this interpretation and finite layering.

One nonempty set component has EGF e^x-1. For n=4,k=2, ordered sizes (1,3),(2,2),(3,1) contribute 4,6,4. Division by 2! gives seven, so $4!\,[x^4](e^x-1)^2/2! = 7$. The four-label counts for k=1,2,3,4 are 1,7,6,1, totaling B_4=15.

Choose k companions of a new label in its block; partition all remaining labels. These reversible choices give

$$B_{n+1} = \sum_{k=0}^{n} \binom{n}{k} B_{n-k},$$

also obtained from $B'(x)=e^xB(x)$ by the EGF product rule.

Repeated letters and parity restrictions

With positions as labels and letter i allowed multiplicities J_i,

$$\#\text{words} = n![x^n] \prod_i \left(\sum_{j \in J_i} x^j / j! \right).$$

For fixed multiplicities summing to n, the labeled product assigns positions in $n!/\prod j_i!$ ways, precisely the repeated-letter arrangement count. OGF multiplicity selection would ignore these orders.

With at most two A's, two B's, one C, length three gives

$$3!\,[x^3](1 + x + x^2/2)^2(1 + x) = 6(1 + 2) = 18.$$

Multiplicities (2,1,0),(1,2,0),(2,0,1),(0,2,1) each give three words; (1,1,1) gives six. Length four gives 4!(1/4+1)=30. Identical letters receive no artificial inventory labels.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Repeated letters and parity restrictions: derivation and application

Even A-count, with B,C unrestricted, gives

$$n![x^n] \frac{e^x + e^{-x}}{2} e^{2x} = (3^n + 1)/2.$$

The even filter cancels odd powers. At n=4, A-counts 0,2,4 contribute 16,24,1, totaling 41. Exponentials here are formal series, without numerical substitution.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The summation operator

For $A(x)=\sum a_n x^n$ and $s_n=\sum_{k=0}^{n} a_k$, convolution with all ones gives

$$A(x)(1+x+x^2+\cdots)=\sum_{n\ge 0}\left(\sum_{k=0}^{n} a_k\right)x^n=S(x)=A(x)/(1-x).$$

Thus 1/(1−x) is the sum operator; repeating it gives higher prefix sums. Weighted sums or a starting index of one require separate shifts. No analytic convergence is needed.

For $a_k=2k+1$, prefix sums are $(n+1)^2$, hence $A(x)/(1-x)=\sum_{n\ge 0}(n+1)^2x^n$. Combining summation with $x(d/dx)$ also derives power-sum identities.

For nonnegative u_1,u_2,u_3 with $u_1+2u_2+3u_3\le n$, add the unique nonnegative slack u_4 so $u_1+2u_2+3u_3+u_4=n$. The four factors give

$$\sum_{n\ge 0} A_n x^n=\frac{1}{(1-x)^2(1-x^2)(1-x^3)}=1+2x+4x^2+7x^3+11x^4+16x^5+23x^6+31x^7+\cdots.$$

Thus $A_7=31$, agreeing with direct enumeration of the triples. The extra factor follows from slack, preserving the inequality's exact meaning.

Fibonacci OGF and both closed-form derivations

For $F_0=0,F_1=1$ and $F_n=F_{n-1}+F_{n-2}$, sum shifted recurrences:

$$F(x)-x=xF(x)+x^2F(x),\qquad F(x)=x/(1-x-x^2).$$

The denominator constant is one, so formal division is valid. The first coefficients are 0,1,1,2,3,5. Comparing $(1-x-x^2)F(x)=x$ gives constant $F_0=0$, linear $F_1-F_0=1$, and for $n\ge 2$ exactly $F_n-F_{n-1}-F_{n-2}=0$, verifying every coefficient and initial shift.

Let $\varphi=(1+\sqrt{5})/2,\psi=(1-\sqrt{5})/2$, the roots of $r^2-r-1=0$. Then $1-x-x^2=(1-\varphi x)(1-\psi x)$, $\varphi-\psi=\sqrt{5}$, and

$$F(x)=\frac{1}{\sqrt{5}}\left(\frac{1}{1-\varphi x}-\frac{1}{1-\psi x}\right).$$

Its numerator is $((1-\psi x)-(1-\varphi x))/\sqrt{5}=x$. Extracting coefficients gives $F_n=(\varphi^n-\psi^n)/\sqrt{5}$.

Fibonacci OGF and both closed-form derivations: derivation and application

Independently, the characteristic-root expression $F_n = A\varphi^n + B\psi^n$ has initial equations A+B=0 and Aϕ+Bψ=1, giving A=1/√5,B=−1/√5 and the same formula. Both routes need both initial values; roots alone do not specify the sequence. Formal identities hold for every n; analytic substitution requires |x|<1/ϕ. Chapter 10 applies the method to general recurrences.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Boundaries and checks

Keep coefficient indices and polynomial degrees aligned, especially at n=0. Distinguish formal identities from analytic evaluation. Multiplicity limits, ordered sequences, and labels each change factors. Check extracted coefficients using a recurrence or small cases. The exercise and solution PDFs provide further derivations. A valid formula follows from a bijection or coefficient identity that can be expanded, rather than resemblance alone.

References and attribution

Lin Cing, handwritten discrete mathematics notes. Ralph P. Grimaldi, Discrete and Combinatorial Mathematics, 5th ed., Ch. 9.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

First-order linear recurrences

For $a_n = pa_{n-1} + q$, p≠1 gives fixed point $a^* = q/(1-p)$. Subtracting it yields $b_n = pb_{n-1}$, hence

$$a_n = p^{n-n_0}a_{n_0} + q\frac{1-p^{n-n_0}}{1-p}.$$

For p=1, $a_n = a_{n_0} + (n-n_0)q$. Specify the initial index. Integer powers use $p^0=1$, including the empty-product convention $0^0=1$, without asserting a two-variable limit.

Unrolling for $n>n_0$ gives $a_n = p^2a_{n-2} + pq + q = \cdots = p^{n-n_0}a_{n_0} + q(1 + p + \cdots + p^{n-n_0-1})$. The geometric sum gives the first formula for p≠1. For p=1 there are $n-n_0$ additions of q; p=0 gives $a_n=q$ after the initial index. Check the initial term, next term, and original recurrence. The fixed-point substitution is a convenient homogenization of the same unrolling proof.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

General forcing terms

For $a_n = pa_{n-1} + f(n)$, p≠0 allows the integrating factor

$$p^{-n}a_n - p^{-(n-1)}a_{n-1} = p^{-n}f(n).$$

Telescoping gives

$$a_n = p^{n-n_0}a_{n_0} + \sum_{k=n_0+1}^{n} p^{n-k}f(k).$$

The final formula no longer divides by p, so remains defined for p=0,1 and can be verified independently by substitution. Only the integrating-factor derivation needed p≠0. Negative p requires retaining parity signs in integer powers, rather than assuming positive-term geometric intuition.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Variable coefficients and applications

For $a_0=1$ and $a_n=na_{n-1}$, unrolling gives $a_n=n(n-1)\dots1a_0=n!$. With $a_0=0$ every term is instead zero, showing why initial data are part of a recurrence specification.

Worst-case bubble-sort comparisons satisfy $h_1=0$,$h_n=h_{n-1}+n-1$: the first pass uses $n-1$ comparisons and leaves $n-1$ elements. Thus $h_n = \sum_{j=1}^{n-1} j = n(n-1)/2 = \Theta(n^2)$. This counts comparisons, not identical elapsed time on every machine; memory hierarchy and branch prediction still matter. Variable coefficients do not directly admit constant-coefficient characteristic roots; try products, telescoping, or substitutions and verify the actual per-round states.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Distinct characteristic roots

For $a_n + Aa_{n-1} + Ba_{n-2} = 0$, $n\geq2$, trial r^n gives characteristic polynomial r^2+Ar+B. Distinct nonzero roots give $a_n = C_1r_1^n + C_2r_2^n$, with two initial conditions fixing constants. Check zero roots and the index domain separately.

Substitution gives $r^{n-2}(r^2 + Ar + B) = 0$, so a nonzero root supplies a solution. For nonzero distinct roots,

$$\det \begin{pmatrix} r_1^n & r_2^n \\ r_1^{n+1} & r_2^{n+1} \end{pmatrix} = r_1^n r_2^n (r_2 - r_1) \neq 0.$$

They are independent. Every solution is uniquely determined by a_0,a_1, so the solution space has dimension two. The initial equations $C_1+C_2=a_0$,$C_1r_1+C_2r_2=a_1$ have determinant $r_2-r_1\neq0$ and uniquely fit all solutions. Normalize the recurrence's signs before forming its polynomial.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

How initial data select one sequence

For $n \geq 2$, let $a_n = 5a_{n-1} - 6a_{n-2}$, with $a_0 = 1, a_1 = 4$. Roots 2,3 give $a_n = C_1 2^n + C_2 3^n$.

Initial data fix the coefficients; each column uses the same n

$a_0 = 1$, $a_1 = 4$

$C_1 + C_2 = 1$

$2C_1 + 3C_2 = 4$

$C_2 = 4 - 2 \cdot 1 = 2$

$C_1 = -1$

Two values → one solution

n	0	1	2	3
-2^n	-1	-2	-4	-8
$2 \cdot 3^n$	2	6	18	54
a_n	1	4	14	46

Add down each column: −4 + 18 = 14; −8 + 54 = 46.

Check. $a_2 = 5 \cdot 4 - 6 = 14$, $a_3 = 5 \cdot 14 - 6 \cdot 4 = 46$. No particular term is needed for this homogeneous recurrence.

Repeated, higher-order, and complex roots

A repeated nonzero root ρ gives $a_n = (C_1 + C_2 n)\rho^n$. To prove the second basis, use $A = -2\rho, B = \rho^2$:

$$n\rho^n + A(n-1)\rho^{n-1} + B(n-2)\rho^{n-2} = \rho^{n-2}[n\rho^2 - 2\rho^2(n-1) + \rho^2(n-2)] = 0.$$

The initial-value matrix $\begin{pmatrix} 1 & 0 \\ \rho & \rho \end{pmatrix}$ has determinant $\rho \neq 0$, proving independence and completeness. If $\rho = 0$, the recurrence only forces $a_n = 0$ for $n \geq 2$, leaving a_0, a_1 arbitrary. The pair $0^n, n0^n$ is not a basis.

For a nonzero root r of multiplicity m in an order-k recurrence, use $r^n, nr^n, \ldots, n^{m-1}r^n$. Handle each root's multiplicity separately; the total basis count must equal k. For zero multiplicity m_0 use prefix sequences $\delta_0, \ldots, \delta_{m_0-1}$, where $\delta_j(n) = 1$ iff $n = j$. Conjugate complex roots can be paired into real trigonometric solutions. Fibonacci's roots are irrational real numbers, not nonreal conjugates.

General proof for higher-order solutions

Write the forward recurrence as P(E)a=0, with $Ea_n=a_{n+1}$, monic degree-k P, and $n \geq 0$. The first k arbitrary values determine the rest uniquely, so dimension is k. For nonzero r and polynomial q,

$$(E - r)(r^n q(n)) = r^{n+1}\Delta q(n).$$

The difference operator lowers positive degree by one and kills constants. Thus multiplicity m makes P(E) annihilate $r^n n^j$ for $j<m$. For another root $\lambda \neq r$,

$$(E - \lambda)(r^n q(n)) = r^n[(r - \lambda)q(n) + r\Delta q(n)].$$

On polynomials of degree below m this map is triangular with nonzero diagonal $r-\lambda$, hence invertible.

In a proposed relation among bases, apply all other roots' factors to isolate the chosen r. Other terms vanish; the surviving polynomial has undergone only invertible maps and must vanish at every nonnegative integer, hence is the zero polynomial. Repeat for each root to prove independence.

Zero roots and dimension

The kernel of E^{m_0} is precisely sequences supported at indices $0,\ldots,m_0-1$. Each $E-\lambda$ for nonzero λ is triangular on this space with nonzero diagonal $-\lambda$. The same isolation argument handles the zero-root prefix bases. All k bases are independent and hence span the full solution space. Factor P over the complex numbers; real coefficients permit conjugate pairing. Roots $\rho e^{\pm i\theta}$ give real solutions $\rho^n(A\cos n\theta + B\sin n\theta)$, with real A,B fitted from real data. For a simple nonreal conjugate pair, take $\rho > 0$ and $\sin\theta \neq 0$. If each root instead has multiplicity m, include $n^j\rho^n\cos n\theta$ and $n^j\rho^n\sin n\theta$ for every $0 \leq j < m$; two constants alone do not cover repeated complex roots.

Nonhomogeneous recurrences and resonance

Linearity gives $a_n = a_n^{(h)} + a_n^{(p)}$. First solve the homogeneous equation, choose a particular trial matching the forcing type, substitute and compare coefficients, then add the homogeneous solution and fit initial data to the total. If its base r is a root of multiplicity m, multiply the trial by n^m.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Which root must a trial be checked against?

Let m be the multiplicity of the root in the last column, taking m=0 when it is not a root.

Forcing	Trial before resonance	Root to check
Constant	Constant	1
Degree-d polynomial	General degree-d polynomial	1
$C r^n$, $r \neq 0$	$A r^n$	r
Degree-d polynomial times r^n, $r \neq 0$	General degree-d polynomial times r^n	r
$A\cos(n\theta) + B\sin(n\theta)$	$\alpha\cos(n\theta) + \beta\sin(n\theta)$	$e^{i\theta}$ and $e^{-i\theta}$

Multiply the entire trial by n^m at resonance. The trigonometric row assumes real coefficients and $\sin\theta \neq 0$; shifts mix sine and cosine, so retain both. For θ an integer multiple of π, the sine sequence vanishes; use the exponential row with base 1 or -1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

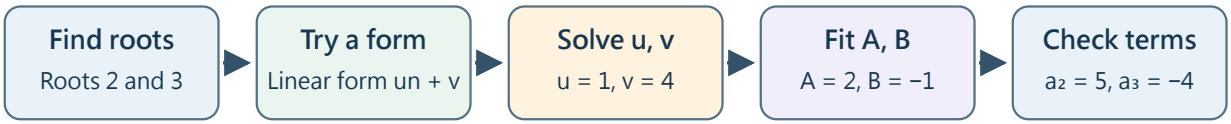
No resonance: find a particular term, then fit initial data

For $a_n=3a_{n-1}+4n, a_0=2$, the homogeneous solution is $C3^n$. Trial $An+B$ gives $An+B=(3A+4)n+(3B-3A)$, so $A=-2, B=-3$. Then $a_n=C3^n-2n-3$, and $a_0=2$ gives $C=5$. Thus $a_n=5\cdot 3^n-2n-3$, with terms 2,10,38; $3\cdot 10+8=38$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A second-order nonhomogeneous recurrence

Solve $a_n-5a_{n-1}+6a_{n-2}=2n+1$ for $n\geq 2$, with $a_0=5, a_1=6$. Solve the corresponding homogeneous recurrence and find one particular solution; impose the initial values on their sum.



The characteristic polynomial $r^2-5r+6=(r-2)(r-3)$ gives $a_n^{(h)}=A2^n+B3^n$. The forcing term is linear, with exponential base 1; since 1 is not a characteristic root, the trial need not be multiplied by n.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Substitute the particular trial

Set $a_n^{(p)} = un + v$. Substitution and collection give

$$(un + v) - 5[u(n-1) + v] + 6[u(n-2) + v] = 2un + 2v - 7u.$$

Matching $2n + 1$ yields $2u = 2$ and $2v - 7u = 1$, so $u = 1, v = 4$. A particular solution is $n + 4$, and the general solution is

$$a_n = A2^n + B3^n + n + 4.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fit the initial values and verify

Apply both initial values to the total solution: $A + B + 4 = 5, 2A + 3B + 5 = 6$. Thus $A + B = 1, 2A + 3B = 1$, giving $A = 2, B = -1$:

$$\boxed{a_n = 2^{n+1} - 3^n + n + 4} \qquad (n \geq 0).$$

The recurrence independently gives

$$a_2 = 5(6) - 6(5) + 5 = 5, \qquad a_3 = 5(5) - 6(6) + 7 = -4.$$

The closed form yields $8 - 9 + 6 = 5$ and $16 - 27 + 7 = -4$ at $n = 2, 3$. For every n, the proof follows from the verified homogeneous and particular solutions and uniqueness from the two initial values.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Resonance at a simple root

In $a_n - a_{n-1} = 2^n$, base two does not collide with root one. In $a_n - 2a_{n-1} = 2^n$ it does, requiring $Cn2^n$. For $a_n = 2a_{n-1} + 5 \cdot 2^n, a_0 = 3$, substitute $An2^n$:

$$An2^n = A(n-1)2^n + 5 \cdot 2^n, \qquad A = 5.$$

Thus $a_n = (C+5n)2^n$ and $C = 3$. Terms $a_1 = 16 = 2 \cdot 3 + 10$ and $a_2 = 52 = 2 \cdot 16 + 20$ check the result.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Multiplicity two: why multiply by n²?

The recurrence $a_n - 4a_{n-1} + 4a_{n-2} = 3 \cdot 2^n$, $n \geq 2$, has characteristic polynomial $(r-2)^2$. Divide by 2^n, putting $u_n = a_n / 2^n$.

Divide by 2ⁿ, then test which trial polynomials vanish

$u_n - 2u_{n-1} + u_{n-2} = 3$ $(n \geq 2)$

Trial u_n	Left side after substitution	Result
A	A − 2A + A	0
An	A[n − 2(n−1) + (n−2)]	0
An²	A[n² − 2(n−1)² + (n−2)²]	2A

The quadratic leaves a nonzero constant: $2A = 3$, so $A = 3/2$.

Both 2^n and $n2^n$ are homogeneous solutions. A particular trial must escape their span; $An^2 2^n$ does, giving $A = 3/2$.

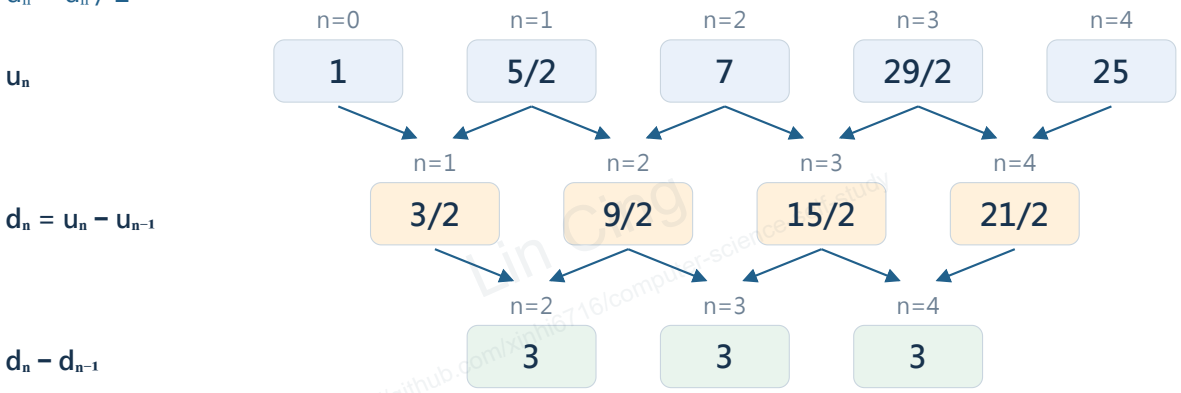
Lin Cing https://github.com/xinhi6716/computer-science-self-study

A numerical second-difference ladder

In the same example, take $a_0 = 1, a_1 = 5$, so $u_0 = 1, u_1 = 5/2$. Compute each term from $u_n = 2u_{n-1} - u_{n-2} + 3$, then subtract adjacent entries.

Each lower entry = upper right − upper left

$u_n = a_n / 2^n$



Second differences are 3: first differences are arithmetic, and u_n is quadratic.

Reconstructing the closed form from differences

Define $d_n = u_n - u_{n-1}$ for $n \geq 1$. Then $d_1 = 5/2 - 1 = 3/2$, and $d_n - d_{n-1} = 3$ for $n \geq 2$. Hence

$$d_n = \frac{3}{2} + 3(n-1) = 3n - \frac{3}{2}, \qquad u_n = 1 + \sum_{k=1}^{n} \left(3k - \frac{3}{2}\right) = 1 + \frac{3}{2}n^2.$$

Thus $a_n = (1 + \frac{3}{2}n^2)2^n$. Equivalently, add the homogeneous term $(C_1 + C_2 n)2^n$ to the particular term $\frac{3}{2}n^2 2^n$, and fit $C_1 = 1, C_2 = 0$. **Initial data constrain the total solution, not the particular term alone.**

n	0	1	2	3	4
a_n	1	5	28	116	400

Check $a_3 = 4 \cdot 28 - 4 \cdot 5 + 3 \cdot 8 = 116$ and $a_4 = 4 \cdot 116 - 4 \cdot 28 + 3 \cdot 16 = 400$. Solution 8 also works through coefficient comparison.

Why the general resonance rule works

Let nonzero r have multiplicity s in P, and let the forcing be $r^n F(n)$, with $\deg F \le d$. The factor $(E - r)^s$ takes s differences of the polynomial part; other factors preserve degree and are invertible on fixed-degree polynomial spaces.

Restrict the trial to $r^n Q(n)$, where $Q(n) = n^s H(n)$ and $\deg H \le d$. For $0 \le j \le d$, $\Delta^s n^{s+j}$ has degree j with nonzero leading coefficient $(s + j)!/j!$. Thus the map from this trial space to degree-at-most-d polynomials is triangular with nonzero diagonal, even after the other invertible factors. It uniquely determines H for the specified forcing.

At s=0 no extra n-factor is needed. This argument assumes constant coefficients over the real or complex numbers and a nonzero base. A particular solution may still gain any homogeneous solution; uniqueness holds only within the restricted trial space. Nonlinear recurrences have no such superposition guarantee.

Generating functions and rationality

Finite-order homogeneous constant-coefficient recurrences give Q(x)A(x)=P(x), Q(0)≠0, by multiplying by x^n and summing. Thus the OGF is rational. Rational forcing OGFs preserve rationality. Linearity alone, especially with variable coefficients, does not suffice.

Conversely, if $A = P/Q$, $Q = 1 + q_1 x + \cdots + q_d x^d$, coefficient comparison past deg P gives $a_n + q_1 a_{n-1} + \cdots + q_d a_{n-d} = 0$. An eventually constant-coefficient recurrence with finitely many exceptions conversely yields a rational OGF. These are formal statements independent of convergence.

For $a_n + C_1 a_{n-1} + C_2 a_{n-2} = f_n$, n≥2, put $F(x) = \sum_{n \ge 2} f_n x^n$. The sums are $A - a_0 - a_1 x$, $x(A - a_0)$,$x^2 A$. Therefore

$$(1 + C_1 x + C_2 x^2) A = a_0 + (a_1 + C_1 a_0) x + F(x).$$

The denominator reflects recurrence coefficients; initial values and forcing determine the numerator.

The same recurrence by two methods

For $a_0=1, a_1=4, a_n=5a_{n-1}-6a_{n-2}$, roots 2,3 give $C_1+C_2=1, 2C_1+3C_2=4$, hence $C_1=-1, C_2=2$ and $a_n=-2^n+2\cdot 3^n$. The OGF route gives

$$(A-1-4x)-5x(A-1)+6x^2A=0, \qquad A=\frac{1-x}{(1-2x)(1-3x)}=-\frac{1}{1-2x}+\frac{2}{1-3x}.$$

Both methods retain the same numerator and constants.

For $a_0=1, a_n=3a_{n-1}+2$, $A-1=3xA+2x/(1-x)$, giving $A=(1+x)/((1-x)(1-3x))$. Partial fractions give $a_n=2\cdot 3^n-1$, with 1,5,17,53.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A linear recurrence with nonrational OGF

For $h_0=0, h_n=h_{n-1}+1/n$, linearity still holds but

$$(1-x)H=\sum_{n\ge 1}x^n/n=-\log(1-x), \qquad H=-\log(1-x)/(1-x).$$

The initial terms are 0,1,3/2,11/6,25/12. Differentiating the forcing series gives $1/(1-x)$; its zero constant fixes $-\log(1-x)$.

To prove nonrationality, suppose H rational. Then $R=(1-x)H=-\log(1-x)$ is rational. At $x=1$ a rational function either has a finite limit or a pole of finite integer order $m\ge 1$. Set $t=-\log(1-x)$. As $x\to 1^-$, $t\to\infty$ but $t/(1-x)^{-m}=te^{-mt}\to 0$ for every fixed m. Logarithmic divergence is neither finite nor any pole order, contradiction. A formal rational representation would have the same Taylor expansion and analytic function, so is excluded too. Both examples require initial corrections; rationality depends on coefficients and forcing, not just linearity.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Nonlinear recurrences

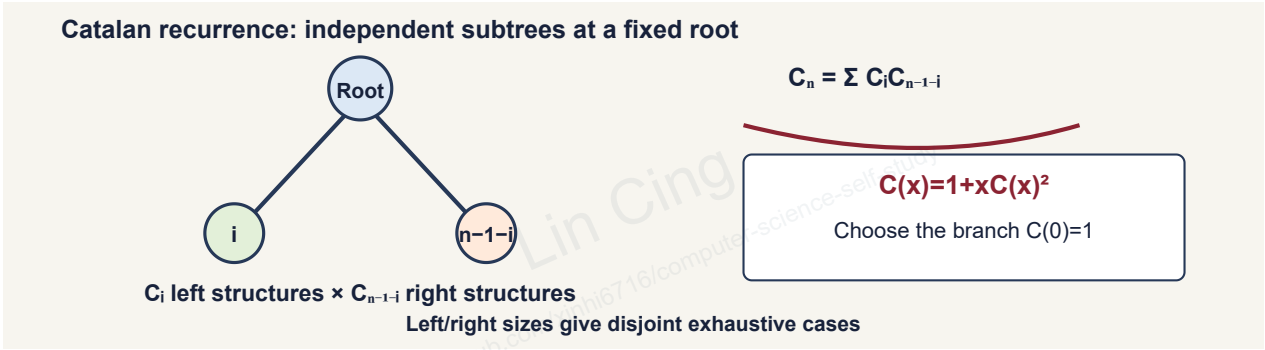
Equations such as $a_{n+1}=a_n+a_n^2$, or recurrences with products and quotients, usually have no fixed linear characteristic polynomial. Seek monotonicity, bounds, fixed points, or a reversible substitution. The change $b_n=\log a_n$ requires $a_n>0$ and converts products into sums. A nonlinear problem may ask for asymptotic bounds or invariants rather than a closed form; specify the target.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Catalan convolution and a nonlinear closed form

Let C_n count ordered rooted binary trees on n vertices, with $C_0=1$ for the empty tree. A root uniquely joins left and right subtrees of sizes i,n−1−i, giving

$$C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i}.$$



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The Catalan generating function

Summing gives $C-1=xC^2$, so $xC^2-C+1=0$. The branch with constant one is $C = (1 - \sqrt{1-4x})/(2x)$.

Construct the formal square root directly. Let $Y(z) = \sum_{j\geq 0} \binom{1/2}{j} z^j$, with descending-product binomials. Coefficient recurrence $(j+1)c_{j+1}=(1/2-j)c_j$ gives $(1+z)Y'=Y/2$,$Y(0)=1$. For $F=Y^2$, $(1+z)F'=F$,$F(0)=1$; coefficient comparison uniquely yields $F=1+z$. Thus Y is the square root with constant one, not the negative branch. Substitute $z=-4x$ and simplify descending products:

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Coefficients and the limits of the example

$$\sqrt{1-4x} = 1 - 2\sum_{k\geq 1} \frac{1}{k}\binom{2k-2}{k-1} x^k.$$

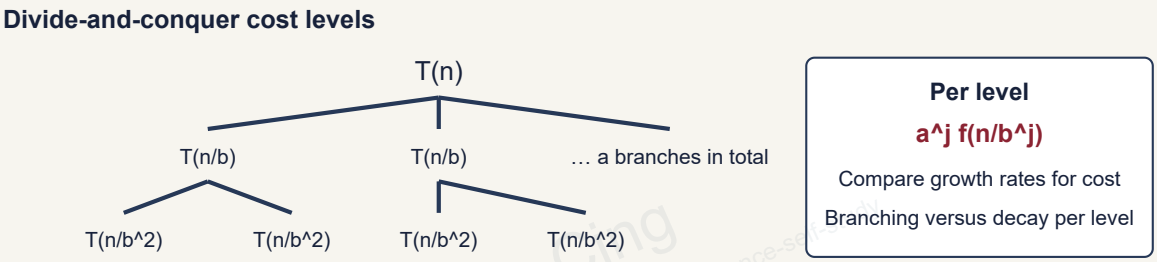
Substitution and $n=k-1$ give $C_n = \binom{2n}{n}/(n+1)$, with 1,1,2,5,14,42. At $n=3$ the five ordered trees biject with five stack outputs of {1,2,3}. Identifying left/right subtrees changes the object and invalidates the same count.

The nonlinear sequence $a_0=1$,$a_{n+1}=a_n+a_n^2$ gives 1,2,6,42,… and is not the Catalan recurrence. Seek transformations, convolution, or growth invariants rather than characteristic roots.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Divide-and-conquer recurrences

For $T(n) = aT(\lceil n/b \rceil) + f(n)$, handle exact floors/ceilings and ensure subproblems strictly decrease past the base threshold. A polynomial-cost master theorem compares a with b^d for $aT(n/b)+n^d$, identifying leaves, equal-cost layers, or the root as dominant.



Expanding powers of b

Fix integers $a \geq 1, b \geq 2, c > 0$ and $n = b^k$. For $f(n) = af(n/b) + c, f(1) = c$,

$$f(b^k) = a^k f(1) + c(1 + a + \cdots + a^{k-1}).$$

If $a = 1$, this is $c(k+1) = \Theta(\log_b n)$. If $a \geq 2$ the sum is $(a^k - 1)/(a - 1)$, yielding $\Theta(a^k) = \Theta(n^{\log_b a})$. Generally,

$$f(b^k) = a^k f(1) + \sum_{j=0}^{k-1} a^j h(b^{k-j}).$$

For fixed $d \geq 0, h(n) = \Theta(n^d) > 0, f(1) = \Theta(1) > 0$, the layer ratio a/b^d gives $\Theta(n^{\log_b a})$ when $d < \log_b a$, $\Theta(n^d \log n)$ at equality, and $\Theta(n^d)$ when $d > \log_b a$. This proof is on powers of b. General integers require floor/ceiling and monotonicity bounds between adjacent powers.

A counterexample with n / log n

For T(n)=2T(n/2)+n/log₂n on n=2ᵏ≥2, a polynomial-forcing theorem cannot replace n/log n by n. Level j costs n/(k−j), so

$$T(2^k) = 2^k T(1) + 2^k \sum_{j=0}^{k-1} \frac{1}{k-j} = n(T(1) + H_k) = \Theta(n \log\log n).$$

Use a separate positive T(1); do not divide by log₂1. At n=16,T(1)=1, internal levels cost 4,16/3,8,16 and leaves cost 16, totaling 148/3=16(1+H₄). A recurrence that still calls itself at n=1 has no termination proof.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Master-theorem examples and limits

For positive constant bases, 2T(n/2)+n gives Θ(n log n), 4T(n/2)+n gives Θ(n²), and 2T(n/4)+n gives Θ(n). Recursion-tree layer sums verify each. Nonpolynomial forcing or unsuitable a,b and division conditions require a broader theorem, recursion trees, Akra–Bazzi, or direct induction. A Θ claim requires both upper and lower bounds; O(n log n) alone does not complete a tight analysis.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Choosing a recurrence method

Use iteration or integrating factors for first order; characteristic roots for homogeneous constant-coefficient linear equations; homogeneous plus particular solutions with resonance checks for forcing; generating functions for coefficient extraction; transformations, invariants, or bounds for nonlinear equations; and exact size recurrences followed by justified tree/theorem/induction analysis for divide and conquer. Characteristic roots outside their linear constant-coefficient family are a mathematical error, not a stylistic choice.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Knockout tournaments

With $n=2^m$ entrants, each game eliminates one player. Every player except the champion is eliminated once, so $n-1$ games suffice and are necessary. Equivalently $T(1)=0, T(n)=2T(n/2)+1$ unrolls to $n-1$. One argument counts eliminations, the other solves a recurrence; agreement checks the base and merge-cost meanings.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Coupled recurrences and simultaneous generating functions

For fixed matrix M and $y_n = My_{n-1} + f_n$, let $G = \sum_{n \geq 0} y_n x^n$, $F = \sum_{n \geq 1} f_n x^n$. Then

$$G - y_0 = xMG + F, \qquad G = (I - xM)^{-1}(y_0 + F).$$

No invertibility of M or analytic convergence is needed. The formal series $\Sigma x^k M^k$ multiplies $I - xM$ to I by cancellation of every positive-degree coefficient. If F has rational components, the adjugate formula and determinant constant one make G's components rational.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Tracing the coupled state

For $a_n = 2a_{n-1} + b_{n-1}$, $b_n = a_{n-1} + b_{n-1}$, $a_0 = 1$, $b_0 = 0$, use the same old pair in both updates.

n	Old pair	New a_n	New b_n
1	(1,0)	2 • 1+0=2	1+0=1
2	(2,1)	2 • 2+1=5	2+1=3
3	(5,3)	2 • 5+3=13	5+3=8
4	(13,8)	2 • 13+8=34	13+8=21

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Solving the generating functions

The OGF system is

$$\begin{pmatrix} 1-2x & -x \\ -x & 1-x \end{pmatrix} \begin{pmatrix} A \\ B \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad A = \frac{1-x}{1-3x+x^2}, \quad B = \frac{x}{1-3x+x^2}.$$

Both components satisfy $u_n=3u_{n-1}-u_{n-2}$ but have distinct initial values. With $r_\pm = (3 \pm \sqrt{5})/2$, partial fractions give

$$b_n = \frac{r_+^n - r_-^n}{\sqrt{5}}, \qquad a_n = \frac{(r_+ - 1)r_+^n + (1 - r_-)r_-^n}{\sqrt{5}}.$$

At n=0,1 these give (1,0),(2,1); uniqueness of each second-order recurrence then proves agreement for all terms. Elimination must preserve each component's own initial values.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Boundaries and verification

Check initial values and substitute into the original recurrence. Verify index floors/ceilings, eventual scope of asymptotic bounds, stopping conditions, and the operation model. Exercises and solutions provide further derivations. Closed forms are one objective; lower-order terms can matter to actual cost or memory and cannot always be discarded.

References and attribution

Lin Cing, handwritten discrete mathematics notes. Ralph P. Grimaldi, Discrete and Combinatorial Mathematics, 5th ed., Ch. 10.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Formal graph models

A finite simple undirected graph is $G = (V, E)$, $E \subseteq \binom{V}{2}$, where the latter consists of unordered pairs of distinct vertices. This excludes loops and parallel edges. A multigraph distinguishes parallel edges by IDs; an undirected loop contributes two to degree. Euler's theorem still applies.

Order is |V| and size |E|. Adjacency concerns endpoints, not their drawn positions. Degree counts incident endpoints with multiplicity. A simple directed graph uses ordered pairs (u, v),u≠v; outdegree d^+ counts departing arcs and indegree d^- arriving arcs. Specify simple/multi, directed/undirected, labeled/unlabeled, and weighted/unweighted models first.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Walks, trails, paths, and cycles

A walk alternates incident vertices and edges; length counts edge uses. A trail repeats no edge ID. A simple path repeats no vertex. In a simple undirected graph a cycle has length at least three, equal first/last vertex, and no other repetition. A closed trail may repeat other vertices and need not be a cycle.

A length-zero walk at one vertex is also a path. Multigraph cycle-length conventions require separate specification. An Euler circuit is a closed trail and is not limited by the simple-cycle minimum length. Undirected path connectivity is reflexive, symmetric, and transitive; its classes are components. One component means connected. A simple path on n vertices has length at most n−1. Euler trails use every edge once; Hamilton paths every vertex once. Their existence conditions differ.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Adjacency matrices count walks

In a fixed vertex order, $A_{ij}=1$ iff v_iv_j is an edge. For a simple undirected graph $A=A^{\mathsf{T}}$, diagonal zero, and row sums are degrees. Directed matrices need not be symmetric.

The entry $(A^r)_{ij}$ counts length-r walks from v_i to v_j. At r=0, $A^0=I$ counts empty walks. Inductively

$$(A^{r+1})_{ij} = \sum_k (A^r)_{ik} A_{kj}$$

classifies walks by their penultimate vertex. Repeated vertices and edges are permitted; powers do not count simple paths. For path 1–2–3,

$$A^2 = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 2 & 0 \\ 1 & 0 & 1 \end{pmatrix}.$$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Two different walks return to the middle vertex

One path

$2 \to 1 \to 2$
$2 \to 3 \to 2$

A: order 1, 2, 3

$$\begin{matrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{matrix}$$

row: start; column: end

A^2: counts of two-step walks

$$\begin{matrix} 1 & 0 & 1 \\ 0 & 2 & 0 \\ 1 & 0 & 1 \end{matrix}$$

$(A^2)_{22} = 1\cdot1 + 0\cdot0 + 1\cdot1 = 2$

The central two counts 2–1–2 and 2–3–2, neither a simple path.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Degree sums and the handshake lemma

$\sum_{v \in V} \deg(v) = 2|E|$, because each edge contributes at both ends. Thus the number of odd-degree vertices is even; this necessary condition does not characterize graphical degree sequences. Sequence (3,1,1) has odd total five and is impossible; (3,3,1,1) has even sum but is impossible: both degree-three vertices must touch every other vertex, forcing the last two degrees to be at least two.

Count parallel edges separately and loops twice, preserving the identity for finite multigraphs. In a digraph each arc contributes one outdegree and one indegree, so both sums equal the arc count.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Subgraphs, complements, and isomorphisms

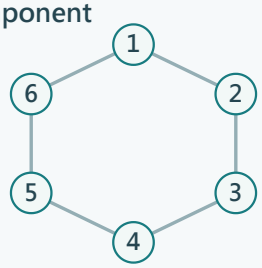
A subgraph H=(W, F) has W⊆V, F⊆E with every retained edge's endpoints in W. The induced subgraph retains all edges between W's vertices. A simple graph's complement uses the same vertices and exactly the absent distinct-vertex edges.

An isomorphism is a bijection $\varphi : V(G) \to V(H)$ preserving adjacency in both directions. Compare order, size, degree multiset, triangles, components first; matching invariants alone is insufficient.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Equal degrees do not prove isomorphism

C_6: one component



6 vertices; 6 edges; degree 2 at every vertex

$C_3 \cup C_3$: two components

6 vertices; 6 edges; degree 2 at every vertex

C_6 and two disjoint triangles each have six vertices, six edges, and all degrees two. One is connected and the other has two components, so they are nonisomorphic. Even matching connectivity would not eliminate all possible distinctions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A relabeling permutes rows and columns together

Let column j of P be e_i when H's vertex j corresponds to G's vertex i. Then an isomorphism's permutation matrix satisfies $A_H = P^T A_G P$; conversely this entrywise equality proves adjacency preservation. Equal spectra alone generally do not suffice.

Complement: reverse adjacency, not the diagonal

For complements, $A_{\overline{G}} = J - I - A_G$ and $d_{\overline{G}}(v) = n - 1 - d_G(v)$. The edge sets partition K_n. The complement of 1–2–3 has only edge 13 and isolated 2; flipping every matrix entry would wrongly introduce loops.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Euler trails and circuits

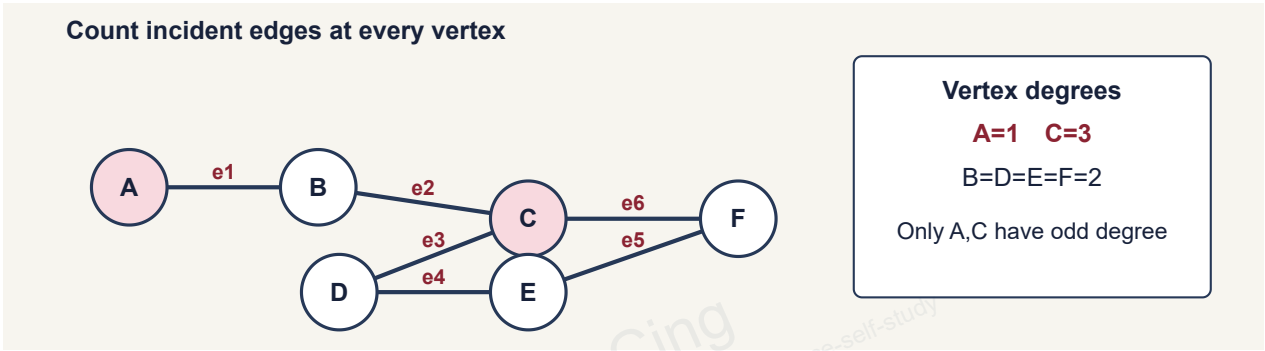
When all nonisolated vertices are connected, an Euler circuit exists iff every degree is even. An open Euler trail exists iff exactly two degrees are odd, at its endpoints. Necessity follows from paired arrivals/departures at internal vertices.

For sufficiency in the even case, follow unused edges from a nonisolated start. No other vertex can be the first dead end, so the trail closes. If edges remain, connectivity supplies a current-trail vertex incident to unused edges; form another closed trail there and splice it in. Each step adds an edge, so finitely many steps exhaust all edges. With two odd vertices, add a temporary edge between them, find a circuit in the even multigraph, then remove that edge to obtain the required trail.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Count degrees before tracing edge IDs

For $e_1,\ldots,e_6$=AB, BC, CD, DE, EF, FC, the degrees are A:1,B:2,C:3,D:2,E:2,F:2. Only A, C are odd.



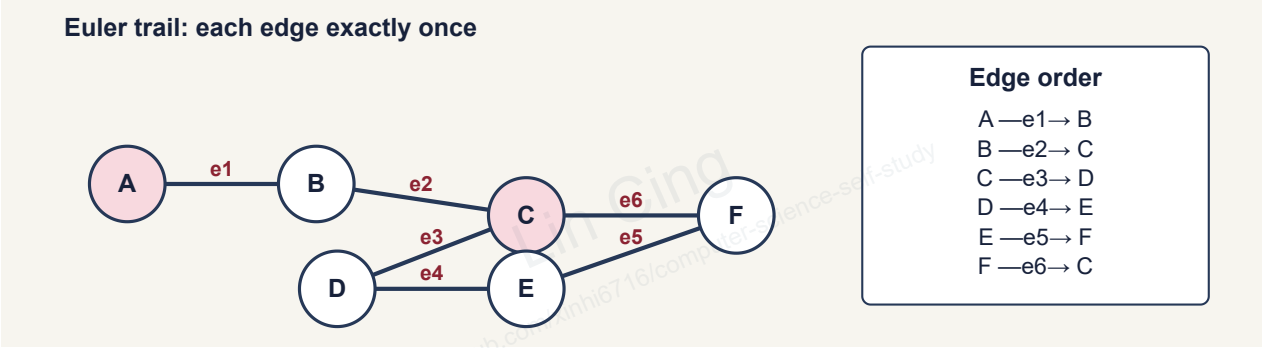
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Every edge in an Euler trail

The figure pairs the same graph with its six-row traversal. Each row uses one edge ID, and every ID appears once:

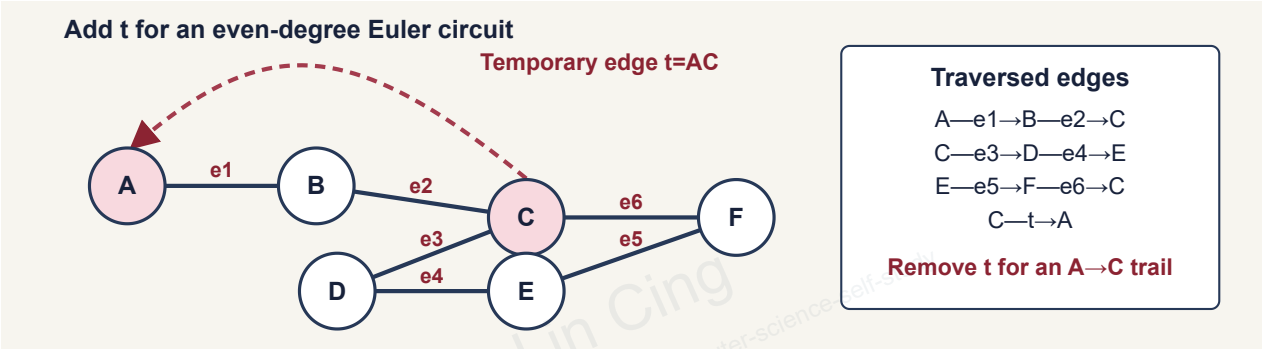
$$A \xrightarrow{e_1} B \xrightarrow{e_2} C \xrightarrow{e_3} D \xrightarrow{e_4} E \xrightarrow{e_5} F \xrightarrow{e_6} C.$$

Every internal visit pairs an arrival/departure; A, C each retain one unmatched incident use.



Temporary edge and cutting the circuit

Add $t = AC$ to make A and C even. The circuit A–B–C–D–E–F–C–A ends with temporary edge t.



Cut immediately after traversing the temporary edge, then remove it. The remaining trail starts at that traversal's destination and ends at its departure point. Reverse an undirected trail to exchange endpoint order. This reduces the two-odd-vertex case to the even case.

Hierholzer's splicing invariant

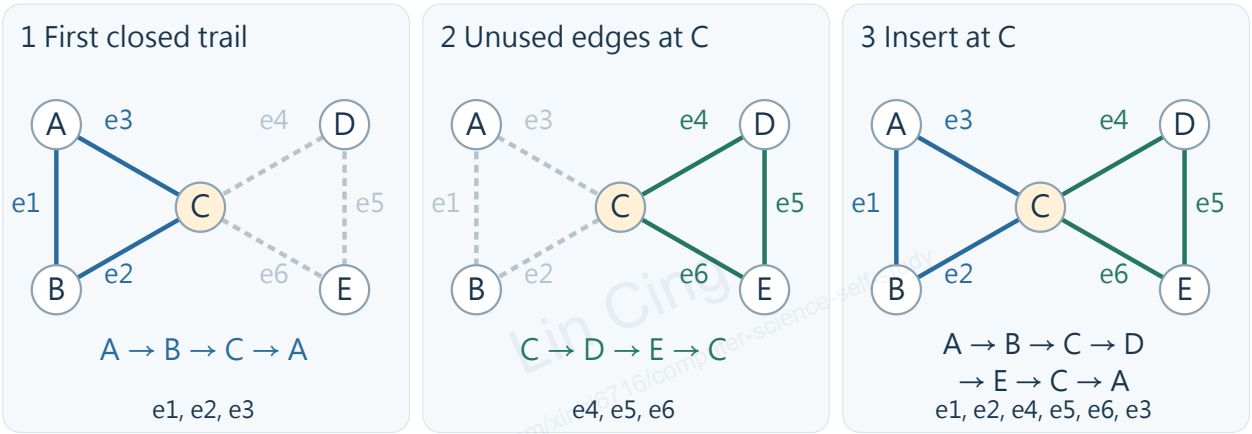
In a finite even-degree undirected multigraph with connected nonisolated part, follow unused edges from s. Stopping at $v \neq s$ would mean an odd number of used incident edges there. Even total degree then leaves an unused edge, contradiction. Thus the first stop is s.

Removing a closed trail preserves even residual degrees. If unused edges remain, original connectivity gives a trail vertex x incident to them. Build another closed trail from x and insert it. The result remains closed, edge-distinct, and contains every previously used edge. Each insertion adds an edge, so all are eventually used.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two closed trails splice at a shared vertex

For triangles sharing C with edges AB, BC, CA, CD, DE, EC, first take A–B–C–A, then C–D–E–C. Splicing at C yields A–B–C–D–E–C–A. C repeats but all six edges occur once.



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Linear-time Euler traversal

Assign one ID to each undirected edge and store it at both endpoint adjacency lists. Loops also use one used-ID flag. A per-vertex cursor skips already used IDs without rescanning the list.

```
S = [start]; answer = []
while S is not empty:
    v = S.top()
    advance cursor[v] past used edge IDs
    if v has no unused edge:
        answer.append(S.pop())
    else:
        take unused e=(v, w); used[e]=true
        S.push(w)
return reverse(answer)
```

The stack retains the not-yet-output trail; backtracking completes inner closed segments, and reversal realizes splicing. Check connectivity and parity first; an open trail starts at an odd vertex. Finally verify m used IDs and output length m+1. Each adjacency entry is scanned once, giving O(n+m) time and additional space.

A dead end at A does not end the search

Scan edge IDs in increasing order on the preceding graph. Initially S=[A], answer=[].

Action	S (top at right)	answer (append order)
e1	`[A,B]`	`[]`
e2	`[A,B,C]`	`[]`
e3	`[A,B,C,A]`	`[]`
pop A	`[A,B,C]`	`[A]`
e4	`[A,B,C,D]`	`[A]`
e5	`[A,B,C,D,E]`	`[A]`
e6	`[A,B,C,D,E,C]`	`[A]`

Pop A after e3; C still has e4, so continue. The first popped A becomes the **last** output vertex.

Backtracking writes the circuit in reverse

All six IDs are used. The remaining pops traverse no additional edge.

Action	S (top at right)	answer (append order)
pop C	`[A,B,C,D,E]`	`[A,C]`
pop E	`[A,B,C,D]`	`[A,C,E]`
pop D	`[A,B,C]`	`[A,C,E,D]`
pop C	`[A,B]`	`[A,C,E,D,C]`
pop B	`[A]`	`[A,C,E,D,C,B]`
pop A	`[]`	`[A,C,E,D,C,B,A]`

Reverse answer: **A–B–C–D–E–C–A**, using **e1,e2,e4,e5,e6,e3**. The second circuit is inserted before e3. Repeated C is legal.

Directed Euler circuits

For m>0, a directed Euler circuit exists iff the underlying undirected nonisolated part is connected and $d^+(v)=d^-(v)$ everywhere. Necessity follows from paired arrivals/departures and one trail's connectivity.

For sufficiency, contract strongly connected components into a DAG. A source component has no entering arcs. Summing $d^+-d^-=0$ inside it cancels internal arcs, forcing no leaving arcs either. More than one component would contradict underlying connectivity; the nonisolated part is strongly connected.

Following unused outgoing arcs cannot stop away from the start: used arrivals exceed departures by one there, and total balance leaves an outgoing arc. Remove the resulting closed trail, retaining balance, and splice remaining trails as before. The same stack algorithm uses outgoing adjacency lists.

Directed open trails and zero-edge cases

An Euler trail from s to distinct t exists iff the nonisolated underlying graph is connected, $d^+(s)-d^-(s)=1$, $d^-(t)-d^+(t)=1$, and every other vertex is balanced. Necessity counts arrivals/departures. Add temporary t→s for sufficiency, find a balanced circuit, cut after that arc and remove it.

For A→B, B→C, C→A, A→C, (out, in) pairs are A:(2,1),B:(1,1),C:(1,2); a trail is A–B–C–A–C. Undirected degree alone does not decide the directed problem.

Zero-edge trails are allowed. If E is empty and V nonempty, any single-vertex trail covers all edges. If V is empty there is no start, and no Euler circuit under this convention. At m>0, extra isolated vertices do not affect traversal of edges.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Seven-edge multigraph and one repair

On A, B, C, D take two separate AD IDs, two BD IDs, and CD, AC, BC once each. The graph is connected with degrees 3,3,3,5, so four odd vertices prevent an Euler trail.

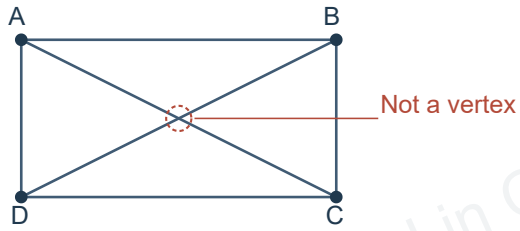
Add t=AB: degrees become 4,4,3,5, leaving C, D odd. The trail C–A–D–A–B–D–B–C–D uses eight edges. Its two AD traversals use different IDs, as do its two BD traversals; AB, AC, BC, CD each occur once. Endpoints are precisely C, D. Using endpoint pairs as used keys would incorrectly suppress the second parallel AD or BD; edge IDs preserve distinct connections.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

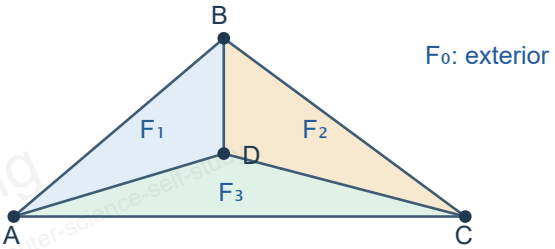
Planarity versus a plane embedding

An abstract graph records only vertices and adjacency; a drawing places them in the plane. A graph is planar if some drawing avoids crossings away from endpoints. Such a drawing is a plane embedding.

The same K_4 redrawn without crossings



V={A,B,C,D} ; E={AB,AC,AD,BC,BD,CD}



Right: 4 faces; 4−6+4=2

Both drawings have four vertices, six edges, and degree three everywhere. The left crossing is not a fifth vertex. Placing D inside outer triangle ABC gives inner faces F_1,F_2,F_3 and outer F_0. A crossing in one drawing does not prove nonplanarity.

Euler's formula by cycle-edge deletion

A connected plane graph without cycles is a tree: e=v−1,f=1, so v−e+f=2. Otherwise delete a cycle edge. Connectivity remains, its two distinct incident faces merge, and both e, f decrease by one. Continue to a tree; the invariant proves the formula. Deleting an arbitrary bridge would disconnect the graph, and its sides belong to one face rather than two.

Operation	v	e	f including exterior	v−e+f
K_4	4	6	4	2
Delete AD; merge F_1,F_3	4	5	3	2
Delete BD; merge inner faces	4	4	2	2
Delete AC; merge inner/outer	4	3	1	2

Remaining AB, BC, CD form a tree. For $c \geq 1$ components, add $c-1$ noncrossing bridges without changing faces, then apply the connected formula to obtain $v-e+f=1+c$. The embedding has one unbounded face; a component need not be incident to it. For the empty vertex set, v=e=c=0 and f=1, so the formula also holds.

Face-boundary multiplicities and edge bounds

Each embedded edge has two sides. A bridge is counted twice in the same face boundary, so $\Sigma \ell(F)=2e$ even with bridges; face boundaries need not be simple cycles.

In a connected simple plane graph with $v \geq 3$, every face boundary has length at least three. Length one requires a loop; length two requires parallel edges or an entire one-edge graph. If there are cycles, boundary cycle portions exclude these cases. If the graph is a tree, its sole boundary has length $2(v-1) \geq 4$. Thus $3f \leq 2e$ and

$$3(2 - v + e) \leq 2e \quad \Rightarrow \quad e \leq 3v - 6.$$

K_5 has $v=5, e=10>9$ and is nonplanar. Satisfying the inequality is only necessary, not sufficient.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Girth and bipartite nonplanarity

Girth g is shortest cycle length. A connected simple plane graph containing a cycle of girth $g \geq 3$ has each face boundary length at least g. Bridge excursions lengthen boundaries without shortening a cycle portion. Hence $gf \leq 2e$ and

$$e \leq \frac{g}{g-2}(v-2).$$

A forest has no finite girth; use $e=v-c$ rather than substituting infinity into this proof. A simple bipartite planar graph with $v \geq 3$ has $g \geq 4$ when cyclic, hence $e \leq 2v-4$. A tree satisfies $v-1 \leq 2v-4$ too. $K_{3,3}$ has $v=6, e=9>8$, proving nonplanarity despite passing the weaker bound. A proof must exclude all embeddings, not one drawing.

For a disconnected triangle-free simple planar graph with $v \geq 3$, place components in disjoint disks and join them in a chain with $c-1$ noncrossing bridges. No new cycle or triangle arises. The connected bound gives $e+c-1 \leq 2v-4$, hence $e \leq 2v-4$ for the original. The general girth proof still retains its cycle premise.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Plane duals depend on the embedding

For each face F of a connected plane graph create vertex F*. Each original edge e gives a dual edge across e between its two incident faces. If these are the same face, the dual edge is a loop, as for an original bridge. Preserve parallel dual edges. Thus v*=f, e*=e, f*=v; both graphs satisfy Euler's formula. This changes edge incidence, not merely matrix row/column order.

For embedded K_4, F_0=ABC, F_1=ABD, F_2=BCD, F_3=CAD:

Edge	AB	BC	CA	AD	BD	CD
Dual endpoints	0–1	0–2	0–3	1–3	1–2	2–3

The dual is again K_4. Adding a leaf X at D inside an existing face makes DX a bridge; the dual gains a loop at that face, not two new faces.

Why does a face boundary traverse a bridge twice?

Fix triangle ABC, put leaf D inside it, and add bridge CD. Number the edges $e_1 = AB, e_2 = BC, e_3 = CA, e_4 = CD$. There are just the exterior F_0 and interior F_1, so $(v, e, f) = (4, 4, 2)$. The bridge does not split the interior: a curve can pass around its leaf endpoint D to the other side.

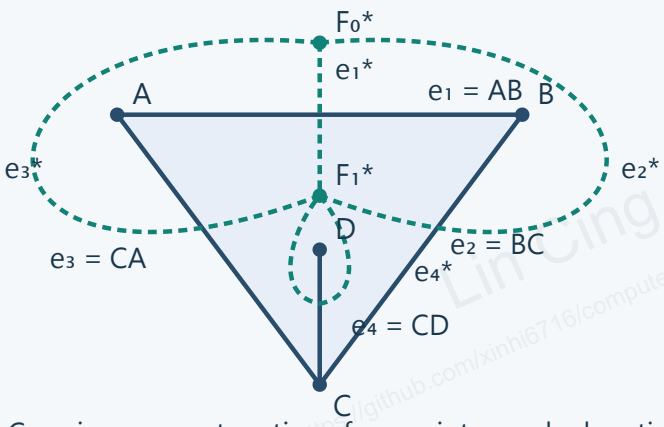
Face	One complete boundary walk	Edge IDs	Length
F_0	$A \to C \to B \to A$	3, 2, 1	3
F_1	$A \to B \to C \to D \to C \to A$	1, 2, 4, 4, 3	5

Boundary length counts occurrences. The interior walk uses the single bridge to reach D and returns on its other side. Thus $3 + 5 = 8 = 2e$. This boundary repeats C and CD and is not a simple cycle.

Draw each primal edge with its matching dual edge

Filled face points are dual vertices. Dashed e_i^* crosses only matching solid e_i; crossings are not vertices.

Primal and dual: dashed edges cross matching solid edges



Solid: primal G

Dashed: dual G*

F_0: exterior

F_1: interior

D in F_1; CD is a bridge.

e_4^* encloses D; both ends are F_1^*.

Crossings are not vertices; face points are dual vertices.

Triangle edges give three parallel dual edges; bridge CD gives a loop at the interior face point.

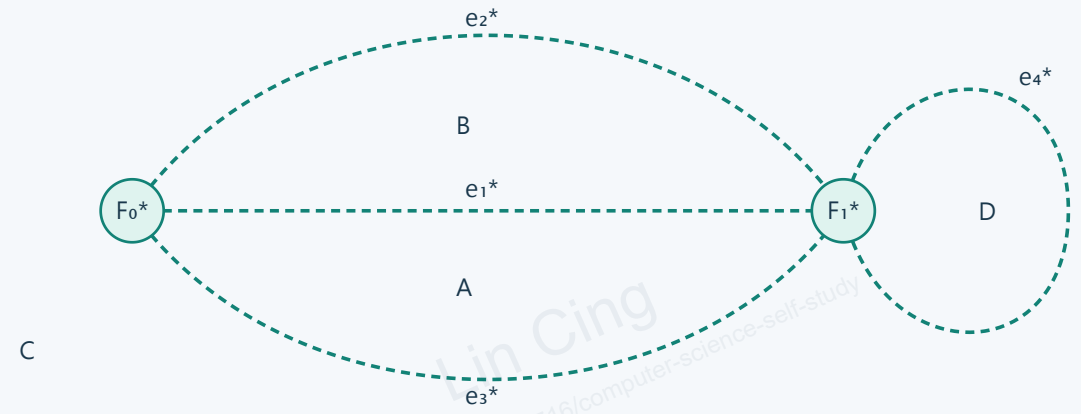
Read the dual edge list from face incidences

Primal edge	Incident faces	Dual edge	Contribution to $\deg(F_0^*)$	Contribution to $\deg(F_1^*)$
$e_1 = AB$	F_0, F_1	$F_0^* F_1^*$	1	1
$e_2 = BC$	F_0, F_1	Same endpoints, distinct ID	1	1
$e_3 = CA$	F_0, F_1	Same endpoints, distinct ID	1	1
$e_4 = CD$	F_1, F_1	Loop at F_1^*	0	2

The degrees are $3, 5$, matching the two boundary lengths. The degree sum is $3 + 5 = 2 \cdot 4$. **A loop contributes two edge ends to one vertex; parallel edges remain distinct.** Collapsing the three parallel edges or deleting the loop destroys the edge-by-edge dual construction.

A simple primal graph can have a nonsimple dual

The dual alone: three parallel edges and one loop



A, B, C, D label four dual faces. A loop contributes two to degree.

Primal vertex degrees are $2, 2, 3, 1$; the dual has two vertices of degrees $3, 5$. Labels A, B, C, D mark the four dual faces, not additional vertices. Its Euler count is $v^* - e^* + f^* = 2 - 4 + 4 = 2$. Simplicity of the primal does not exclude dual loops or parallel edges.

Deleting a cycle edge differs from deleting a bridge

Each row independently starts from the complete triangle with its interior leaf. Keep all four vertices.

Operation	Components c	Faces f	$v - e + f$	Reason
No deletion	1	2	$4 - 4 + 2 = 2$	Triangle separates inside from outside
Delete AB	1	1	$4 - 3 + 1 = 2$	Remaining graph is a tree; the faces merge
Delete CD	2	2	$4 - 3 + 2 = 3$	D is isolated; the triangle still separates faces

The last row needs $v - e + f = 1 + c$. **Removing CD cannot merge distinct faces:** its two sides already belonged to one face. An isolated point encloses no new region; a new component does not bring another exterior face.

Why does a connected primal graph have $f^* = v$?

Fix a connected plane embedding, allowing bridges. One dual vertex per face and one dual edge per primal edge give $v^* = f$ and $e^* = e$.

The dual is connected. A curve joining points in any two faces can avoid the finite vertex set and cross edges transversely finitely many times. Reading successive crossed faces yields a walk in the dual. Euler's formula therefore applies to it:

$$f^* = 2 - v^* + e^* = 2 - f + e = v.$$

The last equality uses primal Euler count $v - e + f = 2$. The bridge-loop correspondence also works in reverse: a nonbridge lies on a simple cycle. The Jordan-curve property separates that cycle's two sides, so the edge cannot have the same face on both sides. Thus a primal edge is a bridge exactly when its dual edge is a loop.

For face-boundary multiplicity and Euler's formula, see Grimaldi, fifth edition, Section 11.4, printed pp. 545–547. The same embedding connects every count in this worked example.

Nonplanarity criteria

Kuratowski's theorem states that a finite graph is planar iff it contains no subdivision of K_5 or $K_{3,3}$. Subdivision replaces edges by internally vertex-disjoint paths, inserting degree-two vertices. Wagner's theorem uses minors instead, allowing vertex/edge deletion and edge contraction.

The sufficiency of these characterizations is cited here, not derived from Euler bounds. Necessity follows because deletion preserves a plane drawing, contraction inside a narrow edge neighborhood adds no crossing, and suppressing a degree-two intermediate vertex also preserves planarity. If these operations produce a known nonplanar obstruction, the original was nonplanar. One found obstruction suffices; unsuccessful manual searching does not prove its absence.

Necessary Hamilton-cycle conditions

A Hamilton path visits each vertex once; a Hamilton cycle also closes, with n≥3 in a simple graph. Every vertex then has degree at least two, and no bridge or cut vertex is possible. These conditions alone are insufficient.

More generally, for every nonempty vertex set S,

$$c(G - S) \leq |S|.$$

Deleting |S| vertices from the cycle leaves at most |S| path segments; extra edges in G−S can join them but not increase component count. For S=V the count is zero. In $K_{2,3}$, deleting the two-vertex part leaves three isolates, violating 3≤2, so no Hamilton cycle exists. Yet b_1–a_1–b_2–a_2–b_3 is a Hamilton path. Euler questions require their separate degree checks.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Ore's edge-addition lemma

For n≥3, nonadjacent u, v with d(u)+d(v)≥n, G has a Hamilton cycle iff G+uv does. The forward direction is immediate. If a cycle in G+uv avoids uv it already belongs to G. Otherwise deleting uv leaves Hamilton path v_1=u, v_2,…,v_n=v. On indices 1,…,n−1 define

$$A = \{i : uv_{i+1} \in E\}, \qquad B = \{i : v_i v \in E\}.$$

Because uv is absent, |A|=d(u),|B|=d(v). Disjointness would bound their sum by n−1, a contradiction. At a common i, the original edges give cycle

$$v_1, v_2, \ldots, v_i, v_n, v_{n-1}, \ldots, v_{i+1}, v_1.$$

This uses two reconnections along a vertex order, without a planarity assumption.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Ore and Dirac sufficient conditions

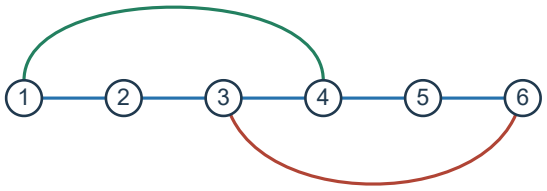
Ore's theorem assumes $d(u)+d(v) \geq n$ for every nonadjacent pair. Add missing edges one by one. Degrees only increase, preserving the premise for remaining pairs; the final K_n has a Hamilton cycle. Applying the lemma backward proves G does too. Dirac's condition $\delta(G) \geq n/2$ implies Ore immediately.

Both are sufficient. Failure does not prove nonexistence: C_6 has degree $2<3$ yet is itself a Hamilton cycle.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Reconnect the path without using the added edge

Ore's lemma: reverse the path suffix to avoid the new edge



Original path: 1 — 2 — 3 — 4 — 5 — 6

At i=3,existing reconnections are
1—4 (green),3—6 (red)

Cycle: 1 → 2 → 3 → 6 → 5 → 4 → 1

Follow 1→2→3,then 6; reverse through 6→5→4 and return to 1.

At i=3, take 1–2–3, edge 3–6, reversed segment 6–5–4, and 4–1. Each vertex occurs once without using the added endpoint edge 1–6.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Hamilton paths from a cycle theorem

For $n \geq 2$, $d(u)+d(v) \geq n-1$ for all nonadjacent pairs ensures a Hamilton path. Add a universal new vertex z, obtaining n+1 vertices. Nonadjacent pairs remain original pairs and gain degree sum two, reaching at least n+1. Ore gives a cycle in the new graph. Delete z and its two incident cycle edges to leave a Hamilton path in G. In particular $\delta(G) \geq (n-1)/2$ suffices.

At n=2, an edgeless pair fails the premise $0 \geq 1$; an edge itself is the required path. This gives existence, not shortestness or prescribed endpoints. Fixed endpoints require another check; deleting z does not freely choose them.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Every tournament has a directed Hamilton path

A tournament has exactly one directed arc between every two distinct vertices. It has a Hamilton path but need not have a cycle; the transitive orientation $i \to j$ iff $i<j$ is acyclic.

Induct on vertices. Given path $v_1 \to \dots \to v_{n-1}$, add x. If $x \to v_1$, prepend it. Otherwise $v_1 \to x$. If a first j with $x \to v_j$ exists, minimality gives $j \geq 2$ and $v_{j-1} \to x$, so insert x there. If none exists, every $v_i \to x$ and append it. Each case preserves a directed path containing all vertices.

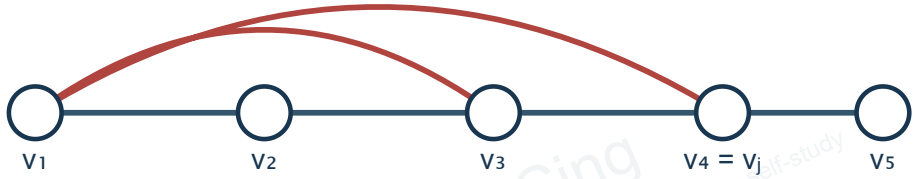
For $a \to b \to c \to d$ with $a \to x$, $b \to x$, $x \to c$, $d \to x$, insert between b, c to get $a \to b \to x \to c \to d$. x need not dominate every later vertex.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A longest path guarantees a long cycle

Let G be a finite simple undirected graph with minimum degree $\delta(G) \geq k \geq 2$. Choose a longest simple path $v_1, v_2, \ldots, v_m$.

Longest path: every endpoint neighbor lies on the path



Local sketch: three endpoint neighbors; v_4 closes a 4-edge cycle

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Count the cycle using the farthest neighbor

Every neighbor of v_1 lies on the path; otherwise prepend it to obtain a longer path. The figure shows three neighbors ($k = 3$) and omits unrelated edges.

Because the graph is simple, v_1 has at least k distinct neighbors, all among $v_2, \ldots, v_m$. Let v_j be the neighbor with largest index. All k indices lie in $2, \ldots, j$, so $j - 1 \geq k$ and $j \geq k + 1$.

The edge $v_j v_1$ closes the path segment $v_1, v_2, \ldots, v_j$ into a simple cycle with exactly $j \geq k + 1$ edges. The assumption $k \geq 2$ gives at least three edges; finiteness ensures that a longest simple path exists.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Vertex coloring

A proper k-coloring maps vertices to {1,…,k} with different colors at adjacent vertices. The chromatic number $\chi(G)$ is minimum k. A nonempty edgeless graph has $\chi=1$; empty vertices have $\chi=0$ by convention. A loop prevents proper coloring.

The bounds are $\omega(G)\leq\chi(G)\leq\Delta(G)+1$, not universal equalities. Odd cycles need three colors; bipartite nonempty graphs need at most two. Greedy coloring depends on vertex order and need not minimize χ.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bipartiteness, odd cycles, and greedy coloring

Two-colorability is equivalent to having no odd cycle. Alternating colors along a cycle forces even length. Conversely BFS each component and color by distance parity. A same-color edge uv together with tree paths from their deepest common ancestor forms an odd cycle, contradiction. Nonempty edgeless graphs need only one color.

Greedy assigns the smallest positive color absent from already colored neighbors. At most Δ colors are forbidden, proving the $\Delta+1$ upper bound, not optimality. On path 1–2–3–4, order 1,4,2,3 gives colors 1,1,2,3; order 1,2,3,4 gives 1,2,1,2. Three greedy colors do not prove $\chi=3$. An edge gives lower bound two and the exhibited coloring upper bound two, so $\chi=2$. The four-color theorem for simple planar graphs is cited, not proved by a face-count inequality.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Greedy order changes the coloring, not the chromatic number

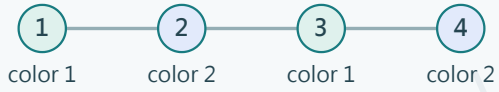
Choose the smallest positive color absent from already-colored neighbors.

Order 1, 4, 2, 3

vertex → prior neighbor colors → choice

1 → {} → 1
4 → {} → 1
2 → {1} → 2
3 → {1, 2} → 3

Order 1, 2, 3, 4



vertex → prior neighbor colors → choice

1 → {} → 1
2 → {1} → 2
3 → {2} → 1
4 → {1} → 2

Order 1,4,2,3 uses colors 1,1,2,3 in that order. Order 1,2,3,4 uses 1,2,1,2. The first run only proves $\chi \leq 3$. The edge lower bound and a valid two-coloring together prove $\chi = 2$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chromatic polynomials

P_G(k) counts proper colorings with k named colors. For a nonloop edge e,

$$P_G(k) = P_{G-e}(k) - P_{G/e}(k).$$

Contraction identifies endpoints and removes e. Parallel edges may merge without changing constraints; an unrelated remaining loop makes the polynomial zero and cannot be silently deleted. Contracting a simple graph this way leaves no loop.

An n-vertex tree has $k(k-1)^{n-1}$ colorings; K_n has $k(k-1)\ldots(k-n+1)$. Values at nonpositive k can be evaluated algebraically but are not direct coloring counts.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Deletion–contraction by a bijection

Partition proper colorings of G−e, e=uv, by whether u, v agree. Different colors give exactly G-colorings; equal colors biject with G/e-colorings. Thus $P_{G-e} = P_G + P_{G/e}$. Inducting on edges from edgeless polynomial k^n proves polynomiality. A simple graph's first terms are $k^n - ek^{n-1}$: each deletion retains leading one while its contraction subtracts leading k^{n-1} once per edge.

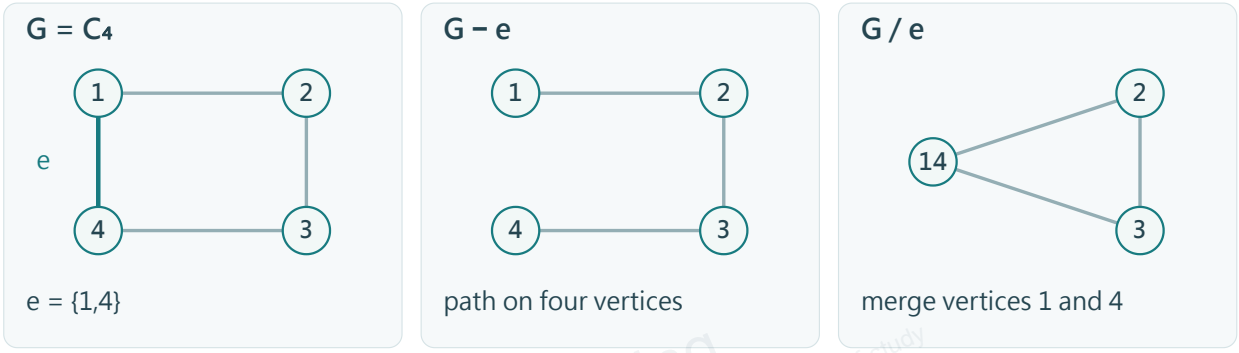
For C_4, deletion gives a four-vertex tree and contraction a triangle:

$$P_{C_4}(k) = k(k-1)^3 - k(k-1)(k-2) = k^4 - 4k^3 + 6k^2 - 3k.$$

Values at two and three are 2,18. Named-color swaps count as different colorings, unlike Chapter 16's symmetry orbits.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Deleting and contracting the same edge of C_4



Take $e = \{1, 4\}$. Deletion keeps all four vertices and removes only e. Contraction merges 1 and 4 and removes e; the remaining three edges form a triangle.

Thus $P_{C_4}(k) = k(k-1)^3 - k(k-1)(k-2)$. The subtraction removes exactly the colorings in which the formerly adjacent endpoints have equal colors.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Labeled graphs and components

On n fixed labels every unordered pair is an optional edge, giving $2^{\binom{n}{2}}$ graphs and $\binom{\binom{n}{2}}{m}$ with m edges. Do not divide uniformly by n!: automorphism counts differ.

Let c_n count connected labeled graphs, $c_1=1$. Classify uniquely by the size k of vertex one's component. Choose its k−1 other labels, a connected graph there, and any graph on the rest; cross edges are absent:

$$2^{\binom{n}{2}} = \sum_{k=1}^{n} \binom{n-1}{k-1} c_k 2^{\binom{n-k}{2}}.$$

Thus $c_2=2-1=1$, $c_3=8-(2+2)=4$, $c_4=64-(8+6+12)=38$. The subtracted terms at four correspond to k=1,2,3, leaving the connected k=4 case.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Inclusion–exclusion for no isolated vertices

If specified j vertices are isolated, every incident edge is forbidden; only edges among the remaining n−j vertices are free. Hence

$$N_n = \sum_{j=0}^{n} (-1)^j \binom{n}{j} 2^{\binom{n-j}{2}}.$$

At n−j=0 or 1 the binomial exponent is zero and one edgeless graph remains. At n=4, 64−4 • 8+6 • 2−4 • 1+1=41. This exceeds 38 connected graphs by three perfect matchings, each two disjoint edges. No isolates is substantially weaker than connectivity.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Boundaries and further exercises

Euler and Hamilton properties imply neither one another. Count the outer face. Isomorphism needs adjacency correspondence, not only degrees. Three-colorability does not require a triangle. Chromatic-polynomial coefficients may be negative; color choices are not independent. The exercise and solution PDFs give full derivations. Chapter 12 builds DFS trees and cut vertices; Chapter 13 adds weights, shortest paths, and flows to the same model.

References and attribution

Lin Cing, handwritten discrete mathematics notes. Ralph P. Grimaldi, Discrete and Combinatorial Mathematics, 5th ed., Ch. 11.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Equivalent definitions of a tree

For a finite nonempty simple undirected graph on $n \geq 1$ vertices, these are equivalent: connected and acyclic; connected with $n-1$ edges; acyclic with $n-1$ edges; exactly one simple path between every two vertices. A forest only requires acyclicity and may have several components. A single vertex has zero edges, and its unique self-path has length zero.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Proofs of the equivalences

An endpoint of a longest tree path is a leaf: an outside neighbor would extend the path, while another neighbor on the path would create a cycle. For $n \geq 2$, deleting a leaf and its edge preserves connected acyclicity. Induction from one vertex proves $e=n-1$. Applying this to every forest component gives $e=n-c$.

Any connected graph can have cycle edges deleted until a spanning tree remains, so it has at least $n-1$ edges. If it initially has exactly $n-1$, no deletion was possible and it was acyclic. An acyclic graph with $n-1$ edges has $c=n-e=1$, hence is connected.

A tree has a path by connectivity. Two distinct simple paths would yield a cycle between their first divergence and next common vertex, contradiction. Conversely, unique paths imply connectivity and exclude cycles, whose two directions would give distinct paths. This proves all equivalences without assuming the edge-count result in advance.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Leaf counts and boundary conventions

For $n \geq 2$, let n_i count degree-i vertices. Subtract $\Sigma n_i = n$ from $\Sigma i n_i = 2(n-1)$ to obtain

$$n_1 = 2 + \sum_{i \geq 3} (i-2) n_i.$$

Thus at least two leaves exist. Degree-two subdivision vertices do not change the leaf count. Two degree-three vertices and one degree-four vertex force $2+2+2=6$ leaves, regardless of degree-two vertices.

The singleton has no degree-one leaf, though it is a rooted leaf under the no-children definition. The formula assumes $n \geq 2$ and undirected degree-one leaves. The empty graph does not satisfy $e = n-1$; trees here are nonempty.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bridges and fundamental cycles

In a connected graph an edge is a bridge iff it belongs to no cycle. Consequently every tree edge is a bridge, and adding an edge to a tree creates exactly one fundamental cycle. A triangle with a dangling edge has both nonbridges and a bridge; being a tree also requires connectivity and every edge to be a bridge. Removing a bridge increases components; removing a nonbridge does not disconnect its component.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rooted trees, depth, and height

A root determines parent, children, and ancestors. Vertex depth is its root distance in edges; height is maximum leaf depth; level d contains depth-d nodes. An m-ary tree has at most m children per internal node; a full m-ary tree has exactly m. Binary trees also distinguish left and right. Fix whether singleton height is zero or one before using formulas; here root depth is zero.

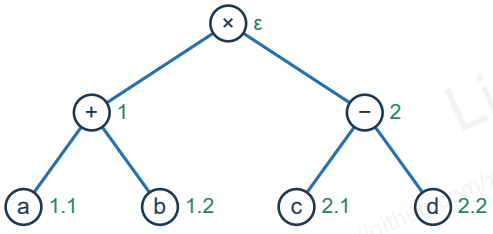
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Ordered trees and addresses

An ordered rooted tree additionally orders each node's children. Append child position i to its parent's address; the root is ε. Address 2.1 has parent 2 and ancestors ε,2; address length is depth. Address numbers are child positions, not key comparisons.

For (a+b)×(c−d), the root × has children +,− and ordered leaves a, b, c, d. Preorder visits root before children; postorder after children; binary inorder visits left, root, right. General ordered trees have no unique inorder convention.

One ordered binary tree: addresses,subtrees,and evaluation



Preorder : × + a b − c d

Parenthesized inorder: (a+b)×(c−d)

Postorder : a b + c d − ×

Substitute 2,3,9,4: result 25.

Root ε has depth 0; leaf addresses have length and depth 2.

Addresses, traversals, and evaluation

Node	×	+	a	b	−	c	d
Address	ε	1	1.1	1.2	2	2.1	2.2
Depth	0	1	2	2	1	2	2

Preorder `× + a b − c d` is prefix; postorder `a b + c d − ×` is postfix. Parenthesized inorder gives `(a+b)×(c−d)`. Removing parentheses yields `a+b×c−d`, generally a different expression; traversal alone does not preserve precedence.

With a=2,b=3,c=9,d=4, postorder computes 2+3=5,9−4=5,then 25. Each node has constant work and each edge is traversed at most twice, giving Θ(n) traversal time and O(h+1) recursion stack, worst-case O(n) in a skew tree.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Binary-tree counts

A full binary tree has exactly two children at every internal node and satisfies $L = I + 1$. With root depth zero and height at most h,

$$I \le 2^h - 1, \qquad L \le 2^h.$$

Both equalities hold only for a perfect tree with every level full and all leaves at depth h. A full tree need not be perfect: give the root one leaf child and one internal child with two leaves. Its height is 2, I=2, and L=3, not I=3 and L=4. Catalan numbers count shapes; applications such as Merkle trees add labels and ordering requirements.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Full, perfect, complete, and m-ary trees

Full means every internal node has exactly m children; perfect additionally puts all leaves at the same depth. Complete binary means all but the final level full and the last filled left to right. Some references call full binary trees complete; use the structural definition.

For full m-ary trees, m≥2, edge counting gives

$$mI = I + L - 1, \qquad L = (m-1)I + 1, \qquad n = mI + 1.$$

With at most m children and height h, level d has at most m^d nodes, total at most $(m^{h+1}-1)/(m-1)$, and leaves at most m^h. The leaf bound follows by height induction over at most m subtrees, each with at most m^{h-1} leaves. Equality requires full levels to one height, not mere fullness.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Binary shapes and Catalan convolution

Let C_n count full ordered binary shapes with n internal nodes; C_0=1 is a single leaf. For n≥1, the root uniquely leaves i internal nodes on the left and n−1−i on the right, so

$$C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i}, \qquad C_0, C_1, C_2, C_3 = 1, 1, 2, 5.$$

Chapter 10 gives $C_n = \binom{2n}{n}/(n+1)$. Encode internal nodes as (and leaves as) in preorder, deleting the final leaf symbol. This gives n balanced pairs: open child slots cannot be exhausted before the final node, ensuring nonnegative prefix balance. Conversely fill the next empty slot from each symbol and restore the final leaf, uniquely recovering the shape. This is a bijection, not just matching initial values. Swapping left/right usually changes the shape; removing order would make this product classification repeat objects.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Comparison-sorting decision trees

A comparison sort on distinct keys has a binary decision tree, with comparisons at internal nodes and output orders at leaves. It must distinguish n! orders. Height H permits at most 2^H leaves, so $H \geq \log_2(n!) = \Omega(n \log n)$. This is a comparison-model lower bound; counting and radix sorts use additional key assumptions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Worst-case and average comparison lower bounds

The estimate $n! \geq (n/2)^{n/2}$ already proves Ω(n log n) without Stirling. This lower bound concerns some input, not every input.

For N=n! equiprobable orders with leaf depths d_i, prefix-free leaf addresses give $\sum_i 2^{-d_i} \leq 1$. Arithmetic–geometric mean gives

$$2^{-\frac{1}{N}\sum_i d_i} \leq \frac{1}{N}\sum_i 2^{-d_i} \leq 1/N.$$

Thus average comparisons are at least $\log_2 N$ too. If comparisons offer equality but keys are distinct, that branch is unreachable and only two branches count. Nonuniform orders require their actual distribution's entropy bound, not the uniform result.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Prefix codes

A binary encoding C must first be **injective**: different source symbols receive different bit strings. Under this condition it is prefix-free if no codeword is a proper prefix of another. Equivalently, for distinct symbols a,b, C(a) is not a prefix of C(b), including equality.

Injectivity cannot be omitted. If a and b both receive 0, no two distinct codeword strings form a proper-prefix pair, yet receiving 0 does not identify the symbol. For a valid prefix code, store one unique symbol at each binary-trie leaf; read bits from the root until a leaf, output its symbol, and restart at the root. For frequencies f_i, mean length is $L = \sum_i f_i |w_i| / \sum_i f_i$. Prefix-freeness ensures immediate unique decoding but is not necessary for every uniquely decodable code.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Kraft's inequality and its converse

For finite prefix-code lengths ℓ_i and maximum H, a word at depth ℓ_i occupies $2^{H-\ell_i}$ descendant leaves at depth H. These sets are disjoint, so $\sum_i 2^{H-\ell_i} \le 2^H$ and $\sum_i 2^{-\ell_i} \le 1$.

Conversely sort nonnegative requested lengths. At depth ℓ, previous codewords cover $\sum_{j<i} 2^{\ell-\ell_j}$ nodes. The Kraft partial sum including the next request guarantees an available node. Select it outside earlier subtrees and forbid its descendants thereafter. This constructs the prefix code.

Lengths 1,2,3,3 sum to Kraft one and permit `0,10,110,111`. Lengths 1,1,2 sum to 5/4 and are impossible. Kraft concerns feasible lengths, not arbitrary proposed strings: `0,00` still conflict.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Huffman: merge the two smallest weights

For at least two symbols with positive frequencies f_i, a binary prefix tree has word lengths equal to leaf depths ℓ_i. Minimize weighted external path length $W = \sum_i f_i \ell_i$. With fixed frequencies, this also minimizes mean word length.

Start with one leaf per symbol. Repeatedly extract the two lightest trees from a min-priority queue, join them below a new parent, and reinsert their total weight. Stop at one tree and label each pair of child edges 0 and 1. An internal weight is a subtree frequency total, not a new source symbol.

Minimum height need not mean minimum W. Longer paths for rare symbols may buy shorter paths for frequent ones. Correctness needs two arguments: why may the two smallest weights be siblings, and why is solving the contracted problem sufficient?

Exchange lemma: moving a smaller weight deeper

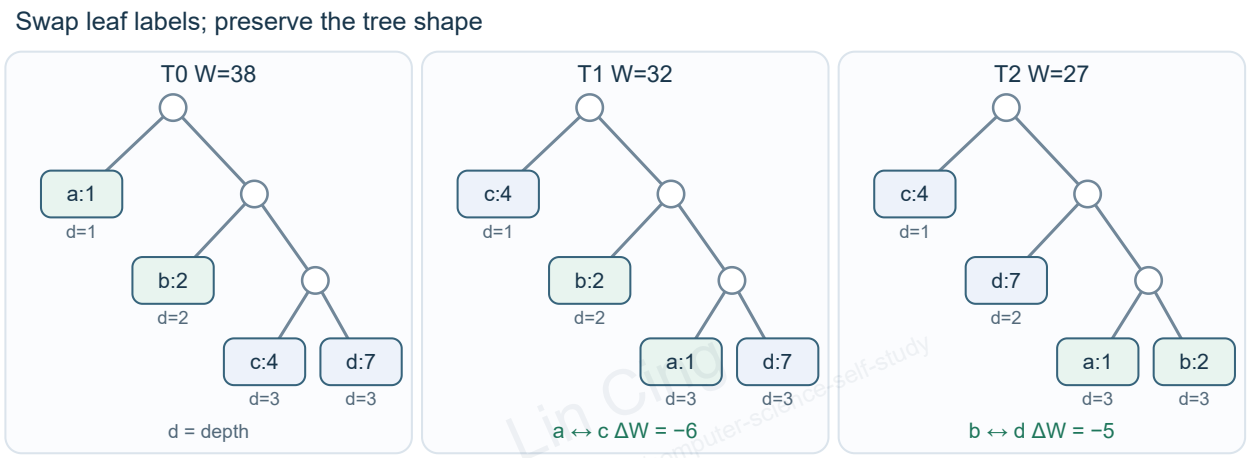
A unary internal node cannot occur in an optimum: contracting it decreases every descendant depth by one and strictly reduces positive-weight cost. Use a full binary tree. A deepest leaf at depth H has a leaf sibling, since an internal sibling would have a child deeper than H.

Let $f_x \le f_u$, with x initially at depth h and u at $H \ge h$. Swap only their leaf labels. Every other term stays fixed, so

$$\Delta W = (f_x H + f_u h) - (f_x h + f_u H) = (f_u - f_x)(h - H) \le 0.$$

Put the minimum-weight symbols x,y in those deepest sibling positions. If one already occupies a target position, keep it and fill the other. Otherwise place x first, then y without moving x. Every remaining symbol weighs at least f_y, so both swaps satisfy the inequality. Starting from an optimum forces equality: a smaller cost would contradict optimality. Some optimum accommodates the chosen merge; optimal trees need not all coincide.

A visible exchange: 38 → 32 → 27



This deliberately starts from a nonoptimal cost-38 tree. Only leaf labels move; edges and leaf positions stay fixed. The minimum weights a,b end as siblings at depth three, yet cost 27 exceeds Huffman's 24. The exchange lemma guarantees nonincrease; two exchanges do not optimize an arbitrary tree.

Which two contributions change in each swap?

Fix frequencies a:1, b:2, c:4, d:7. Each entry gives depth and its frequency-times-depth cost.

Symbol	Original T_0	Swap a,c: T_1	Swap b,d: T_2
a:1	Depth 1; cost 1	Depth 3; cost 3	Depth 3; cost 3
b:2	Depth 2; cost 4	Depth 2; cost 4	Depth 3; cost 6
c:4	Depth 3; cost 12	Depth 1; cost 4	Depth 1; cost 4
d:7	Depth 3; cost 21	Depth 3; cost 21	Depth 2; cost 14
W	38	32	27

First a pays two more and c pays eight less: $\Delta W = (4-1)(1-3) = -6$. Next b pays two more and d pays seven less: $\Delta W = (7-2)(2-3) = -5$. Equal weights or equal depths give zero difference. Distinct symbols must still receive distinct words.

Contracting siblings removes one final level

Replace sibling leaves x,y and their parent by leaf z of weight $f_z = f_x + f_y$. If z has depth k in the contracted tree U, expansion places x,y at k+1. All other leaf depths stay fixed.

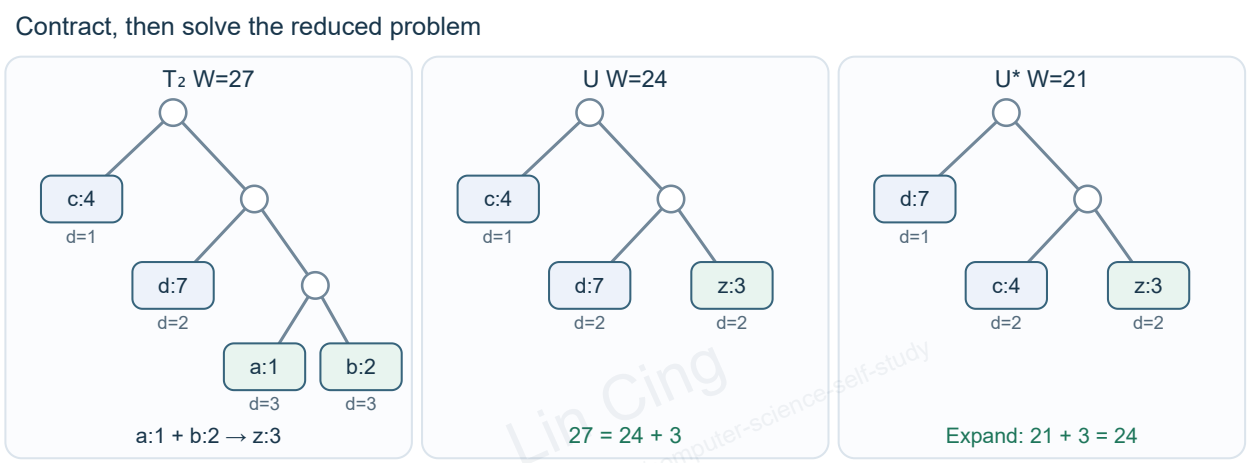
$$f_x(k+1) + f_y(k+1) = (f_x + f_y)k + f_x + f_y.$$

Every tree T with sibling x,y satisfies $W(T) = W(U) + f_x + f_y$. Conversely any legal reduced-problem tree U expands at z into a legal original prefix tree, adding exactly this constant.

If T is optimal, U must be optimal too. Otherwise replace U by a cheaper U* and expand the same pair: the original cost becomes smaller, contradicting optimality of T. This establishes optimal substructure rather than assuming local choices solve the whole problem.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Contraction example: 27 − 3 = 24, then improve



Contracting depth-three a,b replaces cost 9 by 6, so U costs 27−3=24. This is reduced cost, not original cost. For weights 3,4,7, depths 2,2,1 give W(U*)=21. Expanding z recovers cost 21+3=24 and depths 3,3,2,1 for a,b,c,d. Reoptimizing U supplies the improvement missing after the swaps.

Finish the optimality proof by induction

Induct on symbol count n with positive weights. A singleton may use the empty word at cost zero; two symbols use one bit each at cost their weight sum. Assume Huffman is optimal for all smaller alphabets.

Merge chosen minimum-weight x,y into z. The exchange lemma gives some original optimum T* with sibling x,y. Its contraction U* is optimal for the reduced alphabet by the replacement argument. Consequently,

$$\mathrm{OPT}(C) = \mathrm{OPT}(C') + f_x + f_y, \qquad C' = (C \setminus \{x, y\}) \cup \{z\}.$$

Huffman continues on C', attaining $\mathrm{OPT}(C')$ by induction. Expansion at z attains the original optimum. Each merge reduces the symbol count by one. Tied minimum weights and swapped 0/1 edges are allowed; the theorem neither promises unique codewords nor imposes a fixed alphabetic leaf order.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Huffman example with weights 1,2,4,7

Merge 1+2=3,then 3+4=7,then 7+7=14. The weighted external path length is 3+7+14=24. One code assigns frequency 1:`110`,2:`111`,4:`10`,7:`0`, giving average 24/14. No prefix code has lower weighted cost. Edge labels may be swapped if the decoder uses the same trie.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Huffman cost model and boundary cases

Assume positive frequencies and objective $W=\Sigma f_i \ell_i$. An optimal tree has no unary internal node: contraction decreases every positive-weight descendant depth and strictly improves W. With at least two symbols, full binary trees therefore suffice.

A min-priority queue performs n−1 merges in O(n log n) time and O(n) space. Sorted frequencies permit two nondecreasing queues, one for original leaves and one for new nodes, selecting each minimum from their heads in O(n) merge time. Include sorting if input is unsorted.

One symbol may use ε with cost zero if its count is separately known; a format requiring nonempty words instead uses 0 at cost f. Zero-frequency symbols do not affect the objective; remove them or specify required encoding, since the strict-improvement proof uses positivity.

Why merge sums equal external path length

Leaf i contributes f_i to each ancestor's subtree weight, once per depth step. Exchanging finite sums gives

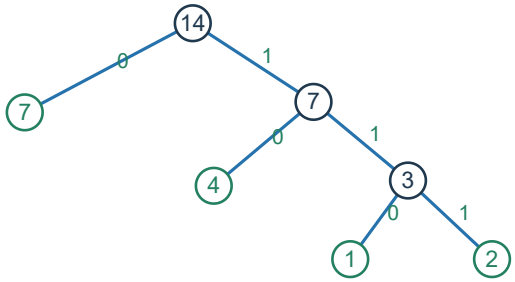
$$\sum_{i} f_i \ell_i = \sum_{\text{internal } v} \text{weight}(v).$$

Round	Extract	Insert	Remaining queue
1	1,2	3	3,4,7
2	3,4	7	7,7
3	7,7	14	14

Depths 3,3,2,1 give $1 \cdot 3+2 \cdot 3+4 \cdot 2+7 \cdot 1=24=3+7+14$. Mean $24/14 \approx 1.7143$ bits is below fixed two-bit coding, excluding codebook storage.

The actual prefix-code tree

Huffman weights,codewords,and weighted cost



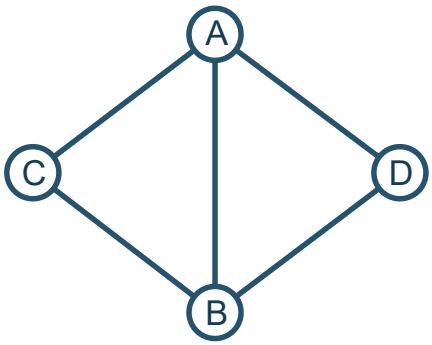
Weight	code	Length	Cost
7	0	1	7
4	10	2	8
1	110	3	3
2	111	3	6

Total cost 24; mean length 24/14 bits.

Internal nodes hold subtree frequency totals; edges hold bits; only leaves represent symbols. A root-to-leaf path spells a codeword. An internal weight seven and a leaf frequency seven may coincide numerically, but treating the internal node as a source symbol would make its code a prefix of descendant codes.

Counting spanning trees with the Laplacian

Consider K_4 after deleting CD. How many spanning trees include all four vertices?



Five edges

AB, AC, AD, BC, BD

Degrees A, B, C, D: (3, 3, 2, 2)

CD is absent

The matrix–tree theorem and incidence matrix

For a finite simple undirected graph with $n \geq 1$ vertices, orient each edge arbitrarily. In its incidence matrix B, an edge column has -1 at its tail, $+1$ at its head, and zero elsewhere. Its Laplacian is

$$L = BB^{\mathsf{T}} = D - A.$$

Matrix–tree theorem: deleting row and column r of L gives $L^{(r)}$, and $\det L^{(r)}$ equals the number $\tau(G)$ of spanning trees. A disconnected graph gives zero; for one vertex the empty determinant is 1.

The orientations only set signs in B: reversing an edge negates one column and leaves BB^{T} unchanged.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why does the cofactor count trees?

Delete row r from B to obtain B_r, so $L^{(r)} = B_r B_r^{\mathsf{T}}$. Cauchy–Binet sums over sets S of $n-1$ edges:

$$\det L^{(r)} = \sum_{|S|=n-1} \det(B_r[:, S])^2.$$

If S contains a cycle, signed incidence columns around that cycle sum to zero, so its determinant vanishes. If S is an acyclic disconnected forest, its $n - c$ edges cannot equal $n - 1$. Thus only spanning trees can contribute.

For a tree, repeatedly choose a leaf other than r. Its row has just one nonzero entry, ± 1; expand on that row and remove its edge. The remaining graph is a tree, ending with empty determinant 1. Every tree therefore contributes $(\pm 1)^2 = 1$, so the sum counts all spanning trees.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Worked example: K_4 without CD

In vertex order A, B, C, D, the five edges are AB, AC, AD, BC, BD. Their degrees are $3, 3, 2, 2$, giving

$$L = \begin{pmatrix} 3 & -1 & -1 & -1 \\ -1 & 3 & -1 & -1 \\ -1 & -1 & 2 & 0 \\ -1 & -1 & 0 & 2 \end{pmatrix}, \qquad L^{(D)} = \begin{pmatrix} 3 & -1 & -1 \\ -1 & 3 & -1 \\ -1 & -1 & 2 \end{pmatrix}.$$

Expanding along the first row gives three signed contributions:

$$3(6-1) \;-\; (-1)(-2-1) \;+\; (-1)(1+3) = 15 - 3 - 4 = \boxed{8}.$$

Thus this five-edge graph has 8 spanning trees; it is not the complete K_4.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Search-tree sorting and its limits

With strict BST invariant left keys<key<right keys, inorder outputs sorted keys; duplicate handling needs its own policy. Sorted insertion without balancing can produce height n−1 and Θ(n) worst-case search. Balanced trees maintain Θ(log n) height through valid rotations and invariants, not visual appearance. Decision-tree lower bounds do not imply that every tree-based sorting algorithm attains optimum.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

DFS and BFS spanning forests

In each component choose a root and retain the first-discovery edge for every new vertex. Each nonroot has one parent; discovery time strictly increases along parent edges, preventing cycles, and every discovered vertex reaches the root. The result is a spanning forest, or a tree when connected.

DFS finishes one neighbor subtree recursively before the next. BFS uses a FIFO queue and nondecreasing distances. Setting d[w]=d[v]+1 on discovery is shortest: a shorter path's penultimate vertex would leave the queue earlier and discover w earlier, contradiction. DFS depth generally is not shortest distance.

For edges AB, BC, CA, CD, DE, EF, FD, DG with alphabetic neighbors:

Method	Discovery order	Tree edges
DFS	A, B, C, D, E, F, G	AB, BC, CD, DE, EF, DG
BFS	A, B, C, D, E, F, G	AB, AC, CD, DE, DF, DG

Equal discovery orders do not imply equal trees. C has BFS depth one and DFS depth two. Adjacency lists cost $O(n+m)$; matrices scan n entries per vertex and cost $\Theta(n^2)$ in the worst case.

Undirected DFS has no sibling cross edges

For a nontree edge xy, assume x is discovered first. If y is undiscovered before x returns, scanning xy puts y in x's subtree. If y was already discovered, it must be an active ancestor: a completed subtree would previously have scanned yx and discovered x, contradicting the later discovery. Thus every nontree edge joins ancestor and descendant.

This underlies low-link proofs. Separate child subtrees can connect around their common parent only through ancestor back edges, not arbitrary cross edges. Directed DFS may have cross edges; these undirected formulas do not directly solve directed strongly connected components.

Merge invariant and complete example

For sorted L, R, keep cursors i, j and initially empty output O. Repeatedly output the smaller remaining head and advance that cursor. Choose the left head on equality for stability. When a side empties, append the other remainder.

Invariant: O is sorted and equals the multiset of removed items, and all remaining items are at least its last item. The smaller head is the minimum of everything remaining, so each step preserves the invariant. Remaining length decreases by one, proving termination and a complete sorted merge.

Merging [2,3,6,8] and [1,4,5,7] outputs 1,2,3,4,5,6,7 by seven comparisons and appends eight. There is no repeated scan of the remainder. Positive lengths p, q require at most p+q−1 comparisons; an initially empty side requires zero.

Merge sort and comparison counts

Split into halves differing by at most one, sort them recursively, then merge. Length zero/one returns immediately. Induction on length gives sorted halves and the merge lemma gives correct complete output.

Stage	Data state	Comparisons
Input	[8,3,6,2,7,1,5,4]	0
Merge singletons	[3,8],[2,6],[1,7],[4,5]	4
Merge pairs	[2,3,6,8],[1,4,5,7]	3+3=6
Final merge	[1,2,3,4,5,6,7,8]	7

Total seventeen. For $n=2^h$, depth-d merges require at most $n-2^d$ comparisons, with root d=0. Thus $M(n) \le \sum_{d=0}^{h-1}(n-2^d) = n\log_2 n - n + 1$, attained here at n=8. General n has depth $\lceil \log_2 n \rceil$ and O(n) work per level, yielding O(n log n); the comparison lower bound makes worst-case Θ(n log n). A reused temporary array uses O(n) extra space; retaining historical copies at every level is a different model.

Articulation points and low values

A cut vertex increases component count on deletion. For DFS discovery times,

$$low[v] = \min\big(disc[v], \min_{\text{back edge } v \to u} disc[u], \min_{\text{tree child } w} low[w]\big).$$

The root is a cut vertex iff it has at least two DFS children. A nonroot v is a cut vertex iff some child w has low[w]≥disc[v]. Exclude the parent edge from back edges.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two different low-link inequalities

For nonroot v, low[w]≥disc[v] means child subtree w has no edge to a proper ancestor of v. With no sibling cross edges, deleting v separates it from the parent side. Conversely, if every child has low[w]<disc[v], each reaches an ancestor and the parent side without v; all remain connected, so v is not a cut vertex.

The root has no parent side. Two or more children separate on its deletion because no cross edges connect them. One child contains all remaining vertices connected by its DFS tree; deleting the root does not increase components. An isolated root's deletion decreases components and is not a cut.

A tree edge vw is a bridge iff low[w]>disc[v]. Equality allows a back edge to v, bypassing vw but not v. Thus the edge need not be a bridge while v may be a cut. The strict and nonstrict inequalities cannot be exchanged.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

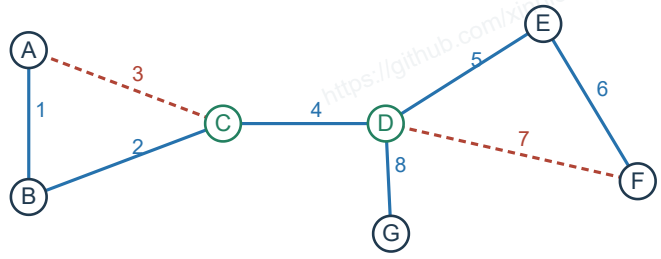
General-graph low values and block boundaries

Use IDs 1:AB,2:BC,3:CA,4:CD,5:DE,6:EF,7:FD,8:DG and alphabetic DFS from A.

Vertex	A	B	C	D	E	F	G
disc	1	2	3	4	5	6	7
final low	1	1	1	4	4	4	7
parent	None	A	B	C	D	E	D

CA sets low[C]=1 and propagates to B; FD sets low[F]=4 and propagates to E. G has no back edge and low[G]=7. Cuts are C, D; bridges CD, DG. Root A has one child, so even a child's low≥disc[A] does not make it a cut.

Eight-edge DFS: two triangles,two bridges,and low values



Vertex	A	B	C	D	E	F	G
disc	1	2	3	4	5	6	7
low	1	1	1	4	4	4	7

Cuts: C,D; A has one child.

Bridges: 4=CD,8=DG.

Dashed back edges: 3=CA,7=FD.

Actual edge stacks and all four pops

Stacks list IDs bottom to top. Push a back edge only from descendant to ancestor, once.

Event	Stack before test or after push	Pop order	Remainder
Enter E subtree and discover FD	1,2,3,4,5,6,7	None	Unchanged
E returns to D: 4≥4	1,2,3,4,5,6,7	7,6,5	1,2,3,4
Push DG; G returns: 7≥4	1,2,3,4,8	8	1,2,3,4
D returns to C: 4≥3	1,2,3,4	4	1,2,3
C returns to B: 1<2	1,2,3	None	1,2,3
B returns to A: 1≥1	1,2,3	3,2,1	Empty

Blocks are {DE, EF, FD},{DG},{CD},{AB, BC, CA}. Every pop includes its triggering tree edge. Even a root not marked as a cut must emit a block when the low condition holds.

Before, during, and after one pop

Return E→D: pop through DE,retain earlier pending edges

Before: low[E]=4,disc[D]=4

Stack bottom Top

Action: pop e7,e6,e5

Stop after FD,EF,DE.

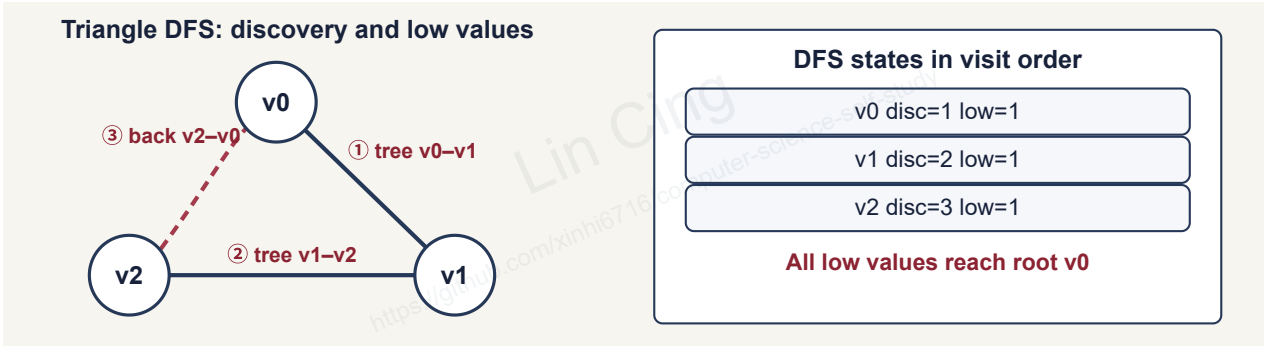
After: emit complete DEF block

The trigger is e_5=DE, so stop only after including e_5. Popping only top e_7 would split the triangle; also popping e_4 would wrongly include the bridge to earlier vertices. This operation emits a completed block from pending edges without modifying the graph.

Vertex-biconnected blocks

A block is a maximal connected subgraph with no articulation point, with maximality under subgraph inclusion. Connectivity excludes disconnected cut-free sets. This chapter treats a bridge plus endpoints K_2 as a block and an isolated K_1 as a singleton block.

The DFS edge stack receives tree edges and back edges to ancestors. On returning along tree edge vw with low[w]≥disc[v], pop a contiguous suffix through and including vw. The whole group is one block; one popped edge alone is insufficient.



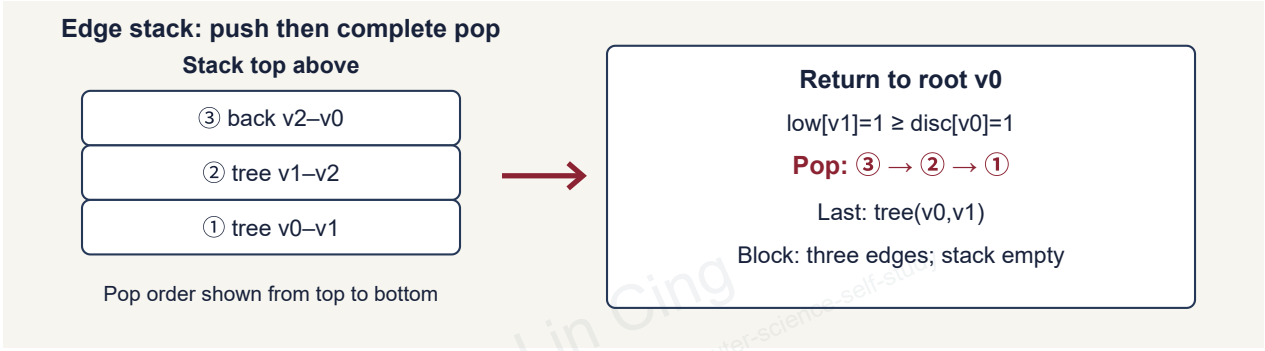
Full stack trace for one triangle

DFS visits v_0,v_1,v_2 then the back edge v_2v_0.

Step	Action	disc	low	Stack	Pop
1	Visit v_0	v_0:1	v_0:1	Empty	None
2	Tree v_0v_1	v_1:2	v_1:2	(v_0,v_1)	None
3	Tree v_1v_2	v_2:3	v_2:3	$(v_0,v_1),(v_1,v_2)$	None
4	Back v_2v_0	Unchanged	v_2:min(3,1)=1	Append (v_2,v_0)	None
5	Return to v_1	Unchanged	v_1:min(2,1)=1	Unchanged	None: 1<2
6	Return to v_0	Unchanged	v_0:1	Unchanged	All three: 1≥1

Stack suffixes and block boundaries

Step six pops $(v_2,v_0),(v_1,v_2),(v_0,v_1)$, ending at the triggering tree edge. These form the full block and empty the stack.



An unfinished subtree owns a continuous stack suffix. If low[w]<disc[v], its back edge reaches a proper ancestor and the suffix remains with the outer block. If low[w]≥disc[v], no such edge exists and deletion of v separates this subtree from the outer part. Popping through vw takes exactly its block's pending edges and excludes already completed blocks. This boundary argument applies at arbitrary depth; the triangle is one trace. Bridges are one-edge blocks; singleton conventions must be explicit. Store edges, not merely vertices, to split blocks at cuts correctly.

Complete DFS block decomposition

Use edge IDs, allowing parallel edges to distinguish the parent edge from another edge to the same parent. The chapter's base input is finite undirected simple graphs without loops.

Start DFS at every unvisited vertex. Emit an isolated vertex separately as a singleton block. Popping cannot depend on cut[v] being true: a one-child root must still emit its final block.

On each return, low values determine articulation points, bridges, and block boundaries; the following procedure implements those rules edge by edge.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

DFS: scan edges and recurse

The disc, low, cut arrays and edge stack are shared across calls; `finish_child` handles one completed child subtree.

```
DFS(v, parent_edge):
    disc[v]=low[v]=++clock; children=0
    for (w,e) in adjacency[v]:
        if e == parent_edge: continue
        if disc[w] is undefined:
            children++; stack.push(e); DFS(w,e)
            finish_child(v,w,e,parent_edge)
        else if disc[w] < disc[v]:
            stack.push(e); low[v]=min(low[v],disc[w])
    if parent_edge is absent: cut[v]=(children>=2)
```

Only the descendant pushes a back edge. Edge IDs preserve a parallel edge to the parent.

Child return: bridges and block boundaries

`finish_child(v,w,e,parent_edge)` receives the just-completed tree edge e=vw; the stack is shared throughout DFS.

```
finish_child(v,w,e,parent_edge):
    low[v]=min(low[v],low[w])
    if low[w] >= disc[v]:
        if parent_edge exists: cut[v]=true
        B=[]
        repeat: z=stack.pop(); B.append(z)
        until z==e
        emit B
    if low[w] > disc[v]: emit bridge e
```

The pop includes e. The root's child count determines whether it is a cut vertex, independently of block output.

Why the emitted blocks are maximal

The stack contains discovered edges not yet assigned to completed blocks. A subtree's pending edges form a suffix; already completed internal blocks have been removed. At low[w]≥disc[v], subtree w reaches the earlier graph only through v. Any connected larger subgraph crossing that boundary would separate on deleting v, so cannot be a larger cut-free block. At low[w]<disc[v], an ancestor bypass exists and the upper boundary has not yet been reached, so retain the suffix.

An emitted suffix has no internal cut z. If z≠v separated a child subtree from the rest, the corresponding low≥disc[z] event would have popped those edges earlier. They could not remain in this suffix. For z=v the emitted set contains one child subtree plus v, and deleting v leaves that subtree's DFS tree connected. Hence the emitted set is cut-free and cannot merge outward, proving maximality. Bridge singletons follow the stated convention.

Each edge is pushed and popped once and each adjacency entry scanned once. Time is O(n+m) and stack/array space O(n+m), including all emitted block edges rather than only the cut set.

Block-cut trees and shared vertices

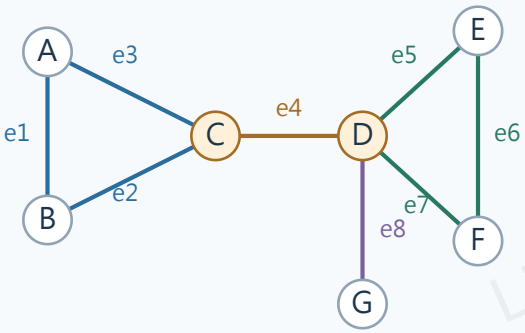
Create one node per block and one per articulation, with incidence edges for membership. For a connected graph this bipartite graph is connected. A cycle would provide bypasses around every supposed cut on it, permitting adjacent blocks to merge into a larger cut-free subgraph, contradicting maximality. Thus it is a tree; disconnected graphs give a forest.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

From the original graph to its block-cut tree

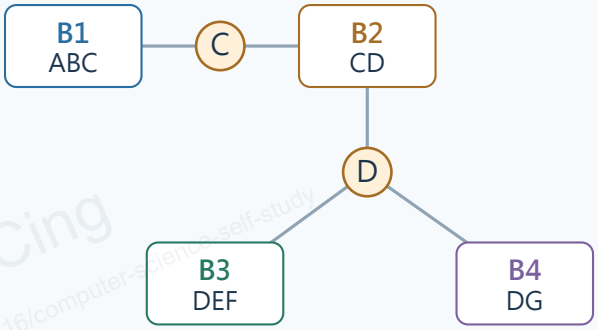
Rectangles represent whole blocks; orange circles represent articulation vertices.

Original graph: four edge blocks



C, D: articulation vertices

Block-cut tree: membership links



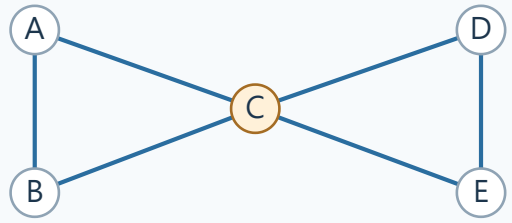
6 nodes · 5 links · no cycle

Membership links: B_1–C–B_2–D, D–B_3, D–B_4. C belongs to two blocks, D to three. Each original edge belongs to exactly one block.

No bridges does not imply no articulation

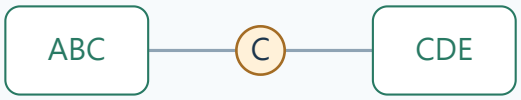
The triangles share C; compare the effects of deleting an edge and deleting a vertex.

Every edge lies on a cycle



No bridges; C separates AB from DE

Two vertex blocks; one cut



Delete all bridges: still one edge component.

Split at articulation C: two vertex blocks.

The two decompositions answer different questions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Compare edge and vertex decompositions

Deleting any one edge leaves its endpoints connected around their triangle. Deleting C instead leaves the two disjoint edges AB and DE. Therefore C is an articulation even though no edge is a bridge. Bridge deletion leaves one edge-biconnected component; vertex-block decomposition gives ABC and CDE, sharing C.

A bridge tests whether one edge can be bypassed. An articulation point tests whether all paths between two remaining regions must use one vertex; these are different tests.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Boundaries and checks

Finite connected simple graphs with n−1 edges are already acyclic. Prefix-free words need not have equal lengths. Huffman optimizes within fixed-frequency prefix codes; other compression families require separate comparison. DFS roots use child counts. Block-cut trees contain block and cut nodes rather than treating every original edge as a block. Exercises and solutions give further derivations. These invariants and cost models support later spanning-tree, flow, and indexing methods.

References and attribution

Lin Cing, handwritten discrete mathematics notes. Ralph P. Grimaldi, Discrete and Combinatorial Mathematics, 5th ed., Ch. 12.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Weighted graphs and shortest paths

In a finite directed weighted graph, path cost sums edge weights. If a source-reachable negative cycle can also reach v, arbitrarily many repetitions make walk cost to v unbounded below. Vertices unreachable from that cycle are not affected. Dijkstra's general correctness premise is nonnegative weights on all source-reachable edges.

Tentative d[v] is an upper bound and π[v] a predecessor. Initialize d[s]=0,others infinity, and predecessors empty. DAGs allow topological relaxation; general negative weights without reachable negative cycles require Bellman–Ford.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Dijkstra's invariant and proof

Extract the unfinalized minimum d[u]. Finalized distances are exact; finite tentative values have realizing source paths, and unreachable vertices retain infinity and no predecessor. Relax u→v by $d[v] = \min(d[v], d[u] + w(u, v))$.

Suppose dist(s, u)<d[u]. On a shortest path, take its first unfinalized x with finalized predecessor y. Earlier relaxation gave d[x]≤dist(s, y)+w(y, x)=dist(s, x). Nonnegativity implies dist(s, x)≤dist(s, u)<d[u],contradicting minimum extraction. If x=s, initial d[s]=0 already prevents the error.

Arrays cost $O(n^2+m)$. A binary heap costs O((n+m)log n). A Fibonacci heap has O(n) initialization/insertion, n amortized O(log n) extractions, m successful amortized O(1) decrease-keys, and O(m) relaxations, for O(m+n log n).

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete Dijkstra trace

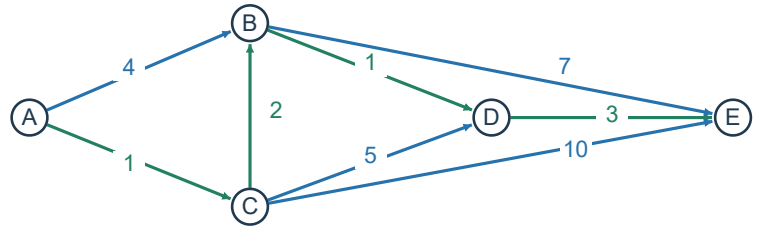
Use A→B:4,A→C:1,C→B:2,B→D:1,C→D:5,D→E:3,C→E:10,B→E:7,source A.

Finalized this round	A	B	C	D	E	Improved predecessors
Initial	0	∞	∞	∞	∞	None
A	0	4	1	∞	∞	B←A, C←A
C	0	3	1	6	11	B←C, D←C, E←C
B	0	3	1	4	10	D←B, E←B
D	0	3	1	4	7	E←D
E	0	3	1	4	7	None

Backtrack E←D←B←C←A to obtain A–C–B–D–E, cost 1+2+1+3=7. Update only on strict improvement. Lazy heaps with duplicate keys must skip stale entries rather than finalize them again.

Edge weights differ from total distances

Dijkstra: arc weights,final distances,and predecessor tree



Finalized: A,C,B,D,E

d(A,B,C,D,E) = (0,3,1,4,7)

Path to E:

A → C → B → D → E

Cost 1+2+1+3 = 7

Arc labels are weights; green arcs form the predecessor tree. The right table gives distances.

C→D has weight five, but d[D]=4 because C→B→D costs three plus A→C's one. An individual arc weight is not the entire distance to its destination.

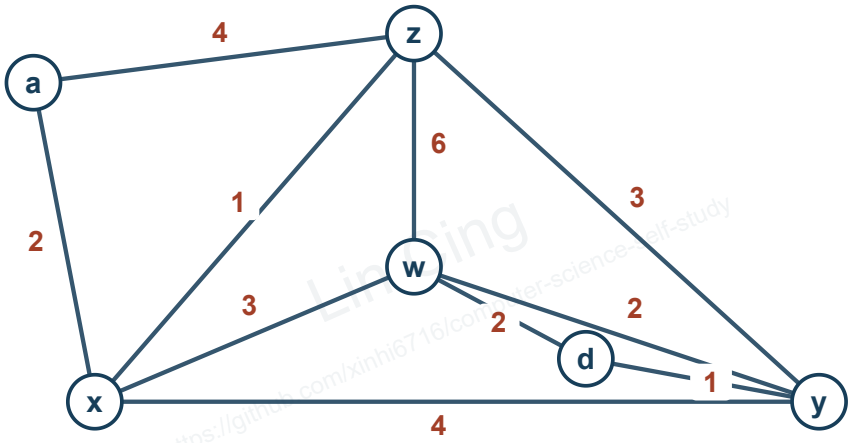
Distance certificates and negative edges

A certificate has d[s]=0,inequality d[v]≤d[u]+w(u, v) for every reachable arc, and a predecessor path realizing each finite d[v]. Applying inequalities along any source path makes d[v] no larger than its cost; the realizing path attains that bound, proving exactness. Graph search separately certifies unreachable vertices. Extracting infinity permits stopping. Do not change finalized predecessors, avoiding unnecessary zero-weight parent cycles.

For s→a:2,s→b:1,a→b:−2,there is no negative cycle, but finalizing b=1 misses true distance zero. Bellman–Ford's round k, reading previous-round distances, computes minimum walk cost with at most k edges. Without reachable negative cycles, a shortest walk removes nonnegative cycles to a path of at most n−1 edges. Thus n−1 rounds suffice; improvement in round n identifies a reachable negative cycle. Search its downstream vertices to identify affected distances, rather than declaring every vertex unbounded.

One graph: shortest paths and a minimum spanning tree

The graph is finite, undirected, and has the nonnegative edge weights shown. Run Dijkstra from a.



Dijkstra: every round and relaxation

Settle equal-distance vertices in alphabetical order. Update a predecessor **only for a strict improvement**; retain it on equal distance. Kruskal processes equal-weight edges in alphabetical order of their endpoints.

Distances are listed in the fixed order a, x, z, w, y, d. An unsuccessful relaxation is still checked.

Settled	Attempts to unsettled vertices	Distances (a, x, z, w, y, d) afterward
a	x:2, z:4	(0,2,4,∞,∞,∞)
x	z:3<4, w:5, y:6	(0,2,3,5,6,∞)
z	w:9>5, y:6=6	(0,2,3,5,6,∞)
w	y:7>6, d:7	(0,2,3,5,6,7)
y	d:7=7	(0,2,3,5,6,7)
d	No unsettled neighbor	(0,2,3,5,6,7)

Kruskal: merge distinct components

For equal weights, order edges alphabetically by their sorted endpoints. Stop after selecting five edges.

Edge	Weight	Decision and reason
dy	1	Select; merge d and y
xz	1	Select; merge x and z
ax	2	Select; add a to the xz component
dw	2	Select; add w to the dy component
wy	2	Skip; w and y are already connected
wx	3	Select; merge the two components to finish

The selected set $\{dy, xz, ax, dw, wx\}$ has total weight $1 + 1 + 2 + 2 + 3 = 9$. Each selected edge crosses two current components and is minimum-weight across that component's cut in the sorted order, so the cut property certifies its safety.

Compare objectives and certificates

The Dijkstra predecessor tree is $\{ax, xz, xw, xy, wd\}$, with total edge weight $2 + 1 + 3 + 4 + 2 = 12$. Its distances from a, in the stated order, are $0, 2, 3, 5, 6, 7$. The Kruskal tree weighs 9, but its path $a \to x \to w \to d \to y$ has length $2 + 3 + 2 + 1 = 8$, greater than the shortest distance 6.

Strict improvements give predecessors $x \leftarrow a, z \leftarrow x, w \leftarrow x, y \leftarrow x, d \leftarrow w$; settlement order is a, x, z, w, y, d. For instance, $a \to x \to z \to y$ and $a \to x \to y$ both have length 6, so y retains predecessor x under the tie rule.

Both are spanning trees. A shortest-path tree minimizes each root-to-vertex distance; a minimum spanning tree minimizes the **sum of selected edge weights**. Ties can produce other valid trees, so this tie-breaking result is not claimed to be unique.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Kruskal and the cut property

Sort undirected edges by weight and accept an edge exactly when it creates no cycle. For any nontrivial cut, each lightest crossing edge belongs to some MST. If an MST T omits such e, the unique T-path between e's endpoints includes a crossing f. Adding e makes a unique cycle; delete that f to preserve a tree. Since w(f)≥w(e),the replacement has no greater total weight and hence is also minimum.

For AB:2,BC:1,CA:4 plus CD:3,Kruskal takes BC, AB, CD, total six. Ties may permit multiple MSTs with the same minimum total.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Prim's invariant and the cycle property

Prim grows one tree by choosing the lightest edge across its current vertex cut. Its accepted edges remain contained in some MST. Kruskal maintains the analogous extendable forest invariant.

A uniquely heaviest edge on a cycle belongs to no MST: delete it from a proposed MST, and replace it across that cut by a lighter cycle edge, strictly improving weight. If the maximum ties, some MST omits a specified heaviest edge, but not necessarily every MST. Disconnected graphs have no spanning tree; detect connectivity or output a minimum spanning forest.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Flow-network model

A finite directed network has distinct source s, sink t, and capacities $c(e) \geq 0$. Each original arc has nonnegative flow $0 \leq f(e) \leq c(e)$. Every internal vertex conserves inflow/outflow. Flow value is source outflow minus source inflow.

An undirected connection modeled as two directed arcs differs from a shared capacity; specify which. Antiparallel original arcs remain distinct records in residual construction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Residual records and augmentation

For each original ID e=(u, v),create forward e^+:u→v of capacity c(e)−f(e),and reverse e^-:v→u of capacity f(e). The latter permits cancellation of existing flow.

An augmenting s-to-t residual path with positive bottleneck Δ increases original flow on forward records and decreases it on reverse records, preserving conservation and capacity limits. Same-direction records may have different origins and must not overwrite one another. Implementations normally store paired residual capacities and update both. A net-flow convention requires its own conversion. Residual paths represent possible adjustments, not necessarily original directed transport paths.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Max-flow min-cut

A cut partitions V into S, T with s∈S, t∈T, and counts only original S→T capacities. Summing conservation over S cancels internal arcs:

$$|f| = \sum_{S\to T} f(e) - \sum_{T\to S} f(e) \le \sum_{S\to T} c(e) = c(S,T).$$

If no augmenting path remains, take S as residual-source-reachable vertices. Then t∉S. Every original S→T arc has zero forward residual and is saturated. Every original T→S arc has zero flow, because its reverse residual would otherwise cross from S to T. Hence |f|=c(S, T),certifying both optimums. The forward residual of an original T→S arc may remain positive; do not say all reverse-crossing records vanish.

Integer capacities and integer bottlenecks from zero flow raise value by at least one per augmentation, with a finite cut bound, so terminate at an integral optimum. Arbitrary real capacities with arbitrary path selection need not terminate in finitely many steps. Edmonds–Karp's shortest-edge-count paths give polynomially many augmentations. Minimum cuts may differ; maximum value is unique.

Bipartite matching and Hall's theorem

A matching in $G=(L \cup R, E)$ has no shared endpoints. Saturating L covers every left vertex; perfect means every vertex on both sides. If $|L|<|R|$,saturating L is not perfect. $N(S)$ is the full right neighborhood of $S \subseteq L$.

Hall's theorem gives a matching saturating L iff $|N(S)| \geq |S|$ for every $S \subseteq L$. Necessity follows because matched neighbors are distinct. For sufficiency, take maximum M and search from all unmatched left vertices along nonmatching $L \to R$ and matching $R \to L$ edges. Reaching an unmatched right vertex would augment, contradiction. Otherwise reachable sets S, T have $N(S)=T$, each T vertex matched into S, and S additionally contains an unmatched starting point. Thus $|N(S)|=|T|<|S|$,contradicting Hall. This is a global subset condition; algorithms search alternating paths instead of enumerating subsets.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Matching as unit-capacity flow

Connect s to every left vertex, orient original edges $L \to R$, and connect each right vertex to t, all capacity one. An integral maximum flow selects a matching, with matching size equal to value. Kőnig equates maximum bipartite matching and minimum vertex-cover sizes. Weighted assignment additionally needs cost optimization, not merely cardinality. Integrality is guaranteed for integer-capacity optima; arbitrary real feasible flows need not be integral.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Common modeling errors

Negative edges invalidate Dijkstra's general theorem, as the s, a, b example shows. Omitting Kruskal's cycle test can produce a cycle and disconnection even after n−1 choices. Cut capacity includes forward capacities only. Singleton-neighborhood checks do not establish Hall. Maximum matching can leave unmatched vertices, and perfect requires both sides. Model assumptions determine which greedy argument applies.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Safe cuts must respect the selected forest

Assume F lies in some MST. A cut respects F when no F-edge crosses it. For its lightest crossing e, take an MST T containing F. If $e \notin T$, adding it creates a cycle with another crossing f. Respect implies $f \notin F$, so replacing f by e preserves all F edges and minimum cost. Thus $F \cup \{e\}$ remains extendable.

For Kruskal's next edge between different components, take one component as S. Any lighter previously scanned crossing edge could not have been rejected: rejection would have joined endpoints already in one component, which would still lie together now. Hence e is lightest for this cut. Prim chooses its current S's lightest crossing edge directly. Start at empty F and preserve the invariant until n−1 connected edges form an MST. Negative or zero MST weights are allowed; the exchange proof uses order comparisons, not Dijkstra's nonnegativity premise.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Kruskal and Prim on the same graph

Use AB:1,AC:4,BC:2,BD:5,CD:3.

Edge	Decision	Components afterward
AB:1	Accept	{A, B},{C},{D}
BC:2	Accept	{A, B, C},{D}
CD:3	Accept	{A, B, C, D}
AC:4	Reject cycle A–B–C–A	Unchanged
BD:5	Reject cycle B–C–D–B	Unchanged

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Prim candidates and algorithm costs

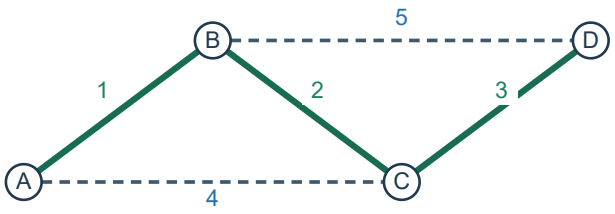
One can stop after three acceptances;remaining rows display cycle checks. Prim at A sees {AB:1,AC:4},then {BC:2,AC:4,BD:5},then {CD:3,BD:5}. It picks AB, BC, CD, total six. The final cut has both CD and BD.

Sorting plus DSU costs $O(n+m \log m+m\alpha(n))$,including initialization even with isolates/disconnection. Binary-heap Prim costs $O((n+m)\log n)$,array Prim $O(n^2+m)$. Disconnected inputs yield one minimum tree per component, not invented edges.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Selected edges and every cut candidate

MST: choose safe cut edges, not source distances



Accept AB → BC → CD

Total weight 1+2+3 = 6

Final cut: {A,B,C} | {D}

Candidates CD:3, BD:5; choose CD.

Thick solid: selected edges. Dashed: other edges. BD crosses the final cut.

CD is uniquely lightest across the final cut and therefore lies in every MST. If omitted, the tree's cut-crossing path must use heavier BD; exchanging strictly improves it. This every-MST conclusion requires unique lightness, not a tied minimum.

Residual IDs and reverse updates

Each original e=(u, v, c) has its own paired forward residual c and reverse residual zero, with reciprocal reverse indices. Sending Δ subtracts it from the traversed record and adds it to its pair. Reverse use cancels old flow rather than making negative capacity.

Antiparallel original arcs require four records, two pairs. A reverse record of one is not the other's forward record. Aggregating same-direction residual capacity for search gives c(u, v) −f(u, v)+f(v, u),but reconstruction still needs origins. Nonnegative original-arc flow and skew-symmetric net flow have different capacity/value conventions and cannot be mixed.

A complete cancellation augmentation

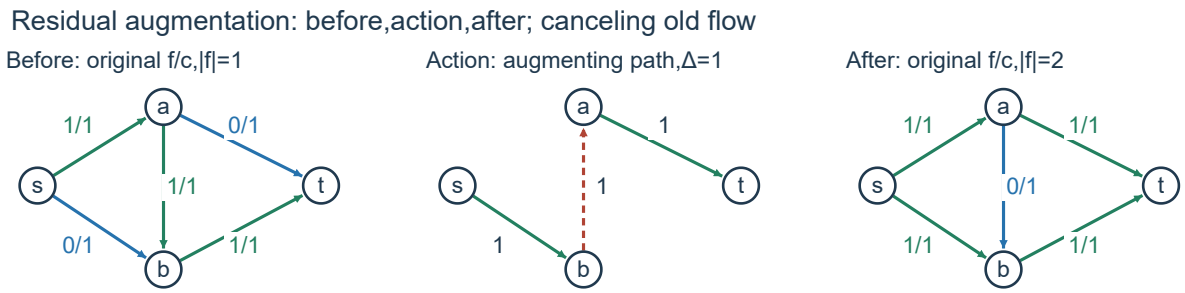
All original capacities are one on sa, sb, ab, at, bt. Sending one along s–a–b–t gives

$$(f_{sa}, f_{sb}, f_{ab}, f_{at}, f_{bt}) = (1, 0, 1, 0, 1).$$

Though sa and bt are saturated, the residual path s–b–a–t exists, using the reverse of ab. Bottleneck one changes the vector to (1,1,0,1,1),raising value from one to two and canceling ab. It does not create original ba transport. Cut {s} has capacity two, certifying optimality. Searching only unsaturated original arcs would wrongly stop at one.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Reverse residual arcs change original flows



Middle labels are residual capacities. Red b→a cancels a→b; afterward a→b is 0/1.

Left and right diagrams show original f/c. The center shows only the three traversed residual records, labeled by residual capacities. Original ab:1/1 creates reverse ba residual one; afterward original ab is 0/1. Vertices and original arcs retain the same positions across states.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Edmonds–Karp's finite augmentation bound

BFS positive residual records to find a minimum-edge-count s-to-t path. If none exists, output flow and reachable cut. Otherwise backtrack predecessor records for the bottleneck, then update their pairs. BFS edge lengths are one, unrelated to capacity size.

Let d(v) be old residual BFS distances. Newly positive records can only reverse path edges; a new u→v has old d(v)=d(u)−1. Existing records satisfy d(v)≤d(u)+1. Summing along any new residual path shows old distance never exceeds its new length, so BFS distances cannot decrease. A newly reachable formerly unreachable vertex would have a first such entry on its path, contradicting the same inequalities.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Edmonds–Karp: bounding saturations

When u→v becomes saturated as a bottleneck,d(v)=d(u)+1. Before it can become a bottleneck again, reverse v→u flow must occur. At that later time,

$$d_{\text{later}}(u) = d_{\text{later}}(v) + 1 \geq d_{\text{old}}(v) + 1 = d_{\text{old}}(u) + 2.$$

Finite reachable distances are at most n−1,so each record is a bottleneck O(n) times. O(m) records and at least one saturated per augmentation give O(nm) augmentations. Excluding irrelevant isolates, BFS costs O(m),giving $O(nm^2)$ core time plus O(n+m) input setup. Exact arithmetic is assumed; integrality is not. Floating tolerances need a separate specification.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Maximum, maximal, and augmenting paths

Maximum means globally most edges; maximal means no edge can simply be added. With a1,a2,b1,M={a1} is maximal but alternating path b–1–a–2 flips to {b1,a2},increasing size.

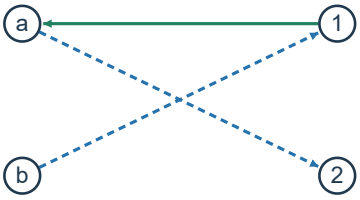
Berge's criterion says M is maximum iff no augmenting path exists. A path gives improvement by symmetric difference. Conversely, for larger M′,M△M′ has degree at most two and splits into alternating paths and even cycles. Cycles and even paths have equal counts from each matching. Since M′ is larger, some path has one extra M′ edge and both endpoints unmatched in M:an augmenting path. Direct additions alone do not suffice.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

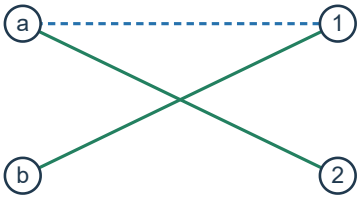
Flipping one alternating path

Alternating path b–1–a–2: flip edges to raise matching size from 1 to 2

Before M={a1}: search alternating edges



After M′={b1,a2}



Green solid edges are matched; blue dashed edges unmatched. Crossings are not vertices.

Traverse matched a1 backward from 1 to a to cancel it; the other edges go left to right. Each internal vertex loses one matched edge and gains one; both unmatched endpoints gain one. The result is a matching with exactly one additional edge.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Constructive Kőnig theorem

For maximum M, search from unmatched left vertices along unmatched L→R and matched R→L edges. Let reachable sets be Z_L, Z_R. Every reachable right vertex is matched, or an augmenting path would exist. Set

$$C = (L \setminus Z_L) \cup Z_R.$$

It covers all edges. A left endpoint outside Z_L is covered directly. If inside, an unmatched edge reaches its right endpoint; for a matched edge, the right endpoint was the step reaching that matched left vertex. Unmatched starting vertices have no matched edge. Thus that right endpoint is in Z_R.

Every Z_R vertex is matched into Z_L; every left vertex outside Z_L is matched, otherwise it would start the search. These two cover parts count each M-edge exactly once, so |C|=|M|. Every cover must choose an endpoint for each disjoint matching edge, so no cover is smaller. Hence minimum cover equals maximum matching.

For {b1,a2},all L is saturated and reachability sets are empty, so C=L of size two. With only a1,b1,start at b and reach 1,a:Z_L={a, b},Z_R={1},giving minimum cover {1}.

Hall deficiency as a failure certificate

At most |N(S)| vertices in S can match, so any matching leaves at least |S|−|N(S)| unmatched left vertices. For maximum size ν,

$$|L| - \nu \geq \max_{S \subseteq L}(|S| - |N(S)|).$$

For the reverse inequality, use alternating reachable S=Z_L, T=Z_R of a maximum matching. T is matched into S, and S additionally contains exactly the k=|L|−ν unmatched left starts. Thus |S|−|T|=k, and N(S)=T, attaining equality. If k=0,the empty set attains zero.

For a, b, c adjacent only to 1,2,and d only to 3,subset {a, b, c} has deficiency one. Matching {a1,b2,d3} has size three and leaves exactly one left vertex, providing both feasibility and impossibility of improvement.

Regular bipartite graphs

In a finite nonempty r-regular bipartite graph, $r \geq 1$,all $r|S|$ edges from S end in N(S),whose vertices each receive at most r edges. Thus $r|S| \leq r|N(S)|$ and Hall holds. Edge counting $r|L|=r|R|$ gives equal sides, so an L-saturating matching is perfect. At r=0 cancellation is invalid and isolated vertices need not have a perfect matching.

Delete a perfect matching; every degree drops by one. Repeating decomposes all edges into r disjoint perfect matchings. In $K_{2,2}$ use {a1,b2} and {a2,b1}. Coloring by matching gives r edge colors, with all incident colors distinct, and degree r requires at least r.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Certificates and complexity checks

Shortest paths use realizing paths plus arc inequalities; MSTs use safe cuts respecting the forest; maximum flows use equal-value cuts; maximum bipartite matchings use equal-size covers or no augmenting path. State representation, priority queue, input distribution, and worst/average cost before comparing speed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Optimization objectives are not interchangeable

MST minimizes total selected-tree weight, not all source-to-vertex tree distances. In triangle AB:2,BC:2,AC:3,the MST uses AB, BC, total four, but its A-to-C route costs four versus shortest distance three.

Maximum cardinality ignores assignment costs. With a1,b2 costing 100 each and a2,b1 costing one each, both perfect matchings have size two but costs 200 and two. Flow value alone cannot choose the cheaper; add cost minimization. Separate exercise and solution PDFs provide further examples.

References and attribution

Lin Cing, handwritten discrete mathematics notes. Ralph P. Grimaldi, Discrete and Combinatorial Mathematics, 5th ed., Ch. 13.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Rings and modular arithmetic

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

14.1 Ring axioms

A nonempty set R, equipped with addition and multiplication, is a **ring** if:

1. $(R, +)$ is an abelian group. Addition is closed, associative and commutative, has identity 0, and gives every element an additive inverse.
2. Multiplication is closed and associative.
3. Both distributive laws hold, $a(b + c) = ab + ac$ and $(a + b)c = ac + bc$.

Following the convention for this chapter, a general ring need not have commutative multiplication or a multiplicative identity. A **ring with identity** has an element 1; a **commutative ring** also satisfies $ab = ba$. Assumptions such as commutativity and the existence of 1 will be stated when discussing modular arithmetic, units, integral domains, fields and CRT. Properties of these special rings cannot silently be used for all rings.

If a ring with identity satisfies $0 = 1$, then every $a = a1 = a0 = 0$. It has just one element and is the **zero ring**. Integral domains and fields exclude it.

14.1 Ring axioms (continued)

The distinction between a subring and an ideal concerns absorption under multiplication. Containing 0 and being closed is not enough to make a subset an ideal.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

14.2 Examples of rings

- $\mathbb{Z}, \mathbb{Q}, \mathbb{R}, \mathbb{C}$ are commutative rings with identity.
- $M_n(\mathbb{R})$, with ordinary matrix addition and multiplication, is a ring with identity. For $n \geq 2$, multiplication is generally not commutative, so steps requiring commutativity need justification.
- The residue classes in $\mathbb{Z}_n$ form a ring under addition and multiplication followed by reduction modulo n.
- Polynomial rings $F[x]$ will be used later; their coefficients come from a field or ring.

Ring elements need not be numbers. Functions also form rings under pointwise addition and multiplication.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two explicit counterexamples in a matrix ring

Let

$$A = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}, \qquad B = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}.$$

Direct multiplication gives

$$AB = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}, \qquad BA = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}.$$

Thus $AB \neq BA$. Also $A \neq 0$ but $A^2 = 0$, so A is a nonzero zero divisor. In general, $AX = AY$ does not allow cancellation of A. Matrix multiplication still satisfies associativity and both distributive laws. Noncommutativity does not prevent a structure from being a ring.

A ring whose elements are sets

For a finite set U, take the subsets of U as elements, symmetric difference $\triangle$ as addition, and intersection $\cap$ as multiplication. The empty set is the additive identity. Since $A \triangle A = \varnothing$, each A is its own additive inverse. The multiplicative identity is U.

For each $u \in U$, use a bit to record membership. Symmetric difference becomes bitwise XOR, and intersection becomes bitwise AND. Each ring axiom holds coordinate by coordinate by the corresponding axiom in $\mathbb{Z}_2$. This is a commutative ring. If $|U| = k$, it has 2^k elements and is isomorphic to $(\mathbb{Z}_2)^k$.

If $k \geq 2$, two distinct singletons A, B satisfy $A \cap B = \varnothing$. They are nonzero zero divisors, so the ring is not a field. Yet every element satisfies $A^2 = A$, the **idempotent** property. Whether division is possible depends on the operations, not on whether the elements look like numbers, sets, matrices or functions.

14.3 Basic consequences of the axioms

For all $a, b \in R$,

$$a0 = 0 = 0a, \qquad a(-b) = -(ab) = (-a)b.$$

Indeed, $a0 = a(0 + 0) = a0 + a0$; cancellation in the additive group gives $a0 = 0$. The argument uses distributivity and additive inverses, rather than assuming the familiar integer rule.

The additive identity is unique, because $0 + 0'$ equals both 0 and $0'$. If b, c are additive inverses of a, then

$$b = b + (a + c) = (b + a) + c = c.$$

Thus additive inverses are unique. Since $ab + a(-b) = a(b - b) = 0$, uniqueness gives $a(-b) = -(ab)$. The other side is analogous, and $(-a)(-b) = ab$ follows. None of these results requires commutative multiplication.

If $ab = 0$ with **both** $a \neq 0$ and $b \neq 0$, then a, b are nonzero zero divisors. Knowing only $b \neq 0$, while allowing $a = 0$, does not establish that b is a zero divisor.

14.3 Basic consequences of the axioms (continued)

In $\mathbb{Z}_6$, $2 \cdot 3 = 0$, so multiplicative cancellation does not hold universally.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

14.4 Units, zero divisors and integral domains

In a commutative ring with identity, u is a **unit** if some v satisfies $uv = 1$. The units form a multiplicative group $R^\times$. In a noncommutative ring, both $uv = 1$ and $vu = 1$ are required. An **integral domain** is a nonzero commutative ring with identity satisfying

$$ab = 0 \implies a = 0 \text{ or } b = 0.$$

The integers form an integral domain. For composite $n > 1$, $\mathbb{Z}_n$ does not.

Proof of the prime-modulus criterion. If p is prime and $ab \equiv 0 \pmod{p}$, Euclid's lemma gives $p \mid a$ or $p \mid b$. Conversely, if $n = rs$ with $1 < r, s < n$, the classes of r, s are nonzero and have product zero.

An integral domain need not be a field. In $\mathbb{Z}$, only $1, -1$ have multiplicative inverses, although other nonzero elements exist.

A **field** is a nonzero commutative ring with identity in which every nonzero element is a unit. Division by nonzero elements is available in a field. A general ring guarantees additive inverses, but not multiplicative inverses.

Cancellation and finite integral domains

In a commutative ring, $a \neq 0$ and $ab = ac$ yield only $a(b - c) = 0$. If a is not a zero divisor, this implies $b = c$. Conversely, cancellation for a particular nonzero a forces $b = 0$ whenever $ab = 0 = a0$. Thus cancellation by a is equivalent to a not being a zero divisor.

In a nonzero commutative ring with identity, a nonzero unit cannot be a zero divisor. Multiplying $ab = 0$ by a^{-1} gives $b = 0$. Every field is therefore an integral domain. The converse fails for $\mathbb{Z}$, where 2 is not invertible.

Every finite integral domain is a field. For $a \neq 0$, the map $m_a : x \mapsto ax$ is injective by cancellation. An injection from a finite set to itself is surjective, so $1 = ax$ for some x. Every nonzero a is invertible. Finiteness is essential. On $\mathbb{Z}$, $x \mapsto 2x$ is injective but not surjective.

The unit group and characteristic

In a ring with identity, v is the multiplicative inverse of u if $uv = vu = 1$. In a commutative ring, one equality suffices. The inverse is unique, since $v = v(uw) = (vu)w = w$. Products of units are units, with

$$(uv)^{-1} = v^{-1}u^{-1}.$$

Consequently all units form a group. In a noncommutative ring, the order of these two inverse factors cannot be reversed without justification.

The **characteristic** is the least positive integer p such that $p \cdot 1 = 0$, or 0 if no such integer exists. An integral domain has $1 \neq 0$, so its characteristic is not 1. If a positive characteristic were composite, $p = ab$ with $1 < a, b < p$, then

$$(a \cdot 1)(b \cdot 1) = 0.$$

Minimality of p makes both factors nonzero, a contradiction. A positive characteristic of an integral domain must therefore be prime. In a finite integral domain, the sequence $0, 1, 2 \cdot 1, \ldots$ repeats, so its characteristic is a prime.

The unit group and characteristic (continued)

For example, $\mathbb{Z}_{12}$ has characteristic 12 and is not an integral domain. The characteristic of $\mathbb{Q}$ is 0. Characteristic measures repeated addition of the identity, rather than the numeral used to name it.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

14.5 Subrings and ideals

A nonempty subset $S \subseteq R$ is a **subring** if it is itself a ring under the restricted operations. Our convention does not require S to contain 1_R. The exact test is

$$a, b \in S \implies a - b \in S \text{ and } ab \in S.$$

For sufficiency, taking $a = b$ gives $0 \in S$, then $0 - a$ gives additive inverses and $a - (-b)$ gives additive closure. Associativity and distributivity are inherited. Necessity follows directly from the subring axioms.

A **unital subring** must additionally contain 1_R. For example, $2\mathbb{Z}$ is a subring and ideal of $\mathbb{Z}$, but does not contain 1 and has no multiplicative identity of its own under the inherited multiplication. Do not change subring conventions midway through an argument.

A **two-sided ideal** I is an additive subgroup satisfying $ri, ir \in I$ for all $r \in R, i \in I$. This is stronger than multiplication being closed only within I. If an ideal in a ring with identity contains 1, then $r = r1 \in I$ for every r, so $I = R$.

Define operations on cosets by

14.5 Subrings and ideals (continued)

$$(a + I) + (b + I) = (a + b) + I, \qquad (a + I)(b + I) = ab + I.$$

Replacing representatives by $a + i, b + j$ changes the product by $aj + ib + ij$. Two-sided absorption puts every term in I; the change in the sum also lies in I. Both operations are well defined, producing the **quotient ring** R/I. In a noncommutative ring, ib cannot arbitrarily become bi. An arbitrary subring lacks the absorption needed to descend multiplication to cosets.

For instance, the constants $\mathbb{Z} \subseteq \mathbb{Z}[x]$ form a subring but not an ideal, since multiplying an integer by x need not give a constant.

Finite subrings and the limits of ideal closure

A nonempty finite subset S closed under addition and multiplication is already a subring. For $a \in S$, some positive multiples satisfy $ia = ja$, with $i < j$. Additive cancellation in R yields $(j-i)a = 0$. This positive multiple lies in S, so $0 \in S$, and

$$-a = (j-i-1)a \in S.$$

If the coefficient is zero, use the zero element just established. Additive inverses and closure make S an additive subgroup.

Finiteness cannot be omitted. Nonnegative integers are closed under ordinary addition and multiplication and contain $0, 1$, but have no additive inverse for 1. They are not a subring. Also, $\mathbb{Z}$ is a unital subring of $\mathbb{Q}$, but not an ideal because $\frac{1}{2} \cdot 1 \notin \mathbb{Z}$.

A nonzero commutative ring with identity having exactly the ideals $\{0\}, R$ is a field. For $a \neq 0$, the principal ideal $(a) = \{ra : r \in R\}$ is nonzero, hence equals R; thus $1 = ra$ and a is invertible. Conversely, any nonzero ideal of a field contains $a \neq 0$; multiplying by a^{-1} puts 1 in the ideal, so it is the whole field. The nonzero hypothesis keeps the two ideals distinct.

Ideals of the integers and of residue rings

Every ideal $I \subseteq \mathbb{Z}$ equals $m\mathbb{Z}$ for a unique $m \geq 0$. If $I = \{0\}$, take $m = 0$. Otherwise take the least positive element m of I. For $a \in I$, division gives $a = qm + r$, $0 \leq r < m$. Since $a, qm \in I$, also $r \in I$. Minimality forces $r = 0$, so $I \subseteq m\mathbb{Z}$. Absorption gives the reverse inclusion.

For an ideal J of $\mathbb{Z}_n$, its preimage

$$I = \{a \in \mathbb{Z} : [a]_n \in J\}$$

is an ideal of $\mathbb{Z}$ containing $n\mathbb{Z}$. Thus $I = d\mathbb{Z}$ with $d \mid n$. Conversely each positive divisor $d \mid n$ gives $J = ([d])$. This is a bijection between ideals of $\mathbb{Z}_n$ and positive divisors of n. Such J has n/d elements; its quotient has d elements and is isomorphic to $\mathbb{Z}_d$.

For example, in $\mathbb{Z}_{12}$, $([4]) = \{0, 4, 8\}$. The quotient has four classes,

$$\{0,4,8\},\ \{1,5,9\},\ \{2,6,10\},\ \{3,7,11\}.$$

Sending each class to its residue modulo 4 is a bijection preserving both operations, so $\mathbb{Z}_{12}/([4]) \cong \mathbb{Z}_4$.

14.6 Operations in $\mathbb{Z}/n\mathbb{Z}$

Representatives a and $a+kn$ must determine the same class. The quotient operations are well defined because

$$(a+kn)+(b+\ell n)\equiv a+b \pmod{n},$$
$$(a+kn)(b+\ell n)\equiv ab \pmod{n}.$$

Every extra term is a multiple of n. Moreover,

$$[a] \text{ is a unit} \iff \gcd(a,n)=1.$$

Proof. If the gcd is 1, Bézout gives $sa+tn=1$, hence $[s][a]=[1]$. Conversely, if $ab+kn=1$, any common divisor of a, n divides 1.

The additive identity is $[0]$ and the multiplicative identity is $[1]$. For $n=1$, these are the same class, producing the zero ring.

Complete classification of units and zero divisors

Let $n>1$ and $[a]\neq[0]$, and put $d=\gcd(a,n)$. If $d=1$, $[a]$ is a unit. If $d>1$, set $b=n/d$. Then $1\leq b<n$ and $n\mid ab$, so $[b]\neq[0]$ and $[a][b]=0$. This explicitly constructs a zero-divisor witness. A unit cannot have zero product with a nonzero class. Thus every nonzero class belongs to exactly one of these two categories.

Euler's function $\varphi(n)$ counts the residue classes coprime to n. If the distinct prime factors are $p_1,\dots,p_r$, inclusion–exclusion on their multiples among $0,\dots,n-1$ gives

$$\varphi(n)=n\prod_{p\mid n}\left(1-\frac{1}{p}\right).$$

There are $n-1-\varphi(n)$ nonzero zero divisors. For $n=18$,

$$\varphi(18)=18(1-1/2)(1-1/3)=6.$$

The units are $1,5,7,11,13,17$; the other 11 nonzero classes are zero divisors. For example, $6\cdot 3\equiv 0$ and $5\cdot 11\equiv 1$.

Complete classification of units and zero divisors (continued)

For $n = 1$, the only class is both 0 and 1, and is invertible by the definition of a unit. This ring is neither a field nor an integral domain. The counting formula above is explicitly restricted to $n > 1$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Linear congruences require a solvability test

To solve $ax \equiv b \pmod{n}$, with $n \geq 1$, put $d = \gcd(a, n)$. A solution implies $n \mid ax - b$, hence $d \mid b$. Conversely, if $d \mid b$, division by d gives

$$\frac{a}{d}x \equiv \frac{b}{d} \pmod{n/d}.$$

The new coefficient is coprime to the new modulus, so there is a unique x_0 modulo n/d. All solutions modulo n are

$$x = x_0 + k(n/d), \qquad k = 0, \ldots, d - 1,$$

which are d distinct classes. If $n/d = 1$, the reduced equation is just $0 \equiv 0$, and all classes modulo n solve the original equation; no ordinary division needs to be defined.

For $6x \equiv 8 \pmod{14}$, $d = 2$ divides 8. Reduction gives $3x \equiv 4 \pmod{7}$. The inverse of 3 is 5, so $x \equiv 20 \equiv 6 \pmod{7}$. The two solutions modulo 14 are $6, 13$. Substitution gives $36 \equiv 8$ and $78 \equiv 8$. Replacing the right side by 9 gives no solution, because $2 \nmid 9$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

14.7 Modular inverses and Euler's theorem

If $\gcd(a, n) = 1$, the extended Euclidean algorithm produces s with $sa \equiv 1 \pmod{n}$. Euler's theorem states that

$$a^{\varphi(n)} \equiv 1 \pmod{n}$$

for a coprime to n. Fermat's little theorem is the prime-modulus case, where $n = p$ is prime and $p \nmid a$:

$$a^{p-1} \equiv 1 \pmod{p}.$$

If $\gcd(a, n) > 1$ and $n > 1$, then a has no multiplicative inverse modulo n. A power congruence happening to have a solution does not make a available for division.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Euler's theorem by permutation and cancellation

Let $n > 1$, with units $u_1, \ldots, u_\varphi$. Since $[a]$ is a unit, multiplication by a permutes the units. Indeed, $au_i = au_j$ implies $u_i = u_j$ after multiplying by a^{-1}; finiteness gives a bijection. Commutativity then gives

$$a^{\varphi(n)} u_1 \cdots u_\varphi = u_1 \cdots u_\varphi.$$

The product on the right is a unit and can be cancelled by its inverse, proving the theorem. The permutation and cancellation fail without coprimality. For example, $2^{\varphi(8)} = 2^4 \equiv 0 \pmod{8}$, not 1. For prime p, $\varphi(p) = p - 1$, yielding Fermat's little theorem. The version also allowing $p \mid a$ is $a^p \equiv a$; division by a is not justified in that case.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Binary modular exponentiation, bit by bit

To compute $a^e \bmod n$ for $n > 1, e \geq 0$, initialize $r = 1$ and scan all binary bits of e from left to right. For each bit b, first set $r \leftarrow r^2 \bmod n$, then, if $b = 1$, set $r \leftarrow ra \bmod n$. After reading a prefix of value q, the invariant is $r \equiv a^q$. Appending b changes the exponent to $2q + b$, proving the update. For $e = 0$, return $1 \bmod n$.

For $a = 5, e = 13 = (1101)_2, n = 17$:

Bit read	Prefix q	After squaring modulo 17	After multiplying by 5 if needed
Initial	0	No squaring	1
1	1	1	5
1	3	8	6
0	6	2	2
1	13	4	3

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Binary modular exponentiation, bit by bit (continued)

Thus $5^{13} \equiv 3 \pmod{17}$. An implementation initialized at $r = a$ must skip the leading significant bit; starting at a and scanning every bit double-counts it. The algorithm uses $O(\log(e + 1))$ modular multiplications. This is not the same number of bit operations; each large-integer modular multiplication has its own cost.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

14.8 Chinese Remainder Theorem

For pairwise coprime positive moduli $m_1, \ldots, m_k$, the system

$$x \equiv a_i \pmod{m_i}$$

has exactly one solution modulo $N = \prod_i m_i$. To construct it, let $N_i = N/m_i$. Choose t_i with $t_i N_i \equiv 1 \pmod{m_i}$, possible since $\gcd(N_i, m_i) = 1$. Then

$$x = \sum_i a_i t_i N_i$$

satisfies every congruence. For uniqueness, the difference of two solutions is divisible by every m_i, hence by N because the moduli are pairwise coprime.

For the example $x \equiv 2 \pmod{3}, x \equiv 3 \pmod{5}$, write $x = 2 + 3t$. Then $2 + 3t \equiv 3 \pmod{5}$, so $3t \equiv 1 \pmod{5}$. Since $3^{-1} \equiv 2 \pmod{5}$, $t \equiv 2 \pmod{5}$, giving $x \equiv 8 \pmod{15}$.

Pairwise coprimality is sufficient for this form of CRT. More general versions use ideals.

CRT as an isomorphism of ring representations

For pairwise coprime moduli greater than 1, define

$$\Theta : \mathbb{Z}_N \longrightarrow \prod_i \mathbb{Z}_{m_i}, \qquad [x]_N \longmapsto ([x]_{m_1}, \ldots, [x]_{m_k}).$$

It is well defined since each $m_i \mid N$, and it preserves addition and multiplication coordinatewise. CRT existence and uniqueness state exactly that it is bijective. Thus it is a ring isomorphism. Product-ring operations are coordinatewise operations, not decimal operations on the entries of a tuple.

For $N = 12 = 3 \cdot 4$, let $e_1 = 4, e_2 = 9$. Their images are $(1, 0), (0, 1)$, and

$$e_1^2 = e_1, \quad e_2^2 = e_2, \quad e_1 e_2 = 0, \quad e_1 + e_2 = 1 \pmod{12}.$$

Hence $\Theta^{-1}(a, b) = 4a + 9b \pmod{12}$.

CRT as an isomorphism of ring representations (continued)

CRT: each (a mod 3, b mod 4) determines one x mod 12

	b=0	b=1	b=2	b=3
a=0	x=0	x=9	x=6	x=3
a=1	x=4	x=1	x=10	x=7
a=2	x=8	x=5	x=2	x=11

Green: units

x=1, 5, 7, 11

4 ↦ (1,0)

9 ↦ (0,1)

Each cell has x = 4a+9b mod 12. All 12 residues occur once; operations are coordinatewise.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Compatibility when moduli are not coprime

For $x \equiv a \pmod{m}, x \equiv b \pmod{n}$, put $g = \gcd(m, n)$. A solution requires $g \mid (b - a)$. If that condition holds, writing $x = a + mk$ reduces the system to

$$(m/g)k \equiv (b - a)/g \pmod{n/g}.$$

The reduced coefficient and modulus are coprime. Thus k is unique modulo n/g, and x is unique modulo $\operatorname{lcm}(m, n)$. This also proves sufficiency of the compatibility condition. Uniqueness is not generally modulo mn.

For $x \equiv 5 \pmod{8}, x \equiv 9 \pmod{12}$, $g = 4$ divides the difference 4. Set $x = 5 + 8k$. Then $2k \equiv 1 \pmod{3}$, so $k \equiv 2 \pmod{3}$, and $x \equiv 21 \pmod{24}$. Indeed, $21 \bmod 8 = 5$ and $21 \bmod 12 = 9$. Replacing the second residue by 10 makes the difference 5, not divisible by 4, so there is no solution.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Deterministic applications of modular arithmetic

A linear congruential generator uses $x_{t+1} = (ax_t + c) \bmod m$. Its finite state space guarantees eventual repetition, but not period m. For $a = 5, c = 1, m = 16, x_0 = 0$, the actual sequence is

$0 \to 1 \to 6 \to 15 \to 12 \to 13 \to 2 \to 11 \to 8 \to 9 \to 14 \to 7 \to 4 \to 5 \to 10 \to 3 \to 0$.

This seed visits all 16 states before returning, so the period is 16. Changing only a to 6 gives $0 \to 1 \to 7 \to 11 \to 3 \to 3$, with a transient followed by a cycle of length 1. Determinism and finite periodicity imply neither independent uniform random values nor cryptographic security.

The hash function $h(k) = k \bmod 13$ maps integers to 13 buckets. The keys $13, 26, 39$ all map to 0; more than 13 distinct keys force a collision. The remainder specifies the bucket, while collision handling and distribution quality are separate issues.

When an affine encoding can be decoded uniquely

On $\mathbb{Z}_n$, $E(x) = ax + b$ is a permutation if and only if $\gcd(a, n) = 1$. In the coprime case,

$$D(y) = a^{-1}(y - b)$$

is its inverse. If $d = \gcd(a, n) > 1$, the distinct classes x and $x + n/d$ have the same image, so E is not injective.

For $n = 26, a = 5, b = 7$, $5^{-1} = 21$, $E(4) = 27 \equiv 1$, and $D(1) = 21(1 - 7) = -126 \equiv 4$. With $a = 6$, both $x = 0$ and $x = 13$ map to 7; unique decoding is impossible. This is an example of algebraic invertibility, not a claim that a simple affine cipher provides secure encryption.

14.9 Ring homomorphisms

A **ring homomorphism** $\varphi : R \to S$ satisfies, for every $a, b \in R$,

$$\varphi(a+b) = \varphi(a) + \varphi(b), \qquad \varphi(ab) = \varphi(a)\varphi(b).$$

Following our convention for general rings, preservation of 1 is not assumed. A **unital ring homomorphism** additionally satisfies $\varphi(1_R) = 1_S$. A map preserving only addition and multiplication may send 1 to another idempotent, so domain units need not become codomain units.

For a bijective homomorphism, the inverse preserves both operations. Write $x = \varphi(a), y = \varphi(b)$; then $\varphi^{-1}(x+y) = a+b$, and similarly for multiplication. A bijective homomorphism is a **ring isomorphism**. Equal cardinalities guarantee only a set bijection, not preservation of operations.

For example, evaluation $F[x] \to F$, $f \mapsto f(a)$, is a unital ring homomorphism.

What a homomorphism preserves

From $\varphi(0) = \varphi(0) + \varphi(0)$, additive cancellation gives $\varphi(0) = 0$. Then $\varphi(a) + \varphi(-a) = \varphi(0)$ yields $\varphi(-a) = -\varphi(a)$. Images of additive subgroups are additive subgroups, and images of subrings are subrings.

If I is an ideal of S, then $\varphi^{-1}(I)$ is closed under subtraction. For i in this preimage and $r \in R$, $\varphi(ri) = \varphi(r)\varphi(i) \in I$ and likewise $\varphi(ir) \in I$. Thus the preimage is an ideal of R. Conversely, the image of an ideal of R is generally only an ideal of $\operatorname{Im}\varphi$. If φ is surjective onto S, any $s \in S$ is $\varphi(r)$, which proves absorption by all of S.

For the inclusion $\mathbb{Z} \to \mathbb{Q}$, the ideal $2\mathbb{Z}$ has the same image, but this is not an ideal of $\mathbb{Q}$, since $\frac{1}{2} \cdot 2 = 1 \notin 2\mathbb{Z}$. Surjectivity is essential in that direction.

If φ is surjective and R has 1, every $s = \varphi(r)$ satisfies $\varphi(1)s = s = s\varphi(1)$. Thus $\varphi(1)$ is the multiplicative identity of S. In the surjective case, identity preservation follows from the axioms.

14.10 Kernel, image and the first isomorphism theorem

The **kernel**

$$\ker \varphi = \{r \in R : \varphi(r) = 0_S\}$$

is an ideal, and the image is a subring. Moreover, φ is injective if and only if $\ker \varphi = \{0\}$. The first isomorphism theorem states

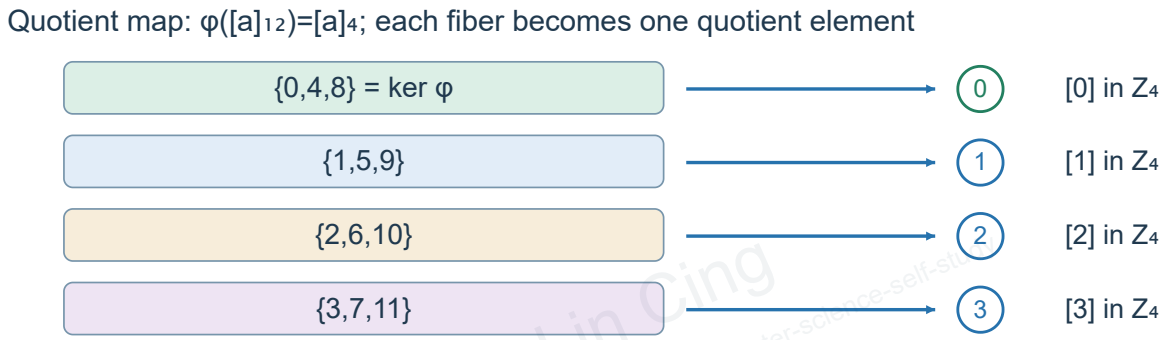
$$R/\ker \varphi \cong \operatorname{Im} \varphi.$$

For $\varphi : \mathbb{Z} \to \mathbb{Z}_n$, $k \mapsto [k]$, the kernel is $n\mathbb{Z}$, giving $\mathbb{Z}/n\mathbb{Z} \cong \mathbb{Z}_n$.

Proof. The map $r + \ker \varphi \mapsto \varphi(r)$ is well defined because representatives of one coset differ by a kernel element. It preserves both operations and surjects onto the image. Equal outputs mean the representatives differ by a kernel element, hence their cosets coincide, proving injectivity. This constructs the isomorphism rather than merely asserting a symbolic equality.

The kernel is an ideal by the preceding preimage result with $I = \{0\}$. If φ is injective, $\varphi(r) = 0 = \varphi(0)$ forces $r = 0$. Conversely, if the kernel is zero and $\varphi(a) = \varphi(b)$, then $\varphi(a - b) = 0$ forces $a = b$. Both directions of the injectivity test are proved.

Four fibers of a quotient map



The natural map $\varphi([a]_{12}) = [a]_4$ is well defined because a difference divisible by 12 is also divisible by 4. Its kernel is $\{0, 4, 8\}$. The three elements in each color group are distinct in $\mathbb{Z}_{12}$ but have one output. The quotient identifies each fiber as a single element; it does not treat the original elements in that fiber as separate quotient elements.

Changing the modulus requires well-definedness

The candidate $[a]_m \mapsto [a]_n$ is well defined if and only if $n \mid m$. Necessity follows because $[0]_m = [m]_m$ must have equal images. For sufficiency, if m divides the difference between representatives, then so does n. Once well defined, the map preserves addition, multiplication and 1.

Thus $\mathbb{Z}_{12} \to \mathbb{Z}_4$ is valid, but the same formula from $\mathbb{Z}_4$ to $\mathbb{Z}_{12}$ is not: $[0]_4 = [4]_4$ would have distinct images. Choosing the least nonnegative representative defines a set function but does not repair additivity. The image of $[3]_4 + [1]_4 = [0]_4$ is 0, whereas $3 + 1 = 4$ in $\mathbb{Z}_{12}$.

A valid nonunital example is

$$\psi : \mathbb{Z}_4 \to \mathbb{Z}_6, \qquad [a] \mapsto [3a].$$

It preserves addition. Multiplication is preserved since $9ab \equiv 3ab \pmod 6$, and $4 \cdot 3 \equiv 0$ ensures independence from representatives. The image $\{0, 3\}$ has its own multiplicative identity 3, rather than 1 of $\mathbb{Z}_6$. Its kernel is $\{0, 2\}$, so $\mathbb{Z}_4/\{0, 2\} \cong \{0, 3\} \cong \mathbb{Z}_2$.

14.11 Boundaries and common errors

- $(\mathbb{Z}_n, +, \cdot)$ is a ring, but nonzero classes need not be units.
- Products of nonzero elements can be zero for composite moduli.
- $\mathbb{Z} \subset \mathbb{Q}$ is a unital subring but not an ideal; $2\mathbb{Z} \subset \mathbb{Z}$ is an ideal lacking 1.
- A kernel must contain zero and absorb multiplication by the ambient ring.
- The pairwise gcd conditions must be checked before applying the coprime form of CRT.

These structures recur in RSA in Chapter 16 and finite fields in Chapter 17. A field requires additional axioms.

Transporting operations changes zero and one

On $\mathbb{Q}$, define

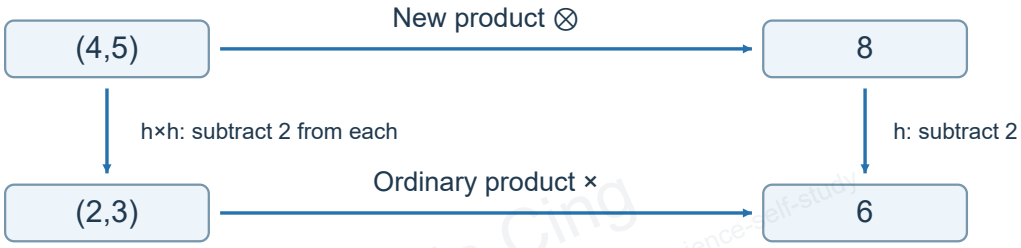
$$a \oplus b = a + b - 2, \qquad a \otimes b = (a-2)(b-2) + 2.$$

The bijection $h(a) = a - 2$ satisfies $h(a \oplus b) = h(a) + h(b)$ and $h(a \otimes b) = h(a)h(b)$. It transforms both new operations into ordinary rational operations. Transporting the ring axioms through h gives a field isomorphic to $\mathbb{Q}$.

The new additive zero is $h^{-1}(0) = 2$, and the new multiplicative identity is $h^{-1}(1) = 3$. The additive inverse of a is $4 - a$. For $a \neq 2$, its new multiplicative inverse is $2 + 1/(a-2)$. For example, the inverse of 4 is $5/2$, and $4 \otimes (5/2) = 3$, the new identity. Requiring this product to equal the ordinary numeral 1 would use the wrong operation structure.

Transporting operations changes zero and one (continued)

Transported multiplication: 4 ⊗ 5 = 8 corresponds to 2 × 3 = 6



New product then h, or h on each input then ordinary product: both paths give 6.

References and attribution

Ralph P. Grimaldi. Discrete and Combinatorial Mathematics: An Applied Introduction, 5th Edition, 2004.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Boolean algebra and switching functions

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.1 Functions, expressions and circuits

A **switching function** is a map $f : \{0,1\}^n \to \{0,1\}$. In input order $x_1, \ldots, x_n$, there are 2^n assignments. A complete truth table specifies an output for each. Since each row has two output choices, there are 2^{2^n} switching functions.

A **Boolean expression** represents a function using AND, OR and NOT. We write $xy = x \wedge y$, $x + y = x \vee y$, and $\bar{x} = \neg x$. Here $+$ means neither integer addition nor XOR. A **literal** is a variable or its complement, a **product term** is an AND of literals, and a **sum of products**, or SOP, is an OR of product terms.

Different expressions can represent the same function, for example $AB + A\bar{B} = A$. Different circuits can also implement one function. First establish function equivalence, then design for a stated cost such as gate count, fan-in or logic depth. These costs need not equal the number of literal occurrences.

15.2 Minterms identify individual rows

Fix the bit order A, B, C and index $4A + 2B + C$. For an assignment $a = (a_1, \ldots, a_n)$, define

$$\ell_i(a) = \begin{cases} x_i, & a_i = 1, \\ \bar{x}_i, & a_i = 0, \end{cases} \qquad m_a = \bigwedge_{i=1}^{n} \ell_i(a).$$

This avoids a special exponent convention such as $x^0 = \bar{x}$.

At input a, every literal is 1, so $m_a = 1$. At any other input, a differing bit makes its literal 0, so $m_a = 0$. Thus m_a is true exactly on row a, and distinct minterms are mutually exclusive.

For $a = 101$, the minterm is $A\bar{B}C = m_5$; for 011, it is $\bar{A}BC = m_3$. Every three-variable minterm contains three literals. Omitting a variable accepts two rows and no longer gives a minterm.

15.3 Existence and uniqueness of canonical DNF

For a fully specified function,

$$f = \bigvee_{a : f(a) = 1} m_a.$$

At any input b, all terms with $a \neq b$ vanish. If $f(b) = 1$, the right side contains $m_b = 1$; otherwise it does not. This proves the identity row by row.

Uniqueness applies to **canonical DNF**, in which every term is a complete minterm, with order and repetitions ignored. Only m_b can be true at b, so $f(b)$ uniquely determines whether that term appears. General DNF is neither unique nor necessarily minimal.

For $f = \Sigma m(3, 5, 7)$:

ABC	000	001	010	011	100	101	110	111
f	0	0	0	1	0	1	0	1

15.3 Existence and uniqueness of canonical DNF (continued)

Thus $f = \bar{A}BC + A\bar{B}C + ABC$, which simplifies to $BC + AC$. We define the empty OR as 0 and the empty AND as 1, covering constant functions and $n = 0$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.4 Maxterms and canonical CNF

For assignment a, choose literals that are false on that row:

$$q_i(a) = \begin{cases} \bar{x}_i, & a_i = 1, \\ x_i, & a_i = 0, \end{cases} \qquad M_a = \bigvee_i q_i(a).$$

All are 0 at a; at another row at least one is 1. Thus M_a is false exactly at a. The canonical CNF identity is

$$f = \bigwedge_{a : f(a) = 0} M_a.$$

At a false input b, the right side includes the zero factor M_b. At a true input, every factor is 1. This proves the identity.

For XOR, the false rows are $00, 11$, so

$$A \oplus B = (A + B)(\bar{A} + \bar{B}) = A\bar{B} + \bar{A}B.$$

DNF lists true rows; CNF lists false rows. Their lengths can differ substantially for a given function.

15.5 Axioms of an abstract Boolean algebra

A **Boolean algebra** is a bounded distributive lattice in which every element has a complement. On a nonempty set B, operations $\vee, \wedge$ and constants $0, 1$ satisfy the following identities for all $x, y, z \in B$.

Property	Join form	Meet form
Commutativity	$x \vee y = y \vee x$	$x \wedge y = y \wedge x$
Associativity	$(x \vee y) \vee z = x \vee (y \vee z)$	$(x \wedge y) \wedge z = x \wedge (y \wedge z)$
Absorption	$x \vee (x \wedge y) = x$	$x \wedge (x \vee y) = x$
Bounds and identities	$x \vee 0 = x,\ x \vee 1 = 1$	$x \wedge 1 = x,\ x \wedge 0 = 0$
Distributivity	$x \vee (y \wedge z) = (x \vee y) \wedge (x \vee z)$	$x \wedge (y \vee z) = (x \wedge y) \vee (x \wedge z)$

Each x has at least one complement y, meaning $x \vee y = 1$ and $x \wedge y = 0$. We will prove uniqueness and denote the complement by $\bar{x}$. The one-element algebra with $0 = 1$ is allowed. A nonzero algebra will explicitly require $0 \neq 1$.

15.6 The partial order induced by the operations

Define $x \leq y \iff x \wedge y = x$. Substituting $y = 0$ in absorption gives $x \wedge x = x$; the dual gives $x \vee x = x$. Thus the relation is reflexive.

If $x \leq y$ and $y \leq x$, then $x = x \wedge y = y \wedge x = y$, proving antisymmetry. If $x \leq y \leq z$, then

$$x \wedge z = (x \wedge y) \wedge z = x \wedge (y \wedge z) = x \wedge y = x,$$

proving transitivity. Absorption also gives

$$x \leq y \iff x \vee y = y.$$

For the forward direction, $x \vee y = (x \wedge y) \vee y = y$. For the reverse direction, $x \wedge y = x \wedge (x \vee y) = x$.

The element $x \wedge y$ is a common lower bound. If $z \leq x, y$, then $z \wedge (x \wedge y) = (z \wedge x) \wedge y = z$, so it is the greatest lower bound. Dually, $x \vee y$ is the least upper bound. Finite joins therefore have a precise least-upper-bound meaning.

15.7 Unique complements, De Morgan and duality

Suppose b, c are complements of a. Distributivity gives

$$b = b \wedge 1 = b \wedge (a \vee c) = (b \wedge a) \vee (b \wedge c) = b \wedge c.$$

Similarly $c = b \wedge c$, hence $b = c$. Since a is also a complement of $\bar{a}$, uniqueness gives $\bar{\bar{a}} = a$. Also $\bar{0} = 1, \bar{1} = 0$.

Put $d = \bar{x} \vee \bar{y}$. Distributing gives

$$(x \wedge y) \wedge d = 0, \qquad (x \wedge y) \vee d = (x \vee d) \wedge (y \vee d) = 1.$$

Uniqueness of complements therefore gives

$$\overline{x \wedge y} = \bar{x} \vee \bar{y}.$$

Dually, $\overline{x \vee y} = \bar{x} \wedge \bar{y}$.

The **duality principle** interchanges $\wedge$ with $\vee$, and 0 with 1. The axiom set remains unchanged, so interchanging these symbols throughout a derivation produces another valid derivation. This operation is different from taking the pointwise complement of a function.

15.8 Simplification needs a derivation and a cost model

For example,

$$\begin{aligned}
(A + B)(A + \bar{B})(\bar{A} + B) &= (A + B\bar{B})(\bar{A} + B) && \text{distributivity} \\
&= A(\bar{A} + B) && \text{complement and identity} \\
&= A\bar{A} + AB && \text{distributivity} \\
&= AB. && \text{complement and identity}
\end{aligned}$$

Every line agrees on all inputs, rather than on just a few tested rows.

If $E = (A + B)C$, its dual is $AB + C$, while its complement is $\bar{E} = \bar{A}\bar{B} + \bar{C}$. At $A = B = C = 1$, the dual equals 1 and the complement equals 0, disproving their equality.

These laws also hold in abstract Boolean algebras. A table of all 2^n rows directly verifies a two-element switching function only. A theorem about abstract structures still needs a derivation from axioms or a valid representation theorem.

15.9 Implicants, primes and minimal SOP

First consider a fully specified function. Exclude identically zero terms containing both x and $\bar{x}$. A product term P is an **implicant** if $P = 1 \implies f = 1$. Deleting a literal expands its accepted assignments. An implicant is **prime** if no literal can be deleted while keeping it legal.

For $f = \Sigma m(3, 5, 7)$, ABC is an implicant but not prime. Deleting A gives BC, and deleting B gives AC; both avoid false rows. Deleting either remaining literal from AC reaches a false row.

Unless stated otherwise, a **minimal SOP** minimizes product-term count first, then literal count, in lexicographic order. A question asking only for minimal literal count uses that cost instead. Any legal term can be expanded to a prime in finitely many deletions without increasing either cost. Thus either cost has an optimum using primes only.

The optimum need not be unique. Greedily taking the largest available group need not produce it.

15.10 Prime charts and essential coverage

For $f = \Sigma m(3, 5, 7)$, the only maximal legal cubes are BC, AC.

Prime	$m_3 = 011$	$m_5 = 101$	$m_7 = 111$	Literals
BC	✓		✓	2
AC		✓	✓	2

Only BC covers m_3, and only AC covers m_5, making both **essential prime implicants**. Every SOP can be expanded to a prime cover, so both terms are needed. The expression

$$f = AC + BC$$

achieves two terms and four literals, proving minimality.

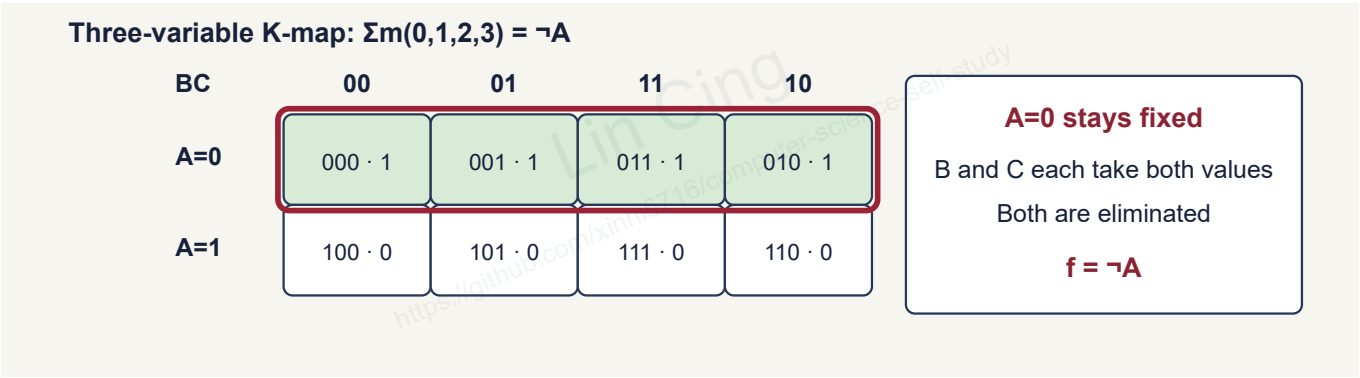
Factoring gives $(A + B)C$, with three literal occurrences, but that expression is not SOP. Both the expression family and the cost must be fixed before claiming a minimum.

15.11 Gray code and a four-cell K-map group

A three-variable Karnaugh map can use rows $A = 0, 1$ and columns $BC = 00, 01, 11, 10$. Gray code makes neighboring columns differ in one bit, including the leftmost and rightmost columns. For

$$f(A, B, C) = \Sigma m(0, 1, 2, 3) = \bar{A},$$

the four cells lie in row $A = 0$. Both B, C vary freely in the group, leaving only $\bar{A}$.



15.11 Gray code and a four-cell K-map group (continued)

The algebra matches the group exactly:

$$\bar{A}(\bar{B}\bar{C} + \bar{B}C + B\bar{C} + BC) = \bar{A}(\bar{B} + B)(\bar{C} + C) = \bar{A}.$$

No cells from a different row are included in this group.

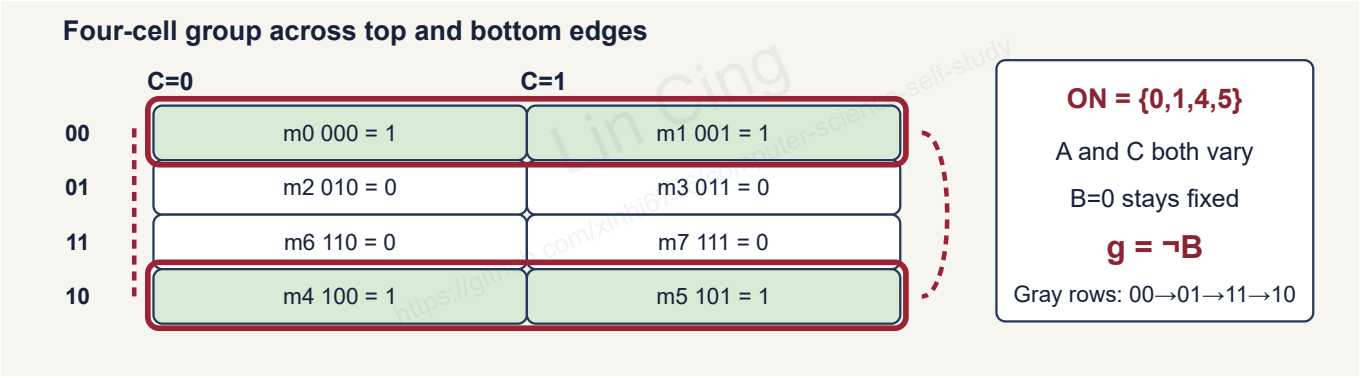
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.12 Wrap-around with a different axis layout

Now use rows $AB = 00, 01, 11, 10$ and columns $C = 0, 1$. For

$$g = \Sigma m(0, 1, 4, 5),$$

m_0, m_1 occupy the top row and m_4, m_5 the bottom row. Opposite top and bottom edges are adjacent because $AB = 00, 10$ differ only in A.



15.12 Wrap-around with a different axis layout (continued)

Within these four cells, A, C vary freely while $B = 0$ is fixed, so $g = \bar{B}$. Wrap-around draws an existing binary-cube adjacency across a planar boundary; it does not join arbitrary nonadjacent inputs.

A legal group is a cube formed by fixing some bits and allowing **all** combinations of the others. It has 2^k cells and eliminates k literals. On a three-variable map, legal groups are rectangles of $1, 2, 4, 8$ cells, allowing wrap-around and overlap. A power-of-two cell count alone is insufficient. An L-shaped incomplete cube is illegal.

15.13 Minimality without essential primes

Let $h = \Sigma m(1,2,3,4,5,6)$, whose only false inputs are $000, 111$. Its six primes are:

Prime	ON indices covered	Prime	ON indices covered
$\bar{A}B$	2,3	$A\bar{B}$	4,5
$\bar{A}C$	1,3	$A\bar{C}$	4,6
$B\bar{C}$	2,6	$\bar{B}C$	1,5

Each ON row is covered by exactly two primes, so none is essential. Every single literal includes 000 or 111, making it illegal. Every legal term covers at most two ON rows, so at least three terms, each with at least two literals, are necessary.

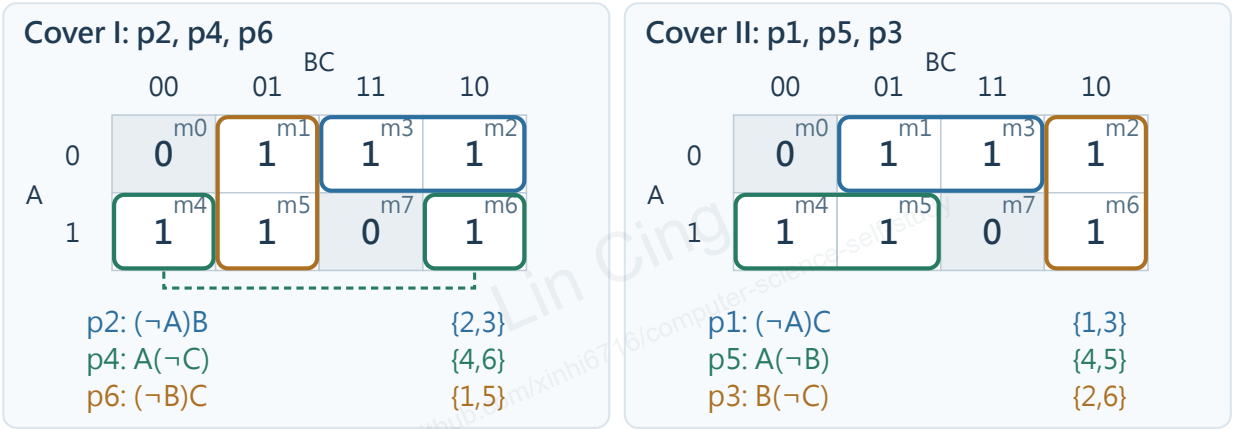
Both covers attain three terms and six literals:

$$h = \bar{A}B + A\bar{C} + \bar{B}C = \bar{A}C + A\bar{B} + B\bar{C}.$$

These are distinct optimal SOPs. Finding all essential primes does not necessarily finish minimization; a remaining chart still requires a cover choice.

Two optimal covers on the same K-map

Gray columns: BC=00,01,11,10. Green end cells form one wrap-around pair.



Each cover uses three two-literal terms and covers all six 1-cells. Exercise 7 proves both optimality and completeness.

15.14 A general cube algorithm

Represent cubes using $0, 1, -$, with $1{-}1$ denoting AC. Two cubes can merge only when their dash positions agree and exactly one remaining fixed bit differs, as in $101, 111 \to 1{-}1$. The merged cube is their union. Cubes with arbitrary mismatched dash positions cannot be combined.

Start with all legal minterms and merge level by level. Mark terms contained in larger cubes. The legal cubes that cannot expand are primes. To prove completeness, induct on the number of free bits. A legal cube with k free bits splits into two legal subcubes by fixing one free bit to 0 and 1. Both halves are generated at the preceding level and merge back to the cube.

Build a prime chart with required ON rows as columns and primes as rows. Select rows covering all columns under the term-count and literal-count objective. Exhaustive search, branch and bound, or exact cover search can solve this selection. Sorting primes by coverage size is only a heuristic and has no general optimality guarantee.

As the variable count grows, the truth table, prime set and cover search may all become large. Convenient small K-maps do not imply a simple polynomial-time minimizer for arbitrary instances.

15.15 Don't-care specifications

Let the ON set F, OFF set R and don't-care set D be pairwise disjoint and partition all assignments. A legal implementation g must equal 1 on F and 0 on R. Choosing either output on each row of D gives a **completion**.

A legal cube for a partial specification need only avoid R. Generate primes using $F \cup D$, but require coverage only of F; a prime covering only don't-care rows need not be selected. This differs from $P \implies f$ for a fully specified function, since no output has yet been assigned to f on D.

For $F = \{0, 2, 4\}, D = \{1, 5\}, R = \{3, 6, 7\}$, one implementation is

$$g = \bar{B} + \bar{A}\bar{C}.$$

The first term accepts $0, 1, 4, 5$; the second accepts $0, 2$. Their union is $0, 1, 2, 4, 5$, which includes all ON rows and no OFF row, and chooses 1 for both don't-care rows.

If later requirements fix outputs on inputs 1 or 5, the specification has changed. The chosen completion must then be checked against those new requirements.

15.16 A literal-count lower bound

For the same specification $F = \{0, 2, 4\}, D = \{1, 5\}, R = \{3, 6, 7\}$, now minimize total literal occurrences.

Among the six single literals, only $\bar{B}$ avoids the OFF set, and it misses ON row $2 = 010$. The other literals $A, \bar{A}, B, C, \bar{C}$ hit OFF rows $6, 3, 3, 3, 6$, respectively.

An SOP using at most two literal occurrences cannot work:

- Two one-literal terms can only repeat $\bar{B}$, still missing 2.
- A single two-literal term cannot cover $0, 2, 4$. Their only common fixed bit is $C = 0$, so there is no second fixed bit. Even the larger cube $\bar{C}$ hits OFF row 6.
- Constant 1 hits the OFF set, while constant 0 misses the ON set.

At least three literals are therefore necessary. The expression $\bar{B} + \bar{A}\bar{C}$ attains this bound. Legality and minimality require different arguments. A truth-table check alone establishes legality.

15.17 Shannon expansion and universal gates

Fix a variable x, and let $f_0 = f|_{x=0}$, $f_1 = f|_{x=1}$. Then

$$f = \bar{x} f_0 + x f_1.$$

At $x = 0$, the right side is f_0; at $x = 1$, it is f_1. These two cases prove the identity on every input. This is also the selection rule of a 2-to-1 multiplexer.

Define NAND by $x \uparrow y = \overline{xy}$. Two-input NAND gates alone give

$$\bar{x} = x \uparrow x, \qquad xy = (x \uparrow y) \uparrow (x \uparrow y), \qquad x + y = (x \uparrow x) \uparrow (y \uparrow y).$$

NAND can implement AND, OR and NOT, and canonical DNF represents every switching function using those operations. Thus NAND is **functionally complete**. The dual argument proves the same for NOR.

This proves that a circuit exists. It does not minimize gate count or model physical loading and propagation delay.

15.18 A circuit with four NAND gates

For $f = A\bar{B} + BC$, define

$$n_B = \mathrm{NAND}(B, B) = \bar{B}, \quad t_1 = \mathrm{NAND}(A, n_B) = \overline{A\bar{B}}, \quad t_2 = \mathrm{NAND}(B, C) = \overline{BC}.$$

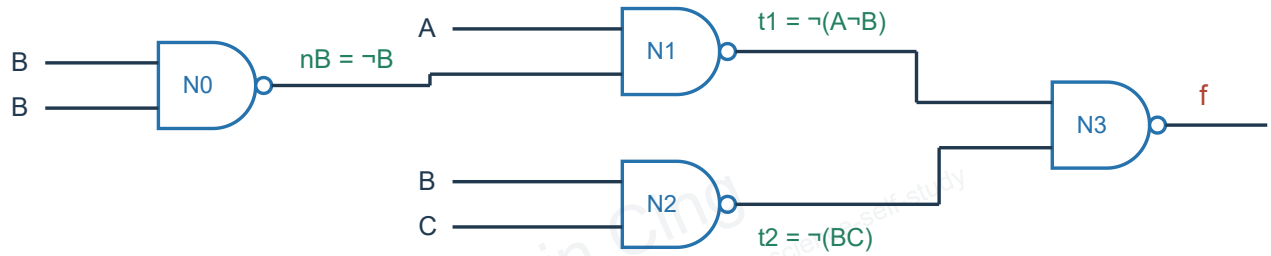
Finally,

$$f = \mathrm{NAND}(t_1, t_2) = \overline{\overline{A\bar{B}}\,\overline{BC}},$$

which is the required OR by De Morgan's law.

15.18 A circuit with four NAND gates (continued)

f = A¬B + BC wired with four two-input NAND gates



Output bubbles invert. Both N0 inputs receive B. Following the wires gives f = ¬(t1·t2).

Both input pins of N0 receive B, so N0 is an inverter. If $\bar{B}$ is externally available, omit N0. The diagram has four two-input NAND gates and maximum logic depth three, along B through N0, N1 and N3. Depth counts gate levels; physical time also depends on each gate's delay model.

15.19 Verify intermediate signals along the wires

Evaluate every wire of the preceding circuit, including each output inversion bubble.

ABC	n_B	t_1	t_2	NAND output	$A\bar{B} + BC$
000	1	1	1	0	0
001	1	1	1	0	0
010	0	1	1	0	0
011	0	1	0	1	1
100	1	0	1	1	1
101	1	0	1	1	1
110	0	1	1	0	0
111	0	1	0	1	1

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.19 Verify intermediate signals along the wires (continued)

The last two columns agree on every row, verifying the function. Four identical gates do not mean four gate delays in series; N1 and N2 lie on parallel branches. A static truth table also makes no guarantee about fan-out, wire delay or transient hazards.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.20 Sets and divisibility as abstract examples

The power set $\mathcal{P}(S)$, with union as join, intersection as meet and complement relative to S, satisfies the axioms. Its zero is $\varnothing$, and its one is S.

Another example uses the positive divisors of 30:

$$B = \{1, 2, 3, 5, 6, 10, 15, 30\}, \qquad x \wedge y = \gcd(x, y), \quad x \vee y = \operatorname{lcm}(x, y).$$

Associate each divisor with the subset of prime factors it contains, for example $6 \leftrightarrow \{2, 3\}$ and $10 \leftrightarrow \{2, 5\}$. Since $30 = 2 \cdot 3 \cdot 5$ has no repeated prime factor, gcd and lcm become intersection and union. The complement is $30/x$.

Here zero is the numeral 1, and one is the numeral 30, not the identities of ordinary integer arithmetic. For example,

$$6 \wedge 10 = 2, \qquad 6 \vee 10 = 30, \qquad \bar{6} = 5.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

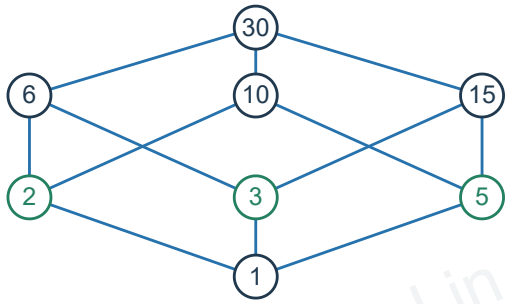
15.21 Atoms in a Hasse diagram

An **atom** is a minimal nonzero element: $a \neq 0$, and $0 < y \leq a \implies y = a$. The atoms of the divisor algebra of 30 are $2, 3, 5$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.21 Atoms in a Hasse diagram (continued)

Divisors of 30: divisibility corresponds to subsets of three atoms



Zero = 1; one = 30 (divisibility order)

Atoms: 2 ↔ {2}, 3 ↔ {3}, 5 ↔ {5}

meet = gcd; join = lcm

6 ∧ 10 = 2; 6 ∨ 10 = 30

¬6 = 5; {2,3} has complement {5}

Every edge is a cover directed upward by divisibility. Only circles are elements; crossings are not vertices.

Moving upward along a cover adds one previously absent prime factor. Not every comparable pair is joined by an edge. The eight subsets of the three atoms correspond exactly to the eight divisors.

Not every divisor lattice is Boolean. A complement b of 2 among divisors of 12 would require $\gcd(2, b) = 1$ and $\operatorname{lcm}(2, b) = 12$. Only $1, 3$ are coprime to 2, and their lcms with 2 are $2, 6$. Thus 2 has no complement.

15.22 Nonzero elements of a finite algebra have atoms below them

Let B be a finite Boolean algebra and $x \neq 0$. The set $\{y : 0 < y \leq x\}$ is nonempty because it contains x. A finite partial order has a minimal element a in this set. If $0 < z \leq a$, transitivity gives $z \leq x$, and minimality forces $z = a$. Hence a is an atom.

Distinct atoms a, b satisfy $a \wedge b = 0$. Otherwise $0 < a \wedge b \leq a, b$, so the atom property forces $a \wedge b = a = b$, a contradiction.

This does not make every nonzero element an atom. The divisor 6 is the join of two atoms and is not an atom. Finiteness matters: in an infinite structure, a descending chain may have no minimal nonzero element, invalidating the selection argument.

15.23 Each element is the join of its atoms

Put $A_x = \{a : a \text{ is an atom and } a \le x\}$, and let $s = \bigvee A_x$, with empty join 0. Since each $a \le x$, we have $s \le x$.

If $s \neq x$, then $x \wedge \bar{s} \neq 0$. Otherwise,

$$x = x \wedge (s \vee \bar{s}) = (x \wedge s) \vee (x \wedge \bar{s}) = s,$$

a contradiction. By the preceding result, an atom a lies below $x \wedge \bar{s}$. Since $a \le x$, it belongs to A_x, so $a \le s$. Also $a \le \bar{s}$, forcing $a \le s \wedge \bar{s} = 0$, another contradiction.

Therefore

$$x = \bigvee_{a \le x,\ a \text{ an atom}} a.$$

In particular, all atoms join to 1. This argument is the core of the finite representation theorem; a few Hasse diagrams do not establish the general statement.

15.24 Atoms and joins

For an atom a, distributivity gives

$$a \wedge (x \vee y) = (a \wedge x) \vee (a \wedge y).$$

Each meet on the right is either 0 or a. Therefore

$$a \le x \vee y \iff a \le x \text{ or } a \le y.$$

The reverse direction follows from the upper-bound property. For the forward direction, if both meets were zero, the displayed expression would contradict $a \neq 0$.

Similarly, $a \le x \wedge y$ exactly when both $a \le x$ and $a \le y$. For complements,

$$a = (a \wedge x) \vee (a \wedge \bar{x}).$$

At least one term is a, and both cannot equal a. Thus

$$a \le \bar{x} \iff a \not\le x.$$

The atom property and distributivity are essential here. The implication $z \le x \vee y \implies z \le x \text{ or } z \le y$ is false for arbitrary elements.

15.25 Representation of finite Boolean algebras

Let A be the set of all atoms. Define $\Phi : B \to \mathcal{P}(A)$ by $\Phi(x) = \{a \in A : a \leq x\}$. The preceding results give

$$\Phi(x \wedge y) = \Phi(x) \cap \Phi(y), \quad \Phi(x \vee y) = \Phi(x) \cup \Phi(y), \quad \Phi(\bar{x}) = A \setminus \Phi(x).$$

Also $\Phi(0) = \varnothing, \Phi(1) = A$, so every operation is preserved.

If $\Phi(x) = \Phi(y)$, the atom-join theorem gives $x = y$, proving injectivity. For any $T \subseteq A$, set $x = \bigvee T$. Each atom of T lies in $\Phi(x)$. If another atom $b \leq x$, repeatedly applying the join property gives $b \leq a$ for some $a \in T$, forcing $b = a$. The empty case has $x = 0$ and also works. Thus $\Phi(x) = T$, proving surjectivity.

Consequently $B \cong \mathcal{P}(A)$. With k atoms, $|B| = 2^k$. This is an isomorphism of operations, not merely an equality of cardinalities. The degenerate algebra has $k = 0$ and corresponds to $\mathcal{P}(\varnothing) = \{\varnothing\}$.

15.26 An infinite algebra without atoms

The finite theorem used an atom below every nonzero element. For an infinite counterexample, take all finite unions of half-open arcs with dyadic endpoints on a circle of circumference 1, together with the empty set. Complements, intersections and unions remain of this form, so this is a Boolean algebra.

Every nonempty element contains a half-open interval of length 2^{-r}. Bisecting it gives a strictly smaller nonempty element in the algebra. Thus there are no atoms. Yet the empty set and the whole circle differ, so the algebra is nonzero.

Without finiteness, neither “no atoms implies a one-element algebra” nor “every Boolean algebra is the power set of its atoms” follows. General representation theorems for infinite Boolean algebras use different objects; the finite proof above does not establish them.

15.27 Boolean algebras and Boolean rings

For sets, symmetric difference corresponds to XOR. In a Boolean algebra, define

$$x \oplus y = (x \wedge \bar{y}) \vee (\bar{x} \wedge y), \qquad xy = x \wedge y.$$

For finite algebras, Φ transforms these into symmetric difference and intersection. They form a commutative ring whose zero and identity are the Boolean $0, 1$, and $x^2 = x$.

Conversely, suppose a ring with identity satisfies $x^2 = x$ for every element. Then $(x + x)^2 = x + x$ gives $4x = 2x$, hence $2x = 0$. Expanding $(x + y)^2 = x + y$ gives $xy + yx = 0$. Since every z satisfies $2z = 0$, this implies $xy = yx$. This includes the degenerate zero ring. Define

$$x \wedge y = xy, \qquad \bar{x} = 1 + x, \qquad x \vee y = x + y + xy.$$

The two uses of addition must be distinguished. Ring $+$ is XOR, while Boolean join is OR. At $x = y = 1$, their results are 0 and 1, respectively, in a nonzero algebra.

15.28 Verifying the Boolean-ring construction

We know the ring is commutative, $2x = 0$ and $x^2 = x$. Define $x \leq y \iff xy = x$. Idempotence proves reflexivity, and commutativity proves antisymmetry. If $xy = x, yz = y$, then $xz = xyz = xy = x$, proving transitivity. Also $0 \leq x \leq 1$.

The product xy is the greatest lower bound: $(xy)x = xy$, and $zx = z, zy = z$ imply $zxy = z$. Put $j = x + y + xy$. Since $xj = x, yj = y$, it is a common upper bound. If $xz = x, yz = y$, then $jz = x + y + xy = j$, so it is the least upper bound. Uniqueness of greatest lower bounds and least upper bounds gives commutativity, associativity and absorption for these lattice operations.

The complement $1 + x$ satisfies $x(1 + x) = 0$ and $x \vee (1 + x) = 1$. Both distributive laws follow by calculation:

$$x(y \vee z) = xy + xz + xyz = (xy) \vee (xz),$$
$$(x \vee y)(x \vee z) = x + yz + xyz = x \vee (yz).$$

The expansions use $x^2 = x$ and $2t = 0$. Thus the construction satisfies the complete Boolean-algebra axioms, rather than merely resembling their notation.

15.29 What to reconstruct independently

- Fix the bit order, derive each minterm and maxterm from its row, and prove the canonical forms equivalent row by row.
- Distinguish legal implicants, primes, essential primes and optimal covers. Give a lower bound and a construction under a fixed expression family and cost.
- Read K-map axes and Gray code, identify the fixed bits of each group, and check that no OFF row is covered.
- Derive all intermediate NAND signals along the actual wires. A static truth table does not establish delay or hazard behavior.
- Derive the order, complements and atoms from the axioms, and prove the finite representation theorem rather than only memorizing 2^k.

The exercises and solutions develop each requirement. Their cost models and finiteness hypotheses are part of the questions.

References and attribution

Ralph P. Grimaldi. Discrete and Combinatorial Mathematics: An Applied Introduction, 5th Edition, 2004.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Groups, coding theory and Pólya counting

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.1 Group axioms and uniqueness

A group is a nonempty set G with a binary operation $G \times G \to G$, associative multiplication, a two-sided identity e, and a two-sided inverse a^{-1} for every a. Closure is included in the operation's codomain.

If e, e' are identities, $e = ee' = e'$. If b, c are inverses of a, then

$$b = eb = (ca)b = c(ab) = ce = c.$$

Left cancellation follows by multiplying $ab = ac$ on the left by a^{-1}. Right cancellation follows similarly.

Associativity changes parentheses, not factor order. In general $(ab)^{-1} = b^{-1}a^{-1}$, because multiplication by this expression on either side gives e.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.1.1 Permutations act on the right first

Use $(\sigma\tau)(i) = \sigma(\tau(i))$. A cycle $(a_1 \cdots a_r)$ sends each point to the next and the last to the first, fixing unlisted points. Disjoint cycles commute. For $g = (13), h = (12)$:

i	$h(i)$	$g(h(i))$	$g(i)$	$h(g(i))$
1	2	2	3	3
2	1	3	2	1
3	3	1	1	2

Thus $gh = (123)$, whereas $hg = (132)$. An r-cycle raised to k fixes all its points exactly when $r \mid k$. For disjoint cycle lengths $r_1, \ldots, r_s$, the permutation order is their lcm. For example, $(123)(45)$ has order six, not five. Operation order also matters for left and right cosets.

16.1.2 Boundaries for matrix and residue multiplication

The invertible real $n \times n$ matrices form $GL_n(\mathbb{R})$. For $n = 2$, take

$$A = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \qquad B = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}.$$

Both determinants are 1, but $AB \neq BA$. For $n > 2$, block-diagonal extension with I_{n-2} preserves this failure. Thus GL_n is nonabelian for $n \geq 2$, whereas $GL_1(\mathbb{R}) \cong \mathbb{R}^\times$ is abelian.

For every $n \geq 2$, all of $\mathbb{Z}_n$ under multiplication is not a group: $[0]$ has no inverse. For prime p, the nonzero classes $\mathbb{Z}_p^\times$ do form a multiplicative group by Bezout.

A multiplicative monoid containing zero is different from its group of units, even at a prime modulus.

16.1.3 Subgroups and orders

A nonempty subset $H \subseteq G$ is a subgroup exactly when $a, b \in H \Rightarrow ab^{-1} \in H$. Setting $a = b$ gives the identity, then taking $a = e$ gives inverses, and applying the test to a, b^{-1} gives products.

Group order means $|G|$. Element order is the least positive m with $g^m = e$, or infinity if none exists. In $(\mathbb{Z}_8, +)$, multiples $2, 4, 6, 0$ show that 2 has order 4 and generates $\{0, 2, 4, 6\}$.

In a finite group, powers repeat, giving a positive power equal to the identity. Infinite order occurs, for example, for $1 \in (\mathbb{Z}, +)$. Lagrange will imply that finite element orders divide group order. The two meanings of order must remain distinct.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.2 Homomorphisms, kernels, and images

A homomorphism $\varphi : G \to H$ satisfies $\varphi(ab) = \varphi(a)\varphi(b)$. It sends identity to identity, because the idempotent $\varphi(e)$ cancels in a group, and sends inverses to inverses.

The kernel $\{g : \varphi(g) = e_H\}$ and image are subgroups, by the subgroup test. Injectivity is equivalent to a trivial kernel. Indeed,

$$\varphi(a) = \varphi(b) \iff \varphi(a^{-1}b) = e_H.$$

A trivial kernel forces $a = b$; injectivity conversely makes the identity's preimage unique.

Preserving an operation does not by itself imply injectivity, surjectivity, or equal cardinalities.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.2.1 Cyclic groups and their images

The subgroup generated by g is $\langle g \rangle = \{g^k : k \in \mathbb{Z}\}$. A cyclic group equals such a subgroup and is abelian, since $g^m g^n = g^{m+n} = g^n g^m$. A finite cyclic group of order n is isomorphic to $\mathbb{Z}_n$ through $[k] \mapsto g^k$.

If $G = \langle g \rangle$, then $\varphi(G) = \langle \varphi(g) \rangle$. The map $\mathbb{Z}_{18} \to \mathbb{Z}_6$, $[k]_{18} \mapsto [k]_6$, is well defined, has kernel $\{0, 6, 12\}$, and is onto.

The Klein four-group $\mathbb{Z}_2 \times \mathbb{Z}_2$ is abelian but not cyclic, since every nonidentity element has order 2.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.2.2 Classification and subgroups of cyclic groups

The map $\psi : \mathbb{Z} \to \langle g \rangle$, $k \mapsto g^k$, is a surjective homomorphism. If g has infinite order, different integers give different powers, so it is an isomorphism. If $o(g) = n$, division with remainder gives $g^k = e \iff n \mid k$. Thus $[k]_n \mapsto g^k$ is a well-defined bijective homomorphism from $\mathbb{Z}_n$.

Every subgroup H of a cyclic group is cyclic. The trivial case is immediate. Otherwise the exponents of elements of H include a positive integer. Let d be the least positive one. For any $g^k \in H$, write $k = qd + r$, $0 \le r < d$. Then $g^r = g^k (g^d)^{-q} \in H$, so minimality forces $r = 0$. Therefore $H = \langle g^d \rangle$. In the finite case the trivial subgroup also has this form with $d = n$.

If $o(g) = n < \infty$, then

$$o(g^k) = \frac{n}{\gcd(n, k)}.$$

Indeed, putting $d = \gcd(n, k)$ gives $n \mid kt \iff n/d \mid t$, because n/d and k/d are coprime. The least positive t is n/d. Thus g^k generates the whole group exactly when $\gcd(n, k) = 1$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.2.3 All homomorphisms between finite cyclic groups

An additive homomorphism $\mathbb{Z}_m \to \mathbb{Z}_n$ is determined by $t = \varphi([1]_m)$, and must satisfy

$$\varphi([a]_m) = [at]_n, \qquad mt \equiv 0 \pmod{n}.$$

Necessity follows from $m[1]_m = 0$. Conversely, when $mt \equiv 0$, changing a representative by qm changes its image by $qmt \equiv 0$, so the map is well defined and additive.

For $d = \gcd(m, n)$, all solutions are $t = j(n/d)$, $j = 0, \dots, d-1$. There are exactly d homomorphisms. The image is $\langle [t]_n \rangle$, with size $n/\gcd(n, t)$. Equal-sized fibers give kernel size $m\gcd(n, t)/n$. Surjectivity requires $\gcd(n, t) = 1$ and $n \mid m$. Cyclicity of both groups alone is insufficient.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.3 Cosets and equality

For $H \leq G$, define left and right cosets

$$aH = \{ah : h \in H\}, \qquad Ha = \{ha : h \in H\}.$$

Unless specified otherwise, use left cosets. The map $h \mapsto ah$ is a bijection $H \to aH$, with inverse $x \mapsto a^{-1}x$. Thus every coset has the same cardinality as H.

$$aH = bH \iff b^{-1}a \in H, \qquad Ha = Hb \iff ab^{-1} \in H.$$

For the first identity, equal cosets put a in bH; conversely $a = bh$ gives $aH = bH$.

Since $e \in H$, every a lies in its own coset. Left and right cosets coincide for abelian groups, but not in general.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.3.1 Why cosets partition the group

If $aH \cap bH \neq \varnothing$, choose $x = ah_1 = bh_2$. Then $a = bh_2h_1^{-1}$, and for every $h \in H$, $ah = b(h_2h_1^{-1}h) \in bH$. Hence $aH \subseteq bH$, and symmetry gives equality.

Distinct cosets are therefore disjoint. Every $g \in G$ belongs to gH, so

$$G = \bigsqcup_{aH \in G/H} aH.$$

For finite G, if there are k cosets, then $|G| = k|H|$.

No normality assumption enters this partition argument. Normality is required for quotient multiplication, not for the coset set itself.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.3.2 Lagrange and its failed converse

For finite G and $H \leq G$,

$$[G : H] = \frac{|G|}{|H|}, \qquad |H| \mid |G|.$$

Apply this to $\langle a \rangle$ to obtain $o(a) \mid |G|$, hence $a^{|G|} = e$.

The converse fails: A_4 has order twelve but no subgroup of order six. A full proof follows after quotient groups are established. Divisibility alone is a necessary subgroup-size condition.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.3.3 Normality

A subgroup H is normal, written $H \trianglelefteq G$, if for every $g \in G$,

$$gH = Hg \iff gHg^{-1} = H \iff g^{-1}Hg = H.$$

Equivalently, $g^{-1}hg \in H$ for every $g \in G, h \in H$; applying inclusion also to g^{-1} yields equality.

Kernels are normal, since

$$\varphi(g^{-1}kg) = \varphi(g)^{-1}\varphi(k)\varphi(g) = e$$

for $k \in \ker \varphi$. Every subgroup of an abelian group is normal.

Normality is preservation under conjugation. It is not a visual symmetry condition on a subgroup diagram.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.3.4 Representative-independent quotient multiplication

Try to define $(aH)(bH) = (ab)H$. If H is normal and $a' = ah_1, b' = bh_2$, then $h_1 b = bh_3$ for some $h_3 \in H$, so

$$a'b' = ah_1bh_2 = abh_3h_2 \in abH.$$

Thus $(a'b')H = (ab)H$, independent of representatives.

Conversely, if multiplication is well defined, replacing representative e of H by $h \in H$ gives $(hg)H = gH$. Hence $hg = gh'$ and $g^{-1}hg = h' \in H$ for every h, g, establishing normality. Associativity then descends from G, identity is H, and inverse of aH is $a^{-1}H$.

Quotient operations act on representatives first. Independence is a necessary mathematical obligation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.3.5 A nonnormal subgroup of S_3

Use right-to-left permutation composition. Let $H = \{e, (12)\}$ and $g = (13)$. Then

$$gH = \{(13), (123)\}, \qquad Hg = \{(13), (132)\}.$$

The sets differ, so H is not normal. The same left coset has representatives (13) and (123). Squaring the first gives $e \in H$, but squaring the second gives $(132) \notin H$. The proposed product of that coset with itself changes with the representatives.

A subgroup always gives cosets, but only a normal subgroup gives this quotient group operation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.3.6 The first isomorphism theorem

Let $\varphi : G \to K$ be a homomorphism and $N = \ker \varphi \trianglelefteq G$. The quotient operation is well defined. Associativity descends from representative multiplication, the identity is N, and the inverse of gN is $g^{-1}N$, so G/N is a group.

Define $\bar{\varphi} : G/N \to \operatorname{Im} \varphi$ by $\bar{\varphi}(gN) = \varphi(g)$. Since

$$gN = hN \iff h^{-1}g \in N \iff \varphi(g) = \varphi(h),$$

the map is both well defined and injective. It is onto by the definition of image. Also

$$\bar{\varphi}((gN)(hN)) = \varphi(gh) = \varphi(g)\varphi(h).$$

Thus $G/\ker \varphi \cong \operatorname{Im} \varphi$. Replacing the image by K requires a separate surjectivity proof.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.3.7 Why the converse to Lagrange fails

The even permutation group A_4 has order $4!/2 = 12$. Suppose H were a subgroup of order 6. Its index would be 2. For every $g \notin H$, both gH and Hg equal $G \setminus H$, so every index-2 subgroup is normal. The natural projection to A_4/H is therefore a homomorphism to a group of order 2.

For any 3-cycle a, its image has order dividing 3 because $a^3 = e$, and also dividing 2 by Lagrange in the quotient. Its order is therefore 1, so every 3-cycle lies in H.

There are $\binom{4}{3} \cdot 2 = 8$ distinct 3-cycles on four points. Each is even because $(abc) = (ac)(ab)$. Together with the identity they force at least nine elements in H, contradicting $|H| = 6$.

16.4 RSA key generation

Choose distinct primes p, q, set $n = pq$, and compute $\varphi(n) = (p-1)(q-1)$. Choose a positive public exponent e coprime to $\varphi(n)$, and use extended Euclid to choose a positive d with $ed \equiv 1 \pmod{\varphi(n)}$.

The public key is (n, e), and the private exponent is d. For a representative $0 \le M < n$,

$$C = M^e \bmod n, \qquad M' = C^d \bmod n.$$

Positive exponents keep powers defined on nonunit messages. This teaching model proves the number-theoretic inverse property. The formulas are deterministic, giving the same ciphertext for the same message, and include neither randomized encoding nor a complete communication protocol.

16.4.1 Correctness for coprime messages

Write $ed = 1 + k(p-1)(q-1)$. If $\gcd(M, n) = 1$, Euler's theorem gives $M^{\varphi(n)} \equiv 1 \pmod{n}$, hence

$$(M^e)^d = M^{ed} = M\big(M^{(p-1)(q-1)}\big)^k \equiv M \pmod{n}.$$

This proves decryption for coprime messages, but alone says nothing about representatives divisible by p or q.

The message space contains every residue class modulo n. Correctness must include noncoprime classes as well.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.4.2 CRT proves correctness for every message

Work modulo p. If $M \equiv 0 \pmod{p}$, then $M^{ed} \equiv M$. Otherwise Fermat gives $M^{p-1} \equiv 1 \pmod{p}$, and since $p - 1 \mid ed - 1$,

$$M^{ed} = M(M^{p-1})^{(ed-1)/(p-1)} \equiv M \pmod{p}.$$

The same cases apply modulo q. Thus both primes divide $M^{ed} - M$. Their coprimality and CRT imply $M^{ed} \equiv M \pmod{pq}$.

This argument includes zero and every message sharing a factor with n, without misapplying Euler's coprime hypothesis.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.4.3 Complete example with $M = 7$

Choose $p = 5, q = 11, e = 17$. Then $n = 55$ and $\varphi(n) = 40$. Euclid gives

$$40 = 2(17) + 6, \quad 17 = 2(6) + 5, \quad 6 = 5 + 1,$$
$$1 = 6 - 5 = 3 \cdot 6 - 17 = 3(40 - 2 \cdot 17) - 17 = 3 \cdot 40 - 7 \cdot 17.$$

Thus $d = 33$. Repeated squares modulo 55 are

$$7^2 \equiv 49, \quad 7^4 \equiv 36, \quad 7^8 \equiv 31, \quad 7^{16} \equiv 26,$$

so $C \equiv 26 \cdot 7 \equiv 17$. For decryption,

$$17^2 \equiv 14, \quad 17^4 \equiv 31, \quad 17^8 \equiv 26, \quad 17^{16} \equiv 16, \quad 17^{32} \equiv 36,$$
$$17^{33} \equiv 36 \cdot 17 = 612 \equiv 7 \pmod{55}.$$

Reduce after every square and multiplication instead of expanding enormous integer powers.

16.4.4 Complete example with noncoprime $M = 15$

Keep $n = 55, e = 17, d = 33$. Since $\gcd(15, 55) = 5$, use CRT. Encryption is zero modulo 5. Modulo 11, $15 \equiv 4$ and

$$4^{17} \equiv 4^7, \qquad 4^2 \equiv 5, \quad 4^4 \equiv 3, \quad 4^7 \equiv 3 \cdot 5 \cdot 4 \equiv 5.$$

Thus $C \equiv 0 \pmod 5, C \equiv 5 \pmod{11}$, giving $C = 5$.

For decryption, $5^{33} \equiv 0 \pmod 5$, while $5^{33} \equiv 5^3 \equiv 4 \pmod{11}$. Write $M' = 5k$. Since $5^{-1} \equiv 9 \pmod{11}$, $k \equiv 9 \cdot 4 \equiv 3$, giving $M' \equiv 15 \pmod{55}$.

This explicitly verifies a noncoprime message. A successful small arithmetic example establishes no cryptographic security.

16.5 Detection and correction guarantees

A length-n block code is $C \subseteq \Sigma^n$, binary when $\Sigma = \{0, 1\}$. Codes here are nonempty. For a singleton set $d(C) = +\infty$, using the empty-pair convention. For at least two codewords, minimum distance is

$$d(C) = \min_{c \neq c'} d(c, c').$$

If $d(C) \geq k + 1$, membership testing detects every nonzero error of weight at most k: its received word lies less than minimum distance from the sent codeword and cannot be a different codeword.

If $d(C) \geq 2t + 1$, radius-t balls around codewords are disjoint. Otherwise one received word in two balls would give $d(c_1, c_2) \leq 2t$ by the triangle inequality. Nearest-codeword decoding therefore uniquely corrects every error of weight at most t.

Detection and correction describe different decoder behavior. Their separate maximum bounds cannot automatically be combined in one protocol.

16.5.1 The distance thresholds are necessary

If two codewords have distance $d \leq k$, an error of weight d changes one directly into the other. A membership detector accepts the received codeword, so it cannot detect every nonzero error of weight at most k.

If $d(c_1, c_2) = d \leq 2t$, construct r by taking the coordinates of c_2 at $\lfloor d/2 \rfloor$ of their differing positions and those of c_1 elsewhere. Then

$$d(r, c_1) = \lfloor d/2 \rfloor \leq t, \qquad d(r, c_2) = \lceil d/2 \rceil \leq t.$$

The same received word can arise from two different transmitted codewords after at most t errors. No decoder can always recover the transmitted word uniquely. Thus $d \geq 2t + 1$ is necessary for the worst-case correction guarantee, not merely sufficient for one algorithm.

Average error rates on a specified probabilistic channel are a separate question.

16.5.2 Why the two guarantees cannot be added

The repetition code $C = \{000, 111\}$ has distance 3. Membership detection detects every one- or two-bit error. Majority decoding corrects every one-bit error. But sending 111 and receiving 100 after two flips gives distance 1 from 000 and distance 2 from 111, so correction misdecodes it as 000.

For a specified correction radius t and additional detection band $s \geq 0$, a bounded-distance decoder can correct errors through t and flag weights $t + 1$ through $t + s$ when $d \geq 2t + s + 1$. If such a heavier received word were within t of another codeword, the codeword distance would be at most $2t + s$, a contradiction.

State the channel model, correction radius, detection band, and behavior outside them. A code's distance is not an unconditional transmission guarantee.

16.6 Hamming distance is a metric

For binary words,

$$wt(x) = |\{i : x_i = 1\}|, \qquad d(x, y) = |\{i : x_i \neq y_i\}| = wt(x + y).$$

Over $\mathbb{F}_2$, a coordinate sum is 1 exactly when the bits differ. Nonnegativity, symmetry, and $d(x, y) = 0 \iff x = y$ follow coordinatewise.

For the triangle inequality,

$$1[x_i \neq z_i] \leq 1[x_i \neq y_i] + 1[y_i \neq z_i].$$

Summing gives $d(x, z) \leq d(x, y) + d(y, z)$.

This metric counts differing positions. Erasures, weighted substitutions, or other channel models require different definitions or guarantees.

16.6.1 Balls and sphere packing

The binary radius-t ball is $B(x,t)=\{y : d(x,y)\le t\}$, of size

$$V(n,t)=\sum_{j=0}^{\min(t,n)}\binom{n}{j}\qquad (t \text{ a nonnegative integer}).$$

Choose the j positions flipped from x. For $t\ge n$, the ball is the whole space. Sums written up to t use $\binom{n}{j}=0$ for $j>n$. A code of size M and minimum distance at least $2t+1$ has disjoint such balls, by the triangle inequality, and therefore

$$M\sum_{j=0}^{t}\binom{n}{j}\le 2^n.$$

Equality defines a perfect t-error-correcting code.

The bound constrains code size. Implementing the correction guarantee still requires an appropriate decoder.

16.7 Linear codes and their matrices

A binary linear code is a subspace $C\le\mathbb{F}_2^n$, with parameters $[n,k,d]$ and $|C|=2^k$. A rank-k, $k\times n$ generator matrix encodes a row message u as $c=uG$.

A rank-$n-k$, $(n-k)\times n$ parity-check matrix satisfies

$$C=\{x : xH^T=0\},\qquad GH^T=0.$$

Rank ensures that containment of the generator row space in this kernel is equality by dimension.

Row operations on G preserve the code but can change the message-to-codeword correspondence. Systematic form $[I_k \mid A]$ preserves message bits in the first k positions.

16.7.1 Why orthogonality also needs rank

For $G = [I_k \mid A]$, take $H = [A^T \mid I_{n-k}]$ over $\mathbb{F}_2$. Then

$$GH^T = [I_k \mid A] \begin{bmatrix} A \\ I_{n-k} \end{bmatrix} = A + A = 0.$$

The identity blocks give ranks k and $n-k$. Orthogonality first proves $\operatorname{rowspan} G \subseteq \ker H$. Rank-nullity makes both spaces have dimension k, proving equality.

Without rank, $GH^T = 0$ is insufficient. For example, $G = [1\ 0]$ and $H = [0\ 0]$ satisfy it, but the row space of G is $\{00, 10\}$, while the kernel of H contains all four words.

A general generator matrix can be put in systematic form by row operations and a coordinate permutation. The permutation relabels coordinates and preserves Hamming distance. It is a separate operation from row reduction.

16.7.2 Minimum distance equals minimum nonzero weight

For a linear code, differences remain codewords and Hamming distance is translation invariant:

$$d(c_1, c_2) = wt(c_1 - c_2).$$

Every nonzero difference is a nonzero codeword, and every nonzero codeword is the difference of itself and zero. Hence

$$d(C) = \min_{c \in C,\, c \neq 0} wt(c).$$

For $C = \{00000, 00011, 01101, 01110\}$, the first two nonzero generators sum to the third. Its nonzero weights are 2, 3, 3, so minimum distance is 2.

This reduction uses linear or suitable group-code structure. For a general nonlinear code, inspect distances between all pairs.

16.8 Syndromes classify cosets

Define the row syndrome $s(r) = rH^T$. Adding a codeword preserves it, since $cH^T = 0$. Conversely,

$$s(r_1) = s(r_2) \iff (r_1 - r_2)H^T = 0 \iff r_1 - r_2 \in C \iff r_1 + C = r_2 + C.$$

Thus syndromes and additive cosets correspond bijectively, with kernel C. Full row rank of H makes every $(n-k)$-bit syndrome attainable.

Changing the row basis of H preserves its kernel but changes syndrome coordinates. More precisely, if $H' = UH$ with U invertible, then $s' = r(H')^T = sU^T$. The zero-syndrome decision is unchanged, but the lookup table must be transformed too; its old bit interpretation cannot simply be reused.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.8.1 Standard arrays and nearest decoding

The first row of a standard array is C. Choose an unused minimum-weight word x as a leader and fill the next row with $x + C$; repeat until all cosets appear.

If received $r = c^* + x$ with a minimum-weight representative x of its coset, then for every $c \in C$, $r + c = x + (c^* + c)$ lies in the same coset. Minimality gives

$$d(c^*, r) = wt(x) \le wt(r + c) = d(c, r).$$

The decoded $c^* = r - x$ is therefore a nearest codeword.

Equal-weight leaders can produce ties. Specify a deterministic choice; the standard array and resulting decoder need not be unique.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.8.2 The $[3, 2]$ parity code

Take $C = \{000, 011, 101, 110\}$ and $H = [1\ 1\ 1]$. Its two cosets are

Coset	Syndrome
$\{000, 011, 101, 110\}$	0
$\{100, 111, 001, 010\}$	1

The nonzero coset has three minimum-weight leaders, 100, 010, 001. Every single-bit error has syndrome 1, so that syndrome cannot identify its position. Minimum distance 2 guarantees single-error detection, but not unique correction of every single error.

Possessing a coset table does not itself supply a single-error correction guarantee. The distances and leader ties determine the limit.

16.9 Hamming parity-check columns

Use every nonzero three-bit column once:

$$H = \begin{bmatrix} 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}.$$

No zero column excludes weight-1 codewords. No repeated columns excludes weight-2 codewords. But columns 1, 2, 3 sum to $110 + 101 + 011 = 000$, so 1110000 is a weight-3 codeword. Thus $d = 3$. The last three columns form the identity, so rank is 3, dimension is 4, and the code has parameters $[7, 4, 3]$.

Permuting columns only relabels coordinates and preserves distances. Always retain the actual column order when locating an error.

16.9.1 Complete systematic encoding

A compatible generator is

$$G = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{bmatrix}.$$

For $u = 1010$, add generator rows 1 and 3 to obtain $c = 1010101$. Parity coordinates satisfy

$$c_5 = c_1 + c_2 + c_4, \quad c_6 = c_1 + c_3 + c_4, \quad c_7 = c_2 + c_3 + c_4.$$

Consequently $cH^T = 0$.

After correction, the first four bits recover the message. Syndrome decoding itself must use all seven positions.

16.9.2 Decoding and perfectness

For $r = 1110101$, compute row syndrome $rH^T = (1, 0, 1)$. Its transpose matches column 2, indicating a single error in position 2. Flip that bit to obtain 1010101.

There are 16 codewords, each with a radius-1 ball of size $1 + 7 = 8$. Their disjoint balls contain $16 \cdot 8 = 128 = 2^7$ words, covering the whole space. The Hamming code is therefore perfect for one-error correction.

A multiple-error syndrome is a column sum and can equal a single column. Interpreting it as one error is justified only within the promised error radius.

16.9.3 A bit-by-bit syndrome trace

The transmitted word $c = 1010101$ becomes $r = 1110101$ after position 2 flips. The three checks, all modulo 2, give

$$s_1 = r_1 + r_2 + r_4 + r_5 = 1 + 1 + 0 + 1 = 1,$$

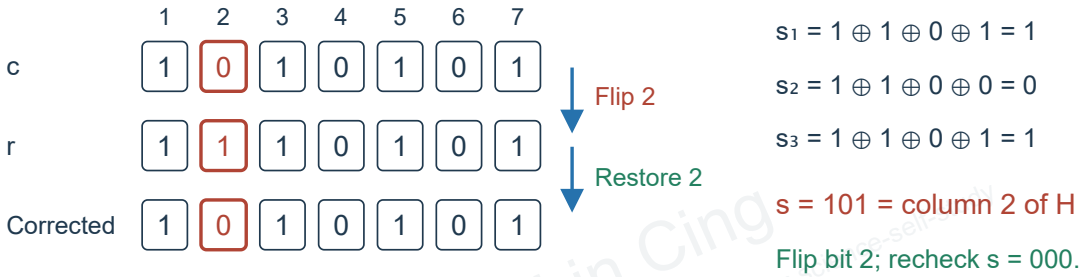
$$s_2 = r_1 + r_3 + r_4 + r_6 = 1 + 1 + 0 + 0 = 0, \qquad s_3 = r_2 + r_3 + r_4 + r_7 = 1 + 1 + 0 + 1 = 1.$$

The syndrome 101 equals column 2. Its numerical binary value 5 is not the error position, because this matrix's columns are not in numerical order.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.9.3 A bit-by-bit syndrome trace (continued)

Position 2 flips: match the syndrome to a column, not its binary value



$s_1 = 1 \oplus 1 \oplus 0 \oplus 1 = 1$

$s_2 = 1 \oplus 1 \oplus 0 \oplus 0 = 0$

$s_3 = 1 \oplus 1 \oplus 0 \oplus 1 = 1$

$s = 101$ = column 2 of H

Flip bit 2; recheck s = 000.

Top labels are bit positions. Red boxes mark position 2. All three sums are modulo 2.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.9.4 Erasures and the Singleton bound

An erasure has a known position. A bit flip does not. For erased positions E, move the known received coordinates to the right-hand side and solve

$$H_E c_E^T = b.$$

Unique recovery holds exactly when the unknown-coordinate columns are linearly independent. If $|E| \le d-1$, a column dependence would create a nonzero codeword of weight below d, a contradiction. Thus every set of at most $d-1$ erasures can be recovered.

For a code of size M, alphabet size q, length n and finite minimum distance d, delete any $d-1$ coordinates. Projection remains injective. Otherwise two distinct codewords could differ only in the deleted coordinates and have distance at most $d-1$. Hence

$$M \le q^{n-d+1}.$$

This is the Singleton bound. A linear $[n,k,d]$ code satisfies $k \le n-d+1$. Its proof differs from the sphere-packing argument.

16.10 Actions, orbits, and stabilizers

A finite group acts on a finite set X if $e \cdot x = x$ and $(gh) \cdot x = g \cdot (h \cdot x)$. Define

$$\mathrm{Orb}(x) = \{g \cdot x : g \in G\}, \quad \mathrm{Stab}(x) = \{g : g \cdot x = x\}.$$

The stabilizer is a subgroup: the identity fixes x, and if g, h fix it, so does gh^{-1}. The relation $y = g \cdot x$ is reflexive, symmetric and transitive by identity, inverses and composition, so orbits partition X. The map $g\,\mathrm{Stab}(x) \mapsto g \cdot x$ is onto the orbit and well defined; equal images mean $g_2^{-1} g_1$ fixes x, hence equal cosets. Thus

$$|G| = |\mathrm{Orb}(x)|\,|\mathrm{Stab}(x)|.$$

An orbit contains configurations; a stabilizer contains group elements. Their cardinalities enter different parts of the theorem.

16.10.1 From positions to colorings

Suppose G permutes a position set Ω. For colorings $f : \Omega \to \{1, \dots, q\}$, define

$$(g \cdot f)(i) = f(g^{-1}i).$$

The color originally at i moves to gi. The inverse is needed because

$$((gh) \cdot f)(i) = f(h^{-1}g^{-1}i) = (g \cdot (h \cdot f))(i).$$

A coloring fixed by g must be constant on each cycle, so every cycle permits one independent color choice. The group moves positions and leaves color names fixed.

Burnside also applies to invariant subsets such as colorings with exactly three red positions. For proper colorings, adjacent vertices must differ, so fixed counts generally require adjacency constraints in addition to $q^{\#\text{cycles}}$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.10.2 Burnside by counting pairs

Let $F = \{(g, x) : g \cdot x = x\}$ and $X^g = \{x : g \cdot x = x\}$. Counting by g gives $|F| = \sum_g |X^g|$. Counting by x gives $|F| = \sum_x |\operatorname{Stab}(x)|$.

On one orbit O, orbit-stabilizer gives each stabilizer size $|G|/|O|$, so

$$\sum_{x \in O} |\operatorname{Stab}(x)| = |O| \frac{|G|}{|O|} = |G|.$$

Every orbit contributes $|G|$, yielding

$$\#(X/G) = \frac{1}{|G|} \sum_{g \in G} |X^g|.$$

The denominator is the group order, not the configuration count. Unequal orbit sizes are handled by their corresponding stabilizers.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.10.3 Two-color square vertices

For four vertices with two named colors, there are 16 raw colorings. The square symmetry group D_4 has eight elements:

Motion	Number	Fixed colorings per motion
Identity	1	16
Quarter turns	2	2
Half turn	1	4
Opposite-vertex reflections	2	8
Opposite-edge reflections	2	4

Thus $(16 + 2 + 2 + 4 + 8 + 8 + 4 + 4)/8 = 6$ orbits. Rotations alone give $(16 + 2 + 2 + 4)/4 = 6$ as well.

The equality for these two groups is specific to this example. Adding reflections can merge orbits in other color sets.

16.11 Cycle index

For a permutation g on n positions, let $j_k(g)$ count its length-k cycles. Define

$$Z_G(x_1, \ldots, x_n) = \frac{1}{|G|} \sum_{g \in G} \prod_{k=1}^{n} x_k^{j_k(g)}.$$

For natural vertex actions,

$$Z_{C_3} = \frac{x_1^3 + 2x_3}{3}, \qquad Z_{C_4} = \frac{x_1^4 + x_2^2 + 2x_4}{4},$$
$$Z_{D_4} = \frac{x_1^4 + x_2^2 + 2x_4 + 2x_1^2 x_2 + 2x_2^2}{8}.$$

The cycle index describes an action before colors are chosen. Equal group orders do not imply equal cycle indices.

16.11.1 A fixed count and an orbit count

Let $c(g) = \sum_k j_k(g)$. A coloring fixed by one specified g is constant on each cycle, so

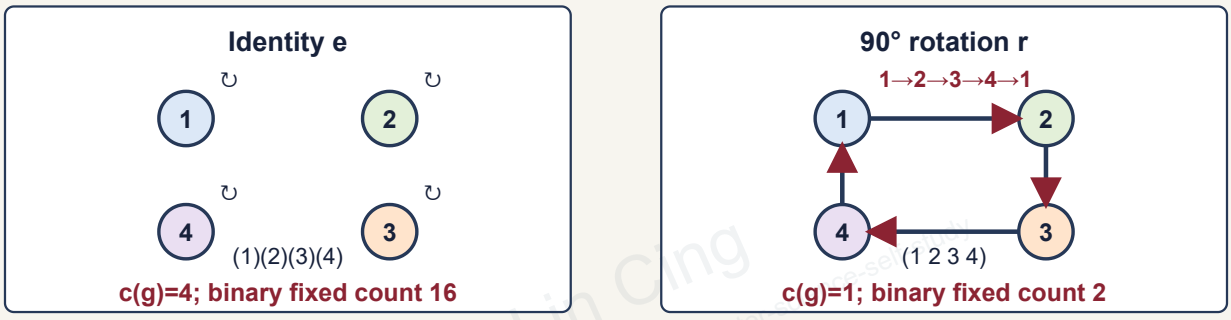
$$|X^g| = q^{c(g)} = \prod_k q^{j_k(g)}.$$

This is one element's fixed count. Only substituting into the already averaged cycle index gives the orbit count

$$Z_G(q, \ldots, q) = \frac{1}{|G|} \sum_g q^{c(g)} = |X/G|.$$

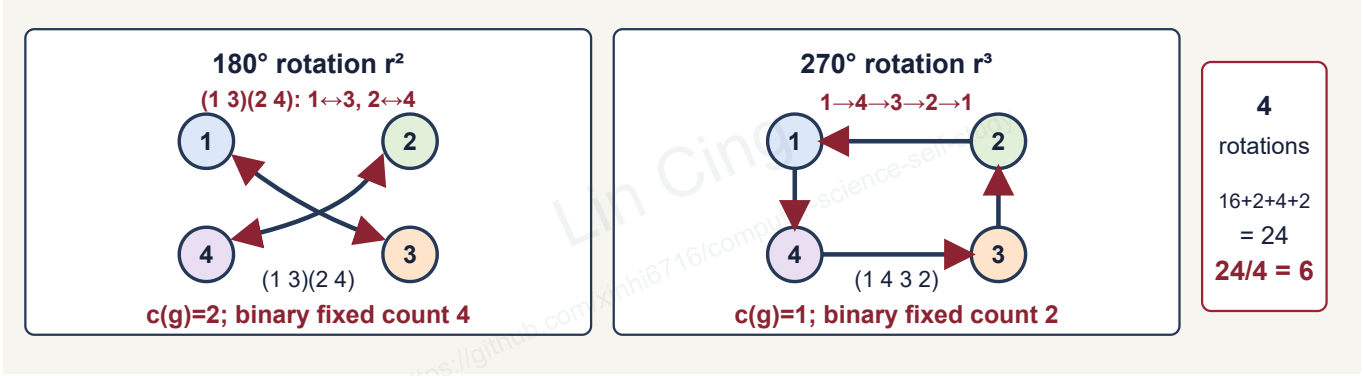
The identity fixes 16 binary configurations, whereas a quarter turn fixes two.

16.11.1 A fixed count and an orbit count (continued)



16.11.2 Half and three-quarter turns

The half turn has two 2-cycles, shown by the double-headed arrows $1 \leftrightarrow 3$ and $2 \leftrightarrow 4$. The three-quarter turn has one 4-cycle. Their fixed binary counts are four and two. The diagram also adds all four counts before averaging.



16.11.3 The fixed-count table and group average

Rotation g	Cycles	$c(g)$	Fixed binary colorings $2^{c(g)}$
e	$(1)(2)(3)(4)$	4	16
r	(1234)	1	2
r^2	$(13)(24)$	2	4
r^3	(1432)	1	2

The group average is $(16 + 2 + 4 + 2)/4 = 6$. Representatives 0000, 0001, 0011, 0101, 0111, 1111 have orbit sizes 1, 4, 4, 2, 4, 1, summing to 16. The identity's fixed count 16 differs from the orbit count 6.

$$Z_{C_4}(q,q,q,q) = \frac{q^4 + q^2 + 2q}{4}, \qquad Z_{D_4}(q,q,q,q) = \frac{q^4 + 3q^2 + 2q^3 + 2q}{8}.$$

For three colors, C_4 gives $(81 + 9 + 6)/4 = 24$ orbits and D_4 gives 21.

16.11.4 The separate three-position necklace

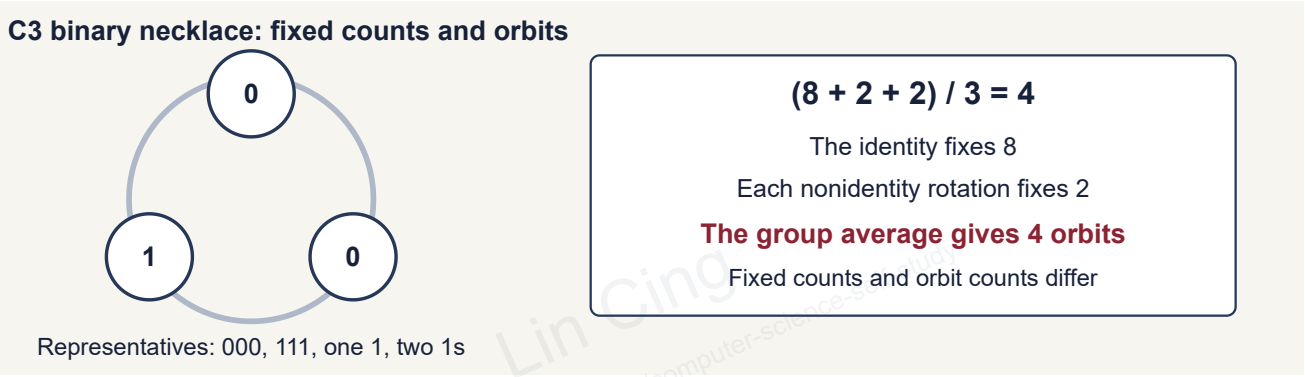
For binary colorings of three positions under C_3, the fixed counts are

$$|X^e| = 2^3 = 8, \qquad |X^r| = |X^{r^2}| = 2,$$

so $|X/C_3| = (8 + 2 + 2)/3 = 4$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.11.4 The separate three-position necklace (continued)



For the preceding binary square under C_4, the average is instead $(16 + 2 + 4 + 2)/4 = 6$. The configuration sets and groups differ. The shared fixed-count averaging method does not make their numerical counts or cycle decompositions interchangeable.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.12 Polya substitution

Give colors formal weights $w_1, \dots, w_m$. A constant color on a length-k cycle contributes w_i^k, so substitute

$$x_k \longmapsto \sum_{i=1}^{m} w_i^k.$$

The resulting pattern inventory counts inequivalent colorings by their color multiplicities. A monomial's coefficient is the number of orbits with those multiplicities. Setting every weight to 1 recovers the total orbit count.

Substituting the same unpowered sum for every x_k loses multiplicity information. The exponent records the number of positions occupied by that cycle's color.

16.12.1 Proof of the weighted substitution

Give a coloring f weight $W(f) = \prod_{i \in \Omega} w_{f(i)}$. Since the group permutes positions, every coloring in an orbit O has the same monomial $W(O)$. Weighted double counting of fixed pairs gives

$$\frac{1}{|G|} \sum_{g} \sum_{f : g \cdot f = f} W(f) = \sum_{O} W(O).$$

Indeed, each of the $|O|$ colorings is fixed by $|G|/|O|$ elements, so averaging contributes its orbit monomial once.

For a fixed g, a length-k cycle in color i contributes w_i^k. Independent choices on different cycles give

$$\sum_{f : g \cdot f = f} W(f) = \prod_{k} \left(\sum_{i} w_i^k \right)^{j_k(g)}.$$

Substituting this expression into the average proves the cycle-index substitution, including the interpretation of every coefficient as an orbit count.

16.12.2 Three colors on a three-position necklace

For rotations C_3, colors r, w, b give

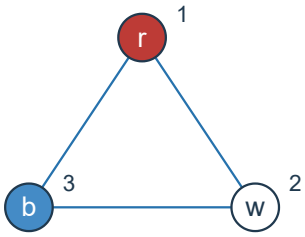
$$\frac{(r+w+b)^3 + 2(r^3+w^3+b^3)}{3} = r^3+w^3+b^3+r^2w+r^2b+w^2r+w^2b+b^2r+b^2w+2rwb.$$

The coefficient 2 on rwb counts two orientations of one red, one white, and one blue bead. Rotations preserve their cyclic order, so reversing it creates another orbit. Total coefficient sum is $3+6+2=11$, matching Burnside's $(27+3+3)/3$.

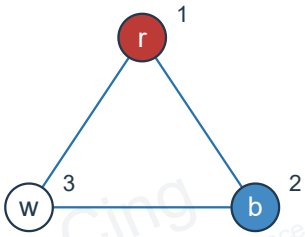
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.12.2 Three colors on a three-position necklace (continued)

One of each color: two rotation orbits; reflections merge them



Representative rwb



Representative rbw

Read colors around 1→2→3

Left: rwb, wbr, brw

Right: rbw, bwr, wrb

The two rotation lists are disjoint.

Reflection swaps positions 2 and 3 and exchanges the drawings. This motion is absent from C_3.

Allowing reflections changes the group and merges the two orientations. A necklace under rotations and a bracelet under rotations and reflections are different orbit problems.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.12.3 Six positions with exactly three red beads

The rotational cycle index is

$$Z_{C_6} = \frac{x_1^6 + x_2^3 + 2x_3^2 + 2x_6}{6}.$$

Substitute $x_k = r^k + b^k$ and extract r^3b^3. The four terms contribute $\binom{6}{3} = 20, 0, 2 \cdot 2 = 4, 0$, respectively. Thus there are $(20 + 4)/6 = 4$ rotational orbits. Dividing all $\binom{6}{3} = 20$ configurations by six fails because the alternating word $rbrbrb$ has only two distinct rotations.

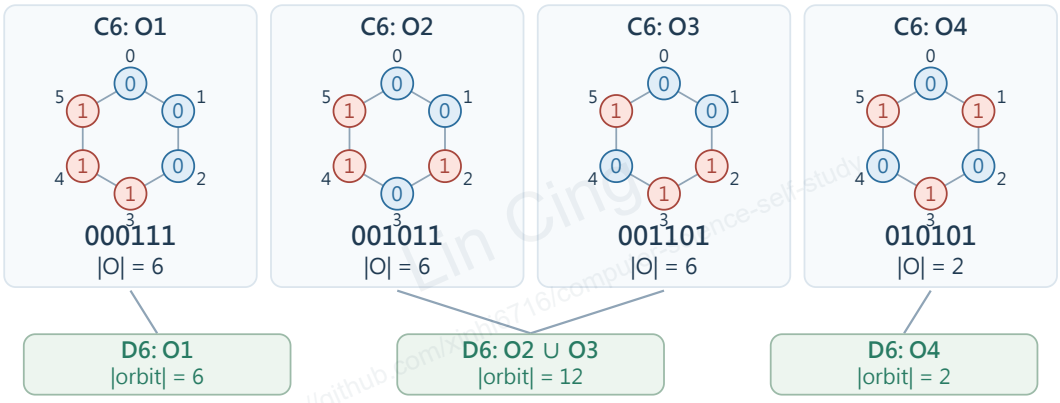
Adding reflections to obtain D_6 adds three elements of type $1^2 2^2$ and three of type 2^3. For the former,

$$[r^3b^3](r + b)^2(r^2 + b^2)^2 = 2 \cdot 2 = 4.$$

The latter allow only even red multiplicities and contribute zero. The bracelet count is $(24 + 3 \cdot 4)/12 = 3$. Changing the permitted reflections can merge equivalence classes.

Four necklaces become three bracelets

Each card shows one representative, not every member of its orbit. Read the six bits clockwise from position 0; r sends position i to i+1 modulo 6.



0 = blue · 1 = red · positions 0–5 run clockwise

Which two rotational classes merge?

For reflection $s:i \mapsto -i \pmod 6$, the word 001011 becomes 011010. A subsequent rotation r gives 001101. Thus reflections merge O2 and O3, which rotations alone cannot relate. O1 and O4 each remain a single class when reflections are added.

Group	Orbit sizes	Check against the 20 labeled configurations
C6	6, 6, 6, 2	6+6+6+2=20
D6	6, 12, 2	6+12+2=20

The alternating word 010101 is fixed by shifts 0,2,4. Its C6 stabilizer has size 3, so its orbit has size 6/3=2. This is why dividing all 20 configurations by 6 fails. Counting orbits uses the specified group action; adding reflections changes equivalence, not the underlying set of configurations.

16.13 Connections and checks

Quotient groups require normality for representative independence. RSA correctness spans the full message space through Fermat and CRT. Code distance and disjoint balls establish correction guarantees, while syndromes identify cosets. Burnside counts fixed pairs; cycle indices encode the action and Polya substitution records color weights.

For each exercise, state the group, code, or action before calculating. Full derivations and executable checks support the finite examples.

Keep hypotheses, counterexamples, and proofs with numerical answers. Memorizing orbit counts or decoding tables does not establish their applicability.

References and attribution

Ralph P. Grimaldi. Discrete and Combinatorial Mathematics: An Applied Introduction, 5th Edition, 2004.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Finite fields and combinatorial designs

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

17.1 Polynomials and polynomial functions

Fix a nonzero field F. An element of $F[x]$ is a finite formal sum $a_0 + a_1 x + \cdots + a_d x^d$. Two polynomials are equal exactly when all their coefficients agree. The symbol x is an indeterminate. Addition is coefficientwise, and multiplication uses convolution:

$$[x^k](fg) = \sum_{i+j=k} a_i b_j.$$

The degree of a nonzero polynomial is the largest index with a nonzero coefficient. Set $\deg 0 = -\infty$. Nonzero constants have degree 0. Since the product of the leading coefficients is nonzero,

$$\deg(fg) = \deg f + \deg g \qquad (f, g \neq 0).$$

Thus $F[x]$ is an integral domain, and its units are precisely the nonzero constants.

Over a finite field, distinct polynomials can define the same function. In $\mathbb{F}_2[x]$, the polynomial $x^2 + x$ is nonzero but evaluates to 0 at both 0 and 1. Equality of polynomials, congruence modulo a polynomial, and equality of evaluation functions are distinct relations.

17.2 The division algorithm

For $a, b \in F[x]$ with $b \neq 0$, there are unique q, r such that

$$a = bq + r, \qquad r = 0 \text{ or } \deg r < \deg b.$$

For existence, initialize $q = 0, r = a$. While $r \neq 0$ and $\deg r \geq \deg b$, set

$$t = \frac{\operatorname{lc}(r)}{\operatorname{lc}(b)} x^{\deg r - \deg b}, \qquad q \leftarrow q + t, \quad r \leftarrow r - tb.$$

The invariant $a = bq + r$ is preserved. Each update cancels the leading term, so the degree strictly decreases and the process terminates.

For uniqueness, suppose $bq + r = bq' + r'$. If $q \neq q'$, then $b(q - q') = r' - r$ has degree at least $\deg b$ on the left and smaller degree on the right, a contradiction. Hence $q = q'$ and $r = r'$.

The field assumption supplies $\operatorname{lc}(b)^{-1}$. In $\mathbb{Z}[x]$, dividing x by $2x$ cannot produce an integer-coefficient quotient and a remainder of degree below 1.

17.3 Division with two cancellation steps

In $\mathbb{F}_5[x]$, take $a = x^3 + 2x^2 + 4$ and $b = x^2 + 2$.

Step	Cancellation term t	Accumulated quotient q	New remainder r
Initial	None	0	$x^3 + 2x^2 + 4$
1	x	x	$2x^2 - 2x + 4 = 2x^2 + 3x + 4$
2	2	$x + 2$	$3x$

The second step subtracts $2(x^2 + 2) = 2x^2 + 4$, so the constant term cancels. The final degree is $1 < 2$, and division stops. Multiplying back gives

$$(x + 2)(x^2 + 2) + 3x = x^3 + 2x^2 + 5x + 4 \equiv x^3 + 2x^2 + 4 \pmod 5.$$

Each subtraction takes place in the coefficients modulo 5. No numerical value is substituted for x.

17.4 The factor theorem and the root bound

Evaluation $ev_\alpha(f) = f(\alpha)$ preserves addition, multiplication, and 1, so it is a ring homomorphism. Division by the monic linear polynomial $x - \alpha$ leaves a constant remainder r. Substituting α gives $r = f(\alpha)$, and therefore

$$f(\alpha) = 0 \iff (x - \alpha) \mid f.$$

A nonzero degree-d polynomial over a field has at most d distinct roots. Prove this by induction. If there is no root, the bound holds. Otherwise choose a root α and write $f = (x - \alpha)g$, where $\deg g = d - 1$. Any other root $\beta \neq \alpha$ satisfies $(\beta - \alpha)g(\beta) = 0$. Cancellation gives $g(\beta) = 0$, so there are at most $d - 1$ other roots.

The bound also holds in an integral domain. It can fail in a ring with zero divisors. In $\mathbb{Z}_8[x]$, $x^2 - 1$ has the four roots 1, 3, 5, and 7, although its degree is 2.

17.5 Polynomial gcd and Bézout coefficients

For a, b not both zero, repeatedly divide:

$$r_{i-1} = q_i r_i + r_{i+1}, \qquad \deg r_{i+1} < \deg r_i.$$

Decreasing degrees guarantee termination. A polynomial divides both r_{i-1}, r_i exactly when it divides both r_i, r_{i+1}. Thus the monic scalar multiple of the last nonzero remainder is the unique monic gcd.

Keep representations $r_i = s_i a + t_i b$ at the same time. The updates are

$$s_{i+1} = s_{i-1} - q_i s_i, \qquad t_{i+1} = t_{i-1} - q_i t_i.$$

Starting with $a = 1a + 0b$ and $b = 0a + 1b$ gives

$$\gcd(a, b) = sa + tb.$$

The final scalar used to make the gcd monic must also multiply s, t.

Consequently, the gcd is 1 if and only if some s, t satisfy $sa + tb = 1$. This algorithm will compute inverses in quotient fields.

17.6 Irreducibility and low degrees

Use the algebra convention from Chapter 14. An irreducible element is nonzero and is not a unit. In $F[x]$, it has degree at least 1, and every factorization $p = ab$ has a nonzero constant factor. Constant units are not irreducible. Every linear polynomial is irreducible.

A polynomial of degree 2 or 3 is reducible if and only if it has a root in the field. A root gives a linear factor by the factor theorem, with a remaining factor of positive degree. Conversely, a factorization into two positive degrees must include degree 1, which gives a root.

In $\mathbb{F}_2[x]$, $x^3 + x + 1$ evaluates to 1 at both 0 and 1, so it is irreducible. However,

$$x^4 + x^2 + 1 = (x^2 + x + 1)^2$$

has no root at 0 or 1 but is reducible. For degree at least 4, absence of roots excludes linear factors, but does not exclude a product of two quadratics.

17.7 Gauss's lemma

An integer polynomial is primitive when its coefficients have gcd 1. The product of two primitive polynomials is primitive. Otherwise some prime p would divide every coefficient of their product. Reduction modulo p would give a zero product in the integral domain $\mathbb{F}_p[x]$, so one factor would be zero modulo p, contrary to primitiveness.

Every nonzero rational polynomial is a rational constant times a primitive integer polynomial. Clear its denominators and divide out the gcd of the coefficients. If primitive $f \in \mathbb{Z}[x]$ factors over $\mathbb{Q}[x]$ into positive degrees, write $f = rGH$, with G, H primitive and $r \in \mathbb{Q}^\times$. Their product is primitive. Comparing the common denominator and coefficient gcd forces $r = \pm 1$. Hence the factorization can be taken over $\mathbb{Z}[x]$.

Thus a primitive integer polynomial is reducible over $\mathbb{Q}[x]$ if and only if it factors into two positive-degree polynomials over $\mathbb{Z}[x]$.

17.8 Eisenstein's criterion

Let $f = a_n x^n + \cdots + a_0 \in \mathbb{Z}[x]$ be primitive. Suppose a prime p satisfies

$$p \nmid a_n, \quad p \mid a_0, \ldots, a_{n-1}, \quad p^2 \nmid a_0.$$

If f were reducible over $\mathbb{Q}[x]$, Gauss's lemma would give $f = GH$ with positive-degree integer factors. Modulo p, the leading coefficients remain nonzero, so both factor degrees are preserved and their product is $a_n x^n$.

In $\mathbb{F}_p[x]$, x divides a product if and only if at least one factor has constant term zero. Repeatedly factoring out x shows that every divisor of a monomial is a monomial. Thus $\bar{G}, \bar{H}$ are positive-degree monomials. Both have constant term zero, giving $p \mid G(0), H(0)$, and hence $p^2 \mid a_0$, a contradiction.

For example, $x^4 + 2x + 2$ satisfies the criterion with $p = 2$, so it is irreducible over $\mathbb{Q}[x]$. This does not assert irreducibility over fields of every characteristic.

17.9 When a polynomial quotient is a field

Let $p \in F[x]$ be nonzero and nonconstant, of degree d. Each class modulo p has a unique representative of degree below d. Add or multiply representatives and reduce by division. The ideal property of (p) makes these operations well-defined.

If p is irreducible, a representative f of a nonzero class satisfies $0 \le \deg f < d$, so $\gcd(f, p) = 1$. Bézout gives $uf + vp = 1$. In the quotient, $[u][f] = [1]$, so every nonzero element has an inverse and the quotient is a field.

Conversely, if $p = ab$ with both factor degrees between 1 and $d - 1$, then $[a], [b] \neq 0$ but $[a][b] = 0$. The quotient cannot be a field. Therefore

$$F[x]/(p) \text{ is a field} \iff p \text{ is irreducible.}$$

If $F = \mathbb{F}_q$, the d free coefficients give q^d classes, whether or not the quotient is a field.

17.10 Coordinates and multiplication in GF(4)

Take the irreducible polynomial $x^2 + x + 1 \in \mathbb{F}_2[x]$ and let $\alpha = [x]$. Then

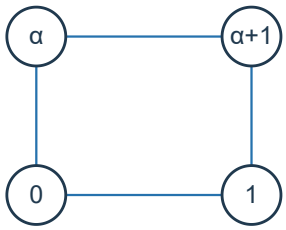
$$GF(4) = \{0, 1, \alpha, \alpha + 1\}, \qquad \alpha^2 = \alpha + 1.$$

Add coefficients modulo 2. For multiplication, expand and then replace α^2.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

17.10 Coordinates and multiplication in GF(4) (continued)

GF(4): two coefficients mod 2, with multiplication given by α² = α + 1



Horizontal: constant; vertical: α.

Addition: α + (α+1) = 1

Multiplication: α(α+1) = α²+α = 1

Multiply by α: 1 → α → α+1 → 1

Each nonzero element has additive order 2.

The two coefficients determine equality. GF(4) differs from integers mod 4: 1+1=0.

Lin Cing

Since $\alpha^3 = \alpha(\alpha + 1) = 1$, the nonzero elements form a multiplicative cycle of length 3. Under addition, $z + z = 0$, so each nonzero element has additive order 2. In $\mathbb{Z}_4$, the element 1 instead has additive order 4.

17.11 Complete operation tables for GF(4)

$+$	**0**	**1**	α	$\alpha+1$
0	0	1	α	$\alpha+1$
1	1	0	$\alpha+1$	α
α	α	$\alpha+1$	0	1
$\alpha+1$	$\alpha+1$	α	1	0

$\cdot$	**0**	**1**	α	$\alpha+1$
0	0	0	0	0
1	0	1	α	$\alpha+1$
α	0	α	$\alpha+1$	1
$\alpha+1$	0	$\alpha+1$	1	α

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

17.11 Complete operation tables for GF(4) (continued)

For example, $(\alpha+1)^2 = \alpha^2 + 2\alpha + 1 = (\alpha+1) + 1 = \alpha$. The inverse table is

$$1^{-1} = 1, \qquad \alpha^{-1} = \alpha + 1, \qquad (\alpha+1)^{-1} = \alpha.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

17.12 A complete Euclidean inverse in GF(8)

Take $p = x^3 + x + 1$, already proved irreducible by its lack of roots. The quotient field has $2^3 = 8$ elements. To find $[x^2]^{-1}$, divide:

$$p = x \cdot x^2 + (x + 1), \qquad x^2 = (x + 1)(x + 1) + 1.$$

Subtraction equals addition in characteristic 2. Back-substitution gives

$$\begin{aligned} 1 &= x^2 + (x + 1)(x + 1) \\ &= x^2 + (x + 1)(p + x \cdot x^2) \\ &= (x^2 + x + 1)x^2 + (x + 1)p. \end{aligned}$$

Thus $[x^2]^{-1} = [x^2 + x + 1]$. Direct multiplication gives $x^4 + x^3 + x^2$. Using $x^3 = x + 1$ and $x^4 = x^2 + x$ reduces this product to 1.

The remainders, Bézout coefficients, and multiplication check all compute the same inverse.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

17.13 Every finite field has prime-power size

In a finite nonzero field K, the sequence $0, 1, 1 + 1, \ldots$ repeats. Let p be the least positive integer with $p1 = 0$. If $p = ab$ with $1 < a, b < p$, then $a1, b1$ would be nonzero with product zero. Hence p is prime.

The map $\mathbb{Z} \to K$, $k \mapsto k1$, has kernel $p\mathbb{Z}$. Its image is $\mathbb{F}_p$, the prime subfield. View K as a vector space over $\mathbb{F}_p$. Since K is finite, it has a finite basis $e_1, \ldots, e_n$, and every element has unique coordinates

$$a_1 e_1 + \cdots + a_n e_n, \qquad a_i \in \mathbb{F}_p.$$

Therefore $|K| = p^n$ with $n \geq 1$.

This proves necessity. To prove existence for every p^n, we cannot assume without proof that an irreducible polynomial of every prescribed degree exists.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

17.14 Splitting a polynomial in finitely many extensions

Every nonconstant polynomial f over a field E has an irreducible factor h. Keep factoring reducible factors. Positive integer degrees decrease, so the process stops. The quotient $E[x]/(h)$ is a field, and $[x]$ is a root of h, hence of f. The embedding of the constants E is injective.

In this extension, use the factor theorem to remove a factor $x - \alpha$. Repeat on the remaining positive-degree factor. Each step removes one linear factor. After at most $\deg f$ steps, a field L contains a full factorization

$$f = a \prod_i (x - \alpha_i).$$

If E is finite, adjoining a root via an irreducible degree-d quotient gives only $|E|^d$ elements. Every extension remains finite, so the final L is finite. This construction uses the quotient theorem without assuming irreducibles of any specified degree.

17.15 Existence for every prime power

Let $q = p^n$. Split $f = x^q - x$ over an extension L of $\mathbb{F}_p$. Formal differentiation gives $f' = qx^{q-1} - 1 = -1$, so there are no repeated roots. Indeed, $(x - a)^2 \mid f$ would force $f'(a) = 0$. Thus f has exactly q distinct roots in L.

Set $K = \{a \in L : a^q = a\}$. In characteristic p, every $\binom{p}{j}$ for $0 < j < p$ is divisible by p. Hence $(a + b)^p = a^p + b^p$, and iteration gives $(a + b)^q = a^q + b^q$.

The set K contains 0 and 1 and is closed under addition, additive inverses, and multiplication. For nonzero $a \in K$,

$$(a^{-1})^q = (a^q)^{-1} = a^{-1},$$

so it is closed under inverses as well. Therefore K is a field with exactly q elements, denoted $GF(q)$.

Together with necessity, this shows that finite field sizes are exactly the prime powers. It does not identify $\mathbb{Z}_{p^n}$ with a field.

17.16 Uniqueness up to isomorphism

Suppose $|K| = |K'| = q = p^n$. Both contain $\mathbb{F}_p$. Lagrange's theorem in the multiplicative group gives $a^{q-1} = 1$ for nonzero a. Including 0, every element is a root of $x^q - x$, so this polynomial splits in both fields.

Start with the identity embedding of $\mathbb{F}_p$. Suppose an intermediate subfield $E \subseteq K$ has been embedded in K'. Choose $a \in K \setminus E$. Finite-dimensional linear dependence among $1, a, a^2, \ldots$ gives a monic minimal polynomial $h \in E[x]$. Division shows that every polynomial vanishing at a is divisible by h, and h is irreducible.

Since $h \mid x^q - x$, transporting its coefficients through the embedding gives $\tilde{h}$ dividing the split polynomial $x^q - x$ in $K'[x]$. Choose a root $b \in K'$. The irreducible quotient gives an embedding

$$\sum c_i a^i \longmapsto \sum \varphi(c_i) b^i$$

of $E(a)$, extending the original embedding.

17.16 Uniqueness up to isomorphism (continued)

Each step strictly enlarges a finite subfield. After finitely many steps this embeds all of K into K'. Equal cardinalities make the injection bijective, hence an isomorphism. Different presentations need not give different finite fields up to isomorphism.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

17.17 An element attaining the exponent

In a finite abelian group, let e be the least common multiple of all element orders. For each prime-power factor ℓ^r of e, choose an element a whose order contains this maximal power. If $o(a) = \ell^r s$ with $\gcd(\ell, s) = 1$, then a^s has order ℓ^r.

Multiply these elements for the different primes. They commute and have pairwise coprime orders. The product has order equal to the product of those orders. For two factors, if $(ab)^t = 1_G$, then $a^t = b^{-t}$. The order of this common element divides both coprime orders, so it is 1. Thus both orders divide t. Their product is also an exponent annihilating ab, proving the claim. Induct on the number of factors.

Consequently, some element has order exactly e. The argument depends on commutativity and cannot be applied to arbitrary nonabelian groups without further hypotheses.

17.18 The multiplicative group is cyclic

For $K = GF(q)$, the group $K^\times$ is abelian of order $q - 1$. Its exponent e divides $q - 1$. All $q - 1$ nonzero elements satisfy $x^e - 1 = 0$, so the polynomial root bound gives

$$q - 1 \le e \le q - 1.$$

Hence $e = q - 1$. The preceding construction supplies an element α of order $q - 1$. Such a primitive element generates $K^\times$.

Now $\mathbb{F}_p[\alpha]$ contains 0 and every nonzero element, so it is K. The minimal polynomial h of α is irreducible, and its quotient field has size $p^{\deg h} = q = p^n$. Thus $\deg h = n$. This also proves that irreducible polynomials of every degree $n \ge 1$ exist over $\mathbb{F}_p$.

When $q = 2$, the multiplicative group has only its identity and has order 1. It is still cyclic, with primitive element 1.

17.19 An irreducible root need not be primitive

In $\mathbb{F}_3[x]$, the values of x^2+1 at 0, 1, 2 are 1, 2, 2. It is irreducible. Let $\alpha^2 = -1 = 2$ in its nine-element quotient field. Then $\alpha^4 = 1$ and $\alpha^2 \neq 1$, so α has multiplicative order 4 rather than 8.

Instead take $\beta = 1 + \alpha$. Then

$$\beta^2 = 1 + 2\alpha + \alpha^2 = 2\alpha, \qquad \beta^4 = (2\alpha)^2 = 2, \qquad \beta^8 = 1.$$

Since $\beta^4 \neq 1$, its order is 8, and it generates all nonzero elements.

Exponent	0	1	2	3	4	5	6	7
β^i	1	$1+\alpha$	2α	$1+2\alpha$	2	$2+2\alpha$	α	$2+\alpha$

For example, $\beta^{-1} = \beta^7 = 2 + \alpha$. Direct multiplication gives $(1+\alpha)(2+\alpha) = 2 + 3\alpha + \alpha^2 = 1$.

17.20 Latin squares and orthogonality

A Latin square of order n is an $n \times n$ array in which every row and every column contains each of the n symbols exactly once. A reduced square has both its first row and first column in a fixed order. Having only the first row in that order is first-row normalization, a weaker condition.

The array $L(x, y) = x + y \pmod{n}$ is Latin for every $n \geq 1$. Fixing one coordinate makes translation in the other coordinate a permutation. Other Latin squares need not be cyclic shifts or satisfy associativity.

Two Latin squares L, M are orthogonal if the n^2 ordered pairs $(L(x, y), M(x, y))$ list all symbol pairs exactly once. Common row and column permutations preserve orthogonality. Independently renaming the symbols in each square also preserves it. Permuting the rows of only one square can destroy it.

17.21 Finite fields construct a complete set of MOLS

Let $F = GF(q)$. For each nonzero a, define

$$L_a(x, y) = ax + y, \qquad x, y \in F.$$

For fixed x, this is translation in y. For fixed y, multiplication by the nonzero a is invertible. Thus each L_a is Latin.

For $a \neq b$, prescribe a symbol pair (u, v) and solve

$$ax + y = u, \qquad bx + y = v.$$

The unique solution is

$$x = (a - b)^{-1}(u - v), \qquad y = u - ax.$$

Every pair occurs exactly once, so L_a, L_b are orthogonal. All $a \in F^\times$ give $q - 1$ mutually orthogonal Latin squares, or MOLS.

The proof needs both a and $a - b$ to be invertible. In $\mathbb{Z}_4$, copying L_1, L_3 fails because their coefficient difference 2 is not invertible. The fact that each square is Latin does not ensure their orthogonality.

17.22 Locating an ordered pair

In $GF(3)$, the nonzero coefficients are 1 and 2. In all three panels below, rows are x and columns are y. The third panel superimposes the two squares cell by cell without moving positions.

Two Latin squares over GF(3): each ordered pair occurs in one cell

$L_1(x,y)=x+y$

	0	1	2
0	0	1	2
1	1	2	0
2	2	0	1

$L_2(x,y)=2x+y$

	0	1	2
0	0	1	2
1	2	0	1
2	1	2	0

Overlay (L_1,L_2)

	0	1	2
0	(0,0)	(1,1)	(2,2)
1	(1,2)	(2,0)	(0,1)
2	(2,1)	(0,2)	(1,0)

Rows x at left, columns y at top. The green cell x=2, y=0 gives (2,1).

To locate the ordered pair $(2, 1)$, compute

$$x = (1 - 2)^{-1}(2 - 1) = 2^{-1} \cdot 1 = 2, \qquad y = 2 - 1 \cdot 2 = 0 \pmod 3.$$

17.22 Locating an ordered pair (continued)

It lies in the green cell $x = 2, y = 0$. All nine ordered pairs are distinct, as predicted by the general unique-solution proof.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

17.23 The upper bound on the number of MOLS

Let $n \geq 2$, and suppose there are k pairwise orthogonal Latin squares. Rename the symbols separately in each square so that its first row is $0, 1, \ldots, n-1$. Symbol renaming preserves cell correspondence and orthogonality.

Examine the entry in the second row and first column of each square. It cannot be 0, because that column already has 0 in its first row. If two squares had the same entry $a \neq 0$ in this cell, their ordered pair (a, a) would occur twice. The other occurrence is in column a of the first row, contradicting orthogonality.

These k entries must therefore be distinct members of the $n-1$ nonzero symbols. Hence $k \leq n-1$. The finite-field construction attains the bound.

Only first-row normalization was used. Independently permuting rows to make every square reduced could change their alignment and invalidate the proof.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

17.24 Affine planes and field coordinates

An affine plane has points, lines, and incidence satisfying three axioms. Two distinct points determine exactly one line. For a line ℓ and a point P off it, exactly one line through P is disjoint from ℓ. At least three points are noncollinear.

Over $F = GF(q)$, take points F^2 and lines $x = c$ or $y = mx + b$. Two points with equal x-coordinates determine their unique vertical line. Otherwise their unique slope is

$$m = (y_2 - y_1)(x_2 - x_1)^{-1},$$

and $b = y_1 - mx_1$ gives the unique intercept.

Distinct lines of the same slope are disjoint. Different slopes give a unique intersection by solving the two equations, and a vertical line meets a nonvertical line exactly once. Thus the unique parallel through an external point has the same direction, with intercept fixed by the point. The points $(0, 0), (1, 0), (0, 1)$ are noncollinear, proving nondegeneracy.

17.25 Directions, parallel classes, and parameters

Each nonvertical direction $m \in F$ has q lines $y = mx + b$. Including the vertical direction gives $q + 1$ parallel classes. The q lines of one class are pairwise disjoint and partition all q^2 points. Lines from different classes meet in exactly one point.

The affine plane $AG(2, q)$ therefore has parameters

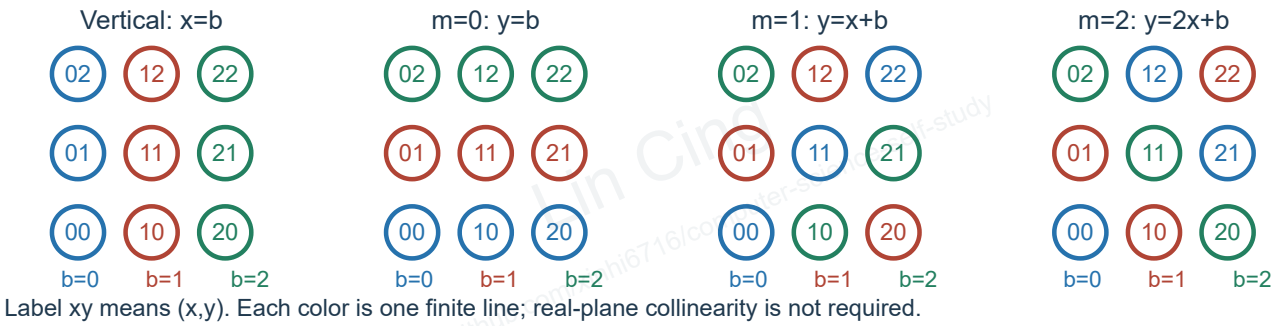
$$v = q^2, \qquad b = q(q + 1), \qquad k = q, \qquad r = q + 1.$$

A vertical line allows any y; every other line allows any x. Thus each line has q points. Exactly one line from each direction passes through a given point, so $r = q + 1$.

Parallel classes partition the set of all lines. Within a single class, the lines partition the set of all points. These are two different partitions.

17.26 All twelve lines of AG(2,3)

Each panel has the same nine points over $GF(3)$; colors distinguish $b = 0, 1, 2$. Label $12 = (1, 2)$, not decimal twelve, satisfies $2 = 2 \cdot 1 + 0$. Incidence follows field equations, not real-plane collinearity.



For $m = 2$, the blocks are $b = 0 : \{00, 12, 21\}$, $b = 1 : \{01, 10, 22\}$, and $b = 2 : \{02, 11, 20\}$.

17.27 Complete MOLS and affine planes

Fix $n \geq 2$. From $n - 1$ MOLS of order n, take the n^2 cells as points. Rows and columns give two parallel classes. In each square, cells with the same symbol form lines in one more class. There are $n + 1$ classes. An order-1 Latin square can be defined, but its single cell cannot give a nondegenerate affine plane.

Each class partitions the cells. Lines of different classes meet exactly once. A row meets a column in its unique cell. A row or column meets a symbol line once by the Latin property. Symbol lines from different squares meet once by orthogonality.

Fix a cell P. Its row and column cover $2(n - 1)$ other cells. Each square's same-symbol line covers $n - 1$ more. Different squares cannot overlap there, since that would repeat an ordered pair. The total is

$$2(n - 1) + (n - 1)^2 = n^2 - 1.$$

Thus every other point shares exactly one line with P. The same class supplies the unique parallel through an external point, giving an affine plane.

17.27 Complete MOLS and affine planes (continued)

Conversely, choose two directions in an affine plane of order n as rows and columns. Their intersections give cells. In each of the other $n-1$ directions, use line labels as symbols. Unique intersections give the Latin property and orthogonality. This correspondence does not claim that every abstract affine plane has field coordinates.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

17.28 Designs and general counting formulas

A $t-(v,k,\lambda)$ design has v points and indexed blocks, each a k-subset. Every t-subset lies in exactly λ blocks. Assume $1 \le t \le k \le v$ and $\lambda \ge 1$. Unless the design is called simple, different block indices may carry the same subset, and counts include multiplicity.

Count pairs (T,B) with $T \subseteq B$ and $|T| = t$:

$$b\binom{k}{t} = \lambda\binom{v}{t}, \qquad b = \lambda\binom{v}{t}\Big/\binom{k}{t}.$$

For any fixed s-subset S, with $0 \le s \le t$, count $S \subseteq T \subseteq B$:

$$\lambda_s\binom{k-s}{t-s} = \lambda\binom{v-s}{t-s}.$$

The number of blocks containing S therefore depends only on s, not on the choice of S. In particular, $\lambda_0 = b$ and $\lambda_1 = r$. All these values must be integers. These are necessary conditions for existence.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

17.29 Two counts for a BIBD

A nondegenerate balanced incomplete block design, or BIBD, is a $2-(v,k,\lambda)$ design with $2 \le k < v$ and $\lambda \ge 1$. Each point lies in exactly r blocks.

Counting point-block incidences gives

$$vr = bk.$$

Fix a point P and count (Q, B) with $Q \neq P$ and $P, Q \in B$. Choosing the block first gives $r(k-1)$, while choosing Q first gives $\lambda(v-1)$. Hence

$$r(k-1) = \lambda(v-1).$$

For $2-(7,3,1)$, this gives $r = 6/2 = 3$ and $b = 7 \cdot 3/3 = 7$. The affine plane $AG(2,q)$ is a $2-(q^2,q,1)$ design. Substitution gives $r = q+1, b = q(q+1)$, agreeing with the direction count.

Parameter conventions vary. The five-parameter notation v, b, r, k, λ must not be read as the notation $t-(v,k,\lambda)$.

17.30 Fisher's inequality

Let M be the $v \times b$ incidence matrix, with points as rows and indexed blocks as columns. Each row has r ones, and two different rows have λ common ones. Over the real numbers,

$$MM^T = (r-\lambda)I + \lambda J.$$

Since $r = \lambda(v-1)/(k-1) > \lambda$, every nonzero real vector z satisfies

$$z^T MM^T z = (r-\lambda)\sum_i z_i^2 + \lambda\left(\sum_i z_i\right)^2 > 0.$$

Thus MM^T has rank v, and

$$v \le \operatorname{rank} M \le b.$$

This proves Fisher's inequality $b \ge v$.

The proposed design $2-(16,6,1)$ gives integral values $r = 3, b = 8$, but $8 < 16$. It therefore does not exist. This explicit obstruction shows why divisibility alone is insufficient.

17.31 Projective plane axioms

A projective plane satisfies three axioms. Any two distinct points lie on exactly one line. Any two distinct lines meet in exactly one point. There are four points with no three collinear. The last axiom excludes degenerate incidence structures. A finite plane has order $q \geq 2$ when every line has $q + 1$ points.

In the field model $PG(2, q)$, points are one-dimensional subspaces $[v] = \{cv : c \in F_q\}$ of F_q^3, with $v \neq 0$. Lines are two-dimensional subspaces, and incidence is containment. The vectors v, cv, with $c \neq 0$, represent the same point and must not be counted separately.

Distinct points have independent representative vectors. Their span is the unique two-dimensional subspace containing them. Two distinct two-dimensional subspaces intersect in dimension 1. The dimension formula gives at least 1, while dimension 2 would make them equal. Thus two lines meet in exactly one point.

The four points $[1, 0, 0], [0, 1, 0], [0, 0, 1], [1, 1, 1]$ have every three independent, with determinant ± 1, proving nondegeneracy.

17.32 Deriving the parameters of PG(2,q)

There are $q^3 - 1$ nonzero vectors in three dimensions. Each one-dimensional subspace contains $q - 1$ nonzero vectors, and these groups are disjoint. Thus

$$v = (q^3 - 1)/(q - 1) = q^2 + q + 1.$$

A two-dimensional line contains $(q^2 - 1)/(q - 1) = q + 1$ points.

Every two-dimensional subspace is the kernel of a nonzero linear functional $a \cdot x$. Two functionals have the same kernel exactly when one is a nonzero scalar multiple of the other. Hence $b = q^2 + q + 1$. Lines through a fixed point P correspond to one-dimensional subspaces of F_q^3/P, so there are $q + 1$ of them.

The result is a $2 - (q^2 + q + 1, q + 1, 1)$ design with $v = b$ and $r = k = q + 1$.

17.32 Deriving the parameters of PG(2,q) (continued)

Duality exchanges points with lines and reverses incidence. The two uniqueness axioms exchange places. Nondegeneracy also needs proof. From four points A, B, C, D with no three collinear, take the lines AB, BC, CD, DA. No three are concurrent. For example, AB, BC meet at B, which lies on neither CD nor DA; the other triples follow cyclically. These four lines are the required four points in the dual. Thus the dual is a projective plane.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

17.33 Adding and deleting a line at infinity

Write an affine point (x, y) as $[x : y : 1]$. Add $P_m = [1 : m : 0]$ for each slope m, and $P_\infty = [0 : 1 : 0]$ for the vertical direction. All new points lie on $\ell_\infty : z = 0$.

Extend $y = mx + b$ to $y = mx + bz$, containing P_m. Extend $x = c$ to $x = cz$, containing P_∞.

Parallel affine lines meet at their unique direction point. Lines of different directions retain their affine intersection. An old point and a direction point determine the unique line of that direction through the point. Two distinct direction points determine ℓ_∞. These cases verify both projective uniqueness axioms. Adding $q + 1$ points and one line, with one new point per old line, gives the projective parameters.

Conversely, delete a projective line ℓ_∞ and its points. Two remaining lines become parallel exactly when their former intersection was on ℓ_∞. For an external point P and remaining line ℓ, let $Q = \ell \cap \ell_\infty$. The unique projective line PQ, with Q removed, is the unique parallel to ℓ through P. The unique join of two remaining points also survives.

17.33 Adding and deleting a line at infinity (continued)

For nondegeneracy, choose $P \notin \ell_\infty$, which the projective axiom guarantees. A line through P meets ℓ_∞ at T. Since it has $q+1 \geq 3$ points, choose $Q \neq P, T$ on it. Choose $S \neq T$ on ℓ_∞, and $R \neq P, S$ on PS. Both Q, R are off ℓ_∞. The lines PS, PQ meet only at P, so P, Q, R are noncollinear. All three affine axioms hold.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

17.34 Fano points, coordinates, and blocks

In $PG(2,2)$, there is no nontrivial nonzero scalar to identify. Label nonzero binary vectors by $1 = 001, \ldots, 7 = 111$. Two distinct vectors u, v span a two-dimensional subspace whose three nonzero vectors are $u, v, u+v$.

The seven line blocks are

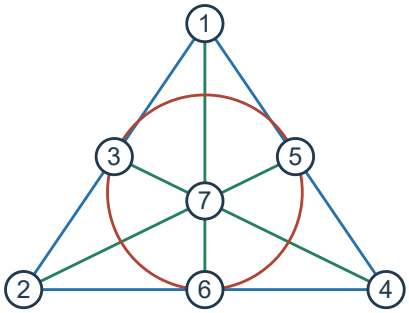
$$123, \quad 145, \quad 167, \quad 246, \quad 257, \quad 347, \quad 356,$$

where, for example, 123 means $\{1,2,3\}$. Every pair u, v has the unique third point $u+v$, so it appears in exactly one block.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

17.34 Fano points, coordinates, and blocks (continued)

Fano plane: seven points and seven blocks; the circle is a line



L1 = {1,2,3}	1 ↔ 001
L2 = {1,4,5}	2 ↔ 010
L3 = {1,6,7}	3 ↔ 011
L4 = {2,4,6}	4 ↔ 100
L5 = {2,5,7}	5 ↔ 101
L6 = {3,4,7}	6 ↔ 110
L7 = {3,5,6}	7 ↔ 111

Right: nonzero binary vectors. The three vectors in each block XOR to 000.

A line is an incidence block. The red circle 356 is also a projective line. Projective lines need not be drawn as straight lines in the real plane.

17.35 Deleting a Fano line

Delete line 123 and its points 1, 2, 3. The affine points are $\{4, 5, 6, 7\}$. Remove the point at infinity from each of the other six lines.

Point at infinity	Projective lines	Affine lines in one parallel class
1	145, 167	$\{4, 5\}, \{6, 7\}$
2	246, 257	$\{4, 6\}, \{5, 7\}$
3	347, 356	$\{4, 7\}, \{5, 6\}$

The two blocks in each class partition the four points, and every pair lies in exactly one block. This is the affine plane of order 2. Lines that previously met at a deleted point are now parallel. Each point at infinity corresponds explicitly to one parallel class.

17.36 Fano lines and Hamming codewords

A Fano line $\{u, v, w\}$ satisfies $u + v + w = 000$. If the parity-check matrix has these seven nonzero vectors as columns, putting 1 in the three corresponding positions and 0 elsewhere gives a weight-3 codeword.

The columns of H in Chapter 16 are not in binary-label order. Their order is

$$110, 101, 011, 111, 100, 010, 001,$$

corresponding to Fano labels $6, 5, 3, 7, 4, 2, 1$. For example, the ones of codeword 1110000 are in positions 1, 2, 3. They give the line $\{6, 5, 3\} = 356$, the red circle in the figure.

The seven lines correspond to the seven weight-3 codewords through the XOR of the same columns. A column permutation must be accompanied by the same permutation of code coordinates and point labels.

17.37 Conditions to check when reconstructing an argument

Reconstruct the calculations from their structures. This includes polynomial degrees and gcds, inverses in irreducible quotients, finite-field existence and isomorphism, unique solutions for MOLS, every point-line incidence case, design double counts, and the rank bound.

Finite-field constructions give planes of prime-power orders. They do not prove that every abstract plane admits field coordinates. Existence of a Latin square does not imply existence of MOLS, and integral design parameters do not supply an incidence structure.

The exercises vary the polynomials, moduli, coefficients, and blocks. They require intermediate computations and general derivations. Each has a complete English solution.

References and attribution

Ralph P. Grimaldi. Discrete and Combinatorial Mathematics: An Applied Introduction, 5th Edition, 2004.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Appendix 1. Exponential and logarithmic functions

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A1.1 Domains of integer powers

For $b \neq 0, n \in \mathbb{Z}_{\geq 0}$, define

$$b^0 = 1, \quad b^n = b \cdot b^{n-1}, \quad b^{-n} = 1/b^n.$$

At $b = 0$, positive integer powers exist, while 0^0 and negative powers require contextual conventions or exclusion. The laws

$$b^{m+n} = b^m b^n, \quad (b^m)^n = b^{mn}, \quad (bc)^n = b^n c^n$$

require all appearing powers to be defined. There is no universal prior evaluation of 0^0; conventions for limits, series, and combinatorial counting are stated where needed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A1.2 Rational powers and principal roots

For $b > 0, p \in \mathbb{Z}, q \in \mathbb{Z}_{>0}$, define $b^{p/q} = (b^{1/q})^p$, where $b^{1/q}$ is the unique positive r satisfying $r^q = b$. Fraction representation does not affect the result, for example $b^{2/4} = b^{1/2}$.

For negative bases, reduce the fraction first. A real rational power is defined through an odd root only when the reduced denominator is odd. Thus $(-8)^{2/6} = (-8)^{1/3} = -2$ is valid despite the unreduced even denominator, while $(-4)^{1/2}$ is not real. The expressions $(x^2)^{1/4}$ and $x^{1/2}$ have different domains; nested-power laws cannot be applied unconditionally to negative bases.

The principal-root convention is part of well-definedness. For $b > 0$, $r^2 = b$ has two real solutions, but its principal square root is the positive one. At zero it is zero.

A1.3 Defining exp by a power series

For every real x, define

$$\exp(x) = E(x) = \sum_{n=0}^{\infty} x^n/n!.$$

The ratio test gives infinite radius of convergence and absolute convergence on bounded intervals. Termwise differentiation gives

$$E'(x) = \sum_{n=1}^{\infty} \frac{nx^{n-1}}{n!} = \sum_{m=0}^{\infty} \frac{x^m}{m!} = E(x),$$

with $E(0) = 1$. Thus E solves $y' = y, y(0) = 1$. After establishing the exponential law, any other solution has $(y(x)E(-x))' = 0$ and initial value one, so $y = E$, proving uniqueness. Infinite radius does not make E a polynomial merely because each term is polynomial.

A1.4 Proving the exponential law

Absolute convergence permits multiplication and rearrangement:

$$\begin{aligned} E(x)E(y) &= \sum_{m,k\geq 0} \frac{x^m y^k}{m!k!} \\ &= \sum_{n=0}^{\infty} \frac{1}{n!} \sum_{m=0}^{n} \binom{n}{m} x^m y^{n-m} \\ &= \sum_{n=0}^{\infty} \frac{(x+y)^n}{n!} = E(x+y). \end{aligned}$$

Taking $y = -x$ gives $E(x)E(-x) = 1$, hence nonvanishing. Since $E(x) = E(x/2)^2$, it is positive and $E(-x) = 1/E(x)$.

Define $e = E(1)$. Induction and roots give $E(r) = e^r$ for rational r; continuity extends this to reals. The binomial expansion proves the identity, and absolute convergence on compact intervals justifies rearrangement.

A1.5 Logarithm as an inverse

Since $E' = E > 0$, E is strictly increasing and injective. For nonnegative x, $E(x) \geq 1 + x$, so it tends to infinity. The reciprocal identity makes $E(x) \to 0$ as $x \to -\infty$. Continuity and the intermediate value theorem therefore give image $(0, \infty)$, and unique inverse functions

$$\exp : \mathbb{R} \to (0,\infty), \qquad \ln : (0,\infty) \to \mathbb{R},$$
$$\ln(e^x) = x, \qquad e^{\ln y} = y.$$

The real logarithm requires positive arguments. Complex logarithms require branch choices beyond this single-valued real definition.

A1.6 Logarithm laws and change of base

For $b > 0$, define every real power by $b^r = E(r \ln b)$. Uniqueness of positive roots makes this agree with the earlier rational definition. The exponential law and inverse give, for $a, b > 0, r \in \mathbb{R}$,

$$\ln(ab) = \ln a + \ln b, \quad \ln(a/b) = \ln a - \ln b, \quad \ln(a^r) = r \ln a.$$

They also prove positive-base real-exponent laws:

$$b^{x+y} = E((x+y)\ln b) = b^x b^y,$$
$$(ab)^x = E(x(\ln a + \ln b)) = a^x b^x,$$
$$(b^x)^y = E(y \ln(E(x \ln b))) = E(xy \ln b) = b^{xy}.$$

The quotient law follows from $b^{-x} = 1/b^x$. All require positive bases and the established inverse.

For $b > 0, b \neq 1$, define $\log_b x = \ln x / \ln b$. Fixed $a, b > 1$ give

$$\log_a x = \frac{\log_b x}{\log_b a},$$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A1.6 Logarithm laws and change of base (continued)

a positive constant factor, hence Θ-equivalence. For a base between zero and one, logarithms are negative at $x > 1$, so positive-cost comparison cannot use the same positive-factor argument directly. Compare absolute values, $|\log_a x| = \Theta(|\log_b x|)$, or retain signs explicitly.

Equal asymptotic classes do not imply equal implementation cost, error, or overflow behavior. Also $\ln(a+b) \neq \ln a + \ln b$; the law concerns products.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A1.7 Derivatives, growth, and boundaries

The inverse derivative gives $(\ln x)' = 1/x$, and $(b^x)' = (\ln b)b^x$. For every real x, $e^x \geq 1 + x$. Nonnegative x permits stronger polynomial bounds from the series. In particular, for fixed n, $x^n/e^x \to 0$, so $x^n = o(e^x)$.

For $b = 0.5$, b^x decreases; it increases only for $b > 1$. State the base range. Mathematical identities need not behave equally in floating point. With $a = b = 10^{200}$, double-precision ab overflows while $\log(a) + \log(b)$ can remain finite. Numerical strategy depends on the input range.

A1.8 Existence and representation independence of rational powers

For $b > 0$ and positive integer q, the function t^q is continuous and strictly increasing on nonnegative reals. For $t > s \geq 0$,

$$t^q - s^q = (t - s)(t^{q-1} + \cdots + s^{q-1}) > 0.$$

The intermediate value theorem between zero and sufficiently large t gives a unique positive q-th root of b.

If $p/q = p'/q'$, raising the two positive candidates to power qq' gives $b^{pq'} = b^{p'q}$. Injectivity of a positive integer power forces equality. Reciprocals handle negative numerators. Thus the definition is independent of fractional representation.

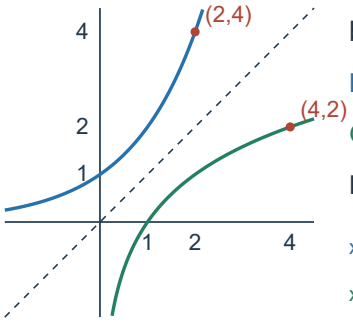
For negative bases, even defined nested powers may fail the usual law:

$$((-8)^2)^{1/2} = 8, \qquad (-8)^{2(1/2)} = -8.$$

Existence of both expressions alone does not extend the positive-base law.

A1.9 Inverse graphs and base-dependent monotonicity

Inversion swaps coordinates across y=x; the base determines monotonicity



b=2

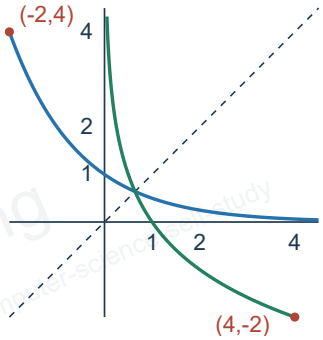
Blue: y=bˣ

Green: y=log_b x

Dashed: y=x

x∈ℝ → y>0

x>0 → y∈ℝ



b=0.5

Blue: y=bˣ

Green: y=log_b x

Dashed: y=x

x∈ℝ → y>0

x>0 → y∈ℝ

Inversion exchanges (x, b^x) and (b^x, x), reflecting the graph in $y = x$. Both functions increase for $b > 1$, where $\ln b > 0$, and decrease for $0 < b < 1$, where $\ln b < 0$. Both remain one-to-one. The exponential range and logarithm domain are the positive reals.

Because the base is below one,

A1.9 Inverse graphs and base-dependent monotonicity (continued)

$$\log_{1/2} x \geq 2 \iff 0 < x \leq (1/2)^2 = 1/4.$$

The inverse reverses the inequality. Substituting $x = 1/8$ gives $3 \geq 2$, while $x = 1$ gives zero and fails.

A1.10 Quantified growth proofs

For fixed integer $k \geq 0, x > 0$, one series term gives

$$e^x \geq \frac{x^{k+1}}{(k+1)!} \Rightarrow 0 \leq \frac{x^k}{e^x} \leq \frac{(k+1)!}{x} \to 0.$$

For fixed real $r \geq 0$, choose integer $k \geq r$; when $x \geq 1$, $x^r \leq x^k$, proving $x^r = o(e^x)$. For fixed $b > 1$, substituting $t = x \ln b$ gives $x^r = o(b^x)$.

For fixed $\varepsilon > 0$, put $u = \varepsilon \ln x \to \infty$. Then

$$\frac{\ln x}{x^\varepsilon} = \frac{1}{\varepsilon} \frac{u}{e^u} \to 0.$$

Thus $\ln x = o(x^\varepsilon)$. Fixing ε, r, b matters; the same conclusion need not hold if b approaches one with x.

A1.11 Halving rounds, bit lengths, and rounding

A positive integer's binary length is $\lfloor \log_2 n \rfloor + 1$. The unique k satisfying $2^k \leq n < 2^{k+1}$ indexes its highest significant bit.

n	Bounding powers	Binary	Length
8	$8 \leq 8 < 16$	1000	4
13	$8 \leq 13 < 16$	1101	4
16	$16 \leq 16 < 32$	10000	5

Replacing workload n by $\lceil n/2 \rceil$ each round until one needs exactly $\lceil \log_2 n \rceil$ rounds for $n \geq 1$, zero at one. After t rounds the value is $\lceil n/2^t \rceil$, by induction using $\lceil \lceil z \rceil / 2 \rceil = \lceil z/2 \rceil$. It first reaches one iff $2^t \geq n$. This differs from bit length, especially at powers of two.

A1.12 Logarithms within numerical range

For $L = \ln(e^a + e^b)$, set $m = \max(a, b)$:

$$L = m + \ln(e^{a-m} + e^{b-m}).$$

Both exponents are nonpositive and one exponential equals one. The sum lies in $[1, 2]$, avoiding overflow in e^a. For $a = 1000, b = 999$, $L = 1000 + \ln(1 + e^{-1}) \approx 1000.313262$. This applies a real identity with input-range control, not the false splitting of a logarithm of a sum.

For the exponential series on $|x| \leq M$, $|x^n/n!| \leq M^n/n!$, with a convergent majorant. The Weierstrass M-test gives uniform convergence. The derivative series has the same bounded-interval estimate. Termwise differentiation uses a power-series theorem, not merely differentiability of individual terms.

Separate exercise and solution PDFs provide domain checks, counterexamples, and quantitative bounds.

References and attribution

Ralph P. Grimaldi. Discrete and Combinatorial Mathematics: An Applied Introduction, 5th Edition, 2004.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Appendix 2. Matrices, matrix operations, and determinants

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A2.1 Matrices and equality

An $m \times n$ matrix is a rectangular array with m rows and n columns,

$$A = (a_{ij})_{1 \le i \le m, 1 \le j \le n}.$$

It is square when $m = n$. Two matrices are equal iff dimensions match and every corresponding entry agrees. Similar-looking entries with different dimensions do not establish equality. A transpose exchanges row and column indices and dimensions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A2.2 Addition and scalar multiplication

For equal dimensions, define $(A + B)_{ij} = a_{ij} + b_{ij}$. For real λ, define $(\lambda A)_{ij} = \lambda a_{ij}$. Entrywise arithmetic gives commutativity and associativity of addition and the distributive laws.

A 2×3 matrix cannot be added to a 3×4 matrix; missing dimensions cannot be padded with zeros. Zero matrices also require matching dimensions, so $0_{2\times 3} + 0_{3\times 2}$ is undefined.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A2.3 Matrix multiplication

For A of size $m \times n$ and B of size $n \times p$, their product has size $m \times p$, with

$$(AB)_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}.$$

Column j of the product is A acting on column j of B. Row i is a linear combination of B's rows with coefficients from A's row i.

For general sizes $m \times n$ and $p \times q$, AB requires $n = p$, and BA requires $q = m$. Both can exist without being equal. To compare them as same-sized arrays requires $m = n$, hence square factors under these simultaneous dimension conditions. Commutativity is not a prerequisite for multiplication and does not generally hold.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A2.4 Associativity, distributivity, and zero divisors

For compatible dimensions, $(AB)C = A(BC)$ and $A(B + C) = AB + AC$. Entrywise,

$$((AB)C)_{ij} = \sum_l \left(\sum_k a_{ik} b_{kl} \right) c_{lj} = \sum_k a_{ik} \left(\sum_l b_{kl} c_{lj} \right) = (A(BC))_{ij}.$$

Finite-sum distribution and rearrangement justify the steps. The matrix ring $M_n(F)$ is noncommutative for $n \geq 2$ and has zero divisors. For example,

$$\begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix} = 0$$

with both factors nonzero. Thus a zero product need not have a zero factor.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A2.5 Transpose, identity, and inverse

Define $(A^T)_{ij} = a_{ji}$; then $(AB)^T = B^T A^T$. The identity I_n has diagonal ones and other entries zero, and acts as identity whenever dimensions are compatible.

For square A, a matrix B satisfying $AB = BA = I_n$ is its unique inverse, written A^{-1}. In size two,

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix}^{-1} = \frac{1}{ad - bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}, \qquad ad - bc \neq 0.$$

Rectangular left or right inverses are separate concepts and are not denoted by an ordinary two-sided A^{-1}.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A2.6 The determinant's domain and definition

For a square $n \times n$ matrix,

$$\det A = \sum_{\sigma \in S_n} \operatorname{sgn}(\sigma) \prod_{i=1}^{n} a_{i,\sigma(i)}.$$

There are $n!$ terms. The determinant is multilinear and alternating in rows, meaning linear in one row with others fixed and sign-changing under row exchange. It does not change sign under arbitrary exchanges of individual entries. For size two it is $ad - bc$.

A nonsquare matrix has no determinant; choosing an arbitrary square block does not define one for the whole matrix. Cofactor expansion is a computational theorem on the same square domain.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A2.7 Minors, cofactors, and expansion

Let M_{ij} be the determinant after deleting row i and column j. Define $C_{ij} = (-1)^{i+j} M_{ij}$. Along any row,

$$\det A = \sum_{j=1}^{n} a_{ij} C_{ij}.$$

A column version also holds. The alternating signs form a checkerboard; missing them is a common error. Expansion gives recursive computation, while elimination is generally preferable for large sizes.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A2.8 Determinant properties and invertibility

For same-sized square matrices,

$$\det(AB) = \det A \det B, \qquad \det(A^T) = \det A.$$

Swapping rows negates the determinant; scaling one row by c multiplies it by c; adding a multiple of one row to another leaves it unchanged. Dependent or repeated rows give zero.

The inverse exists iff $\det A \neq 0$. Then $AX = B$ has unique solution $X = A^{-1}B$, and Cramer's rule also divides by $\det A$. A zero determinant may give no solutions or infinitely many over reals. Invertibility is a criterion; computing the inverse still costs work.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A2.9 Boundary checks

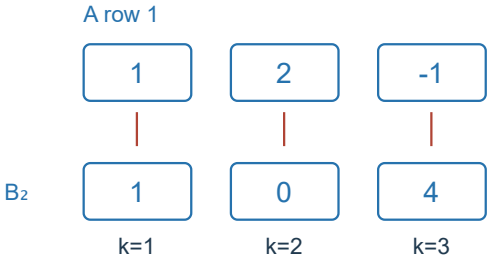
Addition needs identical dimensions; scalar multiplication acts on any fixed-size matrix. Multiplication requires $A_{m\times n}B_{n\times p}$. Transposing a product reverses order. Determinants and ordinary inverses require square matrices, and inverses require nonzero determinant. The formula $A^{-1}B$ for a unique system solution requires these conditions on the coefficient matrix.

State dimensions before numerical calculations. An undefined operation is not a zero result. Group calculations, coding matrices, and signal transformations need the same checks.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A2.10 One contracted index produces one entry

Matrix inner index k: match one row and one column by position



$(AB)_{12}$ = 1×1 + 2×0 + (−1)×4

= 1 + 0 − 4 = −3

All entries: AB = [5 −3 ; 1 8]

(2×3)(3×2) → 2×2; sum over k.

Red lines match the same k; row and column lengths must agree.

$$A = \begin{pmatrix} 1 & 2 & -1 \\ 0 & 3 & 2 \end{pmatrix}, \quad B = \begin{pmatrix} 2 & 1 \\ 1 & 0 \\ -1 & 4 \end{pmatrix}, \quad AB = \begin{pmatrix} 5 & -3 \\ 1 & 8 \end{pmatrix}.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A2.10 One contracted index produces one entry (continued)

Entries are $2 + 2 + 1 = 5$, $1 + 0 - 4 = -3$, $0 + 3 - 2 = 1$, and $0 + 0 + 8 = 8$. Each product term shares inner index k; this is not entrywise multiplication. The product BA exists but is 3×3, unlike AB's 2×2.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A2.11 Alternating multilinearity from the definition

Entries here are real; the proof works over any field. Each Leibniz term uses exactly one entry from a specified row, giving linearity in that row. Exchange rows r, s and pair permutations by exchanging the selected columns in those rows. Their signs reverse. If rows agree, each pair has identical products and opposite signs, summing to zero. This argument never divides by two, so remains valid in characteristic two; merely writing $\det A = -\det A$ would not suffice there.

Expand each row $v_i = \sum_j a_{ij} e_j$ by multilinearity. Repeated basis rows vanish by alternation. Remaining terms permute the basis and have value $\operatorname{sgn}(\sigma)$ times the standard-basis value. Thus any alternating multilinear function with $d(I) = 1$ must be

$$d(A) = \sum_{\sigma} \operatorname{sgn}(\sigma) \prod_{i} a_{i,\sigma(i)} = \det A.$$

A row expressed as a combination of others gives repeated rows in every expansion term, hence zero. This also proves invariance under adding another row's multiple.

A2.12 Product and transpose theorems

Row i of AB is $\sum_k a_{ik} \operatorname{row}_k(B)$. Multilinearity gives

$$\det(AB) = \sum_{k_1,\ldots,k_n} \left(\prod_i a_{ik_i} \right) \det(B_{k_1,*}, \ldots, B_{k_n,*}).$$

Repeated k's vanish. Distinct choices form a permutation $k_i = \sigma(i)$, whose reordered rows have determinant $\operatorname{sgn}(\sigma) \det B$. Therefore

$$\det(AB) = \left(\sum_{\sigma} \operatorname{sgn}(\sigma) \prod_i a_{i,\sigma(i)} \right) \det B = \det A \det B.$$

This proves the theorem for arbitrary size without assuming either matrix invertible.

Replace σ by σ^{-1} in Leibniz, with unchanged sign, and rearrange products to prove $\det(A^T) = \det A$. Row properties therefore hold for columns too. For a triangular matrix, every nonidentity permutation selects a structural zero, leaving the diagonal product.

Separately,

A2.12 Product and transpose theorems (continued)

$$(AB)_{ij}^T = \sum_k a_{jk} b_{ki} = \sum_k (B^T)_{ik} (A^T)_{kj},$$

proving equality of transpose matrices. Equality of determinants alone is a different claim.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A2.13 Cofactors, adjugate, and inverse

Group Leibniz terms according to column j selected in fixed row i. Deleting that row and column changes the remaining permutation sign by $(-1)^{i+j}$, proving row expansion. Transpose gives column expansion. For $n = 1$, use $\det(0 \times 0) = 1$ as the recursion base.

Define $\operatorname{adj}(A)_{ji} = C_{ij}$, reversing indices. Then

$$(A \operatorname{adj} A)_{ij} = \sum_k a_{ik} C_{jk}.$$

For $i = j$ this expands $\det A$. Otherwise it expands along row j after replacing that row by row i, giving two equal rows and determinant zero. Thus $A \operatorname{adj} A = (\det A) I$. Column expansion similarly gives $(\operatorname{adj} A) A = (\det A) I$.

If $\det A \neq 0$, dividing constructs the two-sided inverse. Conversely, an inverse and the product theorem give $\det A \det(A^{-1}) = 1$, so the determinant is nonzero. If B, C are inverses, $B = B(AC) = (BA)C = C$. Likewise $(AB)^{-1} = B^{-1} A^{-1}$, with reversed order.

A2.14 Full elimination of a third-order determinant

$$\begin{pmatrix} 2 & 1 & 0 \\ 0 & 3 & 1 \\ 1 & -1 & 2 \end{pmatrix} \xrightarrow{R_3 - \frac{1}{2}R_1} \begin{pmatrix} 2 & 1 & 0 \\ 0 & 3 & 1 \\ 0 & -\frac{3}{2} & 2 \end{pmatrix} \xrightarrow{R_3 + \frac{1}{2}R_2} \begin{pmatrix} 2 & 1 & 0 \\ 0 & 3 & 1 \\ 0 & 0 & \frac{5}{2} \end{pmatrix}.$$

Both operations preserve the determinant. The triangular result gives $2 \cdot 3 \cdot (5/2) = 15$. Expanding the original first row gives $2(6+1) - 1(-1) = 15$, agreeing.

A zero pivot may require swapping with a nonzero one, recording a minus sign for each swap; never divide by zero. A zero unprocessed column offers no pivot. With all n pivots, the triangular diagonal product is nonzero. Fewer pivots give a zero echelon row and determinant zero. Elimination uses $O(n^3)$ arithmetic operations instead of $n!$ explicit terms; exact rational bit complexity is a separate question.

A2.15 Inversion solves all identity columns together

Apply identical row operations to $[A|I]$:

$$\left[\begin{array}{cc|cc} 2 & 1 & 1 & 0 \\ 1 & 3 & 0 & 1 \end{array}\right] \xrightarrow{R_1/2} \left[\begin{array}{cc|cc} 1 & 1/2 & 1/2 & 0 \\ 1 & 3 & 0 & 1 \end{array}\right] \xrightarrow{R_2 - R_1} \left[\begin{array}{cc|cc} 1 & 1/2 & 1/2 & 0 \\ 0 & 5/2 & -1/2 & 1 \end{array}\right].$$

Scale R_2 by $2/5$, then use $R_1 - R_2/2$:

$$\left[\begin{array}{cc|cc} 1 & 0 & 3/5 & -1/5 \\ 0 & 1 & -1/5 & 2/5 \end{array}\right].$$

Each row operation is invertible left multiplication by an elementary matrix. Their product E satisfies $EA = I$. Since $\det A = 5 \neq 0$, $E = A^{-1}$. For $b = (5, 7)^T$, the solution is $((15-7)/5, (-5+14)/5) = (8/5, 9/5)$, which satisfies both $2x + y = 5$ and $x + 3y = 7$.

A2.16 Cramer and singular-system branches

If A's columns are $a_1, \ldots, a_n$ and $b = \sum_j x_j a_j$, replace column i by b. Expand linearly. Every $j \neq i$ term repeats another column and vanishes; the $j = i$ term is $x_i \det A$. Hence $x_i = \det A_i(b) / \det A$, assuming nonzero determinant and the unique solution.

For zero determinant, inspect the augmented system:

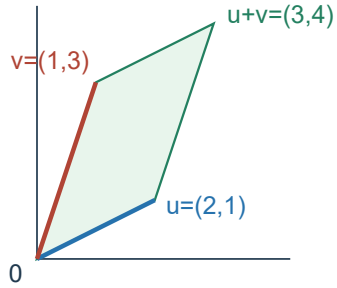
$$\begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix} x = \begin{pmatrix} 3 \\ 6 \end{pmatrix} \Rightarrow x = (3 - 2t, t),\ t \in \mathbb{R}.$$

With the same matrix and right side $(3, 7)^T$, subtract twice the first row from the second to get $0 = 1$, so no solution exists. A zero determinant alone does not select the branch.

Generally an echelon row $[0 \cdots 0 | c]$ with $c \neq 0$ makes a system inconsistent. Otherwise choose free variables arbitrarily and uniquely back-substitute pivot variables. At least one free variable gives infinitely many solutions over reals, but only finitely many over a finite field.

A2.17 Geometric and algebraic determinant values

A columns u=(2,1)、v=(1,3)：signed area det(A)=5



Counterclockwise: 0 → u → u+v → v.

det(A)=2×3−1×1=5 > 0

Swap columns: det=−5; area remains 5.

Dependent columns collapse to a segment; det=area=0.

For columns $u = (2, 1), v = (1, 3)$, the unit square maps to vertices $0, u, u + v, v$. The determinant $6 - 1 = 5$ preserves orientation and multiplies area by five. Swapping columns reverses orientation and gives minus five with the same area. Dependent columns collapse the image to a segment of area zero.

A2.17 Geometric and algebraic determinant values (continued)

In two dimensions, signed area is $u_1v_2 - u_2v_1$, derivable by base times height or subtraction of triangles, matching Leibniz. Ordinary area is its absolute value. Three-dimensional volume also uses absolute determinant; an abstract field need not have a real area interpretation, while the algebraic definition remains valid.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Ralph P. Grimaldi. Discrete and Combinatorial Mathematics: An Applied Introduction, 5th Edition, 2004.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Appendix 3. Countable and uncountable sets

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A3.1 Cardinality and bijections

Write $A \sim B$ if a bijection $A \to B$ exists. Cardinality abstracts this equal-size relation. An infinite set can be equinumerous with a proper subset: $n \mapsto 2n$ bijects $\mathbb{N}^+ = \{1, 2, \ldots\}$ with $2\mathbb{N}^+$. Equal cardinalities do not make these sets equal.

For finite sets, element counts compare size. Infinite cardinality needs existence of bijections rather than the finite intuition that every proper subset is smaller.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A3.2 Countable and countably infinite

A set is finite if it bijects with $\{1, \ldots, n\}$, with the empty set assigned size zero, and infinite otherwise. Countable means finite or equinumerous with $\mathbb{N}^+$. An infinite countable set is countably infinite; uncountable means not countable.

A countable set has an enumeration $A = \{a_1, a_2, \ldots\}$, permitting repetition or termination for finite sets. Equivalently, a nonempty countable set is the image of a surjection from $\mathbb{N}^+$. Treat the empty set separately, since no function maps a nonempty domain to it. For infinite images, remove duplicates to obtain distinct representatives and a bijection. A sequence may repeat terms; a distinct-element listing requires duplicate removal.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A3.3 Integers, pairs, and pairing

The listing $0, 1, -1, 2, -2, \ldots$ enumerates integers, with $f(1) = 0, f(2n) = n, f(2n+1) = -n$. Starting natural numbers at zero, Cantor pairing is

$$\pi(m, n) = \frac{(m+n)(m+n+1)}{2} + m.$$

It bijects $\mathbb{N}_0^2$ with $\mathbb{N}_0$, following diagonals

$$(0,0);\ (0,1),(1,0);\ (0,2),(1,1),(2,0);\ldots.$$

A listing needs a finite index for every object, not a uniquely natural ordering.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A3.4 Closure properties

Subsets of a countable set are countable by retaining their terms in a listing. Products with $\mathbb{N}_0$ and finite products of countable sets are countable by repeated pairing. Countably infinite unions require the conditions on the next section.

Every rational is p/q for $p \in \mathbb{Z}, q \in \mathbb{N}^+$, without requiring reduced form. The map $(p, q) \mapsto p/q$ surjects from a countable product onto $\mathbb{Q}$, including zero via $p = 0$. Remove repeated values to enumerate rationals. Distinct fractions such as $1/2, 2/4$ may represent the same value, so the first map is a surjection.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A3.5 Finite and countably infinite unions

For finitely many countable sets, first use the tagged disjoint union

$$\bigsqcup_{i=1}^{n} A_i = \bigcup_{i=1}^{n} (A_i \times \{i\}).$$

Finite interleaving lists it. Projection $(a, i) \mapsto a$ has image the original union, proving it countable. Original elements are not tagged pairs, so the original union is not automatically a subset of this tagged union. Skip empty sets.

For a countably indexed family with simultaneously specified enumerations $A_n = \{a_{n,1}, a_{n,2}, \ldots\}$, the map $(n, k) \mapsto a_{n,k}$ from a countable pair domain enumerates the union.

If only the separate countability of each A_n is known, simultaneously selecting their enumerations is a choice issue in ZF. Countable Choice suffices; the default background here is standard ZFC unless stated otherwise. A finite family has no infinite simultaneous-choice gap. For an infinite family, uniform enumeration or the relevant choice assumption is part of the proof's conditions.

A3.6 Diagonalizing subsets of natural numbers

Using positive natural numbers, suppose all subsets were listed as $S_1, S_2, \ldots$. Define

$$D = \{n \in \mathbb{N}^+ : n \notin S_n\}.$$

For each k, membership of k in D is opposite to its membership in S_k. Hence $D \neq S_k$ for every k, contradicting completeness of the list.

More generally, Cantor's theorem gives $|A| < |\mathcal{P}(A)|$. Finite sets are countable, but powersets of infinite sets need not be. This proof uses membership directly, without decimal conventions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A3.7 Uncountability and decimal ambiguity

If $(0, 1]$ were countable, its subset $(0, 1)$ would be. Remove endpoint one, and list the remaining values using canonical decimals $r_n = 0.a_{n1}a_{n2}\cdots$, excluding eventually-all-nine alternatives. Define

$$b_k = \begin{cases} 3, & a_{kk} \neq 3, \\ 7, & a_{kk} = 3, \end{cases} \qquad x = 0.b_1 b_2 \cdots .$$

Digits three and seven ensure $0 < x < 1$ with no trailing-nine ambiguity. The k-th digit differs from that of r_k, so x is missing, contradiction. Thus $(0, 1)$, its superset $(0, 1]$, and $\mathbb{R}$ are uncountable.

Without resolving $0.1000\cdots = 0.0999\cdots$, a digit mismatch might concern representations rather than distinct numbers.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A3.8 Directions of countability arguments

Subsets of countable sets are countable. Supersets of uncountable sets are uncountable, but their every subset need not be; finite subsets still exist. Since $\mathbb{Q}$ is countable and $\mathbb{R}$ uncountable, the irrationals are uncountable. Otherwise

$$\mathbb{R} = \mathbb{Q} \cup (\mathbb{R} \setminus \mathbb{Q})$$

would be a union of two countable sets. Infinite cardinal sums and products need bijection arguments rather than unexamined finite arithmetic. State the sets, function direction, and any choice assumptions in each countability claim.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A3.9 Reconstructing the pairing inverse

Let $T_s = s(s+1)/2$, so $\pi(m,n) = T_{m+n} + m$. Diagonal s has $s+1$ pairs and exactly the integer indices in $[T_s, T_{s+1})$.

Given $z \geq 0$, there is a unique s with $T_s \leq z < T_{s+1}$. Put $m = z - T_s, n = s - m$. Interval length gives $0 \leq m \leq s$, so both coordinates are nonnegative and $\pi(m,n) = z$. Every component is unique, proving both onto and one-to-one.

Mathematically $s = \lfloor (\sqrt{8z+1} - 1)/2 \rfloor$. Large-integer code should use integer square root or comparisons instead of rounding floating-point approximations.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A3.9 Reconstructing the pairing inverse (continued)

Cantor pairing: row m, column n; cells contain π(m,n)

	n=0	n=1	n=2	n=3
m=0	0	1	3	6
m=1	2	4	7	11
m=2	5	8	12	17
m=3	9	13	18	24

Green diagonal: m+n=3

π=6,7,8,9 (upper right → lower left)

Invert 18: 15=T_5 ≤18 <T_6=21

m=18−15=3 · n=5−3=2

Window: 0≤m,n≤3. Complete diagonals occupy consecutive, disjoint index intervals.

A3.10 Removing duplicates and listing subsets

For a surjection $f : \mathbb{N}_0 \to A$, take first-occurrence indices

$$I = \{n : \forall k < n, f(k) \neq f(n)\}.$$

The restriction $f|_I$ bijects I with A. Finite I gives finite A. If I is infinite, define $i_0 = \min I$, then $i_{t+1} = \min\{i \in I : i > i_t\}$. Each step exists. Every $i \in I$ has only finitely many smaller natural numbers, so it is eventually listed. Thus $I \sim \mathbb{N}_0$ and A is countable.

For $B \subseteq A$, use a distinct listing of A and retain its indices belonging to B. The same finite-or-infinite argument applies; empty B is handled separately. This proves set existence, not a computable membership test for arbitrary B.

With given union enumerations, map tagged pairs to elements, sort indices by pairing, and remove duplicates. Skip empty A_n instead of inventing enumerations for them.

A3.11 Finite words, finite subsets, and infinite sequences

For a finite alphabet Σ of size k, words of length n number k^n. Length-first, then lexicographic order lists Σ^*, proving countability. It is infinite for $k \geq 1$; for $k = 0$, only the empty word remains.

Finite subsets of $\mathbb{N}_0$ have an explicit bijection with $\mathbb{N}_0$:

$$S \longmapsto \sum_{i \in S} 2^i.$$

Uniqueness of binary expansion proves injectivity, and every integer's finite bit support gives the inverse, with zero corresponding to the empty set.

By contrast, $\{0,1\}^{\mathbb{N}_0}$ is uncountable. Against any listing $s_0, s_1, \ldots$, define $t_i = 1 - s_i(i)$, differing from row i at coordinate i. The infinite product is not the union of finite-length products. Countability of all finite-length layers does not prove countability of infinite sequences.

A3.12 Cantor's theorem for every set

Suppose $f : A \to \mathcal{P}(A)$ is onto and define

$$D = \{a \in A : a \notin f(a)\}.$$

Onto supplies d with $f(d) = D$, giving $d \in D \iff d \notin D$, contradiction. No such surjection or bijection exists. The singleton map $a \mapsto \{a\}$ is injective, hence $|A| < |\mathcal{P}(A)|$.

For empty A, its powerset is the nonempty singleton $\{\varnothing\}$, and no surjection exists either. The proof needs neither decimal representations nor a particular listing order.

A3.12 Cantor's theorem for every set (continued)

Given a proposed list $S_0, S_1, \ldots$, reverse coordinate i of row i to construct D

	i=0	i=1	i=2	i=3
S0	1	0	1	1
S1	0	0	1	0
S2	1	1	1	0
S3	0	1	0	0

First four digits of D: 0,1,0,1

$D(i) = 1 - 1[i \in S_i]$, defined for every i.

D differs from S_0 at coordinate 0,

from S_1 at coordinate 1, and so on.

If D=S_m, coordinate m contradicts it.

The figure shows four coordinates to illustrate reversal. The formal definition covers every index. A finite picture does not claim a finite set has infinite cardinality.

A3.13 Constructing Cantor–Schröder–Bernstein

Given injections $f : A \to B, g : B \to A$, define

$$A_0 = A \setminus g(B), \quad A_{n+1} = g(f(A_n)), \quad C = \bigcup_{n \geq 0} A_n,$$

$$h(a) = \begin{cases} f(a), & a \in C, \\ g^{-1}(a), & a \notin C. \end{cases}$$

The second branch is defined because $a \notin A_0$ implies $a \in g(B)$, with unique preimage under the injection. Each branch is injective. If a cross-branch equality $f(c) = g^{-1}(a)$ occurred for $c \in C, a \notin C$, then $a = g(f(c)) \in C$, contradiction.

For arbitrary $b \in B$, if $g(b) \notin C$, then $h(g(b)) = b$. Otherwise $g(b) \in C$ but not A_0, so it lies in some $A_{n+1} = g(f(A_n))$. Injectivity of g gives $b = f(c) = h(c)$ for some $c \in A_n \subseteq C$. Thus h is onto and a bijection. No arbitrary selection of preimages is needed; injection determines each uniquely.

A3.14 Reals and the powerset of natural numbers

Inject $\mathcal{P}(\mathbb{N}_0)$ into $[0,1] \subseteq \mathbb{R}$ by

$$S \longmapsto \sum_{i \geq 0} \frac{\mathbf{2}_{i \in S}}{3^{i+1}}.$$

If two sets first differ at k, that digit contributes difference $2/3^{k+1}$. The entire later tail can offset at most $1/3^{k+1}$, leaving a positive difference. This proves injectivity, including ternary boundary cases.

Conversely, fix a distinct rational enumeration (q_i), and send x to $\{i : q_i < x\}$. If $x < y$, rational density supplies a q_i between them, distinguishing their sets. To prove density, choose integer N with $N(y-x) > 1$, then integer $Nx < k < Ny$; k/N lies between x, y. CSB gives

$$|\mathbb{R}| = |\mathcal{P}(\mathbb{N}_0)| = 2^{\aleph_0}.$$

Every nonempty open real interval has this cardinality. Map linearly to $(-1,1)$, then use $h(t) = t/(1-|t|)$, whose inverse is $y/(1+|y|)$. This is an explicit bijection rather than an appeal to density.

A3.15 Density, cardinality, and computability

Rationals are countable yet meet every nonempty open interval; density differs from cardinality. Real algebraic numbers are countable. Integer polynomials are finite integer tuples, hence countable, and a nonzero degree-d polynomial has at most d roots by induction using the factor theorem. A countable union of their finite real-root sets is countable. If transcendental reals were also countable, their union with algebraic reals would make all reals countable, contradiction.

Finite programs over a fixed finite alphabet are countable. Each program computes at most one total binary function, so computable total binary functions are countable. All functions $\mathbb{N}_0 \to \{0,1\}$ biject with $\mathcal{P}(\mathbb{N}_0)$ and are uncountable. Thus noncomputable binary functions exist. This existence proof gives no algorithm deciding whether a specified program halts.

If A is uncountable and C countable, then $A \setminus C$ is uncountable. Otherwise $A = (A \setminus C) \cup (A \cap C)$ would be a union of two countable sets. It does not follow that every subset is uncountable.

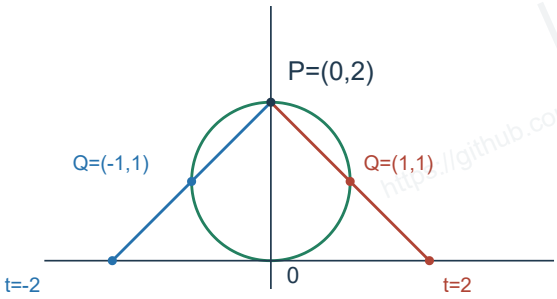
A3.16 A projection between a line and a punctured circle

Let C be $x^2 + (y-1)^2 = 1$, with $P = (0, 2)$. The line from P to $(t, 0)$ meets the circle again at

$$\phi(t) = \left(\frac{4t}{t^2+4}, \frac{2t^2}{t^2+4} \right).$$

A line point is $(st, 2-2s)$. Substitution in the circle equation gives $s^2(t^2+4) - 4s = 0$. Root zero gives P; the other root $4/(t^2+4)$ gives ϕ. At $t = 0$, it still gives $(0, 0)$, without exception.

Projection from P=(0,2): each second circle intersection corresponds to one real t



Circle: x²+(y−1)²=1; P is excluded.

φ(t)=(4t/(t²+4), 2t²/(t²+4))

inverse : t=2x/(2−y) · Q≠P ◦

t=0 uses the y-axis and maps to (0,0).

A3.16 A projection between a line and a punctured circle (continued)

For $Q = (x, y) \in C \setminus \{P\}$, $y < 2$, and the inverse is $t = 2x/(2-y)$. Substituting $x^2 = 2y - y^2$ verifies $\phi(t) = Q$, proving onto and one-to-one.

Adding P back does not change cardinality, as an explicit bijection sends $\mathbb{R}$ plus a new element $*$ to $\mathbb{R}$: send $*$ to zero, each nonnegative integer n to $n+1$, and leave every other real fixed. Combining with ϕ bijects the full circle with the reals. The reason is the construction, rather than treating "adding one changes nothing" as a finite-arithmetic rule.

References and attribution

Ralph P. Grimaldi. Discrete and Combinatorial Mathematics: An Applied Introduction, 5th Edition, 2004.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study